

ZYNQ MPSoC 开发平台 FPGA 教程

2021.1.7 08:33:18

版权声明

Copyright ©2012-2020 芯驿电子科技（上海）有限公司

公司网址:

[Http://www.alinx.com.cn](http://www.alinx.com.cn)

技术论坛:

<http://www.heijin.org>

官方旗舰店:

<http://alinx.jd.com>

邮箱:

avic@alinx.com.cn

电话:

021-67676997

传真:

021-37737073

ALINX 微信公众号:



文档修订记录:

版本	时间	描述
1.01	2020/6/16	初始版本
1.02	2020/10/9	修改 AXU3EG/AXU5EV 的芯片型号
1.03	2021/1/5	修改 RAM 仿真图, 修改 7 寸屏说明

我们承诺本教程并非一劳永逸, 固守不变的文档。我们会根据论坛上大家的反馈意见, 以及实际的开发实践经验积累不断的修正和优化教程。

序

首先感谢大家购买芯驿电子科技（上海）有限公司出品的 ZYNQ 的开发板 AXU3EG、AXU4EV、AXU5EV！您对我们和我们产品的支持和信任,给我们增添了永往直前的信心和勇气。

有人要问零基础能不能学习 ZYNQ？那要看这个零在哪里，如果连原理图都看不懂，C 语言里数组是什么都不知道，对指针完全没有概念，这是负基础，学习 ZYNQ 要有基本的硬件知识，熟练的 C 语言功底。

本教程为 FPGA 部分教程，通过不断练习，掌握 FPGA 开发的基本流程，虽然没有讲解很多大道理，但是熟能生巧，多多练习，逐渐掌握其中的奥秘。

目录

版权声明	2
序	4
目录	5
第一章 Ultrascale+ MPSoC 介绍	11
1.1 PS 和 PL 互联技术	11
1.2 ZYNQ 芯片开发流程的简介	17
1.3 学习 ZYNQ 要具备哪些技能	18
1.3.1 软件开发人员	18
1.3.2 逻辑开发人员	18
第二章 开发板硬件介绍	19
2.1 ACU3EG 核心板	22
2.1.1 简介	22
2.1.2 ZYNQ 芯片	22
2.1.3 DDR4 DRAM	24
2.1.4 QSPI Flash	29
2.1.5 eMMC Flash	30
2.1.6 时钟配置	31
2.1.7 LED 灯	32
2.1.8 电源	33
2.1.9 结构图	35
2.1.10 连接器管脚定义	35
2.2 扩展板	41
2.2.1 简介	41
2.2.2 M.2 接口	42
2.2.3 DP 显示接口	42
2.2.4 USB3.0 接口	43
2.2.5 千兆以太网接口	45
2.2.6 USB Uart 接口	46
2.2.7 SD 卡槽	47
2.2.8 40 针扩展口	48
2.2.9 CAN 通信接口	49
2.2.10 485 通信接口	50
2.2.11 MIPI 接口	51
2.2.12 JTAG 调试口	52
2.2.13 RTC 实时时钟	52
2.2.14 EEPROM 和温度传感器	53

2.2.15 LED 灯	53
2.2.16 按键	54
2.2.17 拨码开关配置	55
2.2.18 电源	56
2.2.19 风扇	57
2.2.20 结构尺寸图	58
第三章 Verilog 基础模块介绍	59
3.1 简介	59
3.2 数据类型	59
3.2.1 常量	59
3.3 变量	60
3.3.1 Wire 型	60
3.3.2 Reg 型	60
3.3.3 Memory 型	61
3.4 运算符	61
3.4.1 算术运算符	61
3.4.2 赋值运算符	61
3.4.3 关系运算符	63
3.4.4 逻辑运算符	63
3.4.5 条件运算符	63
3.4.6 位运算符	63
3.4.7 移位运算符	63
3.4.8 拼接运算符	63
3.4.9 优先级别	63
3.5 组合逻辑	64
3.5.1 与门	64
3.5.2 或门	65
3.5.3 非门	66
3.5.4 异或	66
3.5.5 比较器	67
3.5.6 半加器	68
3.5.7 全加器	69
3.5.8 乘法器	70
3.5.9 数据选择器	70
3.5.10 3-8 译码器	72
3.5.11 三态门	73
3.6 时序逻辑	74
3.6.1 D 触发器	75

3.6.2 两级 D 触发器.....	75
3.6.3 带异步复位的 D 触发器.....	76
3.6.4 带异步复位同步清零的 D 触发器.....	78
3.6.5 移位寄存器.....	79
3.6.6 单口 RAM.....	80
3.6.7 伪双口 RAM.....	80
3.6.8 真双口 RAM.....	81
3.6.9 单口 ROM.....	83
3.6.10 有限状态机.....	84
3.7 总结.....	88
第四章 PL 的“Hello World”LED 实验.....	89
4.1 LED 硬件介绍.....	89
4.2 创建 Vivado 工程.....	90
4.3 创建 Verilog HDL 文件点亮 LED.....	96
4.4 添加管脚约束.....	101
4.5 添加时序约束.....	104
4.6 生成 BIT 文件.....	107
4.7 Vivado 仿真.....	109
4.8 下载.....	114
4.9 在线调试.....	117
4.9.1 添加 ILA IP 核.....	117
4.9.2 MARK DEBUG.....	121
4.10 实验总结.....	124
第五章 Vivado 下 PLL 实验.....	125
5.1 实验原理.....	125
5.2 创建 Vivado 工程.....	126
5.3 仿真.....	131
5.4 板上验证.....	132
第六章 FPGA 片内 RAM 读写测试实验.....	133
6.1 实验原理.....	133
6.2 创建 Vivado 工程.....	133
6.3 RAM 的端口定义和时序.....	136
6.4 测试程序编写.....	137
6.5 仿真.....	139
6.6 板上验证.....	140
第七章 FPGA 片内 ROM 测试实验.....	141
7.1 实验原理.....	141
7.2 程序设计.....	141
7.2.1 创建 ROM 初始化文件.....	141

7.2.2 添加 ROM IP 核	142
7.3 ROM 测试程序编写	145
7.4 仿真	146
7.5 板上验证	146
第八章 FPGA 片内 FIFO 读写测试实验	148
8.1 实验原理	148
8.2 创建 Vivado 工程	149
8.2.1 添加 FIFO IP 核	149
8.2.2 FIFO 的端口定义与时序	151
8.3 FIFO 测试程序编写	153
8.4 仿真	158
8.5 板上验证	159
第九章 Vivado 下按键实验	161
9.1 按键硬件电路	161
9.2 程序设计	161
9.3 创建 Vivado 工程	162
9.4 板上验证	164
第十章 PWM 呼吸灯实验	165
10.1 实验原理	165
10.2 实验设计	165
10.3 下载验证	169
第十一章 UART 实验	170
11.1 程序设计	170
11.1.1 异步串口通信协议	170
11.1.2 波特率	171
11.1.3 接收模块设计	171
11.1.4 发送模块设计	172
11.1.5 波特率的产生	173
11.1.6 测试程序	173
11.2 仿真	175
11.3 实验测试	176
第十二章 RS485 实验	179
12.1 实验原理	179
12.2 程序设计	180
12.3 实验测试	181
第十三章 PL 端 DDR4 读写测试实验	183
13.1 硬件介绍	183
13.2 Vivado 工程建立	183

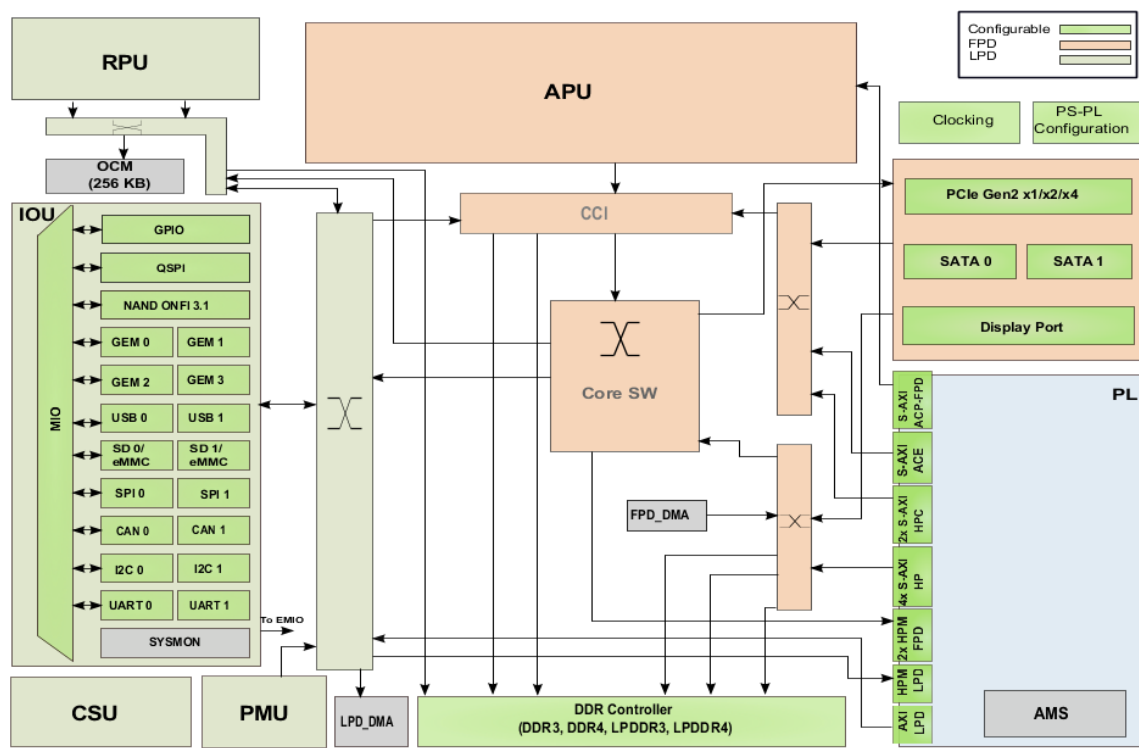
13.2.1 创建一个 PL 端 ddr4 测试工程并配置 ddr4 IP.....	183
13.2.2 添加其他测试代码	185
13.3 下载调试	186
13.4 实验总结	186
第十四章 HDMI 输出实验	187
14.1 硬件介绍	187
14.2 程序设计	188
14.3 添加 XDC 约束文件	190
14.4 下载调试	191
14.5 实验总结	192
第十五章 HDMI 字符显示实验	193
15.1 实验原理	193
15.2 程序设计	193
15.3 实验现象	199
第十六章 7 寸液晶屏显示实验	200
16.1 硬件介绍	200
16.2 程序设计	200
16.3 实验现象	202
第十七章 AD7606 多通道波形显示实验	203
17.1 实验原理	204
17.1.1 AD7606 时序	204
17.1.2 AD7606 配置	205
17.1.3 AD7606 AD 转换	206
17.2 程序设计	206
17.3 实验现象	212
第十八章 AD9238 双通道波形显示实验	213
18.1 硬件介绍	213
18.1.1 两通道 AD 模块说明	213
18.1.2 模块功能说明	214
18.2 程序设计	216
18.3 实验现象	220
第十九章 ADDA 测试实验	222
19.1 硬件介绍	222
19.1.1 数模转换 (DA) 电路	223
19.1.2 模数转换 (AD) 电路	224
19.2 程序设计	225
19.3 实验现象	230
第二十章 AD9767 双通道正弦波产生实验	233

20.1 硬件介绍	233
20.1.1 AN9767 模块的参数说明	234
20.1.2 AN9767 模块的原理框图	234
20.1.3 AD9767 芯片简介	235
20.1.4 电流电压转换及放大	236
20.1.5 电流电压转换及放大	237
20.2 程序设计	237
20.2.1 生成 ROM 初始化文件	238
20.2.2 双通道正弦波发生程序	240
20.3 实验现象	241

第一章 Ultrascale+ MPSoC 介绍

Zynq UltraScale+ MPSoC 系列是 Xilinx 第二代 Zynq 平台。其亮点在于 FPGA 里包含了完整的 ARM 处理子系统 (PS)，包含了四核 Cortex-A53 处理器或双核 Cortex-A53 加双核 Cortex-R5 处理器，整个处理器的搭建都以处理器为中心，而且处理器子系统中集成了内存控制器和大量的外设，使处理器核在 Zynq 中完全独立于可编程逻辑单元，也就是说如果暂时没有用到可编程逻辑单元部分 (PL)，ARM 处理器的子系统也可以独立工作，这与以前的 FPGA 有本质区别，其是以处理器为中心的。

Zynq 就是两大功能块，PS 部分和 PL 部分，说白了，就是 ARM 的 SOC 部分，和 FPGA 部分。其中，PS 集成了 APU ARM Cortex™-A53 处理器，RPU Cortex-R5 处理器，AMBA®互连，内部存储器 (OCM)，外部存储器接口 (DDR Controller) 和外设 (IOU)。这些外设 (IOU) 主要包括 USB 总线接口，以太网接口，SD/eMMC 接口，I2C 总线接口，CAN 总线接口，UART 接口，GPIO 等。高速接口如 PCIe，SATA，Display Port。



ZYNQ MPSoC 芯片的总体框图

PS: 处理系统 (Processing System)，就是与 FPGA 无关的 ARM 的 SoC 的部分。

PL: 可编程逻辑 (Programmable Logic)，就是 FPGA 部分。

1.1 PS 和 PL 互联技术

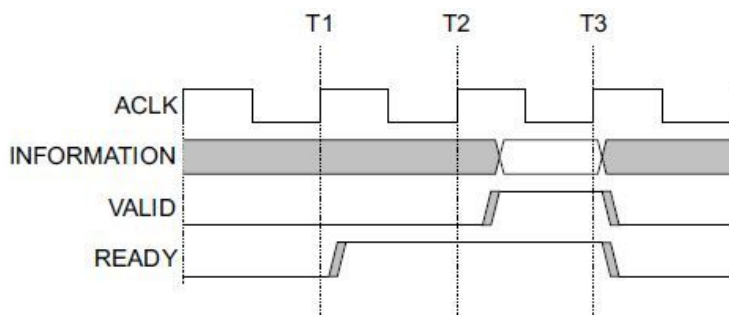
ZYNQ 作为将高性能 ARM Cortex-A53 系列处理器与高性能 FPGA 在单芯片内紧密结合的产

品，为了实现 ARM 处理器和 FPGA 之间的高速通信和数据交互，发挥 ARM 处理器和 FPGA 的性能优势，需要设计高效的片内高性能处理器与 FPGA 之间的互联通路。因此，如何设计高效的 PL 和 PS 数据交互通路是 ZYNQ 芯片设计的重中之重，也是产品设计的成败关键之一。本节，我们就将主要介绍 PS 和 PL 的连接，让用户了解 PS 和 PL 之间连接的技术。

其实，在具体设计中我们往往不需要在连接这个地方做太多工作，我们加入 IP 核以后，系统会自动使用 AXI 接口将我们的 IP 核与处理器连接起来，我们只需要再做一点补充就可以了。

AXI 全称 Advanced eXtensible Interface，是 Xilinx 从 6 系列的 FPGA 开始引入的一个接口协议，主要描述了主设备和从设备之间的数据传输方式。在 ZYNQ 中继续使用，版本是 AXI4，所以我们经常会看到 AXI4.0，ZYNQ 内部设备都有 AXI 接口。其实 AXI 就是 ARM 公司提出的 AMBA（Advanced Microcontroller Bus Architecture）的一个部分，是一种高性能、高带宽、低延迟的片内总线，也用来替代以前的 AHB 和 APB 总线。第一个版本的 AXI（AXI3）包含在 2003 年发布的 AMBA3.0 中，AXI 的第二个版本 AXI（AXI4）包含在 2010 年发布的 AMBA 4.0 之中。

AXI 协议主要描述了主设备和从设备之间的数据传输方式，主设备和从设备之间通过握手信号建立连接。当从设备准备好接收数据时，会发出 READY 信号。当主设备的数据准备好时，会发出和维持 VALID 信号，表示数据有效。数据只有在 VALID 和 READY 信号都有效的时候才开始传输。当这两个信号持续保持有效，主设备会继续传输下一个数据。主设备可以撤销 VALID 信号，或者从设备撤销 READY 信号终止传输。AXI 的协议如图，T2 时，从设备的 READY 信号有效，T3 时主设备的 VALID 信号有效，数据传输开始。



AXI 握手时序图

在 ZYNQ 中，支持 AXI-Lite，AXI4 和 AXI-Stream 三种总线，通过表 5-1,我们可以看到这三中 AXI 接口的特性。

接口协议	特性	应用场合
AXI4-Lite	地址/单数据传输	低速外设或控制
AXI4	地址/突发数据传输	地址的批量传输
AXI4-Stream	仅传输数据，突发传输	数据流和媒体流传输

AXI4-Lite：

具有轻量级，结构简单的特点，适合小批量数据、简单控制场合。不支持批量传输，读写

时一次只能读写一个字（32bit）。主要用于访问一些低速外设和外设的控制。

AXI4:

接口和 AXI-Lite 差不多，只是增加了一项功能就是批量传输，可以连续对一片地址进行一次性读写。也就是说具有数据读写的 burst 功能。

上面两种均采用内存映射控制方式，即 ARM 将用户自定义 IP 编入某一地址进行访问，读写时就像在读写自己的片内 RAM，编程也很方便，开发难度较低。代价就是资源占用过多，需要额外的读地址线、写地址线、读数据线、写数据线、写应答线这些信号线。

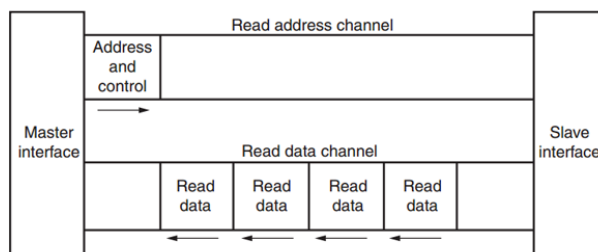
AXI4-Stream:

这是一种连续流接口，不需要地址线（很像 FIFO，一直读或一直写就行）。对于这类 IP，ARM 不能通过上面的内存映射方式控制（FIFO 根本没有地址的概念），必须有一个转换装置，例如 AXI-DMA 模块来实现内存映射到流式接口的转换。AXI-Stream 适用的场合有很多：视频流处理；通信协议转换；数字信号处理；无线通信等。其本质都是针对数值流构建的数据通路，从信源（例如 ARM 内存、DMA、无线接收前端等）到信宿（例如 HDMI 显示器、高速 AD 音频输出，等）构建起连续的数据流。这种接口适合做实时信号处理。

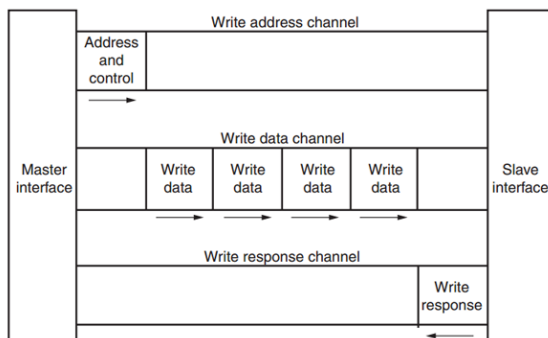
AXI4 和 AXI4-Lite 接口包含 5 个不同的通道：

- Read Address Channel
- Write Address Channel
- Read Data Channel
- Write Data Channel
- Write Response Channel

其中每个通道都是一个独立的 AXI 握手协议。下面两个图分别显示了读和写的模型：



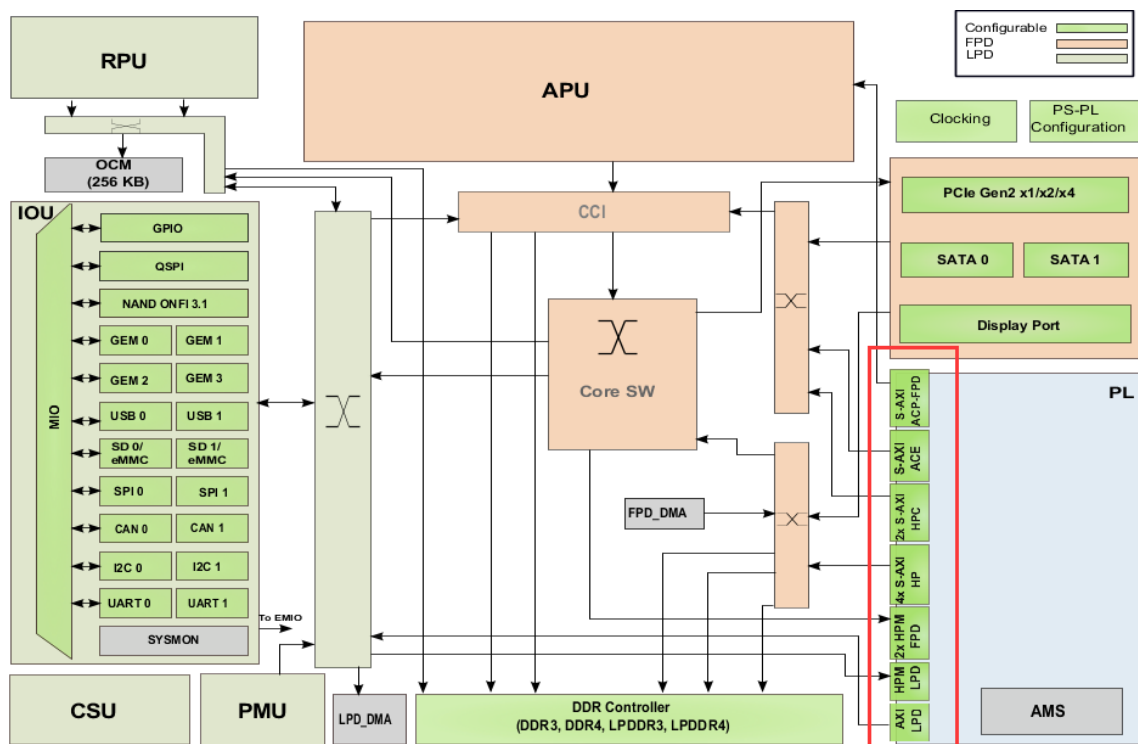
AXI 读数据通道



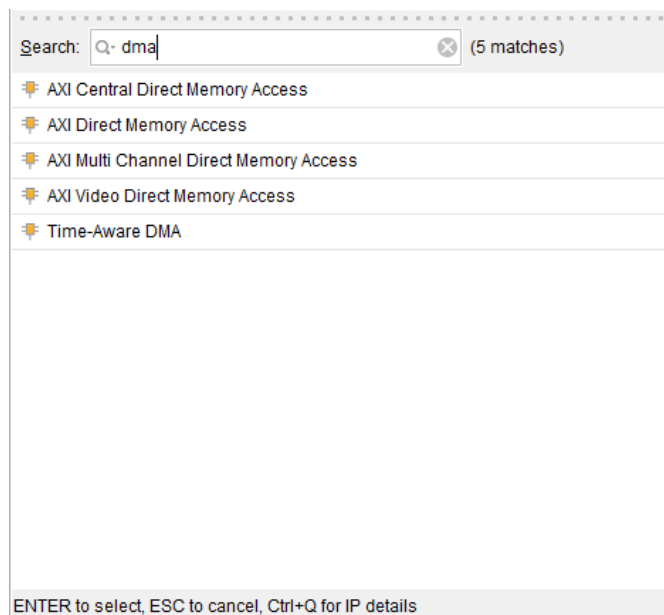
AXI 写数据通道

在 ZYNQ 芯片内部用硬件实现了 AXI 总线协议，包括 12 个物理接口，分别为 S_AXI_HP{0:3}_FPD, S_AXI_LPD, S_AXI_ACE_FPD, S_AXI_ACP_FPD,

M_AXI_HPM0_LPD 接口，低延迟接口总线，PS 为 master，连接 LPD 到 PL，可直接访问 PL 端的 BRAM，DDR 等，也经常用于配置 PL 端的寄存器。



位于 PS 端的 ARM 直接有硬件支持 AXI 接口，而 PL 则需要使用逻辑实现相应的 AXI 协议。Xilinx 在 Vivado 开发环境里提供现成 IP 如 AXI-DMA，AXI-GPIO，AXI-Dataover，AXI-Stream 都实现了相应的接口，使用时直接从 Vivado 的 IP 列表中添加即可实现相应的功能。下图为 Vivado 下的各种 DMA IP：



下面为几个常用的 AXI 接口 IP 的功能介绍：

AXI-DMA：实现从 PS 内存到 PL 高速传输高速通道 AXI-HP<---->AXI-Stream 的转换

AXI-FIFO-MM2S：实现从 PS 内存到 PL 通用传输通道 AXI-HPM<----->AXI-Stream 的转换

AXI-Datamover：实现从 PS 内存到 PL 高速传输高速通道 AXI-HP<---->AXI-Stream 的转换，只不过这次是完全由 PL 控制的，PS 是完全被动的。

AXI-VDMA：实现从 PS 内存到 PL 高速传输高速通道 AXI-HP<---->AXI-Stream 的转换，只不过是专门针对视频、图像等二维数据的。

AXI-CDMA：这个是由 PL 完成的将数据从内存的一个位置搬移到另一个位置，无需 CPU 来插手。

关于如何使用这些 IP，我们会在后面的章节中举例讲到。有时，用户需要开发自己定义的 IP 同 PS 进行通信，这时可以利用向导生成对应的 IP。用户自定义 IP 核可以拥有 AXI4-Lite，AXI4，AXI-Stream，PLB 和 FSL 这些接口。后两种由于 ARM 这一端不支持，所以不用。

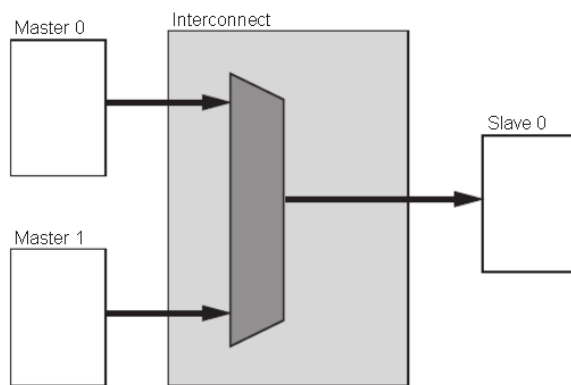
有了上面的这些官方 IP 和向导生成的自定义 IP，用户其实不需要对 AXI 时序了解太多（除非确实遇到问题），因为 Xilinx 已经将和 AXI 时序有关的细节都封装起来，用户只需要关注自己的逻辑实现即可。

AXI 协议严格的讲是一个点对点的主从接口协议，当多个外设需要互相交互数据时，我们需要加入一个 AXI Interconnect 模块，也就是 AXI 互联矩阵，作用是提供将一个或多个 AXI 主设备连接到一个或多个 AXI 从设备的一种交换机制（有点类似于交换机里面的交换矩阵）。

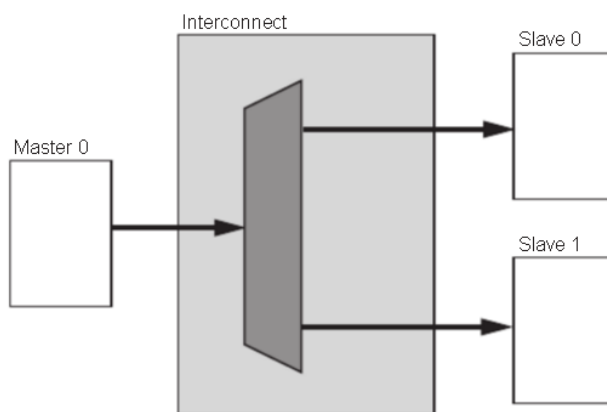
这个 AXI Interconnect IP 核最多可以支持 16 个主设备、16 个从设备，如果需要更多的接口，可以多加入几个 IP 核。

AXI Interconnect 基本连接模式有以下几种：

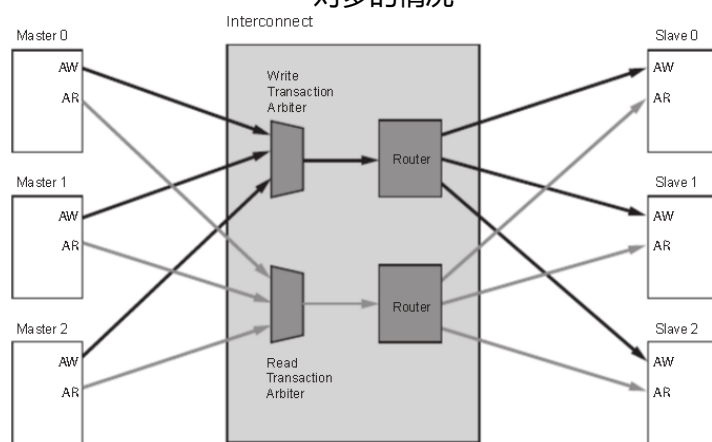
- N-to-1 Interconnect
- to-N Interconnect
- N-to-M Interconnect (Crossbar Mode)
- N-to-M Interconnect (Shared Access Mode)



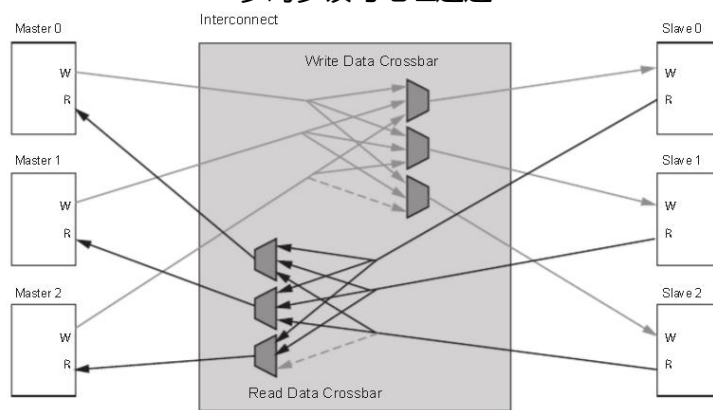
多对一的情况



一对多的情况

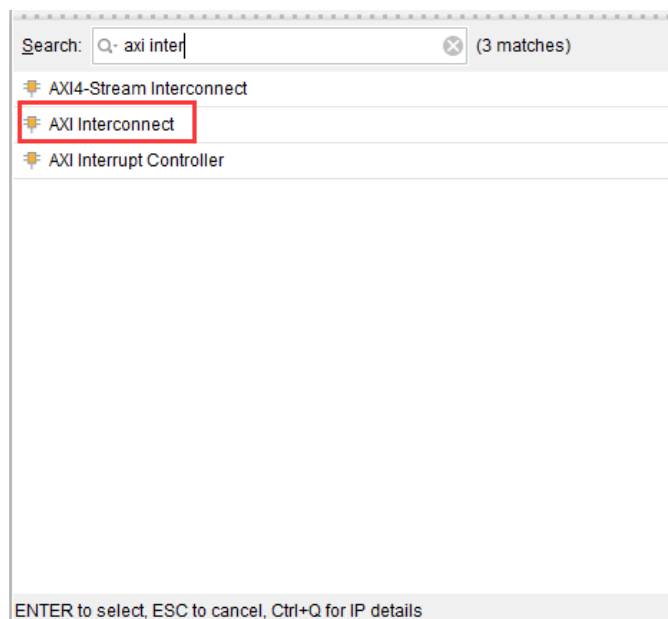


多对多读写地址通道



多对多读写数据通道

ZYNQ 内部的 AXI 接口设备就是通过互联矩阵的方式互联起来的，既保证了传输数据的高效性，又保证了连接的灵活性。Xilinx 在 Vivado 里我们提供了实现这种互联矩阵的 IP 核 axi_interconnect，我们只要调用就可以。



AXI Interconnect IP

1.2 ZYNQ 芯片开发流程的简介

由于 ZYNQ 将 CPU 与 FPGA 集成在了一起，开发人员既需要设计 ARM 的操作系统应用程序和设备的驱动程序，又需要设计 FPGA 部分的硬件逻辑设计。开发中既要了解 Linux 操作系统，系统的构架，也需要搭建一个 FPGA 和 ARM 系统之间的硬件设计平台。所以 ZYNQ 的开发是需要软件人员和硬件人员协同设计并开发的。这既是 ZYNQ 开发中所谓的“软硬件协同设计”。

ZYNQ 系统的硬件系统和软件系统的设计和开发需要用到一下的开发环境和调试工具：Xilinx Vivado。

Vivado 设计套件实现 FPGA 部分的设计和开发，管脚和时序的约束，编译和仿真，实现 RTL 到比特流的设计流程。Vivado 并不是 ISE 设计套件的简单升级，而是一个全新的设计套件。它替代了 ISE 设计套件的所有重要工具，比如 Project Navigator、Xilinx Synthesis Technology、Implementation、CORE Generator、Constraint、Simulator、Chipscope Analyzer、FPGA Editor 等设计工具。

Xilinx SDK (Software Development Kit)，SDK 是 Xilinx 软件开发套件(SDK),在 Vivado 硬件系统的基础上，系统会自动配置一些重要参数，其中包括工具和库路径、编译器选项、JTAG 和闪存设置，调试器连接已经裸机板支持包(BSP)。SDK 也为所有支持的 Xilinx IP 硬核提供了驱动程序。SDK 支持 IP 硬核（FPGA 上）和处理器软件协同调试，我们可以使用高级 C 或 C++语言来开发和调试 ARM 和 FPGA 系统，测试硬件系统是否工作正常。SDK 软件也是 Vivado 软件自

带的，无需单独安装。

ZYNQ 的开发也是先硬件后软件的方法。具体流程如下：

- 1) 在 Vivado 上新建工程，增加一个嵌入式的源文件。
- 2) 在 Vivado 里添加和配置 PS 和 PL 部分基本的外设，或需要添加自定义的外设。
- 3) 在 Vivado 里生成顶层 HDL 文件，并添加约束文件。再编译生成比特流文件 (*.bit)。
- 4) 导出硬件信息到 SDK 软件开发环境，在 SDK 环境里可以编写一些调试软件验证硬件和软件，结合比特流文件单独调试 ZYNQ 系统。
- 5) 在 SDK 里生成 FSBL 文件。
- 6) 在 VMware 虚拟机里生成 u-boot.elf、bootloader 镜像。
- 7) 在 SDK 里通过 FSBL 文件，比特流文件 system.bit 和 u-boot.elf 文件生成一个 BOOT.bin 文件。
- 8) 在 VMware 里生成 Ubuntu 的内核镜像文件 Zimage 和 Ubuntu 的根文件系统。另外还需要要对 FPGA 自定义的 IP 编写驱动。
- 9) 把 BOOT、内核、设备树、根文件系统文件放入到 SD 卡中，启动开发板电源，Linux 操作系统会从 SD 卡里启动。

以上是典型的 ZYNQ 开发流程，但是 ZYNQ 也可以单独做为 ARM 来使用，这样就不需要关系 PL 端资源，和传统的 ARM 开发没有太大区别。ZYNQ 也可以只使用 PL 部分，但是 PL 的配置还是要 PS 来完成的，就是无法通过传统的固化 Flash 方式把只要 PL 的固件固化起来。

1.3 学习 ZYNQ 要具备哪些技能

学习 ZYNQ 比学习 FPGA、MCU、ARM 等传统工具开发要求更高，想学好 ZYNQ 也不是一蹴而就的事情。

1.3.1 软件开发人员

- ✓ 计算机组成原理
- ✓ C、C++语言
- ✓ 计算机操作系统
- ✓ tcl 脚本
- ✓ 良好的英语阅读基础

1.3.2 逻辑开发人员

- ✓ 计算机组成原理
- ✓ C 语言
- ✓ 数字电路基础

第二章 开发板硬件介绍

芯驿电子科技（上海）有限公司基于 XILINX Zynq UltraScale+ MPSoCs 开发平台的开发板（型号：AXU3EG）2020 款正式发布了，为了让您对此开发平台可以快速了解，我们编写了此用户手册。

这款 MPSoCs 开发平台采用核心板加扩展板的模式，方便用户对核心板的二次开发利用。核心板使用 XILINX Zynq UltraScale+ EG 芯片 ZU3EG 的解决方案，它采用 Processing System(PS)+Programmable Logic(PL)技术将双核 ARM Cortex-A53 和 FPGA 可编程逻辑集成在一颗芯片上。另外核心板上 PS 端带有 4 片共 4GB 高速 DDR4 SDRAM 芯片，1 片 8GB 的 eMMC 存储芯片和 1 片 256Mb 的 QSPI FLASH 芯片；核心板上 PL 端带有 1 片 1GB 的 DDR4 SDRAM 芯片。

在底板设计上我们为用户扩展了丰富的外围接口，比如 1 个 FMC LPC 接口、1 路 SATA M.2 接口、1 路 DP 接口、1 个 USB3.0 接口、1 路千兆以太网接口、1 路 UART 串口接口、1 路 SD 卡接口、2 个 40 针扩展接口、2 路 CAN 总线接口、2 路 RS485 接口等等。满足用户各种高速数据交换，数据存储，视频传输处理，深度学习，人工智能以及工业控制的要求，是一款“专业级”的 ZYNQ 开发平台。为高速数据传输和交换，数据处理的前期验证和后期应用提供了可能。相信这样的一款产品非常适合从事 MPSoCs 开发的学生、工程师等群体。



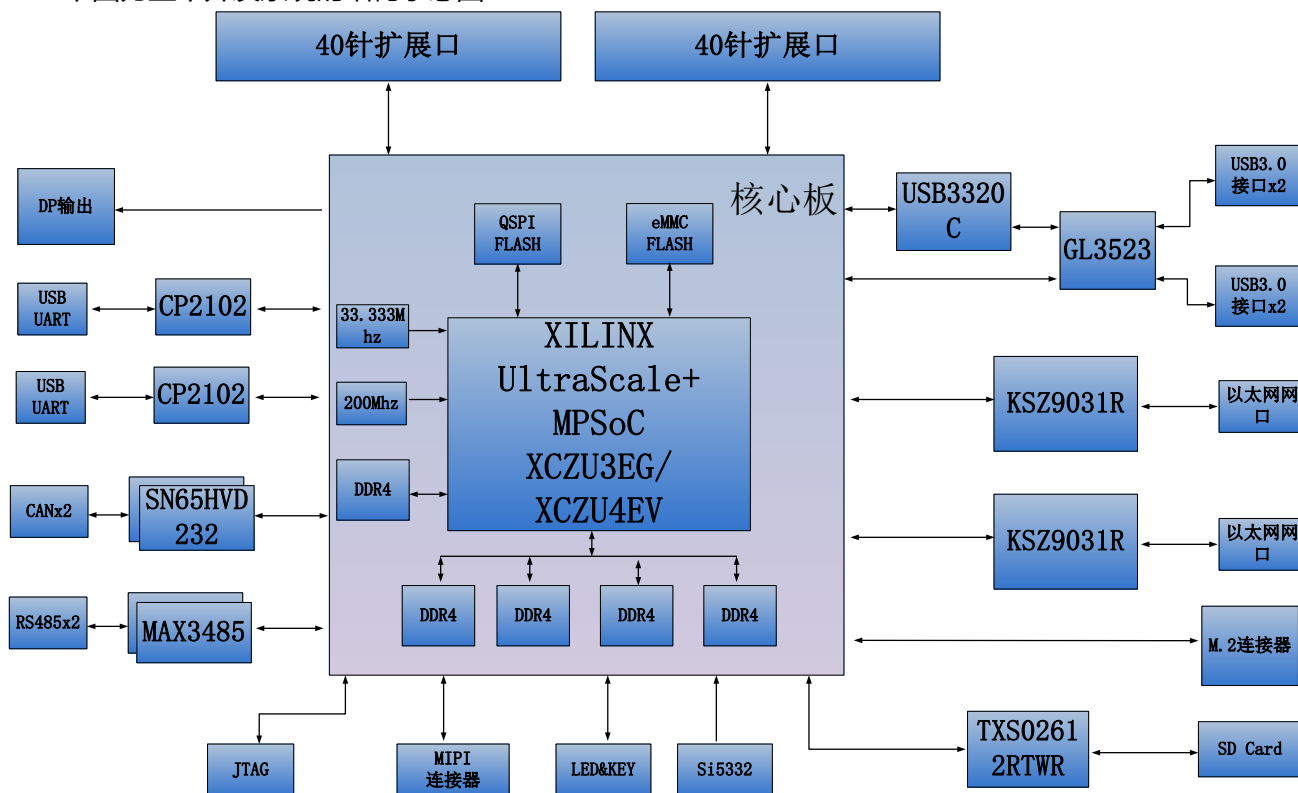
在这里，对这款 AXU3EG MPSoCs 开发平台进行简单的功能介绍。

开发板的整个结构，继承了我们一贯的核心板+扩展板的模式来设计的。核心板和扩展板之间使用高速板间连接器连接。

核心板主要由 ZU3EG + 5 个 DDR4 + eMMC + 1 个 QSPI FLASH 的最小系统构成。ZU3EG 采用 Xilinx 公司的 Zynq UltraScale+ MPSoCs EG 系列的芯片，型号为 XCZU3EG-1SFVC784I。ZU3EG 芯片可分成处理器系统部分 Processor System (PS) 和可编程逻辑部分 Programmable Logic (PL)。在 ZU3EG 芯片的 PS 端和 PL 端分别挂了 4 片和 1 片 DDR4，每片 DDR4 容量高达 1G 字节，使得 ARM 系统和 FPGA 系统能独立处理和存储的数据的功能。PS 端的 8GB eMMC FLASH 存储芯片和 1 片 256Mb 的 QSPI FLASH 用来静态存储 MPSoCs 的操作系统、文件系统及用户数据。

底板为核心板扩展了丰富的外围接口，其中包含 1 路 M.2 接口、1 路 DP 接口、4 路 USB3.0 接口、2 路千兆以太网接口、2 路 UART 串口接口、1 路 SD 卡接口、2 个 40 针扩展接口、2 路 CAN 总线接口，2 路 RS485 接口，1 路 MIPI 接口和一些按键 LED。

下图为整个开发系统的结构示意图：



通过这个示意图，我们可以看到，我们这个开发平台所能含有的接口和功能。

- ZU3EG 核心板

由 ZU3EG+4GB DDR4 (PS) +1GB DDR4 (PL) +8GB eMMC FLASH + 256Mb QSPI FLASH 组成，另外有 2 个晶振提供时钟，一个单端 33.3333MHz 晶振提供给 PS 系统，一个差分 200MHz 晶振提供给 PL 逻辑 DDR 参考时钟。

- M.2 接口

1 路 PCIe x1 标准的 M.2 接口，用于连接 M.2 的 SSD 固态硬盘，通信速度高达 6Gbps。

- DP 输出接口

1 路标准的 Display Port 输出显示接口，用于视频图像的显示。最高支持 4K@30Hz 或者 1080P@60Hz 输出。

- USB3.0 接口

4 路 USB3.0 HOST 接口，USB 接口类型为 TYPE A。用于连接外部的 USB 外设，比如连接鼠标，键盘，U 盘等等。

- 千兆以太网接口

2 路 10/100M/1000M 以太网 RJ45 接口，PS 和 PL 各 1 路。用于和电脑或其它网络设备进行以太网数据交换。

- USB Uart 接口

2 路 Uart 转 USB 接口，PS 和 PL 各 1 路。用于和电脑通信，方便用户调试。串口芯片采用 Silicon Labs CP2102GM 的 USB-UAR 芯片，USB 接口采用 MINI USB 接口。

- Micro SD 卡座

1 路 Micro SD 卡座，用于存储操作系统镜像和文件系统。

- 40 针扩展口

2 个 40 针 2.54mm 间距的扩展口，可以外接黑金的各种模块（双目摄像头，TFT LCD 屏，高速 AD 模块等等）。扩展口包含 5V 电源 1 路，3.3V 电源 2 路，地 3 路，IO 口 34 路。

- CAN 通信接口

2 路 CAN 总线接口，选用 TI 公司的 SN65HVD232 芯片，接口采用 4Pin 的绿色接线端子。

- 485 通信接口

2 路 485 通信接口，选用 MAXIM 公司的 MAX3485 芯片。接口采用 6Pin 的绿色接线端子。

- MIPI 接口

2 个 LANE 的 MIPI 摄像头输入接口，用于连接 MIPI 摄像头模块（AN5641）。

- JTAG 调试口

1 个 10 针 2.54mm 标准的 JTAG 口，用于 FPGA 程序的下载和调试，用户可以通过 XILINX 下载器对 ZU3EG 系统进行调试和下载。

- 温湿度传感器

板载 1 片温湿度传感器芯片 LM75，用于检测板子周围环境的温度和湿度。

- EEPROM

1 片 IIC 接口的 EEPROM 24LC04;

- RTC 实时时钟

1 路内置的 RTC 实时时钟;

- LED 灯

5 个发光二极管 LED，核心板上 2 个，底板上 3 个。核心板上 1 个电源指示灯和 1 个 DONE 配置指示灯。底板上 1 个电源指示灯，2 个用户指示灯。

- 按键

3 个按键，1 个复位按键，2 个用户按键。

2.1 ACU3EG 核心板

2.1.1 简介

ACU3EG(核心板型号,下同)核心板, ZYNQ 芯片是基于 XILINX 公司的 Zynq UltraScale+ MPSoCs EG 系列的 XCZU3EG-1SFVC784I。

这款核心板使用了 5 片 Micron 的 DDR4 芯片 MT40A512M16GE,其中 PS 端挂载 4 片 DDR4,组成 64 位数据总线带宽和 4GB 的容量。PL 端挂载 1 片,为 16 位的数据总线宽度和 1GB 的容量。PS 端的 DDR4 SDRAM 的最高运行速度可达 1200MHz(数据速率 2400Mbps), PL 端的 DDR4 SDRAM 的最高运行速度可达 1066MHz(数据速率 2132Mbps)。另外核心板上也集成了 1 片 256MBit 大小的 QSPI FLASH 和 8GB 大小的 eMMC FLASH 芯片,用于启动存储配置和系统文件。

为了和底板连接,这款核心板的 4 个板对板连接器扩展出了 PS 端的 USB2.0 接口,千兆以太网接口,SD 卡接口及其它剩余的 MIO 口;也扩展出了 4 对 PS MGT 高速收发器接口;以及 PL 端的几乎所有 IO 口(HP I/O: 96 个,HD I/O: 84 个),XCZU3EG 芯片到接口之间走线做了等长和差分处理,并且核心板尺寸仅为 80*60 (mm),对于二次开发来说,非常适合。



ACU3EG 核心板正面图

2.1.2 ZYNQ 芯片

开发板使用的是 Xilinx 公司的 Zynq UltraScale+ MPSoCs EG 系列的芯片, 型号为

XCZU3EG-1SFVC784I。ZU3EG 芯片的 PS 系统集成了 4 个 ARM Cortex™-A53 处理器，速度高达 1.2Ghz，支持 2 级 Cache；另外还包含 2 个 Cortex-R5 处理器，速度高达 500Mhz。

ZU3EG 芯片支持 32 位或者 64 位的 DDR4，LPDDR4，DDR3,DDR3L, LPDDR3 存储芯片，在 PS 端带有丰富的高速接口如 PCIE Gen2, USB3.0, SATA 3.1, DisplayPort；同时另外也支持 USB2.0，千兆以太网，SD/SDIO，I2C，CAN，UART，GPIO 等接口。PL 端内部含有丰富的可编程逻辑单元，DSP 和内部 RAM。ZU3EG 芯片的总体框图如图 2-2-1 所示

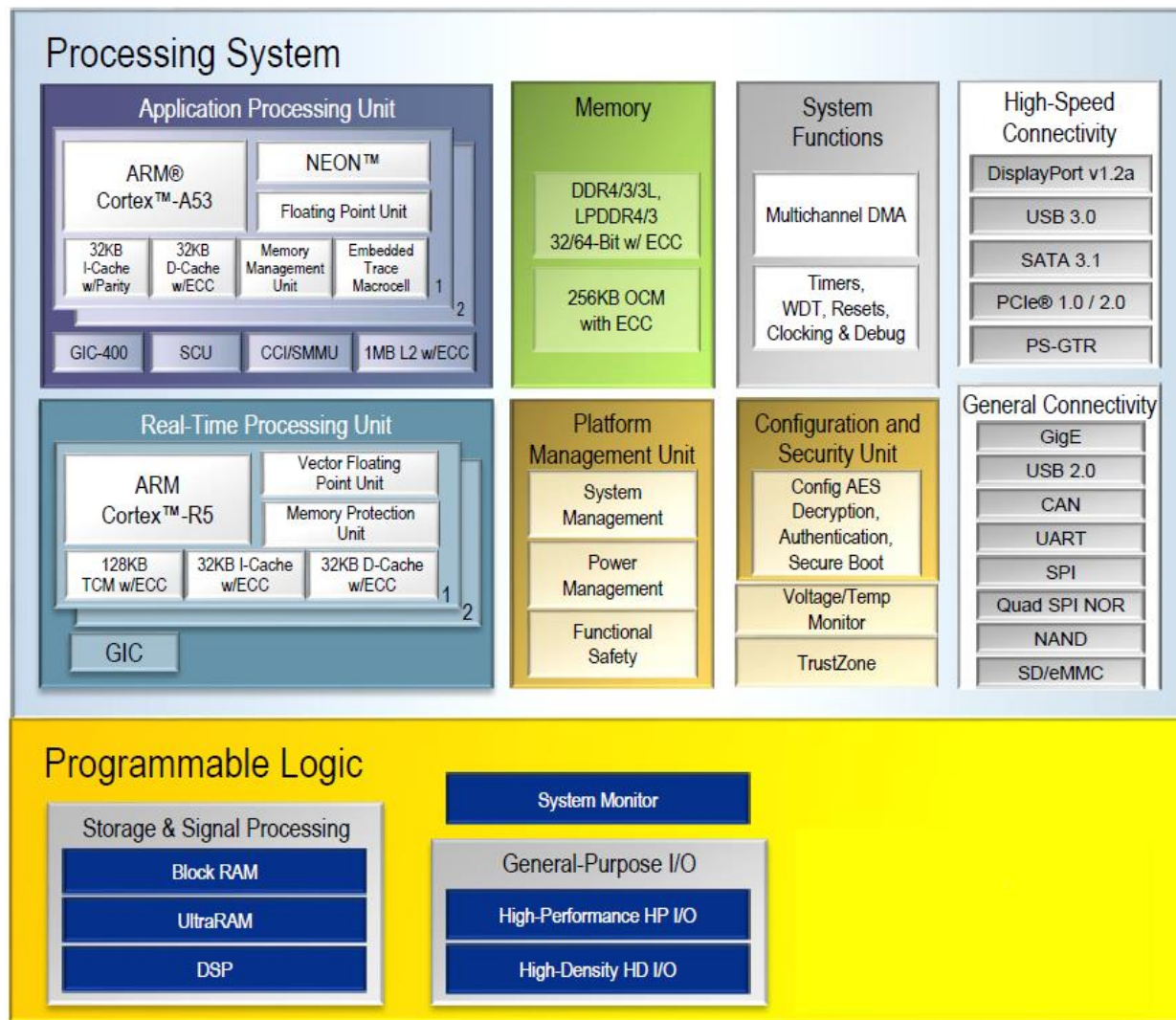


图2-2-1 ZYNQ ZU3EG芯片的总体框图

其中 PS 系统部分的主要参数如下：

- ARM 四核 Cortex™-A53 处理器，速度高达 1.2GHz，每个 CPU 32KB 1 级指令和数据缓存，1MB 2 级缓存 2 个 CPU 共享。
- ARM 双核 Cortex-R5 处理器，速度高达 500MHz，每个 CPU 32KB 1 级指令和数据缓存，及 128K 紧耦合内存。
- 外部存储接口，支持 32/64bit DDR4/3/3L、LPDDR4/3 接口。
- 静态存储接口，支持 NAND，2xQuad-SPI FLASH。
- 高速连接接口，支持 PCIe Gen2 x4，2xUSB3.0，Sata 3.1，DisplayPort，4x Tri-mode Gigabit Ethernet。

- 普通连接接口：2xUSB2.0, 2x SD/SDIO, 2x UART, 2x CAN 2.0B, 2x I2C, 2x SPI, 4x 32b GPIO。
- 电源管理：支持 Full/Low/PL/Battery 四部分电源的划分。
- 加密算法：支持 RSA, AES 和 SHA。
- 系统监控：10 位 1Mbps 的 AD 采样，用于温度和电压的检测。

其中 PL 逻辑部分的主要参数如下：

- 逻辑单元 Logic Cells：154K；
- 触发器(flip-flops)：141K；
- 查找表 LUTs：71K；
- Block RAM：9.4Mb；
- 时钟管理单元 (CMTs)：3
- 乘法器 18x25MACCs：360

XCZU3EG-1SFVC784I芯片的速度等级为-1，工业级，封装为SFVC784。

2.1.3 DDR4 DRAM

ACU3EG核心板上配有5片Micron(美光) 的512MB的DDR4芯片,型号为MT40A512M16GE-083E, 其中PS端挂载4片DDR4, 组成64位数据总线带宽和4GB的容量。PL端挂载1片, 为16位的数据总线宽度和1GB的容量。PS端的DDR4 SDRAM的最高运行速度可达1200MHz(数据速率2400Mbps), 4片DDR4存储系统直接连接到了PS的BANK504的存储器接口上。PL端的DDR4 SDRAM的最高运行速度可达1066MHz(数据速率2133Mbps), 1片DDR4连接到了FPGA的BANK64的接口上。DDR4 SDRAM的具体配置如下表2-3-1所示。

位号	芯片型号	容量	厂家
U12,U14,U15,U16	MT40A512M16GE-083E	512M x 16bit	Micron

表 2-3-1 DDR4 SDRAM 配置

DDR4 的硬件设计需要严格考虑信号完整性，我们在电路设计和 PCB 设计的时候已经充分考虑了匹配电阻/终端电阻,走线阻抗控制，走线等长控制，保证 DDR4 的高速稳定的工作。

PS 端的 DDR4 的硬件连接方式如图 2-3-1 所示：

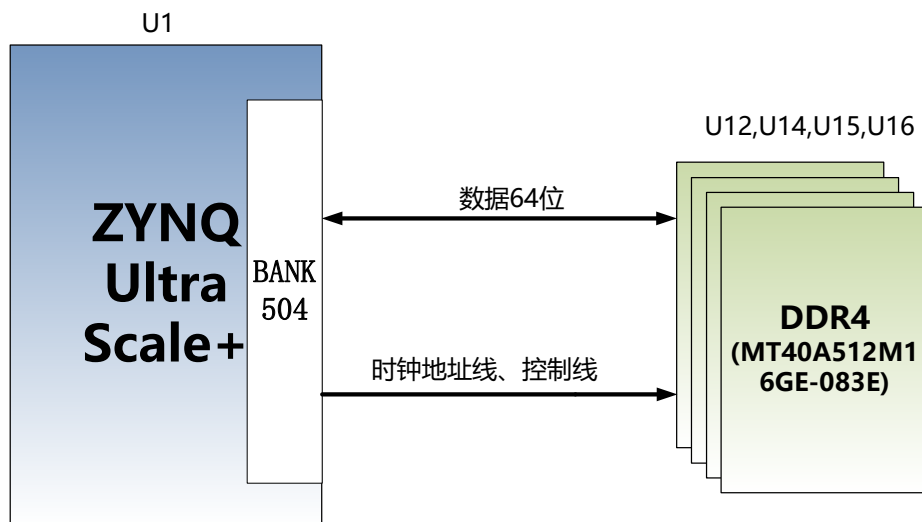


图2-3-1 PS端DDR4 DRAM原理图部分

PL 端的 DDR4 DRAM 的硬件连接方式如图 2-3-2 所示:

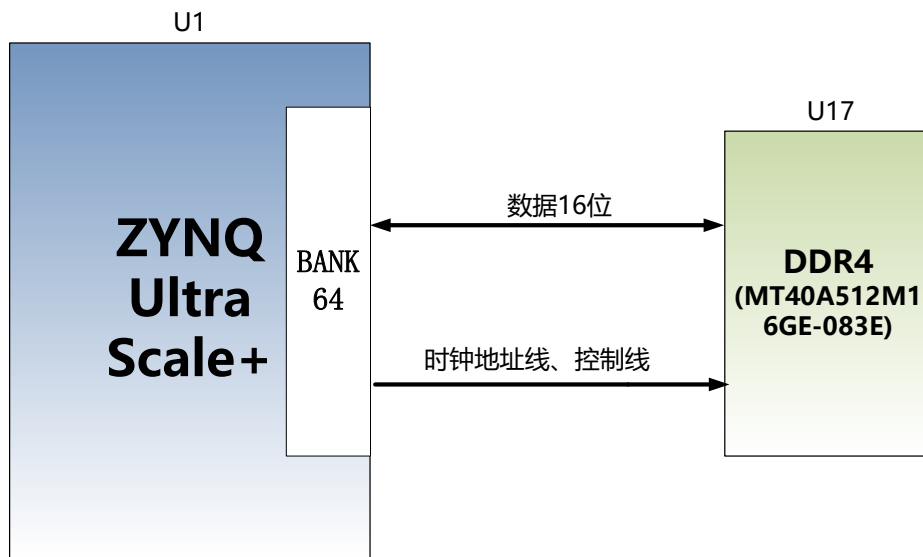


图2-3-2 PL端DDR4 DRAM原理图部分

PS 端 DDR4 SDRAM 引脚分配:

信号名称	引脚名	引脚号
PS_DDR4_DQS0_P	PS_DDR_DQS_P0_504	AF21
PS_DDR4_DQS0_N	PS_DDR_DQS_N0_504	AG21
PS_DDR4_DQS1_P	PS_DDR_DQS_P1_504	AF23
PS_DDR4_DQS1_N	PS_DDR_DQS_N1_504	AG23
PS_DDR4_DQS2_P	PS_DDR_DQS_P2_504	AF25
PS_DDR4_DQS2_N	PS_DDR_DQS_N2_504	AF26
PS_DDR4_DQS3_P	PS_DDR_DQS_P3_504	AE27
PS_DDR4_DQS3_N	PS_DDR_DQS_N3_504	AF27
PS_DDR4_DQS4_P	PS_DDR_DQS_P4_504	N23
PS_DDR4_DQS4_N	PS_DDR_DQS_N4_504	M23
PS_DDR4_DQS5_P	PS_DDR_DQS_P5_504	L23
PS_DDR4_DQS5_N	PS_DDR_DQS_N5_504	K23
PS_DDR4_DQS6_P	PS_DDR_DQS_P6_504	N26
PS_DDR4_DQS6_N	PS_DDR_DQS_N6_504	N27
PS_DDR4_DQS7_P	PS_DDR_DQS_P7_504	J26
PS_DDR4_DQS7_N	PS_DDR_DQS_N7_504	J27
PS_DDR4_DQ0	PS_DDR_DQ0_504	AD21
PS_DDR4_DQ1	PS_DDR_DQ1_504	AE20
PS_DDR4_DQ2	PS_DDR_DQ2_504	AD20
PS_DDR4_DQ3	PS_DDR_DQ3_504	AF20
PS_DDR4_DQ4	PS_DDR_DQ4_504	AH21
PS_DDR4_DQ5	PS_DDR_DQ5_504	AH20
PS_DDR4_DQ6	PS_DDR_DQ6_504	AH19
PS_DDR4_DQ7	PS_DDR_DQ7_504	AG19
PS_DDR4_DQ8	PS_DDR_DQ8_504	AF22

PS_DDR4_DQ9	PS_DDR_DQ9_504	AH22
PS_DDR4_DQ10	PS_DDR_DQ10_504	AE22
PS_DDR4_DQ11	PS_DDR_DQ11_504	AD22
PS_DDR4_DQ12	PS_DDR_DQ12_504	AH23
PS_DDR4_DQ13	PS_DDR_DQ13_504	AH24
PS_DDR4_DQ14	PS_DDR_DQ14_504	AE24
PS_DDR4_DQ15	PS_DDR_DQ15_504	AG24
PS_DDR4_DQ16	PS_DDR_DQ16_504	AC26
PS_DDR4_DQ17	PS_DDR_DQ17_504	AD26
PS_DDR4_DQ18	PS_DDR_DQ18_504	AD25
PS_DDR4_DQ19	PS_DDR_DQ19_504	AD24
PS_DDR4_DQ20	PS_DDR_DQ20_504	AG26
PS_DDR4_DQ21	PS_DDR_DQ21_504	AH25
PS_DDR4_DQ22	PS_DDR_DQ22_504	AH26
PS_DDR4_DQ23	PS_DDR_DQ23_504	AG25
PS_DDR4_DQ24	PS_DDR_DQ24_504	AH27
PS_DDR4_DQ25	PS_DDR_DQ25_504	AH28
PS_DDR4_DQ26	PS_DDR_DQ26_504	AF28
PS_DDR4_DQ27	PS_DDR_DQ27_504	AG28
PS_DDR4_DQ28	PS_DDR_DQ28_504	AC27
PS_DDR4_DQ29	PS_DDR_DQ29_504	AD27
PS_DDR4_DQ30	PS_DDR_DQ30_504	AD28
PS_DDR4_DQ31	PS_DDR_DQ31_504	AC28
PS_DDR4_DQ32	PS_DDR_DQ32_504	T22
PS_DDR4_DQ33	PS_DDR_DQ33_504	R22
PS_DDR4_DQ34	PS_DDR_DQ34_504	P22
PS_DDR4_DQ35	PS_DDR_DQ35_504	N22
PS_DDR4_DQ36	PS_DDR_DQ36_504	T23
PS_DDR4_DQ37	PS_DDR_DQ37_504	P24
PS_DDR4_DQ38	PS_DDR_DQ38_504	R24
PS_DDR4_DQ39	PS_DDR_DQ39_504	N24
PS_DDR4_DQ40	PS_DDR_DQ40_504	H24
PS_DDR4_DQ41	PS_DDR_DQ41_504	J24
PS_DDR4_DQ42	PS_DDR_DQ42_504	M24
PS_DDR4_DQ43	PS_DDR_DQ43_504	K24
PS_DDR4_DQ44	PS_DDR_DQ44_504	J22
PS_DDR4_DQ45	PS_DDR_DQ45_504	H22
PS_DDR4_DQ46	PS_DDR_DQ46_504	K22
PS_DDR4_DQ47	PS_DDR_DQ47_504	L22
PS_DDR4_DQ48	PS_DDR_DQ48_504	M25
PS_DDR4_DQ49	PS_DDR_DQ49_504	M26
PS_DDR4_DQ50	PS_DDR_DQ50_504	L25
PS_DDR4_DQ51	PS_DDR_DQ51_504	L26
PS_DDR4_DQ52	PS_DDR_DQ52_504	K28
PS_DDR4_DQ53	PS_DDR_DQ53_504	L28
PS_DDR4_DQ54	PS_DDR_DQ54_504	M28
PS_DDR4_DQ55	PS_DDR_DQ55_504	N28
PS_DDR4_DQ56	PS_DDR_DQ56_504	J28

PS_DDR4_DQ57	PS_DDR_DQ57_504	K27
PS_DDR4_DQ58	PS_DDR_DQ58_504	H28
PS_DDR4_DQ59	PS_DDR_DQ59_504	H27
PS_DDR4_DQ60	PS_DDR_DQ60_504	G26
PS_DDR4_DQ61	PS_DDR_DQ61_504	G25
PS_DDR4_DQ62	PS_DDR_DQ62_504	K25
PS_DDR4_DQ63	PS_DDR_DQ63_504	J25
PS_DDR4_DM0	PS_DDR_DM0_504	AG20
PS_DDR4_DM1	PS_DDR_DM1_504	AE23
PS_DDR4_DM2	PS_DDR_DM2_504	AE25
PS_DDR4_DM3	PS_DDR_DM3_504	AE28
PS_DDR4_DM4	PS_DDR_DM4_504	R23
PS_DDR4_DM5	PS_DDR_DM5_504	H23
PS_DDR4_DM6	PS_DDR_DM6_504	L27
PS_DDR4_DM7	PS_DDR_DM7_504	H26
PS_DDR4_A0	PS_DDR_A0_504	W28
PS_DDR4_A1	PS_DDR_A1_504	Y28
PS_DDR4_A2	PS_DDR_A2_504	AB28
PS_DDR4_A3	PS_DDR_A3_504	AA28
PS_DDR4_A4	PS_DDR_A4_504	Y27
PS_DDR4_A5	PS_DDR_A5_504	AA27
PS_DDR4_A6	PS_DDR_A6_504	Y22
PS_DDR4_A7	PS_DDR_A7_504	AA23
PS_DDR4_A8	PS_DDR_A8_504	AA22
PS_DDR4_A9	PS_DDR_A9_504	AB23
PS_DDR4_A10	PS_DDR_A10_504	AA25
PS_DDR4_A11	PS_DDR_A11_504	AA26
PS_DDR4_A12	PS_DDR_A12_504	AB25
PS_DDR4_A13	PS_DDR_A13_504	AB26
PS_DDR4_WE_B	PS_DDR_A14_504	AB24
PS_DDR4_CAS_B	PS_DDR_A15_504	AC24
PS_DDR4_RAS_B	PS_DDR_A16_504	AC23
PS_DDR4_ACT_B	PS_DDR_ACT_N_504	Y23
PS_DDR4_ALERT_B	PS_DDR_ALERT_N_504	U25
PS_DDR4_BA0	PS_DDR_BA0_504	V23
PS_DDR4_BA1	PS_DDR_BA1_504	W22
PS_DDR4_BG0	PS_DDR_BG0_504	W24
PS_DDR4_CS0_B	PS_DDR_CS_N0_504	W27
PS_DDR4_ODT0	PS_DDR_ODT0_504	U28
PS_DDR4_PARITY	PS_DDR_PARITY_504	V24
PS_DDR4_RESET_B	PS_DDR_RST_N_504	U23
PS_DDR4_CLK0_P	PS_DDR_CLK0_P_504	W25
PS_DDR4_CLK0_N	PS_DDR_CLK0_N_504	W26
PS_DDR4_CKE0	PS_DDR_CKE0_504	V28

PL 端 DDR4 SDRAM 引脚分配:

信号名称	引脚名	引脚号
------	-----	-----

PL_DDR4_DQS0_P	IO_L22P_T3U_N6_DBC_AD0P_64	AE2
PL_DDR4_DQS0_N	IO_L22N_T3U_N7_DBC_AD0N_64	AF2
PL_DDR4_DQS1_P	IO_L16P_T2U_N6_QBC_AD3P_64	AD2
PL_DDR4_DQS1_N	IO_L16N_T2U_N7_QBC_AD3N_64	AD1
PL_DDR4_DQ0	IO_L24N_T3U_N11_64	AG1
PL_DDR4_DQ1	IO_L24P_T3U_N10_64	AF1
PL_DDR4_DQ2	IO_L23N_T3U_N9_64	AH1
PL_DDR4_DQ3	IO_L23P_T3U_N8_64	AH2
PL_DDR4_DQ4	IO_L21N_T3L_N5_AD8N_64	AF3
PL_DDR4_DQ5	IO_L21P_T3L_N4_AD8P_64	AE3
PL_DDR4_DQ6	IO_L20N_T3L_N3_AD1N_64	AH3
PL_DDR4_DQ7	IO_L20P_T3L_N2_AD1P_64	AG3
PL_DDR4_DQ8	IO_L18N_T2U_N11_AD2N_64	AC1
PL_DDR4_DQ9	IO_L18P_T2U_N10_AD2P_64	AB1
PL_DDR4_DQ10	IO_L17N_T2U_N9_AD10N_64	AC2
PL_DDR4_DQ11	IO_L17P_T2U_N8_AD10P_64	AB2
PL_DDR4_DQ12	IO_L15N_T2L_N5_AD11N_64	AB3
PL_DDR4_DQ13	IO_L15P_T2L_N4_AD11P_64	AB4
PL_DDR4_DQ14	IO_L14N_T2L_N3_GC_64	AC3
PL_DDR4_DQ15	IO_L14P_T2L_N2_GC_64	AC4
PL_DDR4_DM0	IO_L19P_T3L_N0_DBC_AD9P_64	AG4
PL_DDR4_DM1	IO_L13P_T2L_N0_GC_QBC_64	AD5
PL_DDR4_A0	IO_L8N_T1L_N3_AD5N_64	AG8
PL_DDR4_A1	IO_L3P_T0L_N4_AD15P_64	AB8
PL_DDR4_A2	IO_L8P_T1L_N2_AD5P_64	AF8
PL_DDR4_A3	IO_L3N_T0L_N5_AD15N_64	AC8
PL_DDR4_A4	IO_L11P_T1U_N8_GC_64	AF7
PL_DDR4_A5	IO_L4P_T0U_N6_DBC_AD7P_64	AD7
PL_DDR4_A6	IO_L9N_T1L_N5_AD12N_64	AH7
PL_DDR4_A7	IO_L2P_T0L_N2_64	AE9
PL_DDR4_A8	IO_L9P_T1L_N4_AD12P_64	AH8
PL_DDR4_A9	IO_L1P_T0L_N0_DBC_64	AC9
PL_DDR4_A10	IO_L4N_T0U_N7_DBC_AD7N_64	AE7
PL_DDR4_A11	IO_L7N_T1L_N1_QBC_AD13N_64	AH9
PL_DDR4_A12	IO_L6N_T0U_N11_AD6N_64	AC6
PL_DDR4_A13	IO_L1N_T0L_N1_DBC_64	AD9
PL_DDR4_BA0	IO_T1U_N12_64	AH6
PL_DDR4_BA1	IO_L5N_T0U_N9_AD14N_64	AC7
PL_DDR4_RAS_B	IO_T2U_N12_64	AB5
PL_DDR4_CAS_B	IO_L5P_T0U_N8_AD14P_64	AB7
PL_DDR4_WE_B	IO_L11N_T1U_N9_GC_64	AF6
PL_DDR4_ACT_B	IO_L13N_T2L_N1_GC_QBC_64	AD4
PL_DDR4_CS_B	IO_L6P_T0U_N10_AD6P_64	AB6
PL_DDR4_BG0	IO_L2N_T0L_N3_64	AE8
PL_DDR4_RST	IO_L7P_T1L_N0_QBC_AD13P_64	AG9
PL_DDR4_CLK_N	IO_L10N_T1U_N7_QBC_AD4N_64	AG5
PL_DDR4_CLK_P	IO_L10P_T1U_N6_QBC_AD4P_64	AG6
PL_DDR4_CKE	IO_T3U_N12_64	AE4

PL_DDR4_OTD	IO_L19N_T3L_N1_DBC_AD9N_64	AH4
-------------	----------------------------	-----

2.1.4 QSPI Flash

ACU3EG 核心板配有 1 片 256MBit 大小的 Quad-SPI FLASH 芯片组成 8 位带宽数据总线, FLASH 型号为 MT25QU256ABA1EW9, 它使用 1.8V CMOS 电压标准。由于 QSPI FLASH 的非易失特性, 在使用中, 它可以作为系统的启动设备来存储系统的启动镜像。这些镜像主要包括 FPGA 的 bit 文件、ARM 的应用程序代码以及其它的用户数据文件。QSPI FLASH 的具体型号和相关参数见表 2-4-1。

位号	芯片类型	容量	厂家
U5	MT25QU256ABA1EW9	256M bit	Winbond

表2-4-1 QSPI Flash的型号和参数

QSPI FLASH 连接到 ZYNQ 芯片的 PS 部分 BANK500 的 GPIO 口上, 在系统设计中需要配置这些 PS 端的 GPIO 口功能为 QSPI FLASH 接口。为图 4-1 为 QSPI Flash 在原理图中的部分。

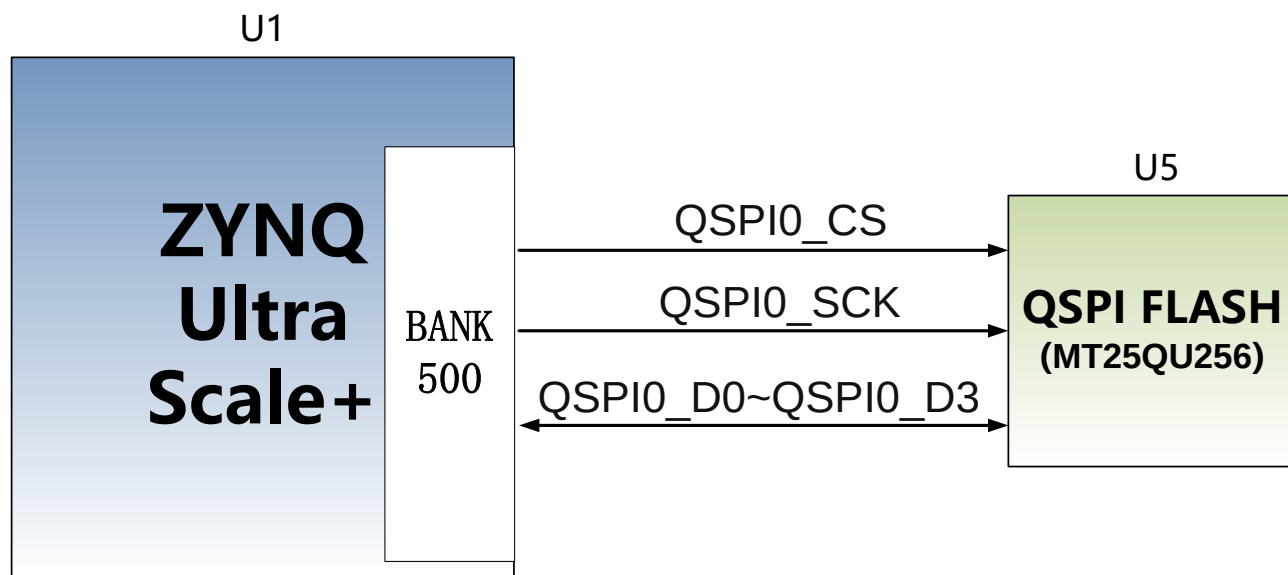


图 2-4-1 QSPI Flash 连接示意图

配置芯片引脚分配:

信号名称	引脚名	引脚号
MIO0_QSPI0_SCLK	PS_MIO0_500	AG15
MIO1_QSPI0_IO1	PS_MIO1_500	AG16
MIO2_QSPI0_IO2	PS_MIO2_500	AF15
MIO3_QSPI0_IO3	PS_MIO3_500	AH15
MIO4_QSPI0_IO0	PS_MIO4_500	AH16
MIO5_QSPI0_SS_B	PS_MIO5_500	AD16

2.1.5 eMMC Flash

ACU3EG 核心板配有一片大容量的 8GB 大小的 eMMC FLASH 芯片，型号为 MTFC8GAKAJCN-4M，它支持 JEDEC e-MMC V5.0 标准的 HS-MMC 接口，电平支持 1.8V 或者 3.3V。eMMC FLASH 和 ZYNQ 连接的数据宽度为 8bit。由于 eMMC FLASH 的大容量和非易失特性，在 ZYNQ 系统使用中，它可以作为系统大容量的存储设备，比如存储 ARM 的应用程序、系统文件以及其它的用户数据文件。eMMC FLASH 的具体型号和相关参数见表 2-5-1。

位号	芯片类型	容量	厂家
U19	MTFC8GAKAJCN-4M	8G Byte	Micron

表2-5-1 eMMC Flash的型号和参数

eMMC FLASH 连接到 ZYNQ UltraScale+ 的 PS 部分 BANK500 的 GPIO 口上，在系统设计中需要配置这些 PS 端的 GPIO 口功能为 EMMC 接口。为图 2-5-1 为 eMMC Flash 在原理图中的部分。

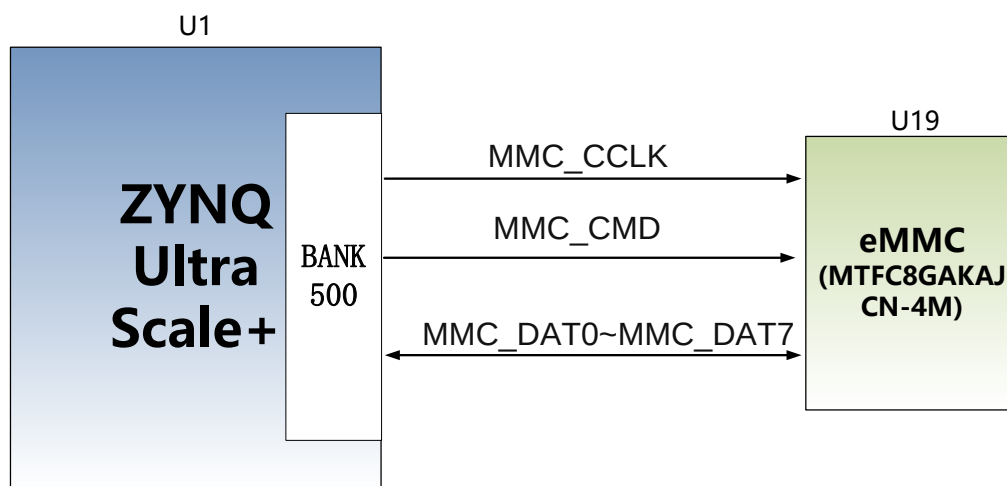


图 2-5-1 eMMC Flash 连接示意图

配置芯片引脚分配:

信号名称	引脚名	引脚号
MMC_DAT0	PS_MIO13_500	AH18
MMC_DAT1	PS_MIO14_500	AG18
MMC_DAT2	PS_MIO15_500	AE18
MMC_DAT3	PS_MIO16_500	AF18
MMC_DAT4	PS_MIO17_500	AC18
MMC_DAT5	PS_MIO18_500	AC19
MMC_DAT6	PS_MIO19_500	AE19
MMC_DAT7	PS_MIO20_500	AD19
MMC_CMD	PS_MIO21_500	AC21
MMC_CCLK	PS_MIO22_500	AB20
MMC_RSTN	PS_MIO23_500	AB18

2.1.6 时钟配置

核心板上分别为 PS 系统, PL 逻辑部分提供了参考时钟和 RTC 实时时钟, 使 PS 系统和 PL 逻辑可以单独工作。时钟电路设计的示意图如下图 2-6-1 所示:

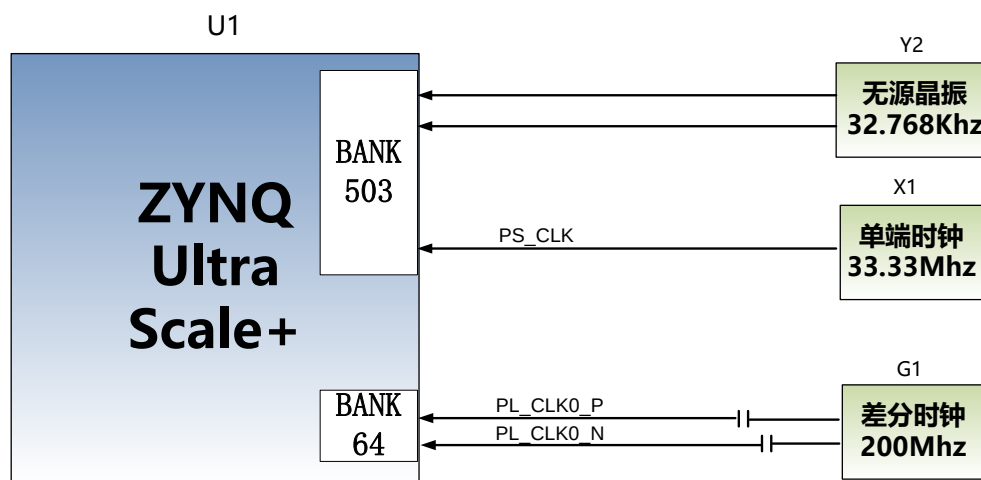


图 2-6-1 核心板时钟源

PS 系统 RTC 实时时钟

核心板上的无源晶体 Y2 为 PS 系统的提供 32.768KHz 的实时时钟源。晶体连接到 ZYNQ 芯片的 BANK503 的 PS_PADI_503 和 PS_PADO_503 的管脚上。其原理图如图 2-6-2 所示:

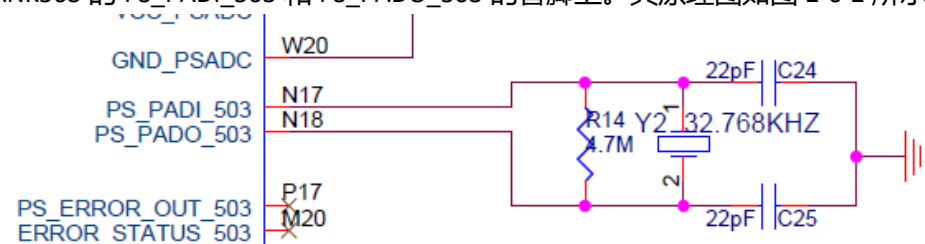


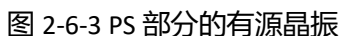
图 2-6-2 RTC 的无源晶振

时钟引脚分配:

信号名称	引脚
PS_PADI_503	N17
PS_PADO_503	N18

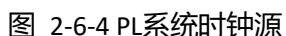
PS 系统时钟源

核心板上的 X1 晶振为 PS 部分提供 33.333MHz 的时钟输入。时钟的输入连接到 ZYNQ 芯片的 BANK503 的 PS_REF_CLK_503 的管脚上。其原理图如图 2-6-3 所示:



信号名称	引脚
PS_CLK	R16

板上提供了一个差分 200MHz 的 PL 系统时钟源，用于 DDR4 控制器的参考时钟。晶振输出连接到 PL BANK64 的全局时钟(MRCC)，这个全局时钟可以用来驱动 FPGA 内的 DDR4 控制器和用户逻辑电路。该时钟源的原理图如图 2-6-4 所示



信号名称	引脚
PL_CLK0_P	AE5
PL_CLK0_N	AF5

ACU3EG 核心板上有 1 个红色电源指示灯(PWR), 1 个是配置 LED 灯(DONE)。当核心板供电后, 电源指示灯会亮起; 当 FPGA 配置程序后, 配置 LED 灯会亮起。LED 灯硬件连接的示意图如图 2-7-1 所示:

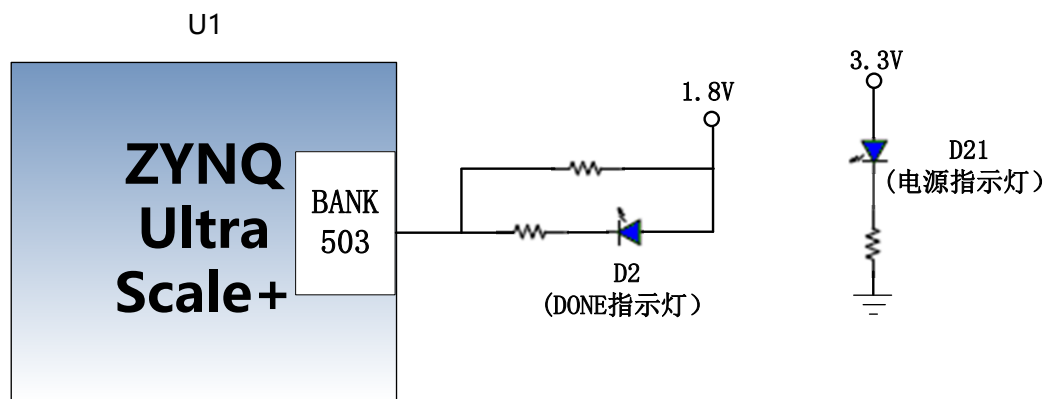
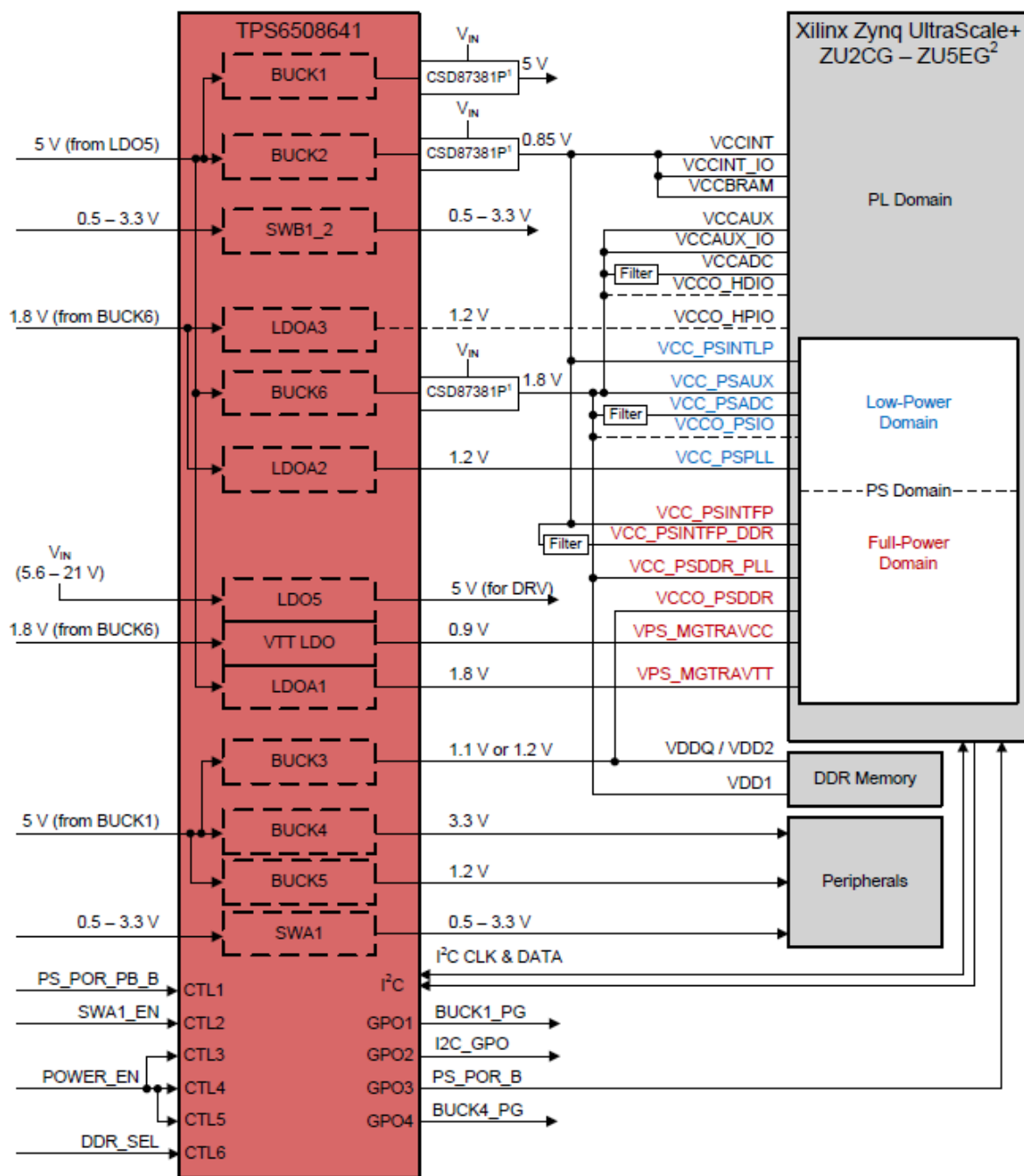


图 2-7-1 核心板 LED 灯硬件连接示意图

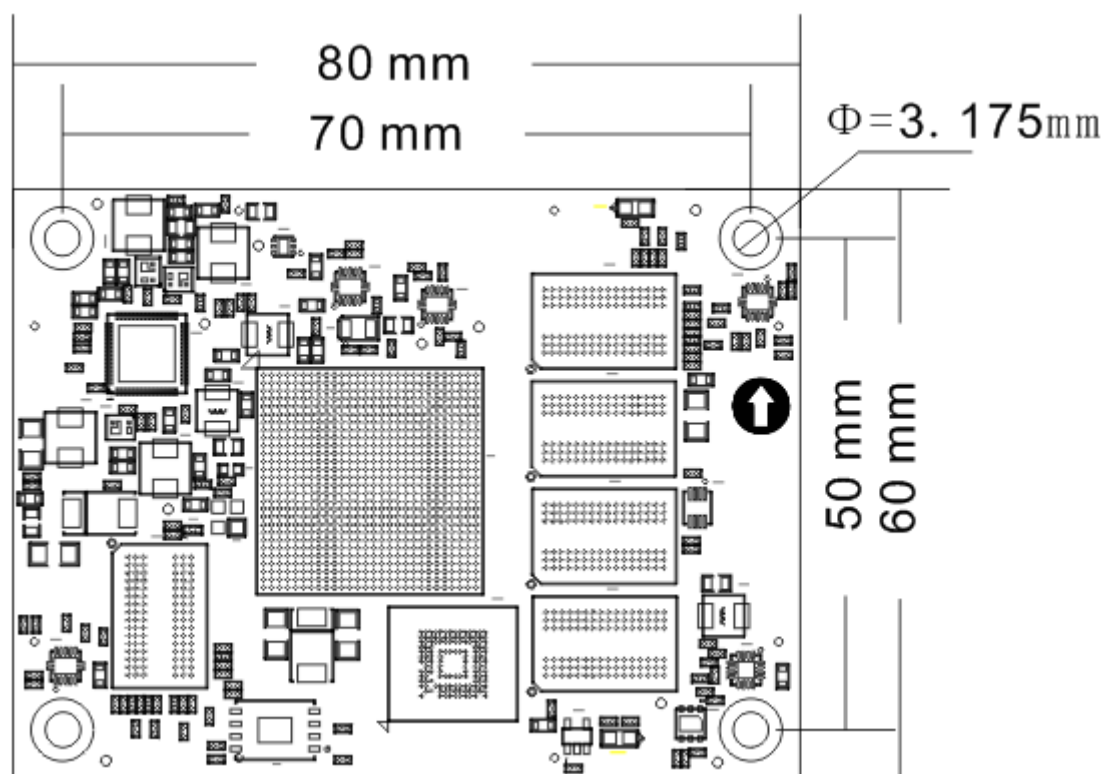
2.1.8 电源

ACU3EG 核心板供电电压为+12V，通过连接底板给核心板供电。核心板上通过一个 PMIC 芯片 TPS6508641 产生 XCZU3EG 芯片所需要的所有电源，TPS6508641 电源设计请参考电源芯片手册，设计框图如下：



另外 XCZU3EG 芯片的 BANK65, BANK66 的 VCCIO 电源是由底板提供, 方便用户修改, 但供电最高不能超过 1.8V。

2.1.9 结构图



正面图 (Top View)

2.1.10 连接器管脚定义

核心板一共扩展出 4 个高速扩展口，使用 4 个 120Pin 的板间连接器 (J29~J32) 和底板连接，连接器使用松下的 AXK5A2137YG，对应底板的连接器型号为 AXK6A2337YG。其中 J29 连接 BANK65, BANK66 的 IO，J30 连接 BANK25, BANK26，BANK66 的 IO 和 BANK505 MGT 的收发器信号，J31 连接 BANK24, BANK44 的 IO，J32 连接 PS 的 MIO，VCCO_65，VCCO_66 和 +12V 电源。

其中 BANK43~46 的 IO 的电平标准为 3.3V，BANK65,66 的电平标准由底板的 VCCO_65，VCCO_66 电源决定，但不能超过 +1.8V；MIO 的电平标准也为 1.8V。

J29 连接器的引脚分配

J29 管脚	信号名称	引脚号	J29 管脚	信号名称	引脚号
1	B65_L2_N	V9	2	B65_L22_P	K8
3	B65_L2_P	U9	4	B65_L22_N	K7
5	GND	-	6	GND	-
7	B65_L4_N	T8	8	B65_L20_P	J6
9	B65_L4_P	R8	10	B65_L20_N	H6
11	GND	-	12	GND	-
13	B65_L1_N	Y8	14	B65_L6_N	T6
15	B65_L1_P	W8	16	B65_L6_P	R6
17	GND	-	18	GND	-
19	B65_L7_P	L1	20	B65_L17_P	N9

21	B65_L7_N	K1	22	B65_L17_N	N8
23	GND	-	24	GND	-
25	B65_L15_P	N7	26	B65_L9_P	K2
27	B65_L15_N	N6	28	B65_L9_N	J2
29	GND	-	30	GND	-
31	B65_L16_P	P7	32	B65_L3_N	V8
33	B65_L16_N	P6	34	B65_L3_P	U8
35	GND	-	36	GND	-
37	B65_L14_P	M6	38	B65_L19_P	J5
39	B65_L14_N	L5	40	B65_L19_N	J4
41	GND	-	42	GND	-
43	B65_L5_N	T7	44	B65_L18_P	M8
45	B65_L5_P	R7	46	B65_L18_N	L8
47	GND	-	48	GND	-
49	B65_L11_N	K3	50	B65_L8_P	J1
51	B65_L11_P	K4	52	B65_L8_N	H1
53	GND	-	54	GND	-
55	B65_L10_N	H3	56	B65_L24_N	H8
57	B65_L10_P	H4	58	B65_L24_P	H9
59	GND	-	60	GND	-
61	B66_L3_P	F2	62	B65_L12_P	L3
63	B66_L3_N	E2	64	B65_L12_N	L2
65	GND	-	66	GND	-
67	B66_L1_P	G1	68	B65_L13_N	L6
69	B66_L1_N	F1	70	B65_L13_P	L7
71	GND	-	72	GND	-
73	B66_L6_P	G5	74	B65_L21_P	J7
75	B66_L6_N	F5	76	B65_L21_N	H7
77	GND	-	78	GND	-
79	B66_L16_P	G8	80	B65_L23_P	K9
81	B66_L16_N	F7	82	B65_L23_N	J9
83	GND	-	84	GND	-
85	B66_L15_P	G6	86	B66_L5_N	E3
87	B66_L15_N	F6	88	B66_L5_P	
89	GND	-	90	GND	-
91	B66_L4_P	G3	92	B66_L2_P	E1
93	B66_L4_N	F3	94	B66_L2_N	D1
95	GND	-	96	GND	-
97	B66_L11_P	D4	98	B66_L20_P	C6
99	B66_L11_N	C4	100	B66_L20_N	B6
101	GND	-	102	GND	-
103	B66_L12_P	C3	104	B66_L7_P	C1
105	B66_L12_N	C2	106	B66_L7_N	B1
107	GND	-	108	GND	-
109	B66_L13_P	D7	110	B66_L10_P	B4
111	B66_L13_N	D6	112	B66_L10_N	A4
113	GND	-	114	GND	-
115	B66_L8_P	A2	116	B66_L9_P	B3
117	B66_L8_N	A1	118	B66_L9_N	A3
119	GND	-	120	GND	-

J30 连接器的引脚分配

J30 管脚	信号名称	引脚号	J30 管脚	信号名称	引脚号
1	B66_L14_P	E5	2	FPGA_TDI	R18
3	B66_L14_N	D5	4	FPGA_TCK	R19
5	GND	-	6	GND	-
7	B66_L22_P	C8	8	FPGA_TDO	T21
9	B66_L22_N	B8	10	FPGA_TMS	N21
11	GND	-	12	GND	-
13	B66_L19_N	A5	14	B66_L21_N	A6
15	B66_L19_P	B5	16	B66_L21_P	A7
17	GND	-	18	GND	-
19	B66_L24_P	C9	20	B66_L17_P	F8
21	B66_L24_N	B9	22	B66_L17_N	E8
23	GND	-	24	GND	-
25	B66_L23_N	A8	26	B25_L9_P	C11
27	B66_L23_P	A9	28	B25_L9_N	B10
29	GND	-	30	GND	-
31	B25_L5_N	F10	32	B25_L10_P	B11
33	B25_L5_P	G11	34	B25_L10_N	A10
35	GND	-	36	GND	-
37	B66_L18_N	D9	38	B25_L12_P	D12
39	B66_L18_P	E9	40	B25_L12_N	C12
41	GND	-	42	GND	-
43	B25_L4_N	H12	44	B25_L11_P	A12
45	B25_L4_P	J12	46	B25_L11_N	A11
47	GND	-	48	GND	-
49	B26_L11_P	K14	50	B25_L6_N	F11
51	B26_L11_N	J14	52	B25_L6_P	F12
53	GND	-	54	GND	-
55	B26_L10_N	H13	56	B26_L6_N	E13
57	B26_L10_P	H14	58	B26_L6_P	E14
59	GND	-	60	GND	-
61	B26_L7_N	F13	62	B26_L3_N	A13
63	B26_L7_P	G13	64	B26_L3_P	B13
65	GND	-	66	GND	-
67	B26_L9_N	G14	68	B26_L2_N	A14
69	B26_L9_P	G15	70	B26_L2_P	B14
71	GND	-	72	GND	-
73	B26_L5_N	D14	74	B26_L4_N	C13
79	B26_L5_P	D15	76	B26_L4_P	C14
77	GND	-	78	GND	-
79	B26_L1_P	B15	80	B26_L12_P	L14
81	B26_L1_N	A15	82	B26_L12_N	L13
83	GND	-	84	GND	-
85	505_CLK2_P	C21	86	505_CLK1_P	E21
87	505_CLK2_P	C22	88	505_CLK1_P	E22
89	GND	-	90	GND	-

91	505_CLK0_P	F23	92	505_CLK3_P	A21
93	505_CLK0_N	F24	94	505_CLK3_N	A22
95	GND	-	96	GND	-
97	505_TX3_P	B23	98	505_TX1_P	D23
99	505_TX3_N	B24	100	505_TX1_N	D24
101	GND	-	102	GND	-
103	505_RX3_P	A25	104	505_TX0_P	E25
105	505_RX3_N	A26	106	505_TX0_N	E26
107	GND	-	108	GND	-
109	505_TX2_P	C25	110	505_RX1_P	D27
111	505_TX2_N	C26	112	505_RX1_N	D28
113	GND	-	114	GND	-
115	505_RX2_P	B27	116	505_RX0_P	F27
117	505_RX2_N	B28	118	505_RX0_N	F28
119	GND	-	120	GND	-

J31 连接器的引脚分配

J31 管脚	信号名称	引脚号	J31 管脚	信号名称	引脚号
1	B24_L10_P	Y14	2	B24_L7_P	AA13
3	B24_L10_N	Y13	4	B24_L7_N	AB13
5	GND	-	6	GND	-
7	B24_L6_P	AC14	8	B44_L6_P	AC12
9	B24_L6_N	AC13	10	B44_L6_N	AD12
11	GND	-	12	GND	-
13	B24_L5_P	AD15	14	B44_L7_P	AD11
15	B24_L5_N	AD14	16	B44_L7_N	AD10
17	GND	-	18	GND	-
19	B24_L1_P	AE15	20	B44_L8_N	AC11
21	B24_L1_N	AE14	22	B44_L8_P	AB11
23	GND	-	24	GND	-
25	B24_L12_P	Y12	26	B24_L2_P	AG14
27	B24_L12_N	AA12	28	B24_L2_N	AH14
29	GND	-	30	GND	-
31	B24_L3_P	AG13	32	-	-
33	B24_L3_N	AH13	34	-	-
35	GND	-	36	GND	-
37	B44_L12_N	AB9	38	B44_L9_P	AA11
39	B44_L12_P	AB10	40	B44_L9_N	AA10
41	GND	-	42	GND	-
43	B44_L10_N	Y10	44	B44_L3_P	AH12
45	B44_L10_P	W10	46	B44_L3_N	AH11
47	GND	-	48	GND	-
49	B24_L11_N	W11	50	B44_L1_N	AH10
51	B24_L11_P	W12	52	B44_L1_P	AG10
53	GND	-	54	GND	-
55	B24_L9_N	W13	56	B24_L4_P	AE13
57	B24_L9_P	W14	58	B24_L4_N	AF13
59	GND	-	60	GND	-
61	B24_L8_P	AB15	62	B44_L5_P	AE12

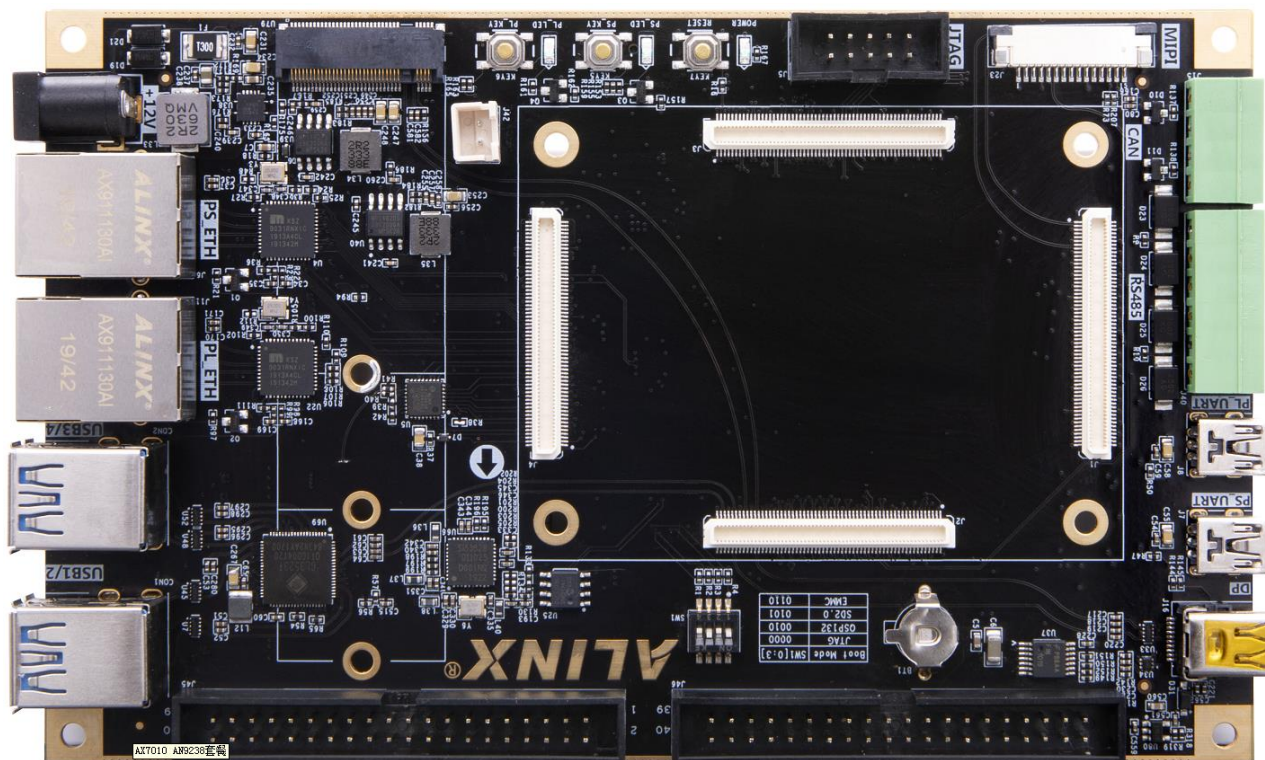
63	B24_L8_N	AB14	64	B44_L5_N	AF12
65	GND	-	66	GND	-
67	B44_L2_N	AG11	68	B44_L4_N	AF10
69	B44_L2_P	AF11	70	B44_L4_P	AE10
71	GND	-	72	GND	-
73	VBAT_IN	Y18	74	B44_L11_P	Y9
75	MR	-	76	B44_L11_N	AA8
77	GND	-	78	GND	-
79	-	-	80	PS_POR_B	P16
81	-	-	82	-	-
83	GND	-	84	GND	-
86	-	-	86	-	-
87	-	-	88	-	-
89	GND	-	90	GND	-
91	224_CLK0_P	Y6	92	224_CLK1_P	V6
93	224_CLK0_N	Y5	94	224_CLK1_N	V5
95	GND	-	96	GND	-
97	224_RX3_P	P2	98	224_TX3_P	N4
99	224_RX3_N	P1	100	224_TX3_N	N3
101	GND	-	102	GND	-
103	224_RX2_P	T2	104	224_TX2_P	R4
105	224_RX2_N	T1	106	224_TX2_N	R3
107	GND	-	108	GND	-
109	224_RX1_P	V2	110	224_TX1_P	U4
111	224_RX1_N	V1	112	224_TX1_N	U3
113	GND	-	114	GND	-
115	224_RX0_P	Y2	116	224_TX0_P	W4
117	224_RX0_N	Y1	118	224_TX0_N	W3
119	GND	-	120	GND	-

J32 连接器的引脚分配

J32 管脚	信号名称	引脚号	J32 管脚	信号名称	引脚号
1	PS_MIO35	H17	2	PS_MIO30	F16
3	PS_MIO29	G16	4	PS_MIO31	H16
5	GND	-	-	GND	-
7	-	-	8	PS_MIO58	F18
9	-	-	10	PS_MIO53	D16
11	GND	-	12	GND	-
13	PS_MODE0	P19	14	PS_MIO52	G18
15	PS_MODE1	P20	16	PS_MIO55	B16
17	GND	-	18	GND	-
19	PS_MODE2	R20	20	PS_MIO56	C16
21	PS_MODE3	T20	22	PS_MIO57	A16
23	GND	-	24	GND	-
25	PS_MIO36	K17	26	PS_MIO54	F17
27	PS_MIO37	J17	28	PS_MIO27	J15
29	GND	-	30	GND	-
31	-	-	32	PS_MIO28	K15
33	PS_MIO77	F20	34	PS_MIO59	E17

35	GND	-	36	GND	-
37	PS_MIO76	B20	38	PS_MIO60	C17
39	-	-	40	PS_MIO61	D17
41	GND	-	42	GND	-
43	PS_MIO39	H19	44	PS_MIO62	A17
45	PS_MIO38	H18	46	PS_MIO63	E18
47	GND	-	48	GND	-
49	-	-	50	PS_MIO65	A18
51	PS_MIO40	K18	52	PS_MIO66	G19
53	GND	-	54	GND	-
55	PS_MIO44	J20	56	PS_MIO67	B18
57	PS_MIO45	K20	58	PS_MIO68	C18
59	GND	-	60	GND	-
61	PS_MIO47	H21	62	PS_MIO64	E19
63	PS_MIO48	J21	64	PS_MIO69	D19
65	GND	-	66	GND	-
67	PS_MIO41	J19	68	PS_MIO74	D20
69	PS_MIO32	J16	70	PS_MIO73	G21
71	GND	-	72	GND	-
73	PS_MIO46	L20	74	PS_MIO72	G20
75	PS_MIO50	M19	76	PS_MIO71	B19
77	GND	-	78	GND	-
79	PS_MIO49	M18	80	PS_MIO75	A19
81	PS_MIO34	L17	82	PS_MIO70	C19
83	GND	-	84	GND	-
85	PS_MIO26	L15	86	PS_MIO43	K19
87	PS_MIO24	AB19	88	PS_MIO51	L21
89	GND	-	90	GND	-
91	PS_MIO25	AB21	92	PS_MIO42	L18
93	-	-	94	PS_MIO33	L16
95	GND	-	96	GND	-
97	-	-	98	-	-
99	VCCO_65	-	100	VCCO_66	-
101	VCCO_65	-	102	VCCO_66	-
103	VCCO_65	-	104	VCCO_66	-
105	GND	-	106	GND	-
107	+12V	-	108	+12V	-
109	+12V	-	110	+12V	-
111	+12V	-	112	+12V	-
113	+12V	-	114	+12V	-
115	+12V	-	116	+12V	-
117	+12V	-	118	+12V	-
119	+12V	-	120	+12V	-

2.2 扩展板



2.2.1 简介

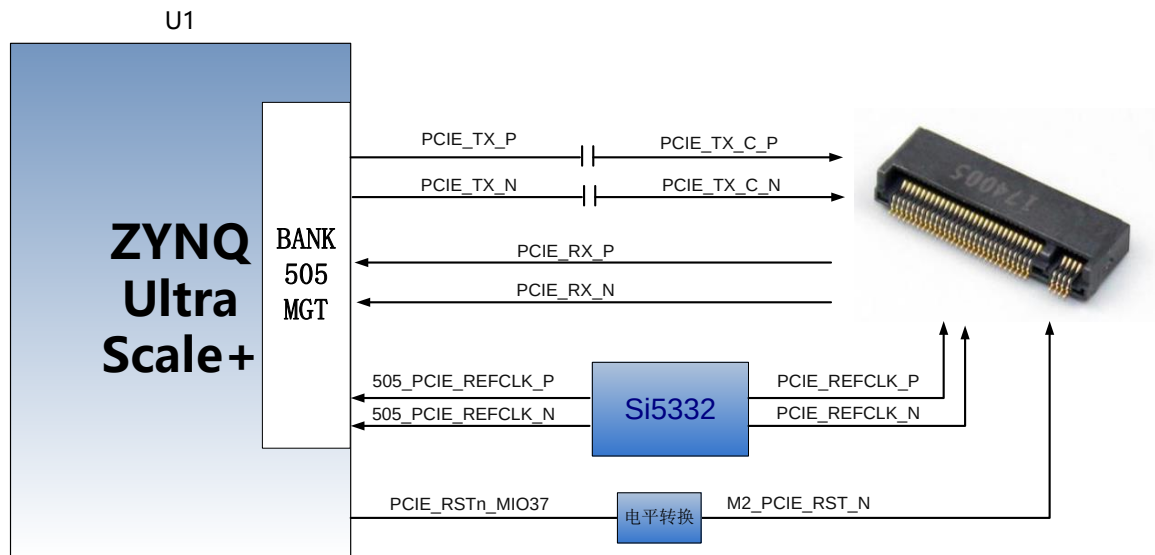
通过前面的功能简介，我们可以了解到扩展板部分的功能

- 1 路 M.2 接口
- 1 路 DP 输出接口
- 4 路 USB3.0 接口
- 2 路千兆以太网接口
- 2 路 USB Uart 接口
- 1 路 Micro SD 卡座
- 1 路 MIPI 摄像头接口
- 2 个 40 针扩展口
- 2 路 CAN 通信接口
- 2 路 485 通信接口
- JTAG 调试口
- 1 路温度传感器
- 1 路 EEPROM
- 1 路 RTC 实时时钟;
- 3 个 LED 灯
- 3 个按键

2.2.2 M.2 接口

AXU3EG 开发板配备了一个 PCIe x1 标准的 M.2 接口，用于连接 M.2 的 SSD 固态硬盘，通信速度高达 6Gbps。M.2 接口使用 M key 插槽，只支持 PCI-E，不支持 SATA，用户选择 SSD 固态硬盘的时候需要选择 PCIe 类型的 SSD 固态硬盘。

PCIe 信号直接跟 ZU3EG 的 BANK505 PS MGT 收发器相连接，1 路 TX 信号和 RX 信号都是以差分信号方式连接到 MGT 的 LANE1。PCIe 的时钟有 Si5332 芯片提供，频率为 100Mhz，M.2 电路设计的示意图如下图 3-2-1 所示：



3-2-1 M.2 接口设计示意图

M.2 接口 ZYNQ 引脚分配如下：

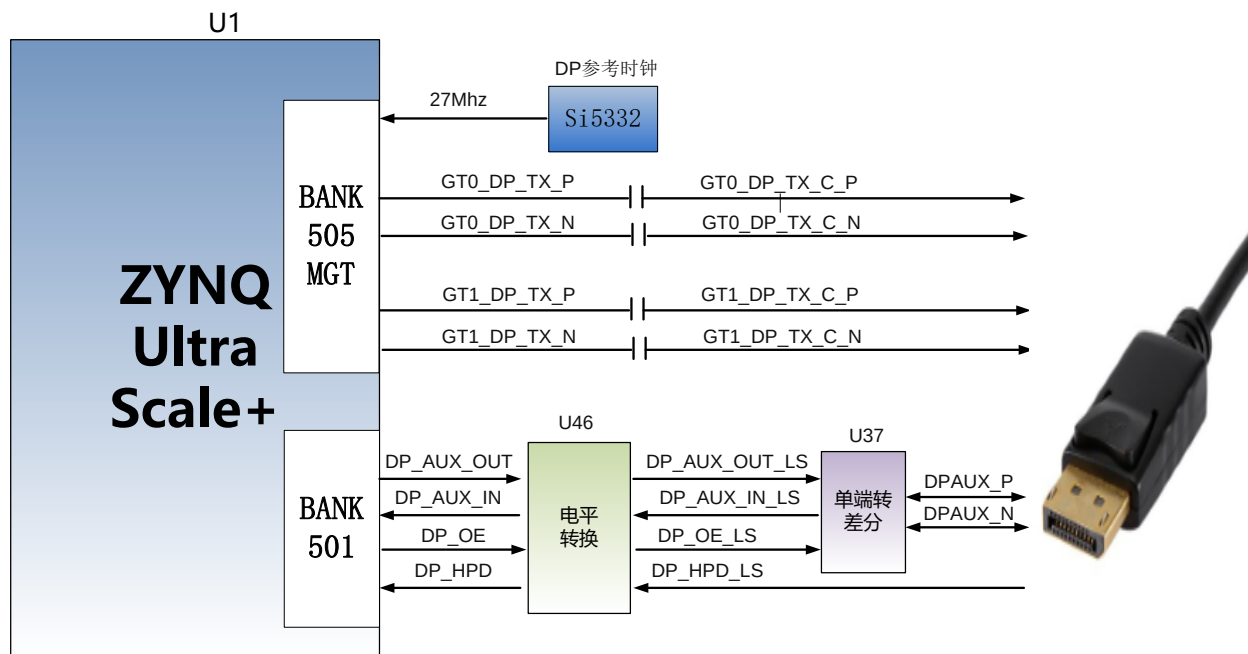
信号名称	引脚名	引脚号	备注
PCIe_TX_P	505_TX0_P	E25	PCIe 数据发送正
PCIe_TX_N	505_TX0_N	E26	PCIe 数据发送负
PCIe_RX_P	505_RX0_P	F27	PCIe 数据接收正
PCIe_RX_N	505_RX0_N	F28	PCIe 数据接收负
505_PCIE_REFCLK_P	505_CLK0_P	F23	PCIe 参考时钟正
505_PCIE_REFCLK_N	505_CLK0_N	F24	PCIe 参考时钟负
PCIe_RSTn_MIO37	PS_MIO37_501	J17	PCIe 复位信号

2.2.3 DP 显示接口

AXU3EG 开发板带有 1 路标准的 DisplayPort 输出显示接口，用于视频图像的显示。接口支持 VESA DisplayPort V1.2a 输出标准，最高支持 4K x 2K@30Fps 输出，支持 Y-only, YCbCr444, YCbCr422, YCbCr420 和 RGB 视频格式，每种颜色支持 6, 8, 10, 或者 12 位。

DisplayPort 数据传输通道直接用 ZU3EG 的 BANK505 PS MGT 驱动输出，MGT 的 LANE2 和

LANE3 TX 信号以差分信号方式连接到 DP 连接器。DisplayPort 辅助通道连接到 PS 的 MIO 管脚上。DP 输出接口设计的示意图如下图 3-3-1 所示:



3-3-1 DP 接口设计示意图

DisplayPort 接口 ZYNQ 引脚分配如下:

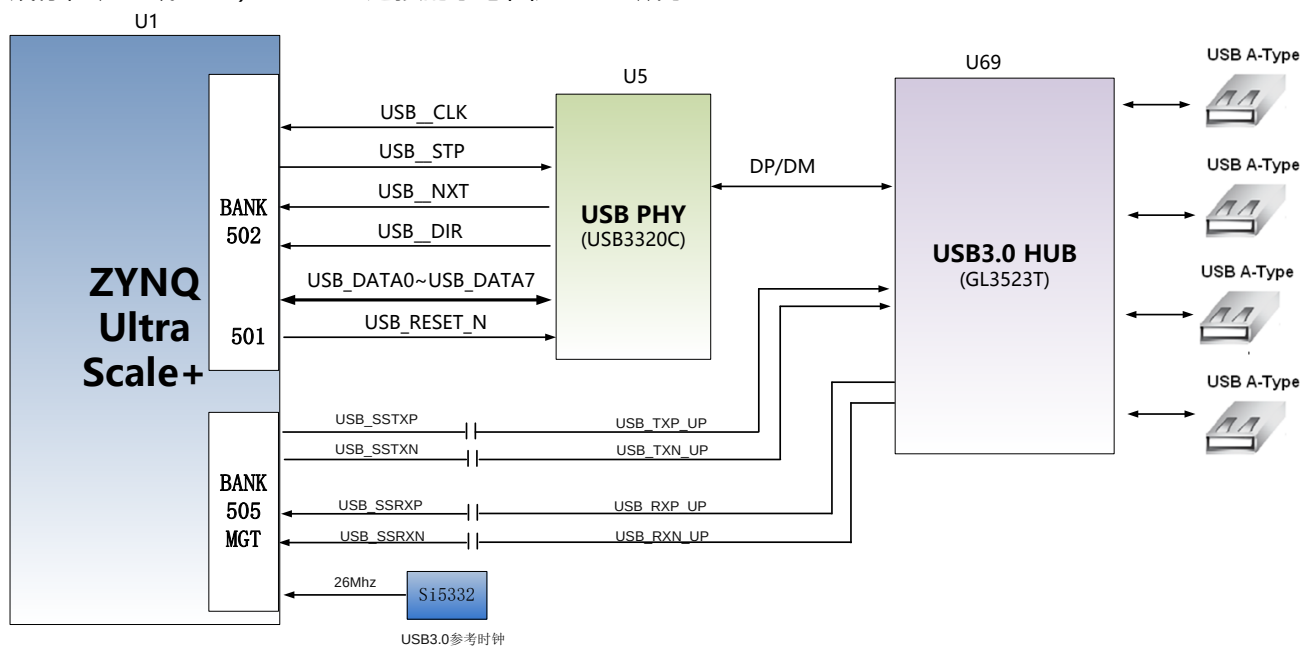
信号名称	ZYNQ 引脚名	ZYNQ 引脚号	备注
GT0_DP_TX_P	505_TX3_P	B23	DP 数据低位发送正
GT0_DP_TX_N	505_TX3_N	B24	DP 数据低位发送负
GT1_DP_TX_P	505_TX2_P	C25	DP 数据高位发送正
GT1_DP_TX_N	505_TX2_N	C26	DP 数据高位发送负
505_CLK1_P	505_CLK2_P	C21	DP 参考时钟正
505_CLK1_N	505_CLK2_N	C22	DP 参考时钟负
DP_AUX_OUT	PS_MIO27	J15	DP 辅助数据输出
DP_AUX_IN	PS_MIO30	F16	DP 辅助数据输入
DP_OE	PS_MIO29	G16	DP 辅助数据输出使能
DP_HPDP	PS_MIO28	K15	DP 插入信号检测

2.2.4 USB3.0 接口

AXU3EG 扩展板上有 4 个 USB3.0 接口, 支持 HOST 工作模式, 数据传输速度高达 5.0Gb/s。USB3.0 通过 PIPE3 接口连接, USB2.0 通过 ULPI 接口连接外部的 USB3320C 芯片, 实现高速的 USB3.0 和 USB2.0 的数据通信。

USB 接口为扁型 USB 接口(USB Type A), 方便用户同时连接不同的 USB Slave 外设(比如 USB

鼠标，键盘或 U 盘)。USB3.0 连接的示意图如 3-4-1 所示：



3-4-1 USB3.0 接口示意图

USB 接口引脚分配：

信号名称	引脚名	引脚号	备注
USB_SSTXP	505_TX1_P	D23	USB3.0 数据发送正
USB_SSTXN	505_TX1_N	D24	USB3.0 数据发送负
USB_SSRXP	505_RX1_P	D27	USB3.0 数据接收正
USB_SSRXN	505_RX1_N	D28	USB3.0 数据接收负
USB_DATA0	PS_MIO56	C16	USB2.0 数据 Bit0
USB_DATA1	PS_MIO57	A16	USB2.0 数据 Bit1
USB_DATA2	PS_MIO54	F17	USB2.0 数据 Bit2
USB_DATA3	PS_MIO59	E17	USB2.0 数据 Bit3
USB_DATA4	PS_MIO60	C17	USB2.0 数据 Bit4
USB_DATA5	PS_MIO61	D17	USB2.0 数据 Bit5
USB_DATA6	PS_MIO62	A17	USB2.0 数据 Bit6
USB_DATA7	PS_MIO63	E18	USB2.0 数据 Bit7
USB_STP	PS_MIO58	F18	USB2.0 停止信号
USB_DIR	PS_MIO53	D16	USB2.0 数据方向信号
USB_CLK	PS_MIO52	G18	USB2.0 时钟信号
USB_NXT	PS_MIO55	B16	USB2.0 下一数据信号
USB_RESET_N	PS_MIO31	H16	USB2.0 复位信号

2.2.5 千兆以太网接口

AXU3EG 扩展板上有 2 路千兆以太网接口，1 路连接到 PS 端，另 1 路连接到 PL 端。GPHY 芯片采用 Micrel 公司的 KSZ9031RNX 以太网 PHY 芯片为用户提供网络通信服务。KSZ9031RNX 芯片支持 10/100/1000 Mbps 网络传输速率，通过 RGMII 接口跟 ZU3EG 系统的 MAC 层进行数据通信。KSZ9031RNX 支持 MDI/MDX 自适应，各种速度自适应，Master/Slave 自适应，支持 MDIO 总线进行 PHY 的寄存器管理。

KSZ9031RNX 上电会检测一些特定的 IO 的电平状态，从而确定自己的工作模式。表 3-5-1 描述了 GPHY 芯片上电之后的默认设定信息。

配置 Pin 脚	说明	配置值
PHYAD[2:0]	MDIO/MDC 模式的 PHY 地址	PHY Address 为 011
CLK125_EN	使能 125Mhz 时钟输出选择	使能
LED_MODE	LED 灯模式配置	单个 LED 灯模式
MODE0~MODE3	链路自适应和全双工配置	10/100/1000 自适应，兼容全双工、半双工

表 3-5-1PHY 芯片默认配置值

当网络连接到千兆以太网时，ZYNQ 和 PHY 芯片 KSZ9031RNX 的数据传输时通过 RGMII 总线通信，传输时钟为 125Mhz，数据在时钟的上升沿和下降样采样。

当网络连接到百兆以太网时，ZYNQ 和 PHY 芯片 KSZ9031RNX 的数据传输时通过 RMII 总线通信，传输时钟为 25Mhz。数据在时钟的上升沿和下降样采样。

图 3-5-1 为 ZYNQ 以太网 PHY 芯片连接示意图：

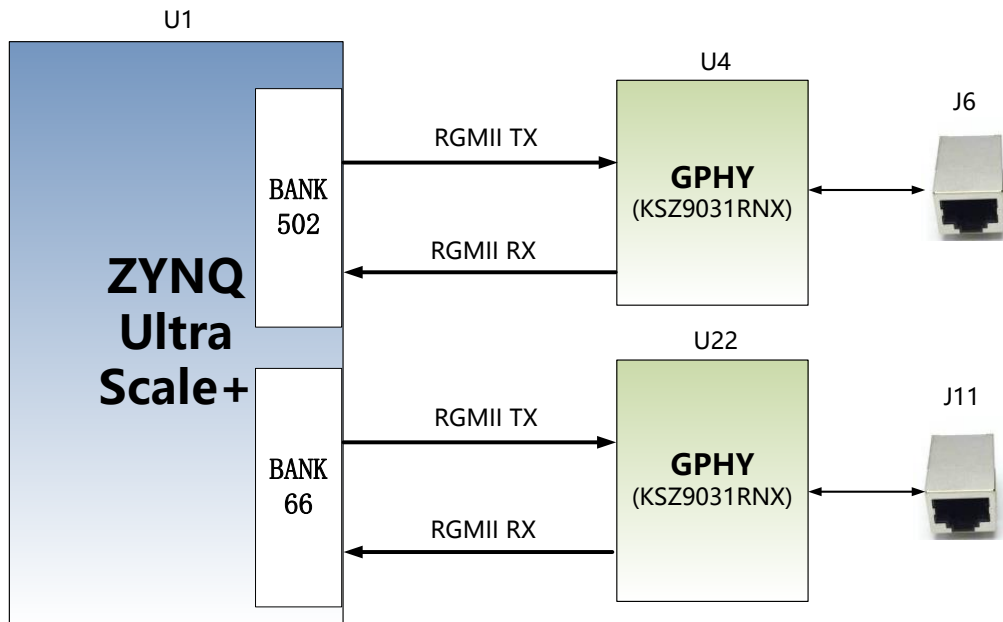


图 3-6-1 ZYNQ 与 GPHY 连接示意图

千兆以太网引脚分配如下：

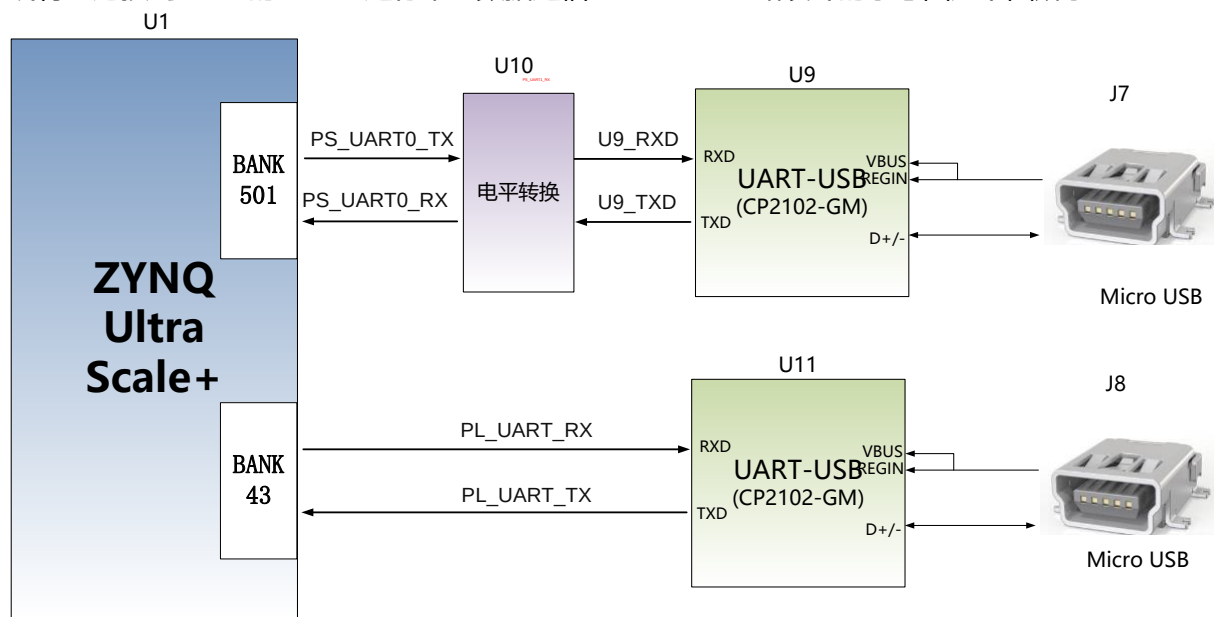
信号名称	引脚名	引脚号	备注
------	-----	-----	----

PHY1_TXCK	PS_MIO64	E19	以太网 1RGMII 发送时钟
PHY1_TXD0	PS_MIO65	A18	以太网 1 发送数据 bit 0
PHY1_TXD1	PS_MIO66	G19	以太网 1 发送数据 bit1
PHY1_TXD2	PS_MIO67	B18	以太网 1 发送数据 bit2
PHY1_TXD3	PS_MIO68	C18	以太网 1 发送数据 bit3
PHY1_TXCTL	PS_MIO69	D19	以太网 1 发送使能信号
PHY1_RXCK	PS_MIO70	C19	以太网 1RGMII 接收时钟
PHY1_RXD0	PS_MIO71	B19	以太网 1 接收数据 Bit0
PHY1_RXD1	PS_MIO72	G20	以太网 1 接收数据 Bit1
PHY1_RXD2	PS_MIO73	G21	以太网 1 接收数据 Bit2
PHY1_RXD3	PS_MIO74	D20	以太网 1 接收数据 Bit3
PHY1_RXCTL	PS_MIO75	A19	以太网 1 接收数据有效信号
PHY1_MDC	PS_MIO76	B20	以太网 1MDIO 管理时钟
PHY1_MDIO	PS_MIO77	F20	以太网 1MDIO 管理数据
PHY2_TXCK	B66_L17_N	E8	以太网 2 RGMII 发送时钟
PHY2_TXD0	B66_L18_P	E9	以太网 2 发送数据 bit 0
PHY2_TXD1	B66_L18_N	D9	以太网 2 发送数据 bit1
PHY2_TXD2	B66_L23_P	A9	以太网 2 发送数据 bit2
PHY2_TXD3	B66_L23_N	A8	以太网 2 发送数据 bit3
PHY2_TXCTL	B66_L24_N	B9	以太网 2 发送使能信号
PHY2_RXCK	B66_L14_P	E5	以太网 2 RGMII 接收时钟
PHY2_RXD0	B66_L19_N	A5	以太网 2 接收数据 Bit0
PHY2_RXD1	B66_L19_P	B5	以太网 2 接收数据 Bit1
PHY2_RXD2	B66_L17_P	F8	以太网 2 接收数据 Bit2
PHY2_RXD3	B66_L24_P	C9	以太网 2 接收数据 Bit3
PHY2_RXCTL	B66_L22_N	B8	以太网 2 接收数据有效信号
PHY2_MDC	B66_L21_N	A6	以太网 2 MDIO 管理时钟
PHY2_MDIO	B66_L22_P	C8	以太网 2 MDIO 管理数据
PHY2_RESET	B66_L14_N	D5	以太网 2 复位信号

2.2.6 USB Uart 接口

AXU3EG 扩展板上配备了 2 个 Uart 转 USB 接口，1 个连接到 PS 端，一个连接到 PL 端。转换芯片采用 Silicon Labs CP2102GM 的 USB-UAR 芯片，USB 接口采用 MINI USB 接口，可以用 USB

线将它连接到上 PC 的 USB 口进行串口数据通信。USB Uart 电路设计的示意图如下图所示:



3-6-1 USB 转串口示意图

USB 转串口的 ZYNQ 引脚分配:

信号名称	引脚名	引脚号	备注
PS_UART0_TX	PS_MIO43	K19	PS Uart 数据输出
PS_UART0_RX	PS_MIO42	L18	PS Uart 数据输入
PL_UART_TX	B43_L3_P	AH12	PL Uart 数据输出
PL_UART_RX	B43_L3_N	AH11	PL Uart 数据输入

2.2.7 SD 卡槽

AXU3EG扩展板包含了一个Micro型的SD卡接口，以提供用户访问SD卡存储器，用于存储ZU3EG芯片的BOOT程序，Linux操作系统内核，文件系统以及其它的用户数据文件。

SDIO信号与ZU3EG的PS BANK501的IO信号相连，因为501的VCCIO设置为1.8V，但SD卡的数据电平为3.3V，我们这里通过TXS02612电平转换器来连接。ZU3EG PS和SD卡连接器的原理图如图3-7-1所示。

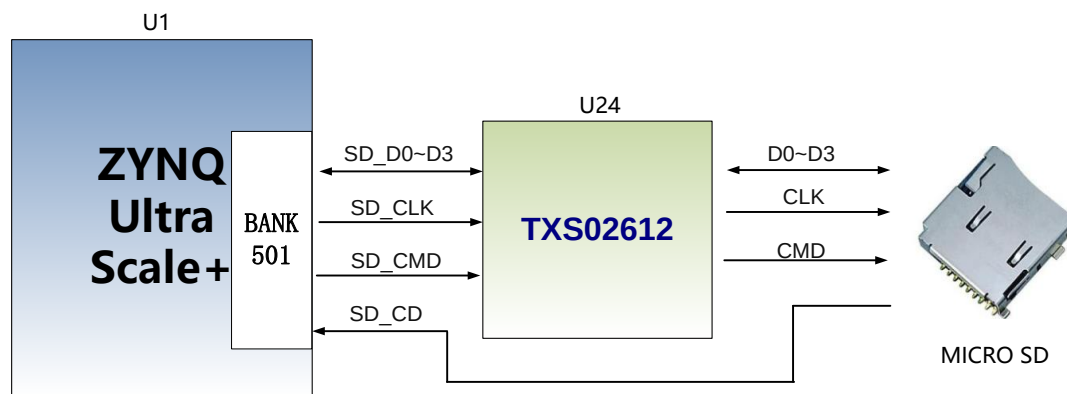


图 3-7-1 SD 卡连接示意图

SD 卡槽引脚分配

信号名称	引脚名	引脚号	备注
SD_CLK	PS_MIO51	I21	SD 时钟信号
SD_CMD	PS_MIO50	M19	SD 命令信号
SD_D0	PS_MIO46	L20	SD 数据 Data0
SD_D1	PS_MIO47	H21	SD 数据 Data1
SD_D2	PS_MIO48	J21	SD 数据 Data2
SD_D3	PS_MIO49	M18	SD 数据 Data3
SD_CD	PS_MIO45	K20	SD 卡检测信号

2.2.8 40 针扩展口

AXU3EG 扩展板预留了 2 个 2.54mm 标准间距的 40 针的扩展口 J45 和 J46，用于连接黑金的各个模块或者用户自己设计的外面电路，扩展口有 40 个信号，其中，5V 电源 1 路，3.3V 电源 2 路，地 3 路，IO 口 34 路。扩展口的 IO 连接的 ZYNQ 芯片 BANK44,24,25,26 的 IO 上，电平标准为 3.3V。

J45 扩展口 ZYNQ 的引脚分配如下：

J45管脚	信号名称	引脚号	J17管脚	信号名称	引脚号
1	GND	-	2	+5V	-
3	B45_L9_N	B10	4	B45_L9_P	C11
5	B45_L5_N	F10	6	B45_L5_P	G11
7	B45_L12_N	C12	8	B45_L12_P	D12
9	B45_L11_N	A11	10	B45_L11_P	A12
11	B45_L6_N	F11	12	B45_L6_P	F12
13	B46_L6_N	E13	14	B46_L6_P	E14
15	B46_L3_N	A13	16	B46_L3_P	B13
17	B46_L2_N	A14	18	B46_L2_P	B14
19	B46_L4_N	C13	20	B46_L4_P	C14
21	B46_L12_N	L13	22	B46_L12_P	L14
23	B45_L4_N	H12	24	B45_L4_P	J12

25	B46_L11_N	J14	26	B46_L11_P	K14
27	B46_L10_N	H13	28	B46_L10_P	H14
29	B46_L7_N	F13	30	B46_L7_P	G13
31	B46_L9_N	G14	32	B46_L9_P	G15
33	B46_L5_N	D14	34	B46_L5_P	D15
35	B46_L1_N	A15	36	B46_L1_P	B15
37	GND	-	38	GND	-
39	+3.3V	-	40	+3.3V	-

J46 扩展口 ZYNQ 的引脚分配如下:

J46管脚	信号名称	引脚号	J13管脚	信号名称	引脚号
1	GND	-	2	+5V	-
3	B43_L2_N	AG11	4	IO2_1P	Y14
5	B44_L8_N	AB14	6	IO2_2P	AA13
7	B44_L9_N	W13	8	IO2_3P	AC12
9	B44_L11_N	W11	10	IO2_4P	AD11
11	B43_L10_N	Y10	12	IO2_5P	AB11
13	B43_L12_N	AB9	14	IO2_6P	AG14
15	B44_L3_N	AH13	16	IO2_7P	AC14
17	B44_L12_N	AA12	18	IO2_8P	AD15
19	B44_L1_N	AE14	20	IO2_9P	AH12
21	B44_L5_N	AD14	22	IO2_10P	AA11
23	B44_L6_N	AC13	24	IO2_11P	Y9
25	B44_L10_N	Y13	26	IO2_12P	AE10
27	B44_L2_N	AH14	28	IO2_13P	AE12
29	B43_L8_N	AC11	30	IO2_14P	AG10
31	B43_L7_N	AD10	32	IO2_15P	AF11
33	B43_L6_N	AD12	34	IO2_16P	W10
35	B44_L7_N	AB13	36	IO2_17P	AB10
37	GND	-	38	GND	-
39	+3.3V	-	40	+3.3V	-

2.2.9 CAN 通信接口

AXU3EG 扩展板上有 2 路 CAN 通信接口, 连接在 PS 系统端 BANK501 的 MIO 接口上。
CAN 收发芯片选用了 TI 公司的 SN65HVD232C 芯片为用户 CAN 通信服务。

图 3-9-1 为 PS 端 CAN 收发芯片的连接示意图

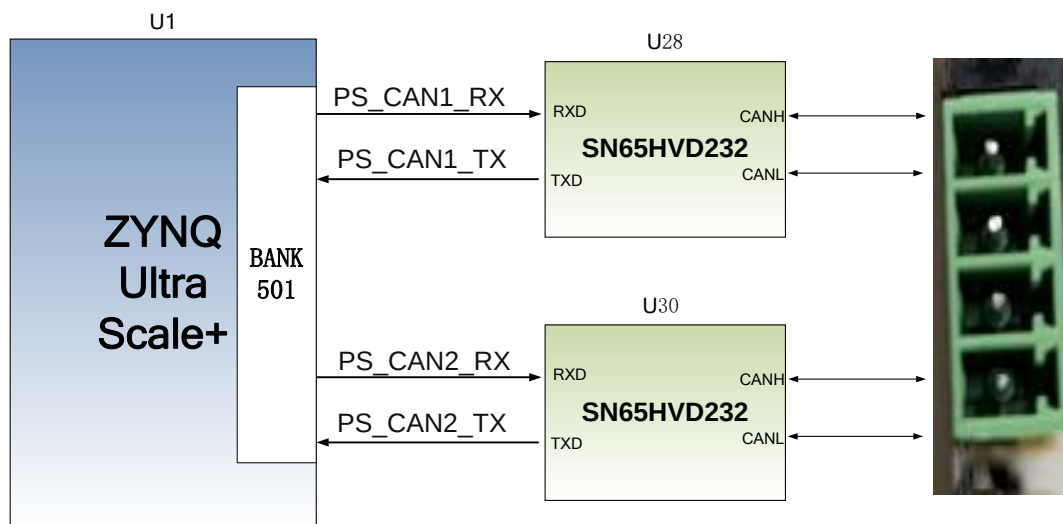


图 3-10-1 PS 端 CAN 收发芯片的连接示意图

CAN 通信引脚分配如下:

信号名称	引脚名	引脚号	备注
PS_CAN1_TX	PS_MIO32	J16	CAN1 发送端
PS_CAN1_RX	PS_MIO33	L16	CAN1 接收端
PS_CAN2_TX	PS_MIO39	H19	CAN2 发送端
PS_CAN2_RX	PS_MIO38	H18	CAN2 接收端

2.2.10 485 通信接口

AXU3EG 扩展板上有 2 路 485 通信接口，485 通信端口连接在 PL 端 BANK43~45 的 IO 接口上。485 收发芯片选用 MAXIM 公司的 MAX3485 芯片为用户 485 通信服务。

图 3-11-1 为 PL 端 485 收发芯片的连接示意图

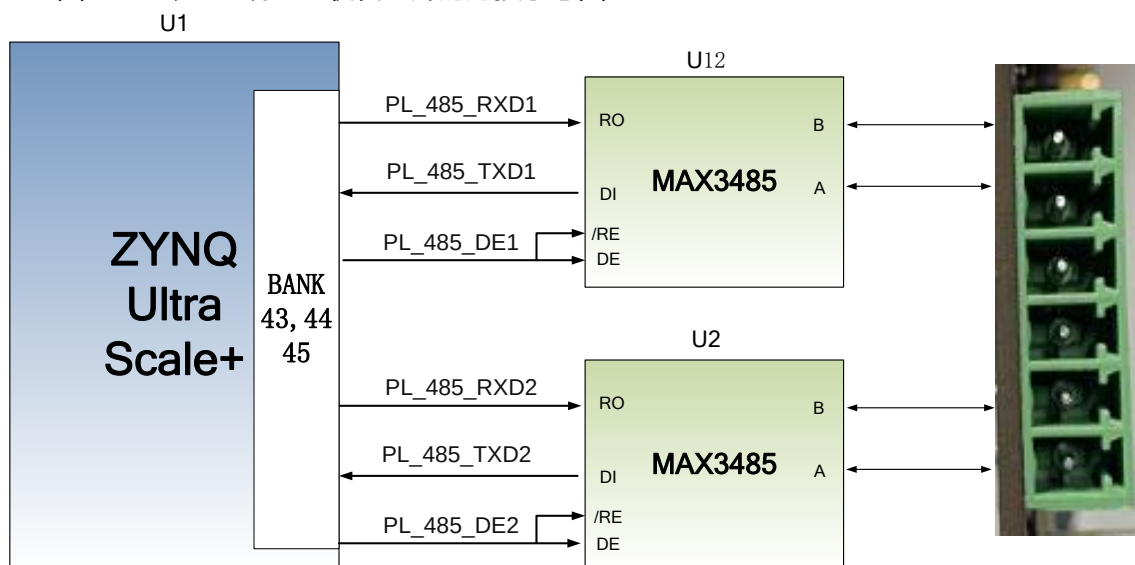


图 3-11-1 PL 端 485 通信的连接示意图

RS485 通信引脚分配如下:

信号名称	引脚名	引脚号	备注
PL_485_TXD1	B43_L1_N	AH10	第一路485发送端
PL_485_RXD1	B44_L4_P	AE13	第一路485接收端
PL_485_DE1	B45_L10_P	B11	第一路485发送使能
PL_485_TXD2	B43_L1_N	AG10	第二路485发送端
PL_485_RXD2	B44_L4_N	AF13	第二路485接收端
PL_485_DE2	B45_L10_N	A10	第二路485发送使能

2.2.11 MIPI 接口

底板上包含了一个 MIPI 摄像头接口，可以用来接我们的 MIPI OV5640 像头模块 (AN5641)。MIPI 接口 15PIN 的 FPC 连接器，为 2 个 LANE 的数据和 1 对时钟，连接到 BANK65 的差分 IO 管脚上，电平标准为 1.2V；其它的控制信号连接到 BANK43 的 IO 上，电平标准为 3.3V。

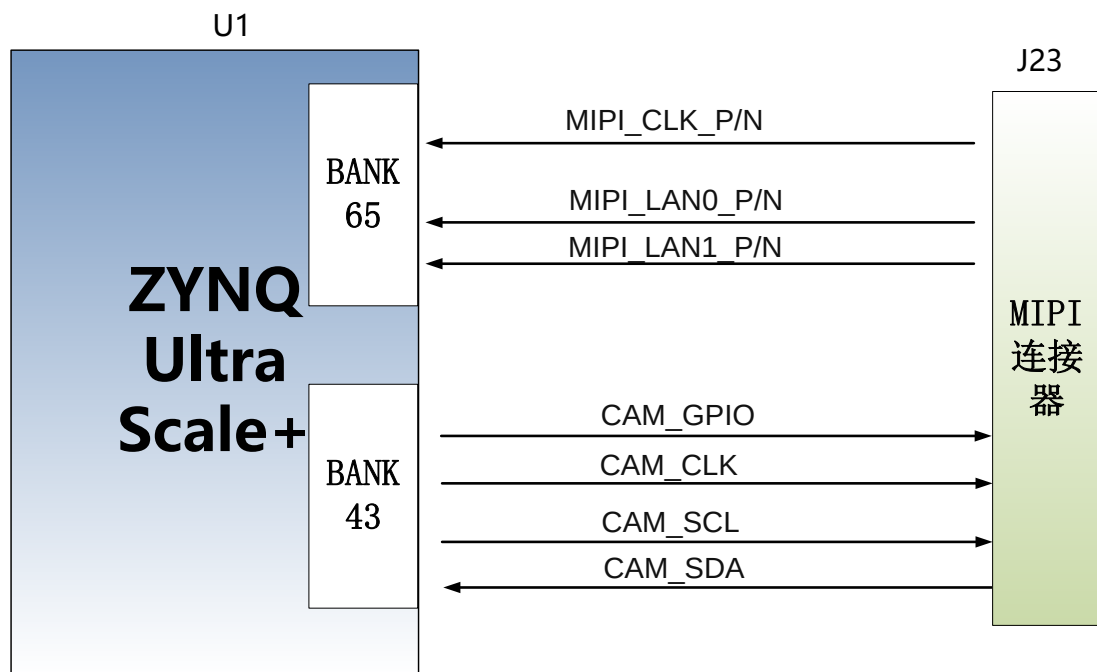


图 3-11-1 HDMI 接口设计原理图

MIPI 接口引脚分配

信号名称	ZYNQ 引脚名	ZYNQ 引脚号	备注
MIPI_CLK_P	B65_L1_P	W8	MIPI输入时钟正
MIPI_CLK_N	B65_L1_N	Y8	MIPI输入时钟负
MIPI_LAN0_P	B65_L2_P	U9	MIPI输入的数据LANE0正
MIPI_LAN0_N	B65_L2_N	V9	MIPI输入的数据LANE0负

MIPI_LAN1_P	B65_L3_P	U8	MIPI输入的数据LANE1正
MIPI_LAN1_N	B65_L3_N	V8	MIPI输入的数据LANE1负
CAM_GPIO	B43_L4_P	AE10	摄像头的GPIO控制
CAM_CLK	B43_L4_N	AF10	摄像头的时钟输入
CAM_SCL	B43_L11_P	Y9	摄像头的I2C时钟
CAM_SDA	B43_L11_N	AA8	摄像头的I2C数据

2.2.12 JTAG 调试口

在 AXU3EG 扩展板上预留了一个 JTAG 接口，用于下载 ZYNQ UltraScale+程序或者固化程序到 FLASH。为了带电插拔造成对 ZYNQ UltraScale+芯片的损坏，我们在 JTAG 信号上添加了保护二极管来保证信号的电压在 FPGA 接受的范围，避免 ZYNQ UltraScale+芯片的损坏。

JTAG Connector

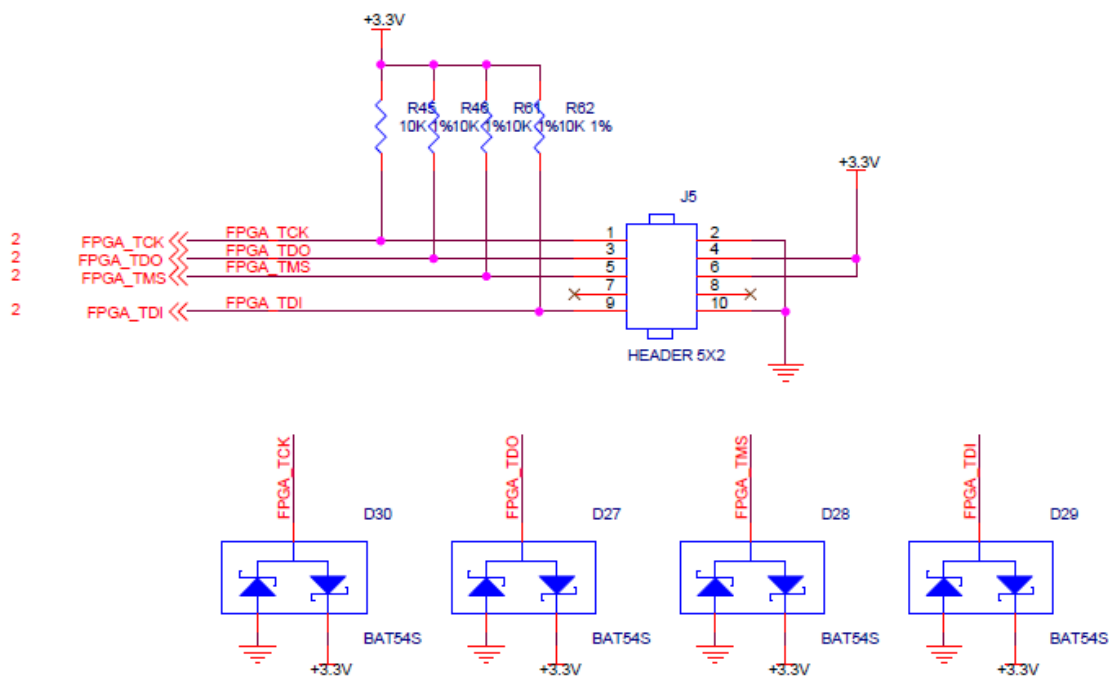


图3-12-1 原理图中JTAG接口部分

2.2.13 RTC 实时时钟

ZU3EG 芯片内部带有 RTC 实时时钟的功能，有年月日时分秒还有星期计时功能。外部需要接一个 32.768KHz 的无源时钟，提供精确的时钟源给内部时钟电路，这样才能让 RTC 可以准确的提供时钟信息。同时为了产品掉电以后，实时时钟还可以正常运行，一般需要另外配一个电池给时钟芯片供电。开发板上的 BT1 为 1.5V 的纽扣电池（型号 LR1130，电压为 1.5V），当系统掉电，纽扣电池还可以给 RTC 系统供电，可以提供持续不断的时间信息。图 3-12-1 为 RTC

实时时钟原理图

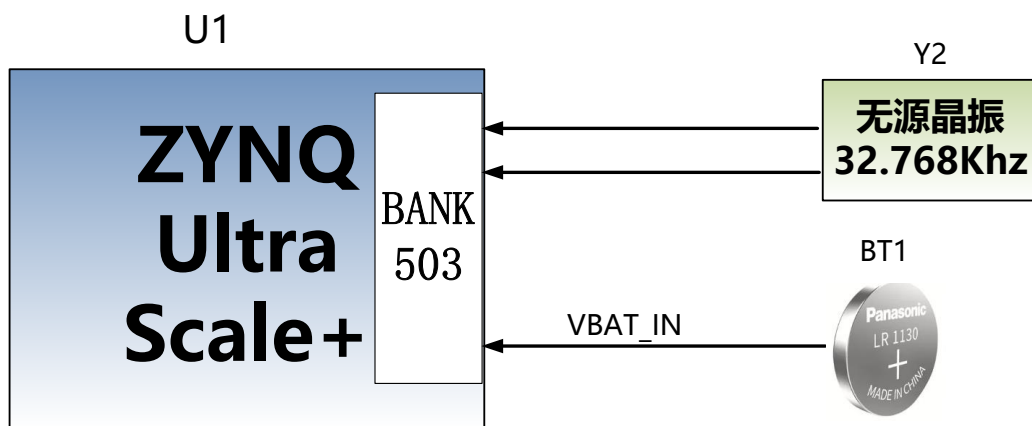


图 3-13-1 为 RTC 实时时钟原理图

2.2.14 EEPROM 和温度传感器

AXU3EG开发板板载了一片EEPROM，型号为24LC04,容量为：4Kbit (2*256*8bit)，通过IIC总线连接到PS端进行通信。另外板上还带有一个高精度、低功耗、数字温度传感器芯片，型号为ON Semiconductor公司的LM75，LM75芯片的温度精度为0.5度。EEPROM和温度传感器通过I2C总线挂载到ZYNQ UltraScale+的Bank500 MIO上。图3-14-1为EEPROM和温度传感器的原理图

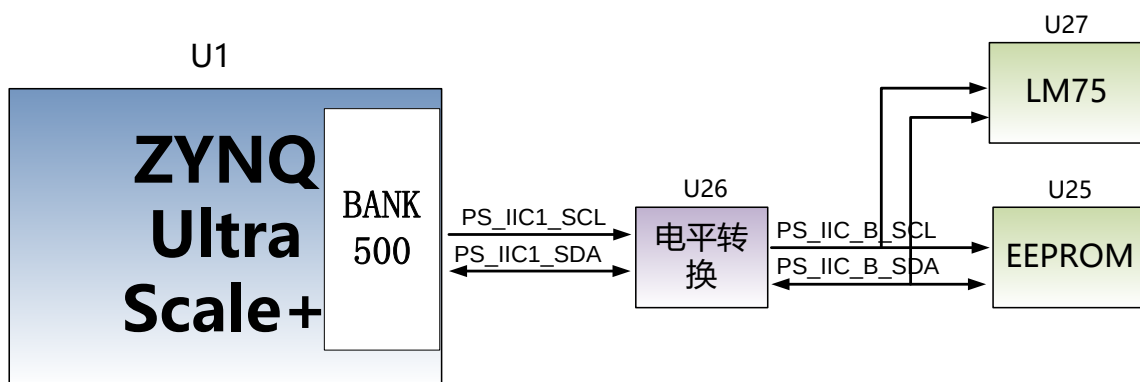


图 3-14-1 EEPROM 和传感器的原理图

EEPROM 通信引脚分配如下：

信号名称	引脚名	引脚号	备注
PS_IIC1_SCL	PS_MIO24	AB19	I2C时钟信号
PS_IIC1_SDA	PS_MIO25	AB21	I2C数据信号

2.2.15 LED 灯

AXU3EG 扩展板上有 3 个发光二极管 LED。包含 1 个电源指示灯，1 个 PS 控制指示灯，1

个 PL 控制指示灯。用户可以通过程序来控制亮和灭，当连接用户 LED 灯的 IO 电压为低时，用户 LED 灯熄灭，当连接 IO 电压为高时，用户 LED 会被点亮。用户 LED 灯硬件连接的示意图如图 3-15-1 所示：

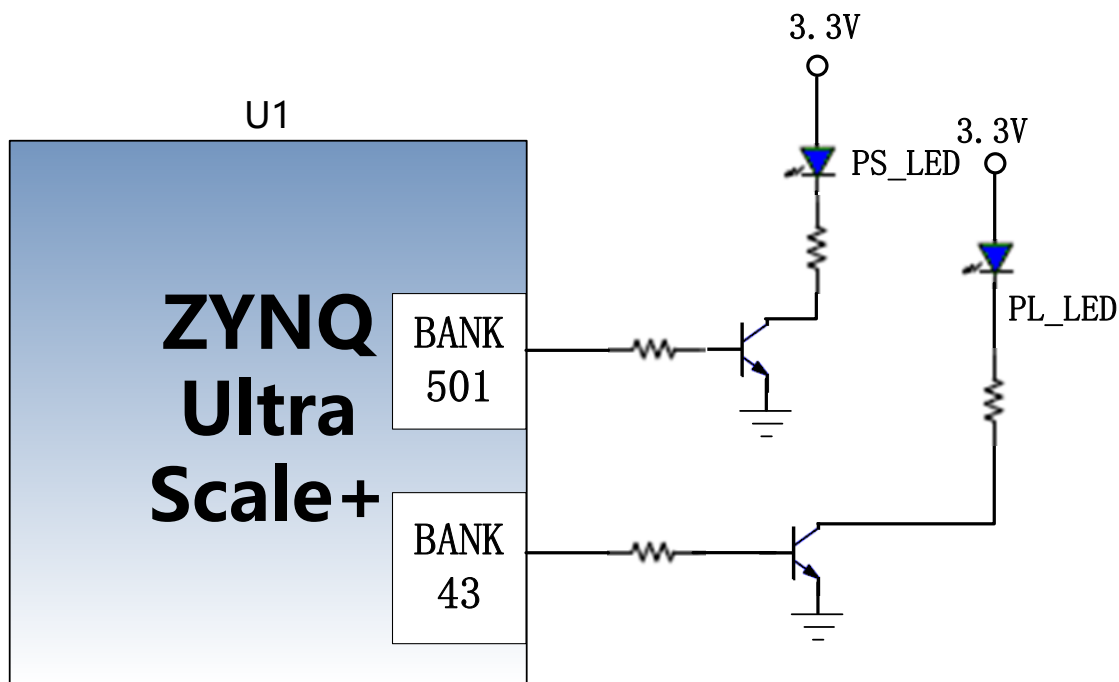


图 3-15-1 用户 LED 灯硬件连接示意图

用户 LED 灯的引脚分配

信号名称	引脚名	管脚号	备注
PS_LED1	PS_MIO40	K18	用户 PS LED 灯
PL_LED1	B43_L5_P	AE12	用户 PL LED 灯

2.2.16 按键

AXU3EG 扩展板上有 1 个复位按键 RESET 和 2 个用户按键。复位信号连接到核心板的复位芯片输入，用户可以使用这个复位按键来复位 ZYNQ 系统。用户按键 1 个连接到 PS 的 MIO 上，1 个是连接到 PL 的 IO 上。复位按键和用户按键都是低电平有效，用户按键的连接示意图如图 3-16-1 所示：

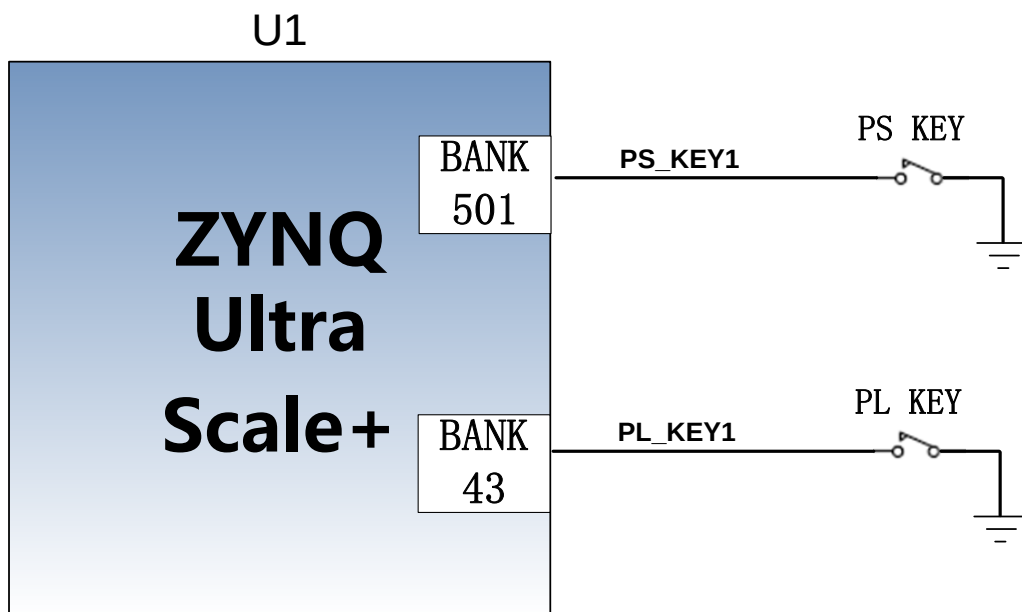


图 3-16-1 复位按键连接示意图

按键的 ZYNQ 管脚分配

信号名称	引脚名	引脚号	备注
PS_KEY1	PS_MIO26	L15	PS按键1输入
PL_KEY1	B43_L5_N	AF12	PL按键1输入

2.2.17 拨码开关配置

开发板上有一个 4 位的拨码开关 SW1 用来配置 ZYNQ 系统的启动模式。AXU3EG 系统开发平台支持 4 种启动模式。这 4 种启动模式分别是 JTAG 调试模式, QSPI FLASH, EMMC 和 SD2.0 卡启动模式。ZU3EG 芯片上电后会检测 (PS_MODE0~3) 的电平来决定那种启动模式。用户可以通过扩展板上的拨码开关 SW1 来选择合适的启动模式。SW1 启动模式配置如下表 3-17-1 所示。

SW1	拨码位置 (1, 2, 3, 4)	MODE[3:0]	启动模式
	ON, ON, ON, ON	0000	PS JTAG
	ON, ON, OFF, ON	0010	QSPI FLASH
	ON, OFF, ON, OFF	0101	SD卡

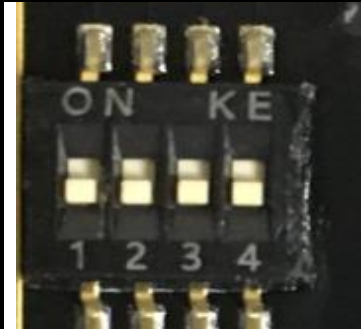
	ON, OFF, OFF, ON	0110	EMMC
---	------------------	------	------

表3-17-1 SW1启动模式配置

2.2.18 电源

AXU3EG 开发板的电源输入电压为 DC12V。底板上通过 1 路 DC/DC 电源芯片 TPS54620 和 2 路 DC/DC 电源芯片 MP1482 转换成+5V, +3.3V, +1.8V。另外底板通过 LDO 产生+1.2V 给核心板 BANK65 供电, BANK66 的供电为+1.8V。板上的电源设计示意图如下图 3-18-1 所示:

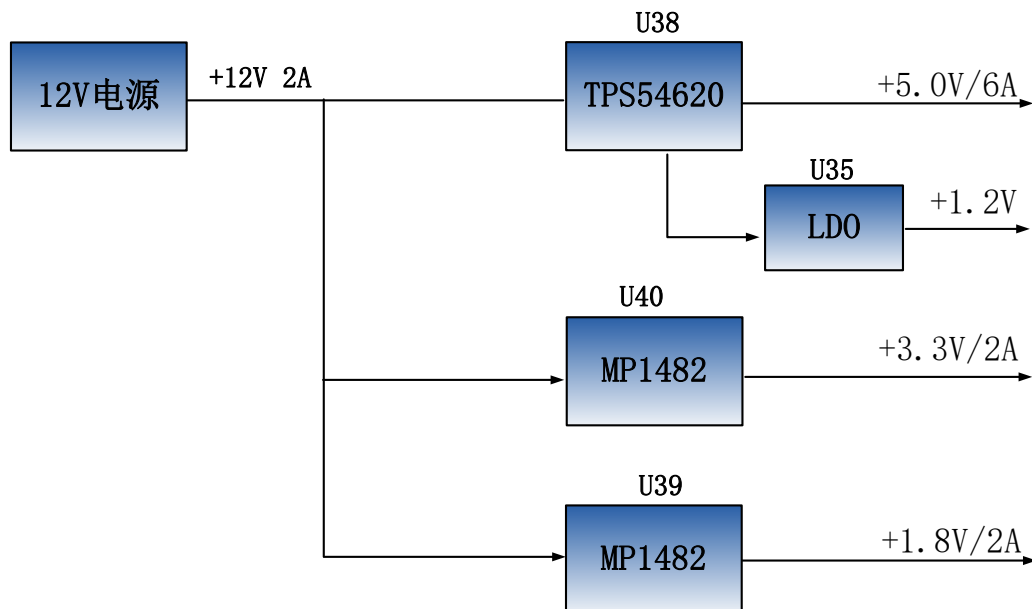


图 3-18-1 原理图中电源接口部分

各个电源分配的功能如下表所示:

电源	功能
+5.0V	USB 供电电源
+1.8V	以太网, USB2.0, 核心板 BANK66
+3.3V	以太网, USB2.0, SD, DP, CAN, RS485
+1.2V	核心板 BANK65

2.2.19 风扇

因为 ZU3EG 正常工作时会产生大量的热量，我们在板上为芯片增加了一个散热片和风扇，防止芯片过热。风扇的控制由 ZYNQ 芯片来控制，控制管脚连接到 BANK43 的 IO 上 (AA11)，如果 IO 电平输出为低，MOSFET 管导通，风扇工作，如果 IO 电平输出为高，风扇停止。板上的风扇设计图如下图 3-19-1 所示：

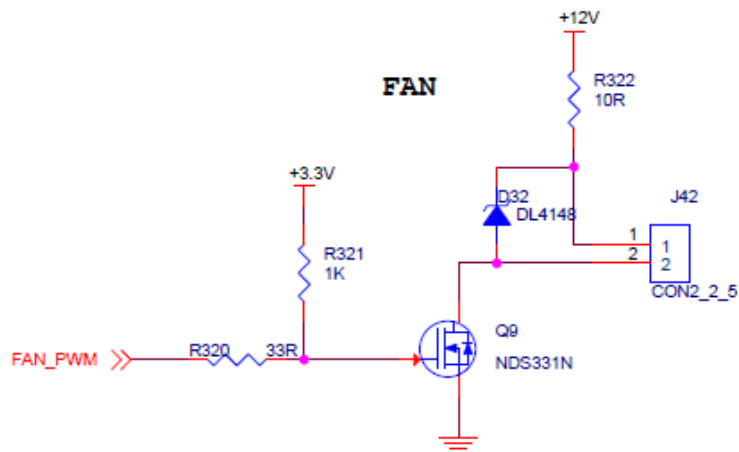


图 3-19-1 开发板原理图中风扇设计

风扇出厂前已经用螺丝固定在开发板上，风扇的电源连接到了 J42 的插座上，红色的为正极，黑色的为负极。

2.2.20 结构尺寸图

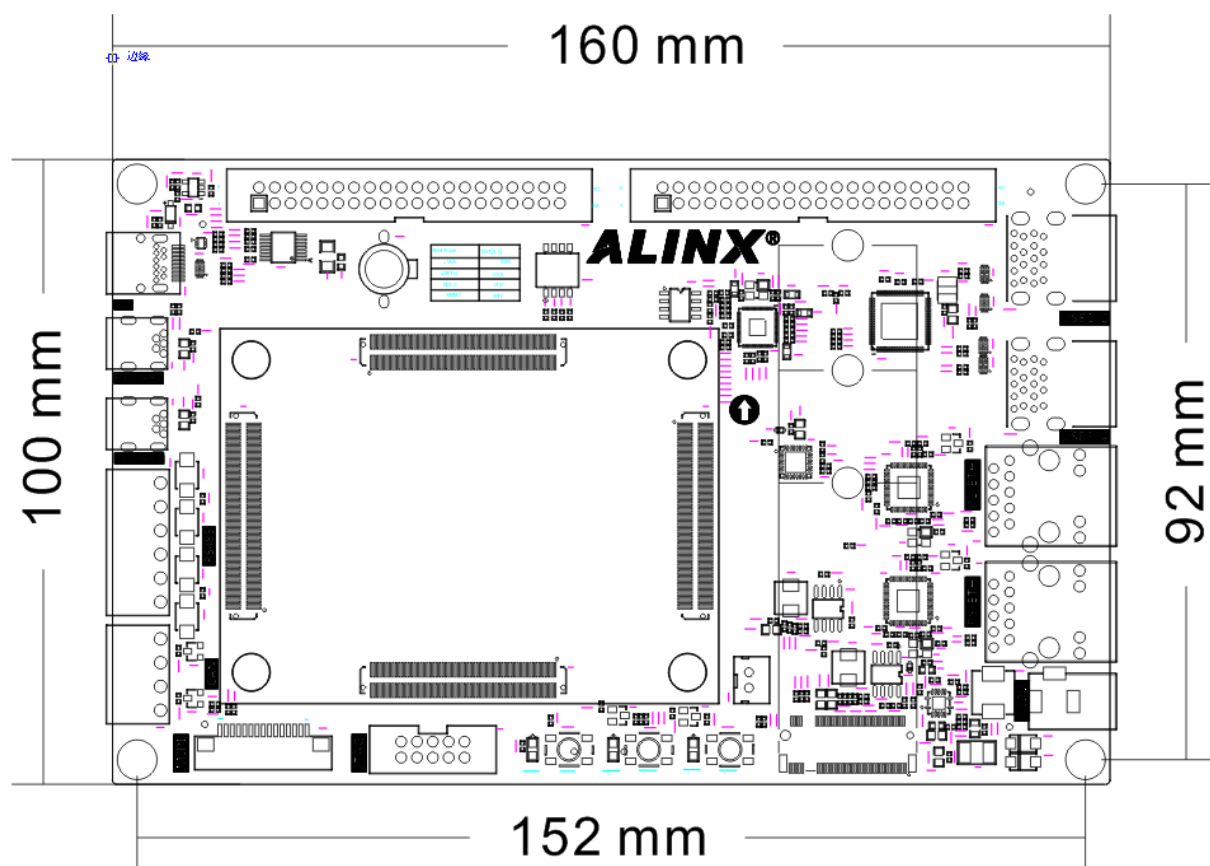


图 3-20-1 正面图 (Top View)

第三章 Verilog 基础模块介绍

3.1 简介

本文主要介绍 verilog 基础模块，夯实基础，对深入学习 FPGA 会有很大帮助。

3.2 数据类型

3.2.1 常量

整数：整数可以用二进制 b 或 B，八进制 o 或 O，十进制 d 或 D，十六进制 h 或 H 表示，例如，8'b00001111 表示 8 位位宽的二进制整数，4'ha 表示 4 位位宽的十六进制整数。

x 和 z：x 代表不定值，z 代表高阻值，例如，5'b00x11，第三位不定值，3'b00z 表示最低位为高阻值。

下划线：在位数过长时可以用来分割位数，提高程序可读性，如 8'b0000_1111

参数 parameter：parameter 可以用标识符定义常量，运用时只使用标识符即可，提高可读性及维护性，如定义 parameter width = 8；定义寄存器 reg [width-1:0] a；即定义了 8 位宽度的寄存器。

参数的传递：在一个模块中如果有定义参数，在其他模块调用此模块时可以传递参数，并可以修改参数，如下所示，在 module 后用# () 表示。

例如定义模块如下

调用模块

```
module rom
#(
    parameter depth =15,
    parameter width = 8
)
(
    input [depth-1:0] addr ,
    input [width-1:0] data ,
    output result
) ;

endmodule
```

```
module top() ;

wire [31:0] addr ;
wire [15:0] data ;
wire result ;

rom
#(
    .depth(32),
    .width(16)
)
r1
(
    .addr(addr) ,
    .data(data) ,
    .result(result)
) ;

endmodule
```

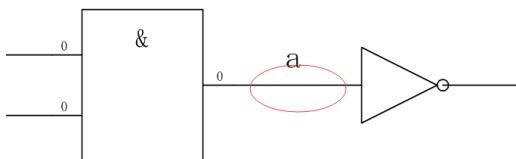
Parameter 可以用于模块间的参数传递，而 localparam 仅用于本模块内使用，不能用于参数传递。Localparam 多用于状态机状态的定义。

3.3 变量

变量是指程序运行时可以改变其值的量，下面主要介绍几个常用了变量类型。

1.1.1 Wire 型

Wire 类型变量，也叫网络类型变量，用于结构实体之间的物理连接，如门与门之间，不能储存值，用连续赋值语句 assign 赋值，定义为 wire [n-1:0] a；其中 n 代表位宽，如定义 wire a；assign a = b；是将 b 的结点连接到连线 a 上。如下图所示，两个实体之间的连线即是 wire 型变量。



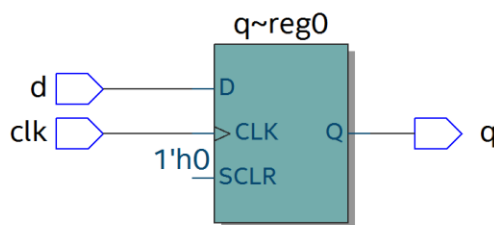
1.1.2 Reg 型

Reg 类型变量，也称为寄存器变量，可用来储存值，必须在 always 语句里使用。其定义为

reg [n-1:0] a；表示 n 位位宽的寄存器，如 reg [7:0] a；表示定义 8 位位宽的寄存器 a。如下所示定义了寄存器 q，生成的电路为时序逻辑，右图为其结构，为 D 触发器。

```
module top(d, clk, q) ;
input d ;
input clk ;
output reg q ;

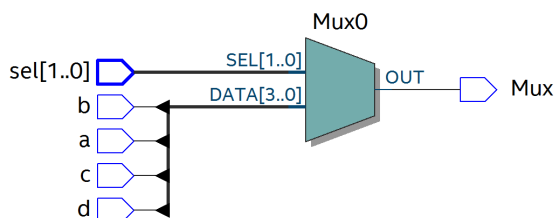
always @(posedge clk)
begin
q <= d ;
end
endmodule
```



也可以生成组合逻辑，如数据选择器，敏感信号没有时钟，定义了 reg Mux，最终生成电路为组合逻辑。

```
module top(a, b, c, d, sel, Mux) ;
input a ;
input b ;
input c ;
input d ;
input [1:0] sel ;
output reg Mux ;

always @(sel or a or b or c or d)
begin
case(sel)
2'b00 : Mux = a ;
2'b01 : Mux = b ;
endcase
end
```



```

2'b10 : Mux = c ;
2'b11 : Mux = d ;
endcase
end

endmodule

```

1.1.3 Memory 型

可以用 memory 类型来定义 RAM,ROM 等存储器, 其结构为 reg [n-1:0] 存储器名[m-1:0], 意义为 m 个 n 位宽度的寄存器。例如, reg [7:0] ram [255:0]表示定义了 256 个 8 位寄存器, 256 也即是存储器的深度, 8 为数据宽度。

3.4 运算符

运算符可分为以下几类:

- (1) 算术运算符 (+, -, *, /, %)
- (2) 赋值运算符 (=, <=)
- (3) 关系运算符 (>, <, >=, <=, ==, !=)
- (4) 逻辑运算符 (&&, ||, !)
- (5) 条件运算符 (? :)
- (6) 位运算符 (~, |, ^, &, ^~)
- (7) 移位运算符 (<<, >>)
- (8) 拼接运算符 ({})

3.4.1 算术运算符

"+"(加法运算符), "-"(减法运算符), "*" (乘法运算符), "/" (除法运算符, 如 7/3=2), "%" (取模运算符, 也即求余数, 如 7%3=1, 余数为 1)

3.4.2 赋值运算符

"="阻塞赋值, "<="非阻塞赋值。阻塞赋值为执行完一条赋值语句, 再执行下一条, 可理解为顺序执行, 而且赋值是立即执行; 非阻塞赋值可理解为并行执行, 不考虑顺序, 在 always 块语句执行完成后, 才进行赋值。如下面的阻塞赋值:

代码如下:

```
module top(din,a,b,c,clk);
```

激励文件如下

```
`timescale 1 ns/1 ns
```

```

input din;
input clk;
output reg a,b,c;

always @(posedge clk)
begin
    a = din;
    b = a;
    c = b;
end

endmodule

module top_tb() ;
reg din ;
reg clk ;
wire a,b,c ;

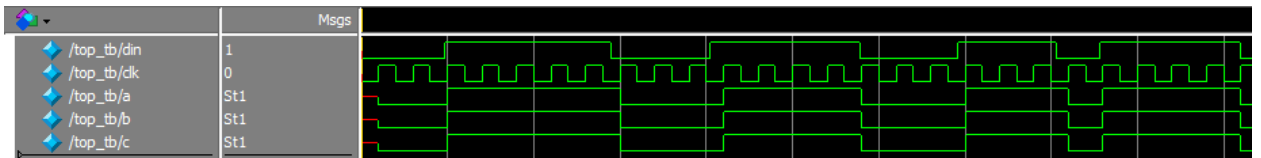
initial
begin
    din = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        din = ~din ;
    end
end

always #10 clk = ~clk ;

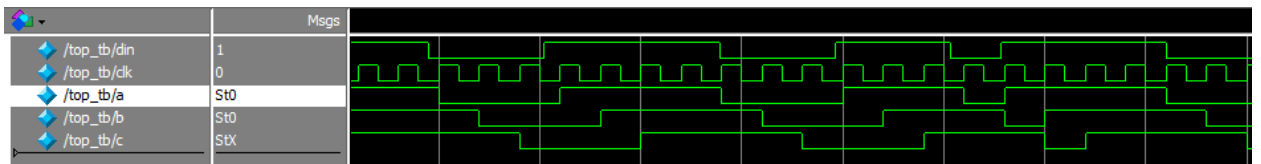
top
t0(.din(din),.a(a),.b(b),.c(c),.clk(clk)) ;
endmodule

```

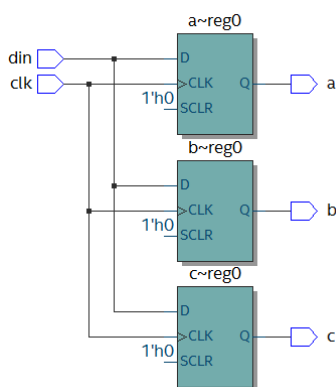
可以从仿真结果看到，在 clk 的上升沿，a 的值等于 din，并立即赋给 b，b 的值赋给 c。



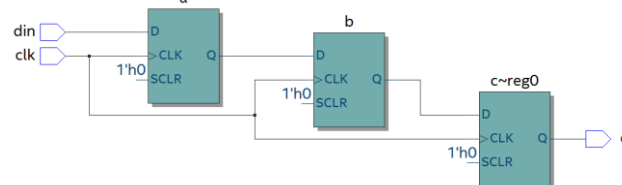
如果改为非阻塞赋值，仿真结果如下，在 clk 上升沿，a 的值没有立即赋值给 b，b 为 a 原来的值，同样，c 为 b 原来的值



可以从两者的 RTL 图看出明显不同：



阻塞赋值 RTL 图



非阻塞赋值 RTL 图

一般情况下，在时序逻辑电路中使用非阻塞赋值，可避免仿真时出现竞争冒险现象；在组合逻辑中使用阻塞赋值，执行赋值语句后立即改变；在 assign 语句中必须用阻塞赋值。

3.4.3 关系运算符

用于表示两个操作数之间的关系，如 $a > b$, $a < b$, 多用于判断条件，例如：

```
if (a >= b) q <= 1'b1 ;  
else q <= 1'b0 ;
```

表示如果 a 的值大于等于 b 的值，则 q 的值为 1，否则 q 的值为 0

3.4.4 逻辑运算符

"&&" (两个操作数逻辑与), "||" (两个操作数逻辑或), "!" (单个操作数逻辑非) 例如：

if ($a > b$ && $c < d$) 表示条件为 $a > b$ 并且 $c < d$; if (! a) 表示条件为 a 的值不为 1，也就是 0。

3.4.5 条件运算符

"?:" 为条件判断，类似于 if else，例如 assign $a = (i > 8) ? 1'b1 : 1'b0$; 判断 i 的值是否大于 8，如果大于 8 则 a 的值为 1，否则为 0。

3.4.6 位运算符

"~" 按位取反，"|" 按位或，"^" 按位异或，"&" 按位与，"^^" 按位同或，除了 "~" 只需要一个操作数外，其他几个都需要两个操作数，如 $a \& b$, $a | b$ 。具体应用在后面的组合逻辑一节中有讲解。

3.4.7 移位运算符

"<<" 左移位运算符，">>" 右移位运算符，如 $a < < 1$ 表示，向左移 1 位， $a > > 2$ ，向右移两位。

3.4.8 拼接运算符

"{" 拼接运算符，将多个信号按位拼接，如 $\{a[3:0], b[1:0]\}$ ，将 a 的低 4 位， b 的低 2 位拼接成 6 位数据。另外， $\{n\{a[3:0]\}\}$ 表示将 n 个 $a[3:0]$ 拼接， $\{n\{1'b0\}\}$ 表示 n 位的 0 拼接。如 $\{8\{1'b0\}\}$ 表示为 $8'b0000_0000$ 。

3.4.9 优先级别

各种运算符的优先级别如下：

! ~	高优先级别
/ * %	
+ -	
<< >>	
< <= > >=	
== !=	
&	
^ ^~	
&&	
?:	
	低优先级别

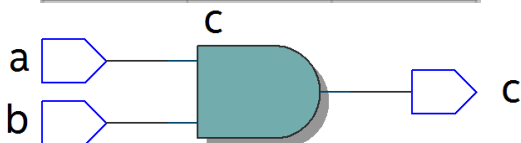
3.5 组合逻辑

本节主要介绍组合逻辑，组合逻辑电路的特点是任意时刻的输出仅仅取决于输入信号，输入信号变化，输出立即变化，不依赖于时钟。

3.5.1 与门

在 verilog 中以 “&” 表示按位与，如 $c=a\&b$ ，真值表如下，在 a 和 b 都等于 1 时结果才为 1，RTL 表示如右图

输入		输出
a	b	c
0	0	0
0	1	0
1	0	0
1	1	1



代码实现如下：

```
module top(a, b, c) ;
input a ;
input b ;
output c ;

assign c = a & b ;
endmodule
```

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire c ;

initial
begin
a = 0 ;
b = 0 ;
forever
begin
```

```

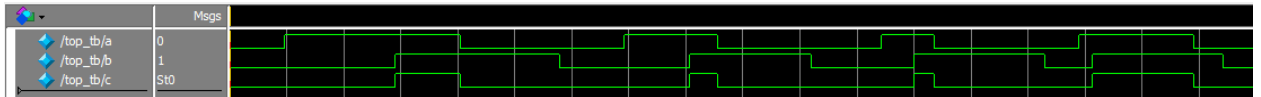
        #({$random}%100)
        a = ~a ;
        #({$random}%100)
        b = ~b ;
    end
end

top t0(.a(a), .b(b), .c(c)) ;

endmodule

```

仿真结果如下：

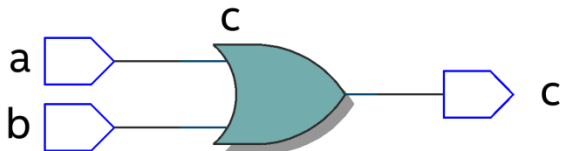


如果 a 和 b 的位宽大于 1，例如定义 input [3:0] a, input [3:0]b，那么 a&b 则指 a 与 b 的对应位相与。如 a[0]&b[0],a[1]&b[1]。

3.5.2 或门

在 verilog 中以 “|” 表示按位或，如 c = a|b，真值表如下，在 a 和 b 都为 0 时结果才为 0。

输入		输出
a	b	c
0	0	0
0	1	1
1	0	1
1	1	1



代码实现如下：

```

module top(a, b, c) ;
    input a ;
    input b ;
    output c ;

    assign c = a | b ;
endmodule

```

激励文件如下

```

`timescale 1 ns/1 ns
module top_tb() ;
    reg a ;
    reg b ;
    wire c ;

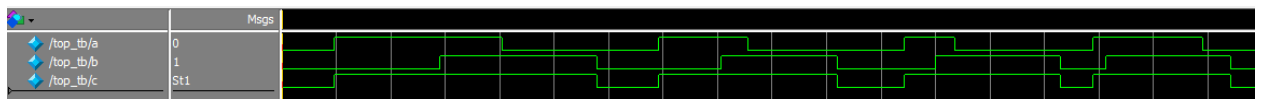
    initial
    begin
        a = 0 ;
        b = 0 ;
        forever
        begin
            #({$random}%100)
            a = ~a ;
            #({$random}%100)
            b = ~b ;
        end
    end
end

```

```
top t0(.a(a), .b(b), .c(c)) ;

endmodule
```

仿真结果如下:

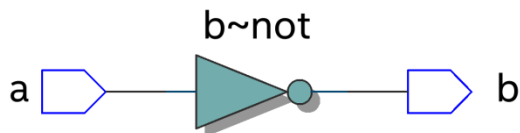


同理，位宽大于 1，则是按位或。

3.5.3 非门

在 verilog 中以 “~” 表示按位取反，如 $b = \sim a$ ，真值表如下，b 等于 a 的相反数。

输入	输出
a	b
0	1
1	0



代码实现如下:

```
module top(a, b) ;
input a ;
output b ;

assign b = ~a ;
endmodule
```

激励文件如下:

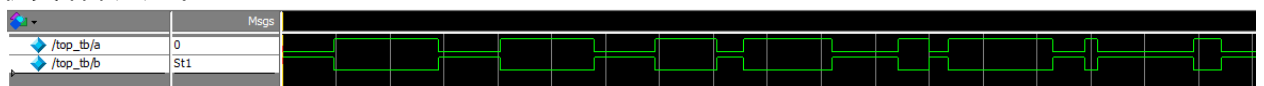
```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
wire b ;

initial
begin
a = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
end
end

top t0(.a(a), .b(b)) ;

endmodule
```

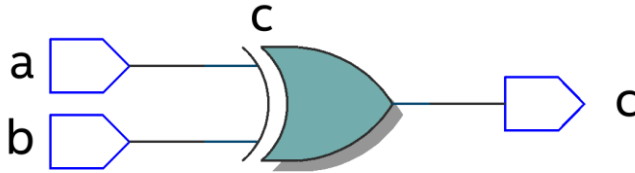
仿真结果如下:



3.5.4 异或

在 verilog 中以 “^” 表示异或，如 $c = a \wedge b$ ，真值表如下，当 a 和 b 相同时，输出为 0。

输入		输出
a	b	c
0	0	0
0	1	1
1	0	1
1	1	0



代码实现如下:

```
module top(a, b, c) ;
input a ;
input b ;
output c ;

assign c = a ^ b ;
endmodule
```

激励文件如下:

```
`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire c ;

initial
begin
a = 0 ;
b = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
end
end

top t0(.a(a), .b(b), .c(c)) ;

endmodule
```

仿真结果如下:



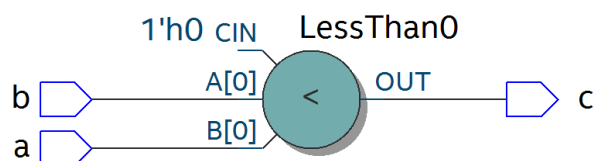
3.5.5 比较器

在 verilog 中以大于 ">", 等于"==", 小于"<", 大于等于">=", 小于等于"<=", 不等于"!=" 表示, 以大于举例, 如 $c = a > b$; 表示如果 a 大于 b, 那么 c 的值就为 1, 否则为 0。真值表如下:

输入		输出
a	b	c
0	0	0
0	1	0
1	0	1
1	1	0

代码实现如下:

```
module top(a, b, c) ;
```



激励文件如下:

```
`timescale 1 ns/1 ns
```

```

input a ;
input b ;
output c ;

assign c = a > b ;
endmodule

```

```

module top_tb() ;
reg a ;
reg b ;
wire c ;

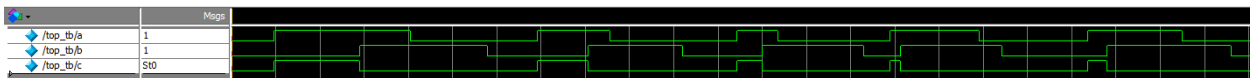
initial
begin
a = 0 ;
b = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
end
end

top t0(.a(a), .b(b), .c(c)) ;

endmodule

```

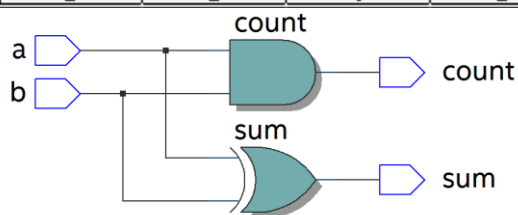
仿真结果如下:



3.5.6 半加器

半加器和全加器是算术运算电路中的基本单元，由于半加器不考虑从低位来的进位，所以称之为半加器，sum 表示相加结果，count 表示进位，真值表可表示如下：

输入		输出	
a	b	sum	count
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



可根据真值表写出代码如下：

```

module top(a, b, sum, count) ;
input a ;
input b ;
output sum ;
output count ;

assign sum = a ^ b ;
assign count = a & b ;

endmodule

```

激励文件如下：

```

`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
wire sum ;
wire count ;

initial
begin
a = 0 ;
b = 0 ;
forever
begin

```

```

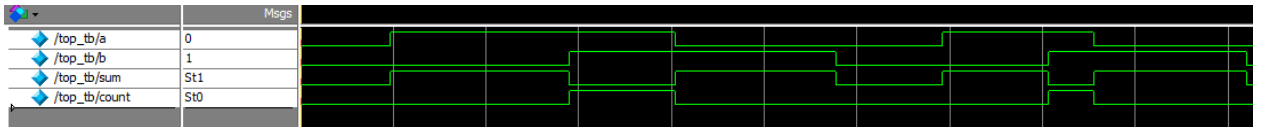
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
end
end

top t0(.a(a), .b(b),
.sum(sum), .count(count)) ;

endmodule

```

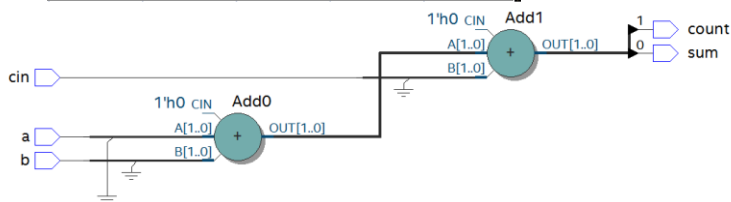
仿真结果如下:



3.5.7 全加器

而全加器需要加上低位来的进位信号 cin，真值表如下：

输入			输出	
cin	a	b	sum	count
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



代码如下：

```

module top(cin, a, b, sum, count) ;
input cin ;
input a ;
input b ;
output sum ;
output count ;

assign {count,sum} = a + b + cin ;

endmodule

```

激励文件如下：

```

`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
reg cin ;
wire sum ;
wire count ;

initial
begin
a = 0 ;
b = 0 ;
cin = 0 ;
forever
begin
#({$random}%100)
a = ~a ;
#({$random}%100)
b = ~b ;
#({$random}%100)
cin = ~cin ;

end

end

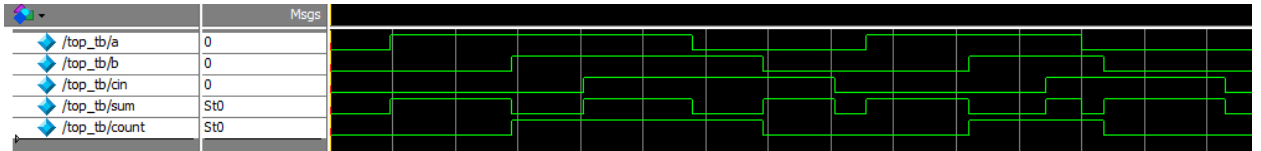
```

```
end
```

```
top t0(.cin(cin),.a(a),.b(b),  
.sum(sum),.count(count));
```

```
endmodule
```

仿真结果如下：

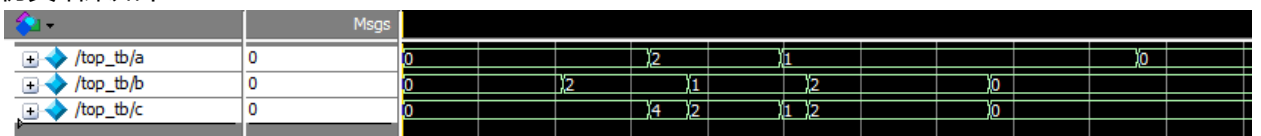


3.5.8 乘法器

乘法的表示也很简单，利用“*”即可，如 $a*b$ ，举例代码如下：

```
module top(a, b, c) ;  
    input [1:0] a ;  
    input [1:0] b ;  
    output [3:0] c ;  
  
    assign c = a * b ;  
endmodule  
  
`timescale 1 ns/1 ns  
module top_tb() ;  
    reg [1:0] a ;  
    reg [1:0] b ;  
    wire [3:0] c ;  
  
    initial  
    begin  
        a = 0 ;  
        b = 0 ;  
        forever  
        begin  
            #({$random}%100)  
            a = ~a ;  
            #({$random}%100)  
            b = ~b ;  
        end  
    end  
  
    top t0(.a(a),.b(b),.c(c)) ;  
  
endmodule
```

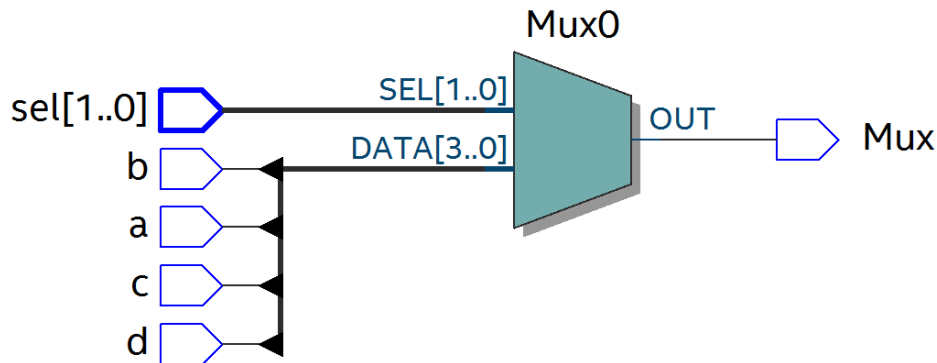
仿真结果如下：



3.5.9 数据选择器

在 verilog 中经常会用到数据选择器，通过选择信号，选择不同的输入信号输出到输出端，如下图真值表，四选一数据选择器，sel[1:0]为选择信号，a,b,c,d 为输入信号，Mux 为输出信号。

选择信号		输入				输出
sel[0]	sel[1]	a	b	c	d	Mux
0	0	x	x	x	x	a
0	1	x	x	x	x	b
1	0	x	x	x	x	c
1	1	x	x	x	x	d



代码如下:

```

module top(a, b, c, d, sel, Mux) ;
input  a ;
input  b ;
input  c ;
input  d ;

input [1:0] sel ;

output reg Mux ;

always @(sel or a or b or c or d)
begin
    case(sel)
        2'b00 : Mux = a ;
        2'b01 : Mux = b ;
        2'b10 : Mux = c ;
        2'b11 : Mux = d ;
    endcase
end

endmodule

```

激励文件如下:

```

`timescale 1 ns/1 ns
module top_tb() ;
reg a ;
reg b ;
reg c ;
reg d ;
reg [1:0] sel ;
wire Mux ;

initial
begin
    a = 0 ;
    b = 0 ;
    c = 0 ;
    d = 0 ;
    forever
    begin
        #({$random}%100)
        a = {$random}%3 ;
        #({$random}%100)
        b = {$random}%3 ;
        #({$random}%100)
        c = {$random}%3 ;
        #({$random}%100)
        d = {$random}%3 ;
    end
end

initial
begin
    sel = 2'b00 ;
    #2000 sel = 2'b01 ;
    #2000 sel = 2'b10 ;
    #2000 sel = 2'b11 ;
end

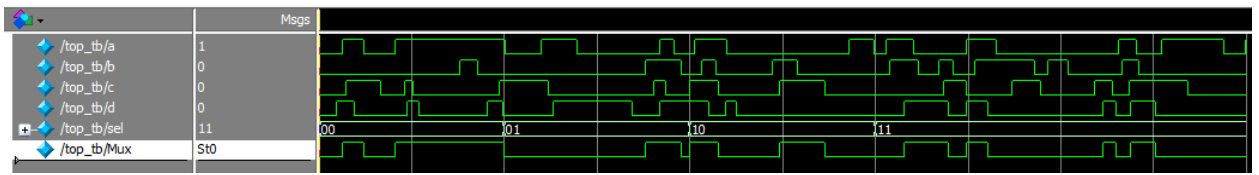
top
t0(.a(a), .b(b), .c(c), .d(d), .sel(sel),
1),

```

```
.Mux (Mux) ) ;
```

```
endmodule
```

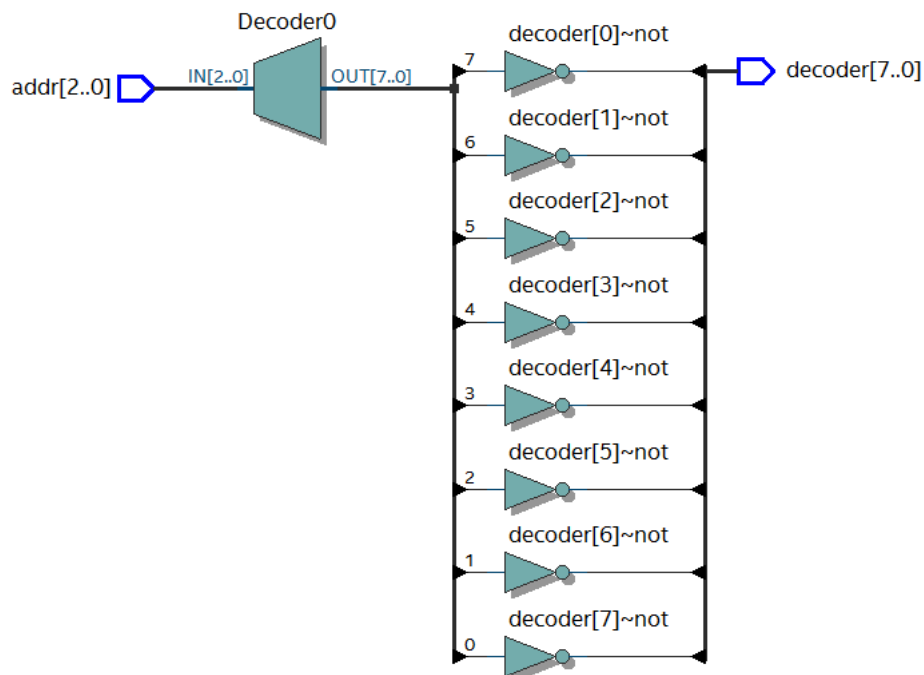
仿真结果如下



3.5.10 3-8 译码器

3-8 译码器是一个很常用的器件，其真值表如下所示，根据 A2,A1,A0 的值，得出不同的结果。

输入			输出							
A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0



代码如下：

```
module top(addr, decoder) ;
input  [2:0] addr ;
output reg [7:0] decoder ;

always @ (addr)
begin
case (addr)
3'b000 : decoder =
```

激励文件如下：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg  [2:0] addr ;
wire  [7:0] decoder ;

initial
begin
addr = 3'b000 ;
```

```

8'b1111_1110 ;
    3'b001 : decoder =
8'b1111_1101 ;
    3'b010 : decoder =
8'b1111_1011 ;
    3'b011 : decoder =
8'b1111_0111 ;
    3'b100 : decoder =
8'b1110_1111 ;
    3'b101 : decoder =
8'b1101_1111 ;
    3'b110 : decoder =
8'b1011_1111 ;
    3'b111 : decoder =
8'b0111_1111 ;
    endcase
end

endmodule

#2000 addr = 3'b001 ;
#2000 addr = 3'b010 ;
#2000 addr = 3'b011 ;
#2000 addr = 3'b100 ;
#2000 addr = 3'b101 ;
#2000 addr = 3'b110 ;
#2000 addr = 3'b111 ;
end

top
t0(.addr(addr),.decoder(decoder)) ;

endmodule

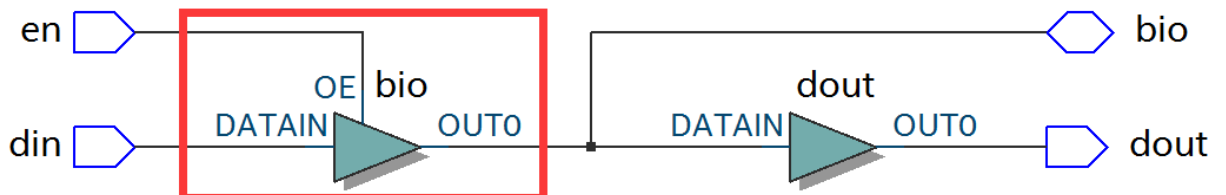
```

仿真结果如下:

	Msgs								
/top_tb/addr	111	000	001	010	011	100	101	110	111
/top_tb/decoder	01111111	11111110	11111101	11111011	11110111	11101111	11011111	10111111	01111111

3.5.11 三态门

在 FPGA 使用中, 经常会用到双向 IO, 需要用到三态门, 如 $bio = en ? din : 1'bz$; 其中 en 为使能信号, 用于打开关闭三态门, 下面的 RTL 图即是实现了双向 IO, 可参考代码。激励文件实现两个双向 IO 的对接。



```

module top(en, din, dout, bio) ;
input din ;
input en ;
output dout ;
inout bio ;

assign bio = en? din : 1'bz ;
assign dout = bio ;

endmodule

```

```

`timescale 1 ns/1 ns
module top_tb() ;
reg en0 ;
reg din0 ;
wire dout0 ;
reg en1 ;
reg din1 ;
wire dout1 ;
wire bio ;

initial
begin
    din0 = 0 ;
    din1 = 0 ;
    forever
    begin
        #({$random}%100)
        din0 = ~din0 ;
        #({$random}%100)
        din1 = ~din1 ;
    end
end

```

```

end
end

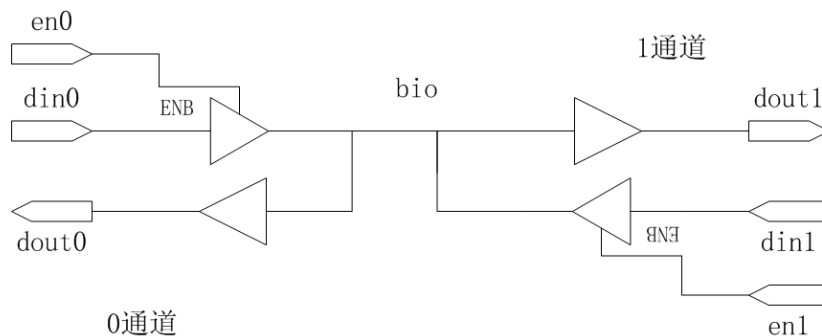
initial
begin
    en0 = 0 ;
    en1 = 1 ;
    #100000
    en0 = 1 ;
    en1 = 0 ;
end

top
t0(.en(en0),.din(din0),.dout(dout0),
.bi
o(bio)) ;
top
t1(.en(en1),.din(din1),.dout(dout1),
.bi
o(bio)) ;

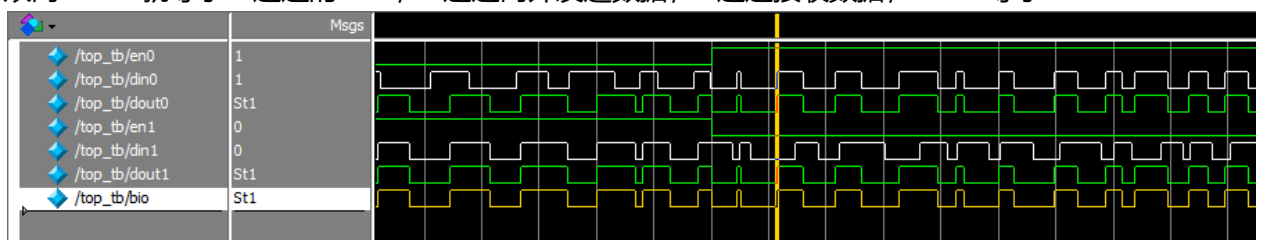
endmodule

```

激励文件结构如下图



仿真结果如下，en0 为 0，en1 为 1 时，1 通道打开，双向 IO bio 就等于 1 通道的 din1，1 通道向外发送数据，0 通道接收数据，dout0 等于 bio；当 en0 为 1，en1 为 0 时，0 通道打开，双向 IO bio 就等于 0 通道的 din0，0 通道向外发送数据，1 通道接收数据，dout1 等于 bio



3.6 时序逻辑

组合逻辑电路在逻辑功能上特点是任意时刻的输出仅仅取决于当前时刻的输入，与电路原来的状态无关。而时序逻辑在逻辑功能上的特点是任意时刻的输出不仅仅取决于当前的输入信号，而且还取决于电路原来的状态。下面以典型的时序逻辑分析。

3.6.1 D 触发器

D 触发器在时钟的上升沿或下降沿存储数据，输出与时钟跳变之前输入信号的状态相同。

代码如下

```
module top(d, clk, q) ;
input d ;
input clk ;
output reg q ;
always @(posedge clk)
begin
q <= d ;
end
endmodule
```

激励文件如下

```
`timescale 1 ns/1 ns
module top_tb() ;
reg d ;
reg clk ;
wire q ;

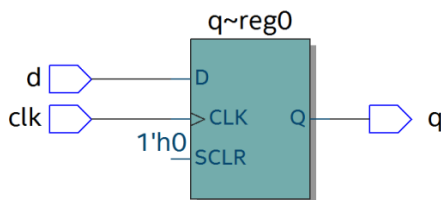
initial
begin
d = 0 ;
clk = 0 ;
forever
begin
#({$random}%100)
d = ~d ;
end
end

always #10 clk = ~clk ;

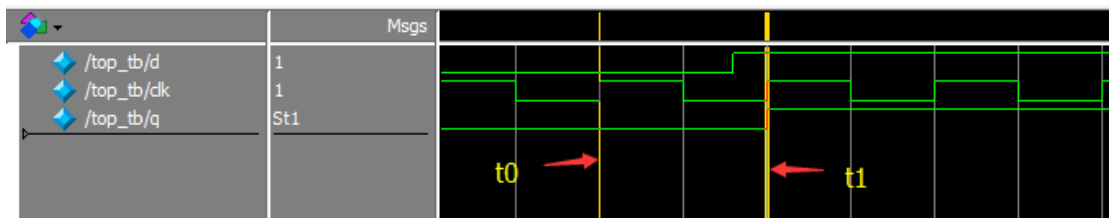
top t0(.d(d),.clk(clk),.q(q)) ;

endmodule
```

RTL 图表示如下

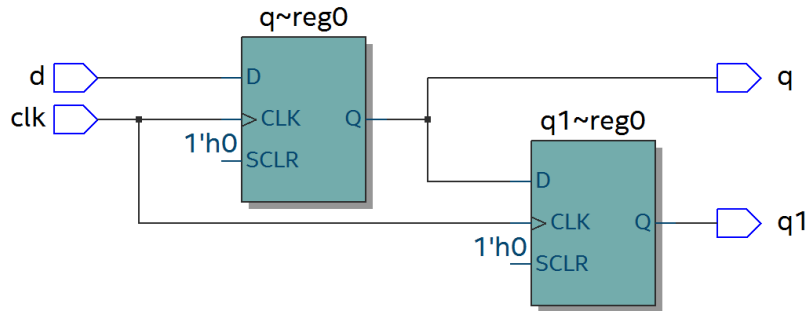


仿真结果如下，可以看到在 t_0 时刻时， d 的值为 0，则 q 的值也为 0；在 t_1 时刻 d 发生了变化，值为 1，那么 q 相应也发生了变化，值变为 1。可以看到在 t_0 - t_1 之间的一个时钟周期内，无论输入信号 d 的值如何变化， q 的值是保持不变的，也就是有存储的功能，保存的值为在时钟的跳变沿时 d 的值。



3.6.2 两级 D 触发器

软件是按照两级 D 触发器的模型进行时序分析的，具体可以分析在同一时刻两个 D 触发器输出的数据有何不同，其 RTL 图如下：



代码如下:

```
module top(d, clk, q, q1) ;
input d ;
input clk ;
output reg q ;
output reg q1 ;

always @(posedge clk)
begin
q <= d ;
end

always @(posedge clk)
begin
q1 <= q ;
end

endmodule
```

激励文件如下:

```
`timescale 1 ns/1 ns
module top_tb() ;
reg d ;
reg clk ;
wire q ;
wire q1 ;

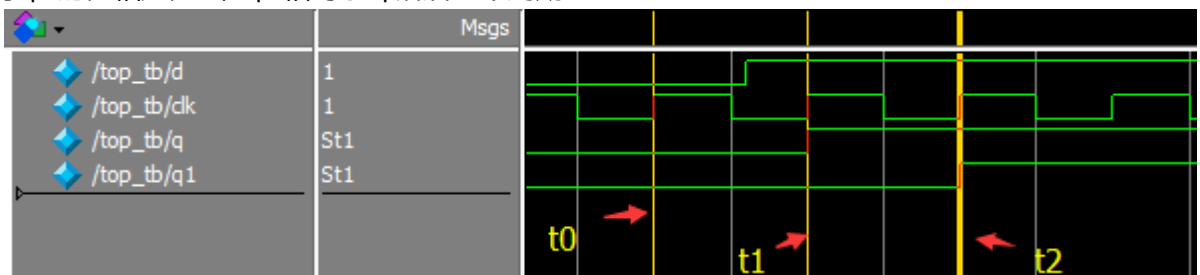
initial
begin
d = 0 ;
clk = 0 ;
forever
begin
#({$random}%100)
d = ~d ;
end
end

always #10 clk = ~clk ;

top
t0(.d(d),.clk(clk),.q(q),.q1(q1)) ;

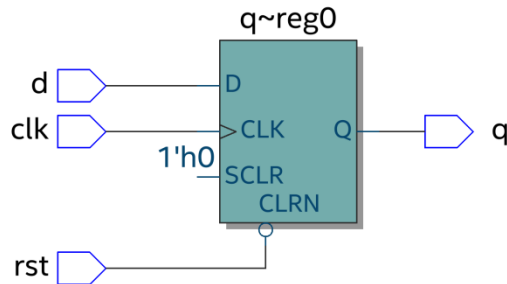
endmodule
```

仿真结果如下, 可以看到 t_0 时刻, d 为 0, q 输出为 0, t_1 时刻, q 随着 d 的数据变化而变化, 而此时钟跳变之前 q 的值仍为 0, 那么 q_1 的值仍为 0, t_2 时刻, 时钟跳变前 q 的值为 1, 则 q_1 的值相应为 1, q_1 相对于 q 落后一个周期。



3.6.3 带异步复位的 D 触发器

异步复位是指独立于时钟, 一旦异步复位信号有效, 就触发复位操作。这个功能在写代码时会经常用到, 用于给信号复位, 初始化。其 RTL 图如下:



代码如下，注意要把异步复位信号放在敏感列表里，如果是低电平复位，即为 negedge，如果是高电平复位，则是 posedge

```

module top(d, rst, clk, q) ;
input d ;
input rst ;
input clk ;
output reg q ;

always @(posedge clk or negedge
rst)
begin
    if (rst == 1'b0)
        q <= 0 ;
    else
        q <= d ;
    end
endmodule

`timescale 1 ns/1 ns
module top_tb() ;
reg d ;
reg rst ;
reg clk ;
wire q ;

initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

initial
begin
    rst = 0 ;
    #200 rst = 1 ;
end

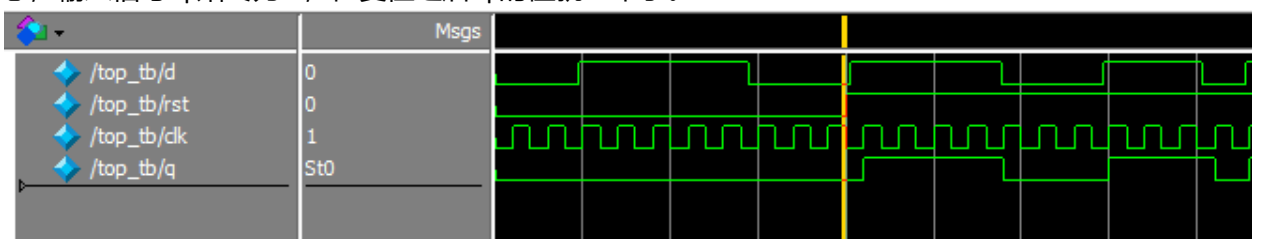
always #10 clk = ~clk ;

top
t0(.d(d), .rst(rst), .clk(clk), .q(q))
;

endmodule

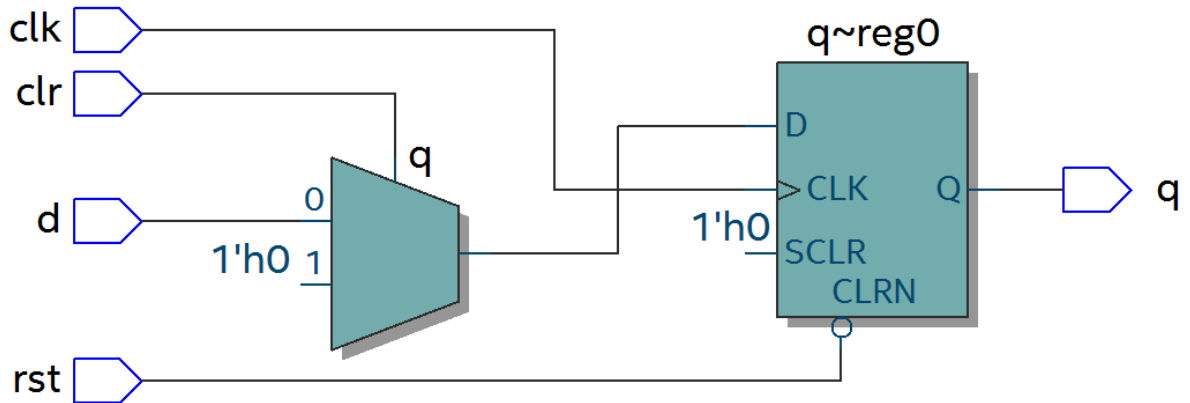
```

仿真结果如下，可以看到在复位信号之前，虽然输入信号 d 数据有变化，但由于正处于复位状态，输入信号 q 始终为 0，在复位之后 q 的值就正常了。



3.6.4 带异步复位同步清零的 D 触发器

前面讲到异步复位独立于时钟操作，而同步清零则是同步于时钟信号下操作的，当然也不仅限于同步清零，也可以是其他的同步操作，其 RTL 图如下：



代码如下，不同于异步复位，同步操作不能把信号放到敏感列表里

```

module top(d, rst, clr, clk, q) ;
    input d ;
    input rst ;
    input clr ;
    input clk ;
    output reg q ;

    always @(posedge clk or negedge
rst)
    begin
        if (rst == 1'b0)
            q <= 0 ;
        else if (clr == 1'b1)
            q <= 0 ;
        else
            q <= d ;
    end
endmodule

`timescale 1 ns/1 ns
module top_tb() ;
    reg d ;
    reg rst ;
    reg clr ;
    reg clk ;
    wire q ;

    initial
    begin
        d = 0 ;
        clk = 0 ;
        forever
        begin
            #({$random}%100)
            d = ~d ;
        end
    end

    initial
    begin
        rst = 0 ;
        clr = 0 ;
        #200 rst = 1 ;
        #200 clr = 1 ;
        #100 clr = 0 ;
    end

    always #10 clk = ~clk ;

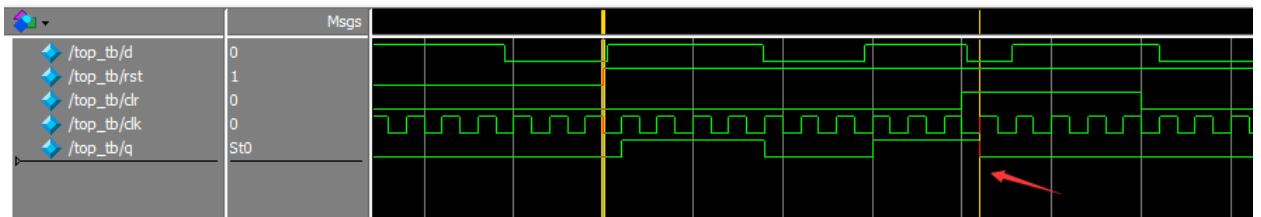
    top
    t0(.d(d),.rst(rst),.clr(clr),.clk(cl
k),
    .q(q)) ;

endmodule

```

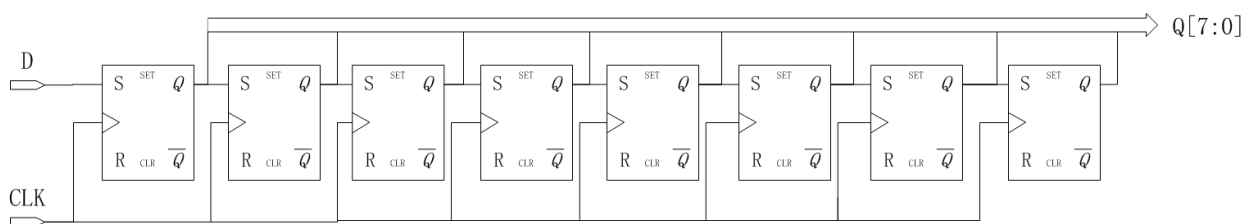
仿真结果如下，可以看到 clr 信号拉高后，q 没有立即清零，而是在下个 clk 上升沿之后执

行清零操作，也就是 clr 同步于 clk。



3.6.5 移位寄存器

移位寄存器是指在每个时钟脉冲来时，向左或向右移动一位，由于 D 触发器的特性，数据输出同步于时钟边沿，其结构如下，每个时钟来临，每个 D 触发器的输出 q 等于前一个 D 触发器输出的值，从而实现移位的功能。



代码实现：

```
module top(d, rst, clk, q) ;
input d ;
input rst ;
input clk ;
output reg [7:0] q ;

always @(posedge clk or negedge
rst)
begin
    if (rst == 1'b0)
        q <= 0 ;
    else
        q <= {q[6:0], d} ; //向左移位
        //q <= {d, q[7:1]} ; //向右移位
    end
endmodule
```

激励文件：

```
`timescale 1 ns/1 ns
module top_tb() ;
reg d ;

reg rst ;
reg clk ;
wire [7:0] q ;

initial
begin
    d = 0 ;
    clk = 0 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

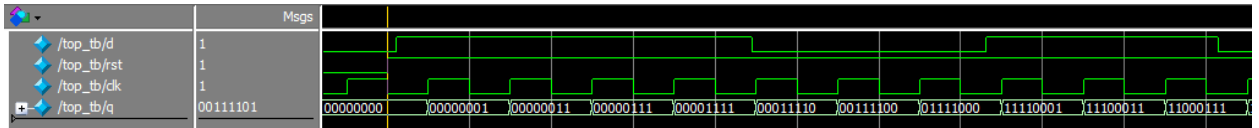
initial
begin
    rst = 0 ;
    #200 rst = 1 ;
end

always #10 clk = ~clk ;

top
t0(.d(d), .rst(rst), .clk(clk), .q(q))
;

endmodule
```

仿真结果如下，可以看到复位之后，每个 clk 上升沿左移一位



3.6.6 单口 RAM

单口 RAM 的写地址与读地址共用一个地址，代码如下，其中 reg [7:0] ram [63:0]意思是定义了 64 个 8 位宽度的数据。其中定义了 addr_reg，可以保持住读地址，延迟一周期之后将数据送出。

```

module top
(
    input [7:0] data,
    input [5:0] addr,
    input wr,
    input clk,
    output [7:0] q
);

    reg [7:0] ram[63:0]; //declare ram
    reg [5:0] addr_reg; //addr
    register

    always @ (posedge clk)
    begin
        if (wr) //write
            ram[addr] <= data;

        addr_reg <= addr;
    end

    assign q = ram[addr_reg]; //read
data
endmodule

`timescale 1 ns/1 ns
module top_tb() ;
    reg [7:0] data ;
    reg [5:0] addr ;
    reg wr ;
    reg clk ;
    wire [7:0] q ;

    initial
    begin
        data = 0 ;
        addr = 0 ;
        wr = 1 ;
        clk = 0 ;
    end

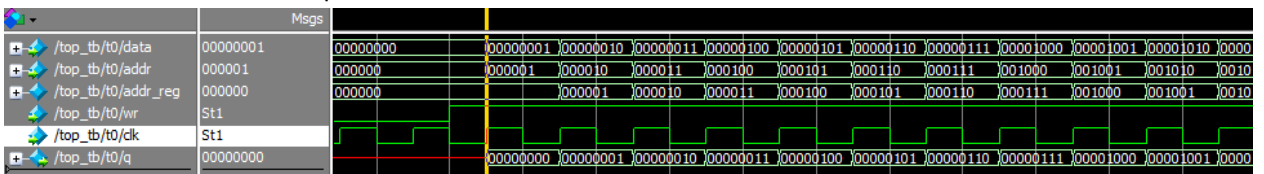
    always #10 clk = ~clk ;

    always @(posedge clk)
    begin
        data <= data + 1'b1 ;
        addr <= addr + 1'b1 ;
    end

    top t0(.data(data),
        .addr(addr),
        .clk(clk),
        .wr(wr),
        .q(q)) ;
endmodule

```

仿真结果如下，可以看到 q 的输出与写入的数据一致



3.6.7 伪双口 RAM

伪双口 RAM 的读写地址是独立的，可以随机选择写或读地址，同时进行读写操作。代码如下，在激励文件中定义了 en 信号，在其有效时发送读地址。

```

module top
(
    input [7:0] data,
    input [5:0] write_addr,

```

```

`timescale 1 ns/1 ns
module top_tb() ;
    reg [7:0] data ;
    reg [5:0] write_addr ;

```

```

    input [5:0] read_addr,
    input wr,
    input rd,
    input clk,
    output reg [7:0] q
);

reg [7:0] ram[63:0]; //declare ram
reg [5:0] addr_reg; //addr
register

always @ (posedge clk)
begin
    if (wr) //write
        ram[write_addr] <= data;
    if (rd) //read
        q <= ram[read_addr];
end

endmodule

reg [5:0] read_addr ;
reg wr ;
reg clk ;
reg rd ;
wire [7:0] q ;

initial
begin
    data = 0 ;
    write_addr = 0 ;
    read_addr = 0 ;
    wr = 0 ;
    rd = 0 ;
    clk = 0 ;
    #100 wr = 1 ;
    #20 rd = 1 ;
end

always #10 clk = ~clk ;

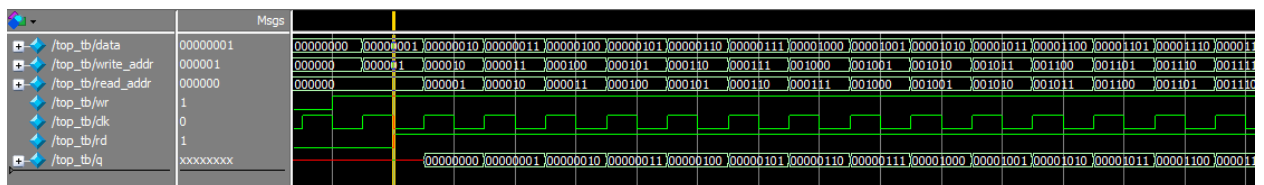
always @(posedge clk)
begin
    if (wr)
    begin
        data <= data + 1'b1 ;
        write_addr <= write_addr +
1'b1 ;
    if (rd)
        read_addr <= read_addr +
1'b1 ;
    end
end

top t0(.data(data),
        .write_addr(write_addr),
        .read_addr(read_addr),
        .clk(clk),
        .wr(wr),
        .rd(rd),
        .q(q)) ;

endmodule

```

仿真结果如下，可以看到在 rd 有效时，对读地址进行操作，读出数据



3.6.8 真双口 RAM

真双口 RAM 有两套控制线，数据线，允许两个系统对其进行读写操作，代码如下：

```

module top
(
    input [7:0] data_a, data_b,
    input [5:0] addr_a, addr_b,
    input wr_a, wr_b,
    input rd_a, rd_b,
    input clk,

    `timescale 1 ns/1 ns
    module top_tb() ;
    reg [7:0] data_a, data_b ;
    reg [5:0] addr_a, addr_b ;
    reg wr_a, wr_b ;
    reg rd_a, rd_b ;
    reg clk ;

```

```

    output reg [7:0] q_a, q_b
);

reg [7:0] ram[63:0]; //declare ram

//Port A
always @ (posedge clk)
begin
    if (wr_a) //write
    begin
        ram[addr_a] <= data_a;
        q_a <= data_a ;
    end
    if (rd_a)
//read
        q_a <= ram[addr_a];
end

//Port B
always @ (posedge clk)
begin
    if (wr_b) //write
    begin
        ram[addr_b] <= data_b;
        q_b <= data_b ;
    end
    if (rd_b)
//read
        q_b <= ram[addr_b];
end

endmodule

```

```

wire [7:0] q_a, q_b ;

initial
begin
    data_a = 0 ;
    data_b = 0 ;
    addr_a = 0 ;
    addr_b = 0 ;
    wr_a = 0 ;
    wr_b = 0 ;
    rd_a = 0 ;
    rd_b = 0 ;
    clk = 0 ;
    #100 wr_a = 1 ;
    #100 rd_b = 1 ;
end

always #10 clk = ~clk ;

always @(posedge clk)
begin
    if (wr_a)
    begin
        data_a <= data_a + 1'b1 ;
        addr_a <= addr_a + 1'b1 ;
    end
    else
    begin
        data_a <= 0 ;
        addr_a <= 0 ;
    end
end

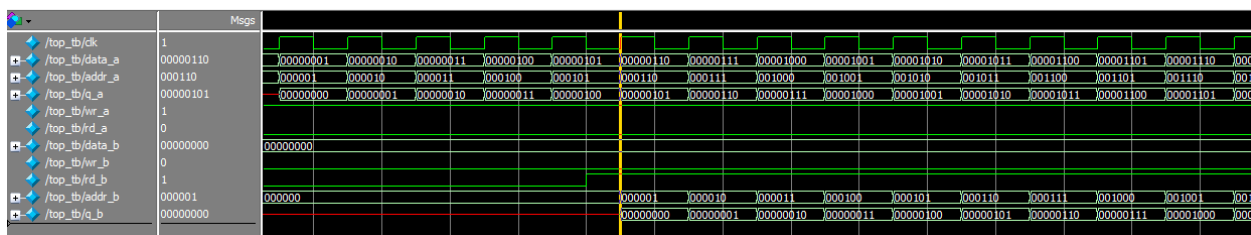
always @(posedge clk)
begin
    if (rd_b)
    begin
        addr_b <= addr_b + 1'b1 ;
    end
    else addr_b <= 0 ;
end

end

top
t0(.data_a(data_a), .data_b(data_b),
    .addr_a(addr_a), .addr_b(addr
    _b
    ),
    .wr_a(wr_a), .wr_b(wr_b),
    .rd_a(rd_a), .rd_b(rd_b),
    .clk(clk),
    .q_a(q_a), .q_b(q_b)) ;
endmodule

```

仿真结果如下



3.6.9 单口 ROM

ROM 是用来存储数据的，可以按照下列代码形式初始化 ROM，但这种方法处理大容量的 ROM 就比较麻烦，建议用 FPGA 自带的 ROM IP 核实现，并添加初始化文件。

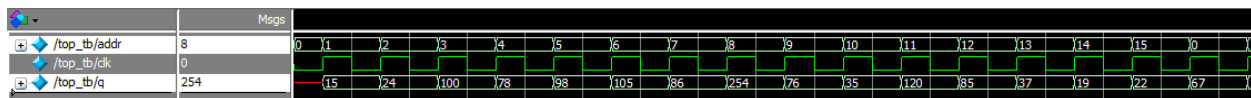
代码实现

```
module top
(
    input [3:0] addr,
    input clk,
    output reg [7:0] q
);

always @(posedge clk)
begin
    case(addr)
        4'd0 : q <= 8'd15 ;
        4'd1 : q <= 8'd24 ;
        4'd2 : q <= 8'd100 ;
        4'd3 : q <= 8'd78 ;
        4'd4 : q <= 8'd98 ;
        4'd5 : q <= 8'd105 ;
        4'd6 : q <= 8'd86 ;
        4'd7 : q <= 8'd254 ;
        4'd8 : q <= 8'd76 ;
        4'd9 : q <= 8'd35 ;
        4'd10 : q <= 8'd120 ;
        4'd11 : q <= 8'd85 ;
        4'd12 : q <= 8'd37 ;
        4'd13 : q <= 8'd19 ;
        4'd14 : q <= 8'd22 ;
        4'd15 : q <= 8'd67 ;
        default: q <= 8'd0 ;
    endcase
end

endmodule
```

仿真结果如下



激励文件

```
`timescale 1 ns/1 ns
module top_tb() ;
    reg [3:0] addr ;
    reg clk ;
    wire [7:0] q ;

    initial
    begin
        addr = 0 ;
        clk = 0 ;
    end

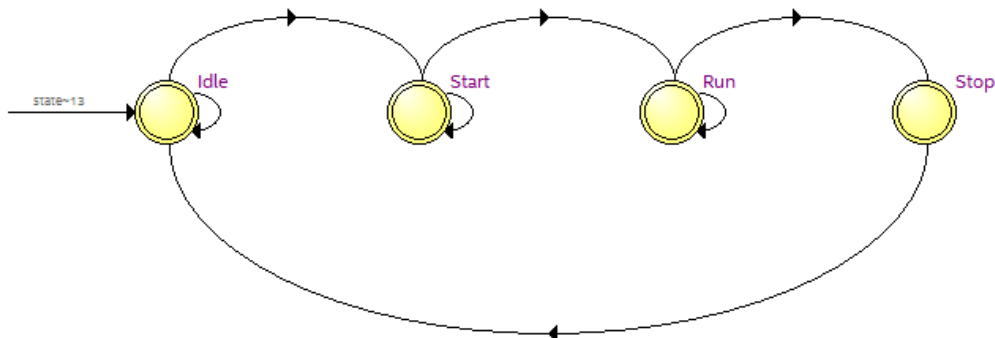
    always #10 clk = ~clk ;

    always @(posedge clk)
    begin
        addr <= addr + 1'b1 ;
    end

    top t0(.addr(addr),
           .clk(clk),
           .q(q)) ;
endmodule
```

3.6.10 有限状态机

在 verilog 里经常会用到有限状态机，处理相对复杂的逻辑，设定好不同的状态，根据触发条件跳转到对应的状态，在不同的状态下做相应的处理。有限状态机主要用到 always 及 case 语句。下面以一个四状态的有限状态机举例说明。



在程序中设计了 8 位的移位寄存器，在 Idle 状态下，判断 shift_start 信号是否为高，如果为高，进入 Start 状态，在 Start 状态延迟 100 个周期，进入 Run 状态，进行移位处理，如果 shift_stop 信号有效了，进入 Stop 状态，在 Stop 状态，清零 q 的值，再跳转到 Idle 状态。

Mealy 有限状态机，输出不仅与当前状态有关，也与输入信号有关，在 RTL 中会与输入信号有连接。

```

module top
(
    input shift_start,
    input shift_stop,
    input rst,
    input clk,
    input d,
    output reg [7:0] q
);

parameter Idle = 2'd0 ; //Idle state
parameter Start = 2'd1 ; //Start state
parameter Run = 2'd2 ; //Run state
parameter Stop = 2'd3 ; //Stop state

reg [1:0] state ; //statement
reg [4:0] delay_cnt ; //delay counter

always @(posedge clk or negedge rst)
begin
    if (!rst)
    begin
        state <= Idle ;
        delay_cnt <= 0 ;
        q <= 0 ;
    end
    else
    case(state)
        Idle : begin
            if (shift_start)
                state <= Start ;
        end
        Start : begin
            delay_cnt <= delay_cnt + 1 ;
            if (delay_cnt == 100)
                state <= Run ;
        end
        Run : begin
            q <= q << 1 | d ;
            if (shift_stop)
                state <= Stop ;
        end
        Stop : begin
            q <= 0 ;
            state <= Idle ;
        end
    endcase
end

```

```

        if (delay_cnt == 5'd99)
        begin
            delay_cnt <= 0 ;
            state <= Run ;
        end
        else
            delay_cnt <= delay_cnt + 1'b1 ;
        end
    Run : begin
        if (shift_stop)
            state <= Stop ;
        else
            q <= {q[6:0], d} ;
        end
    Stop : begin
        q <= 0 ;
        state <= Idle ;
    end
    default: state <= Idle ;
endcase
end
endmodule

```

Moore 有限状态机，输出只与当前状态有关，与输入信号无关，输入信号只影响状态的改变，不影响输出，比如对 delay_cnt 和 q 的处理，只与 state 状态有关。

```

module top
(
    input shift_start,
    input shift_stop,
    input rst,
    input clk,
    input d,
    output reg [7:0] q
);

parameter Idle = 2'd0 ; //Idle state
parameter Start = 2'd1 ; //Start state
parameter Run = 2'd2 ; //Run state
parameter Stop = 2'd3 ; //Stop state

reg [1:0] current_state ; //statement
reg [1:0] next_state ;
reg [4:0] delay_cnt ; //delay counter
//First part: statement transition
always @(posedge clk or negedge rst)
begin
    if (!rst)
        current_state <= Idle ;
    else
        current_state <= next_state ;
end
//Second part: combination logic, judge statement transition
condition
always @(*)
begin
    case(current_state)
        Idle : begin
            if (shift_start)
                next_state <= Start ;
            else
                next_state <= Idle ;

```

```

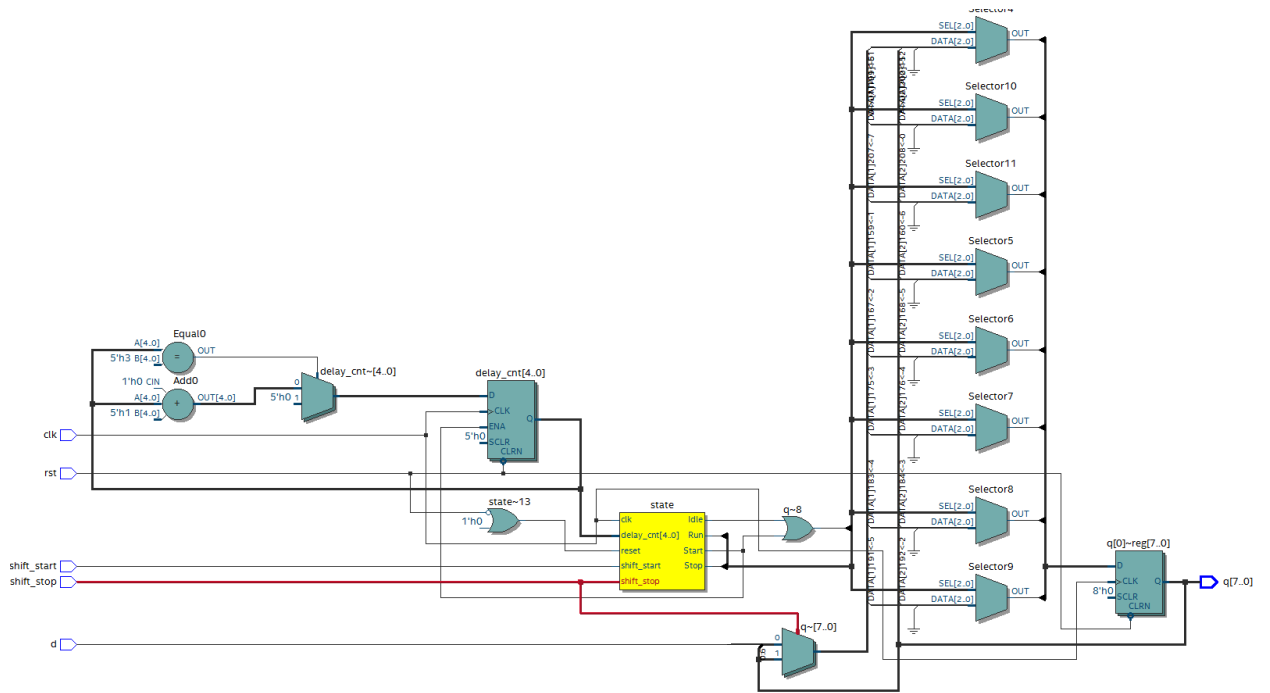
end
Start : begin
    if (delay_cnt == 5'd99)
        next_state <= Run ;
    else
        next_state <= Start ;
    end
Run : begin
    if (shift_stop)
        next_state <= Stop ;
    else
        next_state <= Run ;
    end
Stop :    next_state <= Idle ;
default:  next_state <= Idle ;
endcase
end
//Last part: output data
always @(posedge clk or negedge rst)
begin
    if (!rst)
        delay_cnt <= 0 ;
    else if (current_state == Start)
        delay_cnt <= delay_cnt + 1'b1 ;
    else
        delay_cnt <= 0 ;
end

always @(posedge clk or negedge rst)
begin
    if (!rst)
        q <= 0 ;
    else if (current_state == Run)
        q <= {q[6:0], d} ;
    else
        q <= 0 ;
end

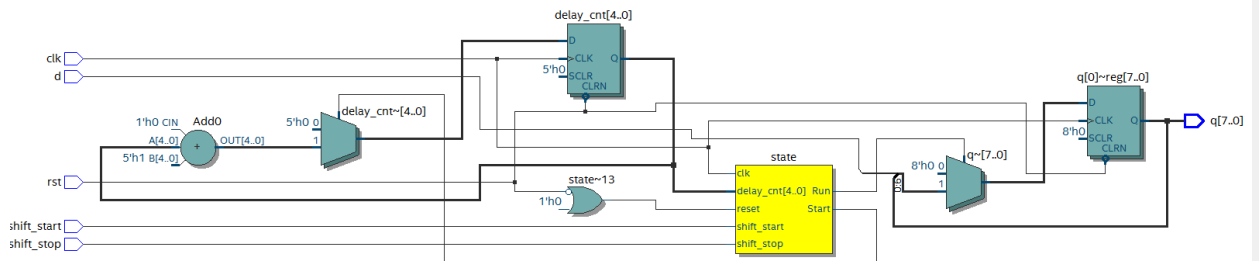
endmodule

```

在上面两个程序中用到了两种方式的写法，第一种 Mealy 状态机，采用了一段式的写法，只用了一个 always 语句，所有的状态转移，判断状态转移条件，数据输出都在一个 always 语句里，缺点是如果状态太多，会使整段程序显的冗长。第二个 Moore 状态机，采用了三段式的写法，状态转移用了一个 always 语句，判断状态转移条件是组合逻辑，采用了一个 always 语句，数据输出也是单独的 always 语句，这样写起来比较直观清晰，状态很多时也不会显得繁琐。



Mealy 有限状态机 RTL 图



Moore 有限状态机 RTL 图

激励文件如下：

```

`timescale 1 ns/1 ns
module top_tb() ;
reg shift_start ;
reg shift_stop ;
reg rst ;
reg clk ;
reg d ;
wire [7:0] q ;

initial
begin
    rst = 0 ;
    clk = 0 ;
    d = 0 ;
    #200 rst = 1 ;
    forever
    begin
        #({$random}%100)
        d = ~d ;
    end
end

```

```

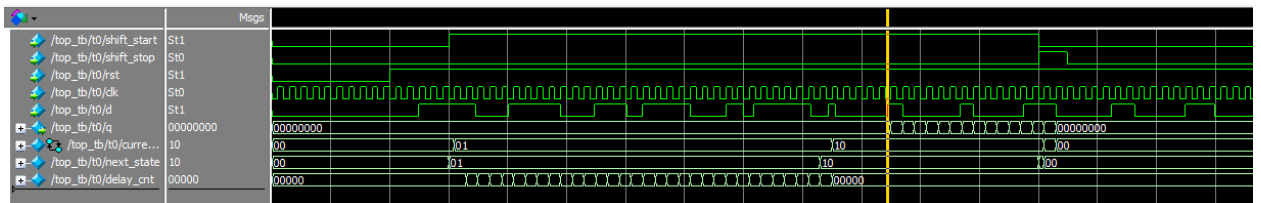
initial
begin
    shift_start = 0 ;
    shift_stop = 0 ;
    #300 shift_start = 1 ;
    #1000 shift_start = 0 ;
        shift_stop = 1 ;
    #50 shift_stop = 0 ;
end

always #10 clk = ~clk ;

top t0
(
    .shift_start(shift_start),
    .shift_stop(shift_stop),
    .rst(rst),
    .clk(clk),
    .d(d),
    .q(q)
);
endmodule

```

仿真结果如下:



3.7 总结

本文档介绍了组合逻辑以及时序逻辑中常用的模块，其中有限状态机较为复杂，但经常用到，希望大家能够深入理解，在代码中多运用，多思考，有利于快速提升水平。

第四章 PL 的“Hello World”LED 实验

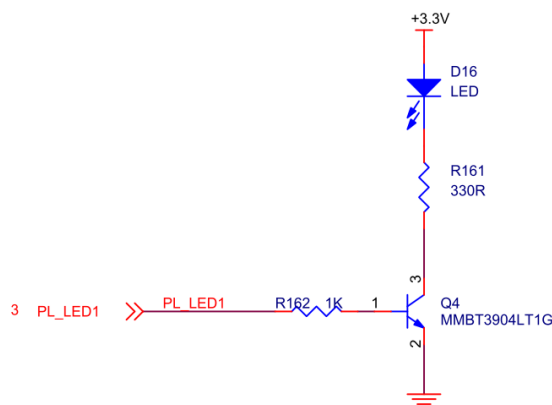
实验 Vivado 工程为 “led”。

对于 ZYNQ 来说 PL (FPGA) 开发是至关重要的，这也是 ZYNQ 比其他 ARM 的有优势的地方，可以定制化很多 ARM 端的外设，在定制 ARM 端的外设之前先让我们通过一个 LED 例程来熟悉 PL (FPGA) 的开发流程，熟悉 Vivado 软件的基本操作，这个开发流程和不带 ARM 的 FPGA 芯片完全一致。

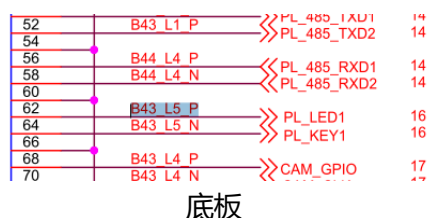
在本例程中，我们要做的是 LED 灯控制实验，每秒钟控制开发板上的 LED 灯翻转一次，实现亮、灭、亮、灭的控制。会控制 LED 灯，其它外设也慢慢就会了。

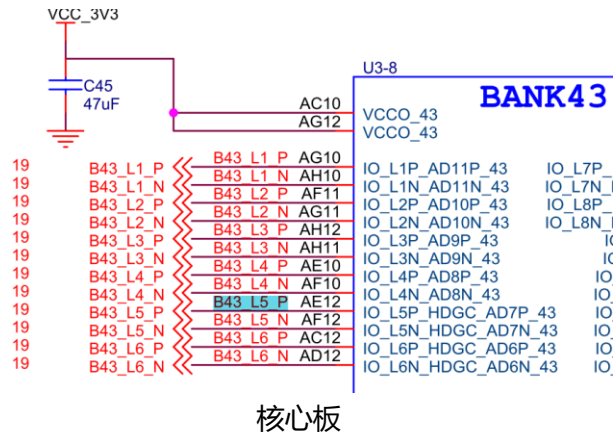
4.1 LED 硬件介绍

- 1) 开发板的 PL 部分连接了 1 个红色的用户 LED 灯。这 1 个灯完全由 PL 控制。如果 PL_LED1 为高电平，三级管导通，灯则会亮，否则会灭。



- 2) 我们可以根据原理图的连线关系确定 LED 和 PL 管脚的绑定关系。





3) 原理图中以 PS_MIO 开头的 IO 都是 PS 端 IO，不需要绑定，也不能绑定

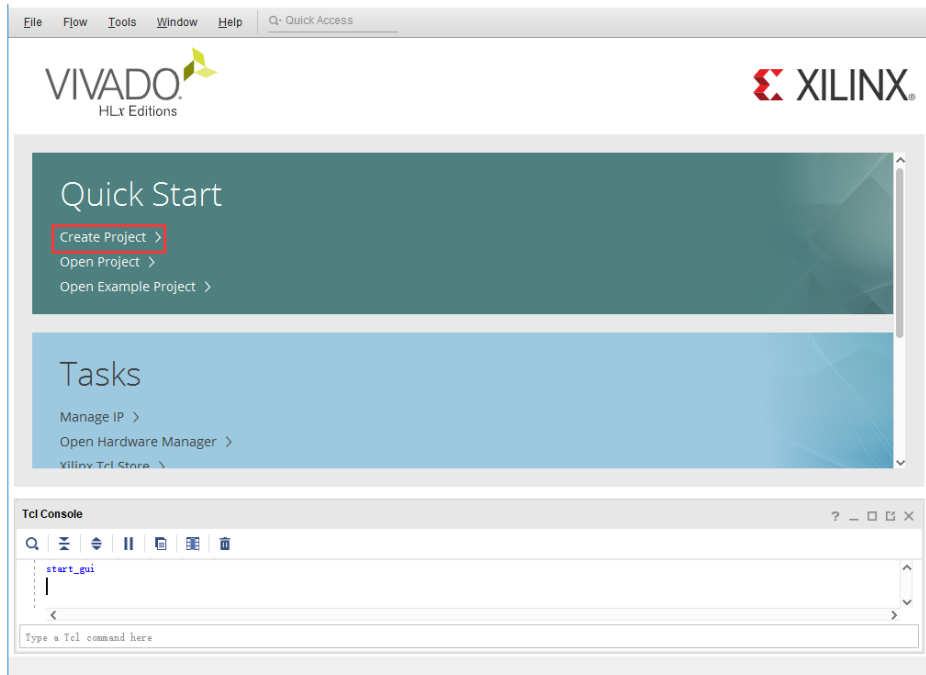
PS_MIO0_500	E6
PS_MIO1_500	A7
PS_MIO2_500	B8
PS_MIO3_500	D6
PS_MIO4_500	B7
PS_MIO5_500	A6
PS_MIO6_500	A5 R1
PS_MIO7_500	D8
PS_MIO8_500	D5
PS_MIO9_500	B5
PS_MIO10_500	E9
PS_MIO11_500	C6
PS_MIO12_500	D9
PS_MIO13_500	E8
PS_MIO14_500	C5
PS_MIO15_500	C8
PS_POR_B_500	C7
PS_CLK_500	E7

4.2 创建 Vivado 工程

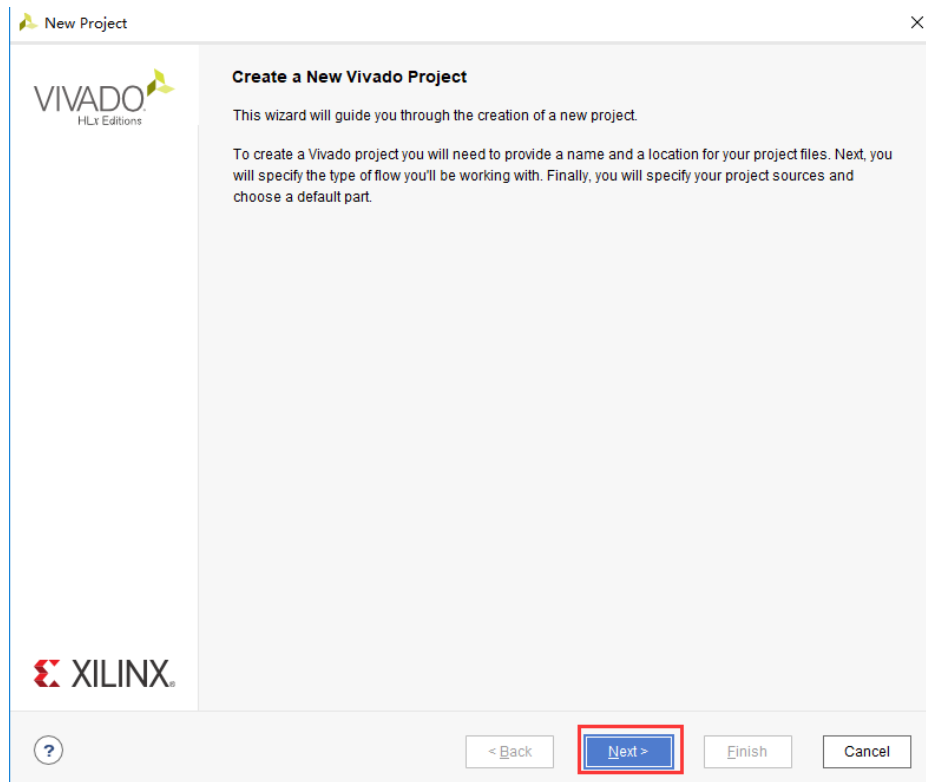
1) 启动 Vivado，在 Windows 中可以通过双击 Vivado 快捷方式启动



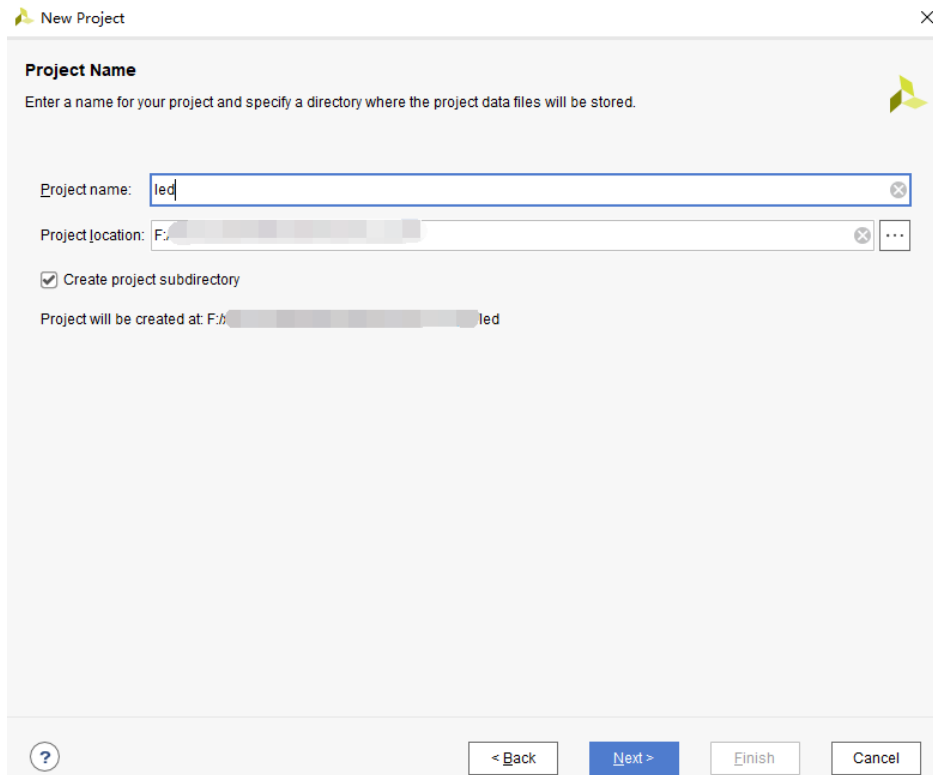
2) 在 Vivado 开发环境里点击 “Create New Project”，创建一个新的工程。



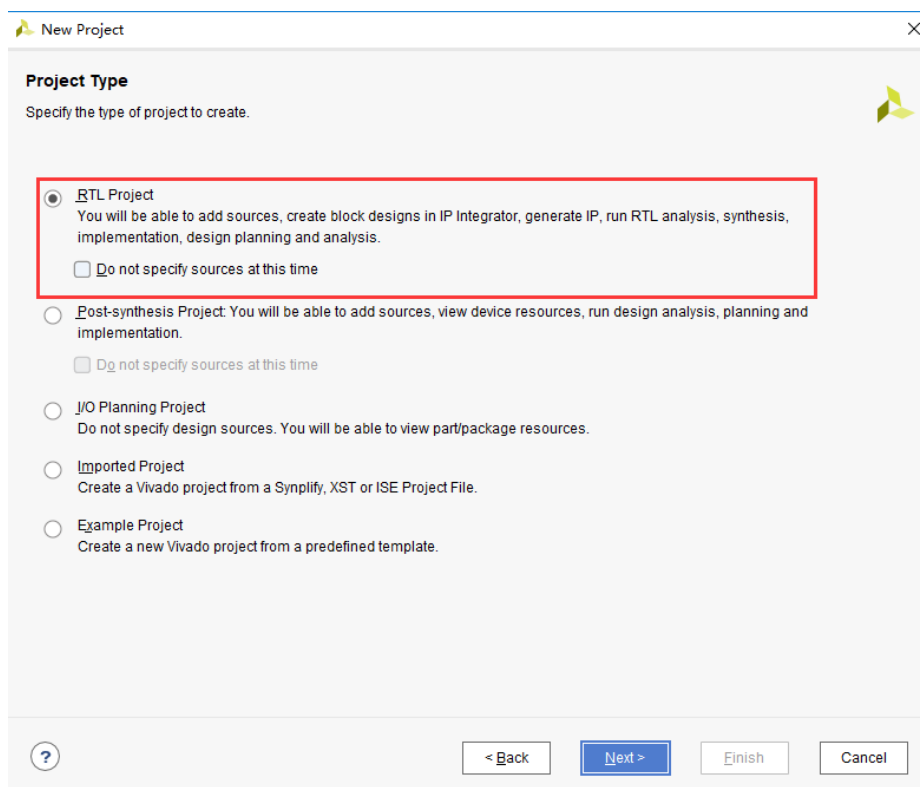
- 3) 弹出一个建立新工程的向导，点击 “Next”



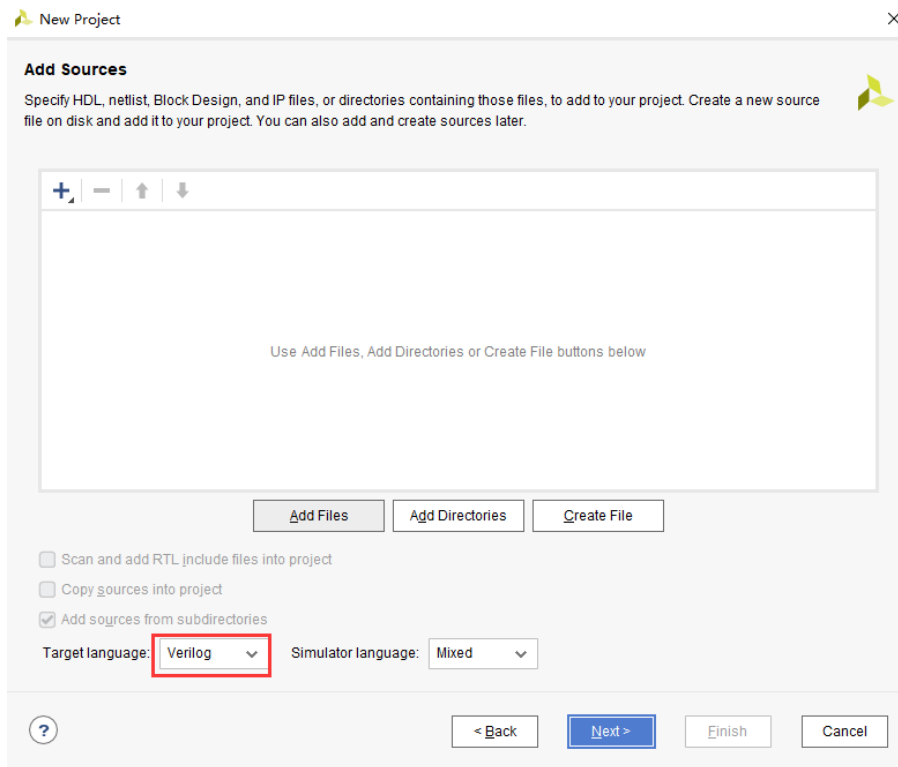
- 4) 在弹出的对话框中输入工程名和工程存放的目录，我们这里取一个 led 的工程名。需要注意工程路径 “Project location” 不能有中文空格，路径名称也不能太长。



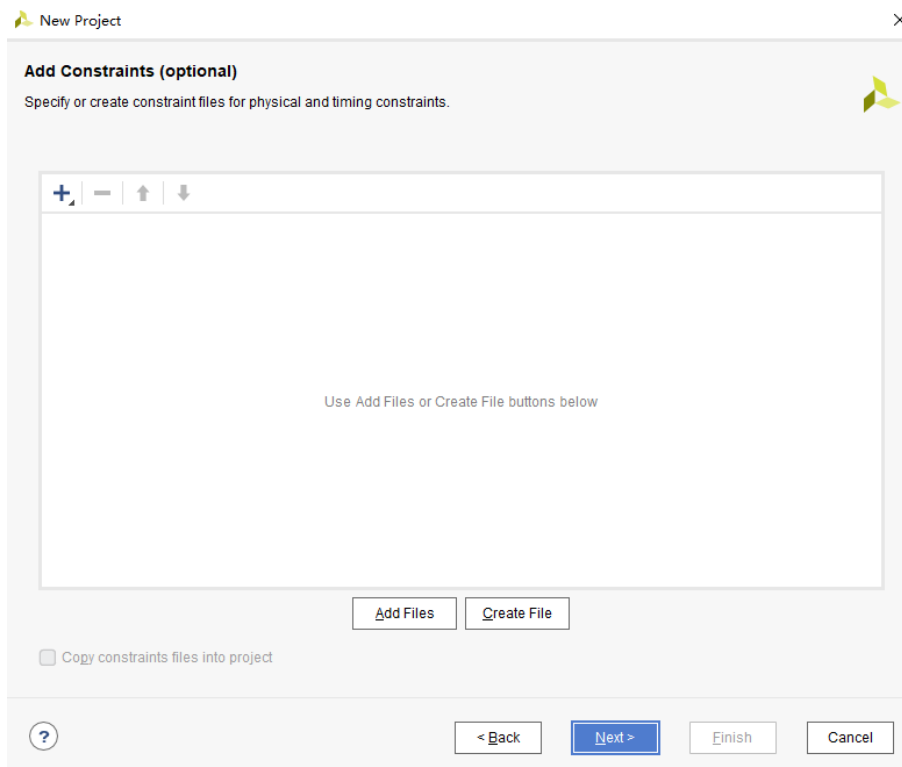
5) 在工程类型中选择 “RTL Project”



6) 目标语言 “Target language” 选择 “Verilog”，虽然选择 Verilog，但 VHDL 也可以使用，支持多语言混合编程。



7) 点击 “Next”，不添加任何文件



8) 以 AXU3EG 开发板为例，在 “Part” 选项中，器件家族 “Family” 选择 “Zynq UltraScale+ MPSoCs”，封装类型 “Package” 选择 “sfvc784”，Speed 选择 “-1”，Temperature 选择 “I” 减少选择范围。在下拉列表中选择 “xczu3eg-sfvc784-1-i”，“-1” 表示速率等级，数字越大，性能越好，速率高的芯片向下兼容速率低的芯片。（例程中选择的型号为 “xazu3eg-

sfvc784-1-i”，两者是兼容的)

New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts | Boards

Reset All Filters

Category: All
Family: Zynq UltraScale+ MPSoCs

Package: sfvc784
Speed: -1
Temperature: I
Static power: All Remaining

Search: Q-

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	Gb Transceivers	GTPE2 Transceivers
xczu2eg-sfvc784-1-i	784	252	47232	94464	150	0	240	0	0
xczu3cg-sfvc784-1-i	784	252	70560	141120	216	0	360	0	0
xczu3eg-sfvc784-1-i	784	252	70560	141120	216	0	360	0	0
xczu4cg-sfvc784-1-i	784	252	87840	175680	128	48	728	4	0
xczu4eg-sfvc784-1-i	784	252	87840	175680	128	48	728	4	0
xczu4ev-sfvc784-1-i	784	252	87840	175680	128	48	728	4	0
xczu5cg-sfvc784-1-i	784	252	117120	234240	144	64	1248	4	0
xczu5eg-sfvc784-1-i	784	252	117120	234240	144	64	1248	4	0
xczu5ev-sfvc784-1-i	784	252	117120	234240	144	64	1248	4	0

< Back Next > Finish Cancel

AXU4EV 开发板，在“Part”选项中，器件家族“Family”选择“Zynq UltraScale+ MPSoCs”，封装类型“Package”选择“sfvc784”，Speed 选择“-1”，Temperature 选择“I”减少选择范围。在下拉列表中选择“xczu4ev-sfvc784-1-i”；

New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts | Boards

Reset All Filters

Category: All
Family: Zynq UltraScale+ MPSoCs

Package: sfvc784
Speed: -1
Temperature: I
Static power: All Remaining

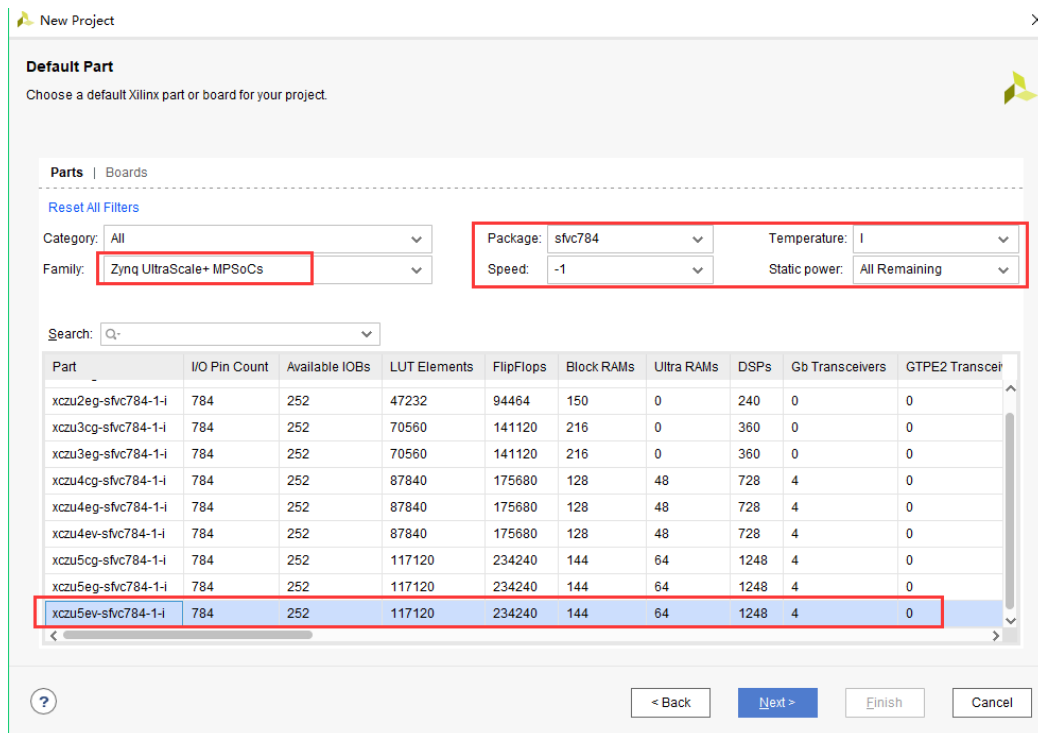
Search: Q-

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs
xczu3cg-sfvc784-1-i	784	252	70560	141120	216	0	360
xczu3eg-sfvc784-1-i	784	252	70560	141120	216	0	360
xczu4cg-sfvc784-1-i	784	252	87840	175680	128	48	728
xczu4eg-sfvc784-1-i	784	252	87840	175680	128	48	728
xczu4ev-sfvc784-1-i	784	252	87840	175680	128	48	728
xczu5cg-sfvc784-1-i	784	252	117120	234240	144	64	1248
xczu5eg-sfvc784-1-i	784	252	117120	234240	144	64	1248
xczu5ev-sfvc784-1-i	784	252	117120	234240	144	64	1248

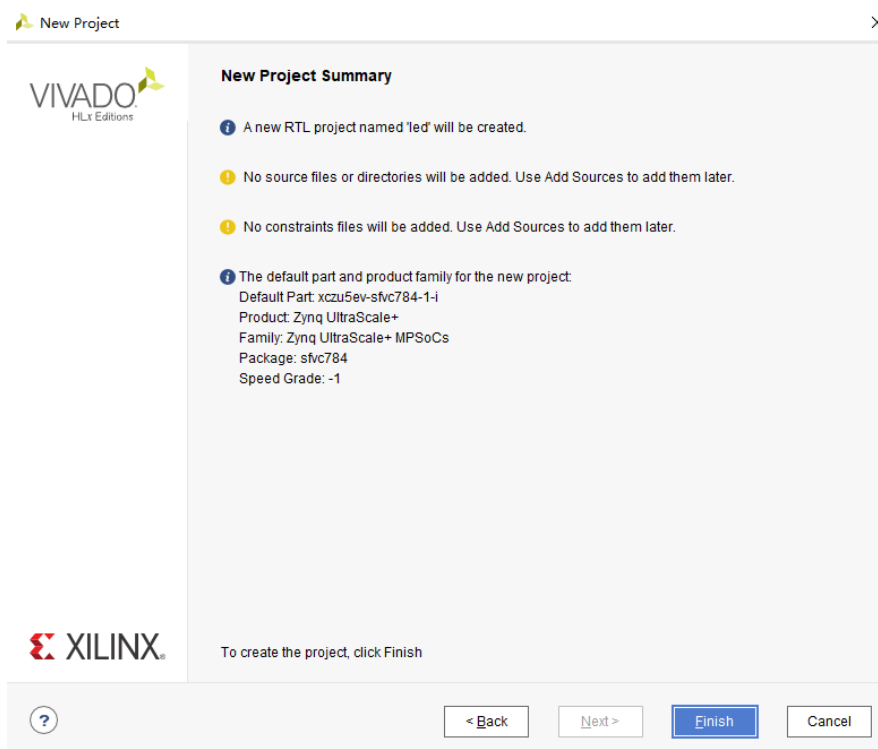
< Back Next > Finish Cancel

AXU5EV 开发板，在“Part”选项中，器件家族“Family”选择“Zynq UltraScale+ MPSoCs”，封装类型“Package”选择“sfvc784”，Speed 选择“-1”，Temperature 选择“I”减少选择范围。在下拉列表中选择“xczu5ev-sfvc784-1-i”；（例程中选择的型号为“xazu5ev-sfvc784-1-i”，两者是

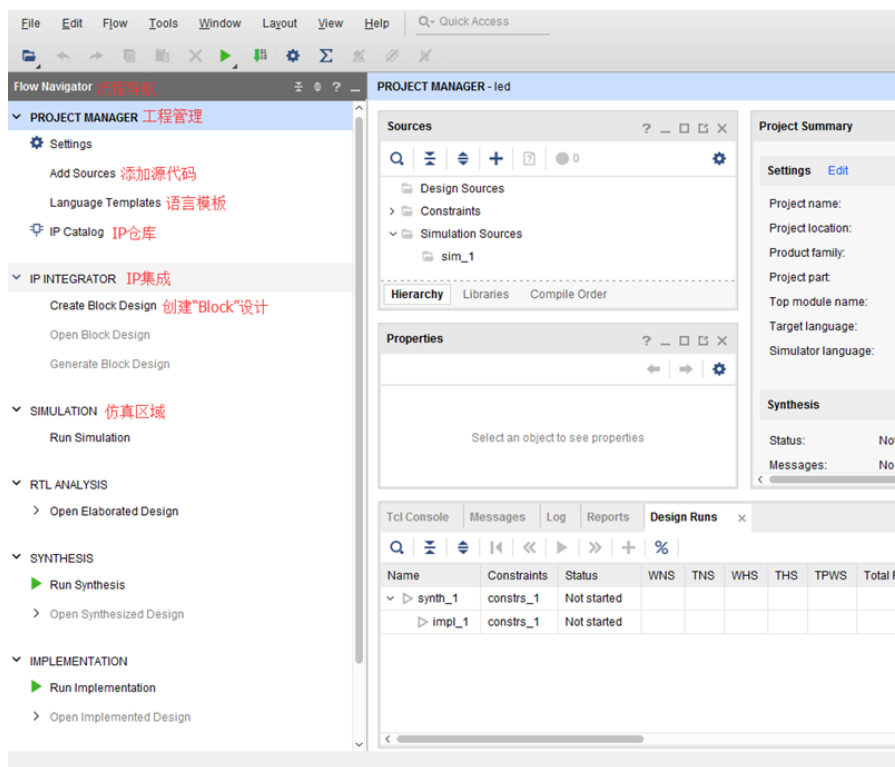
兼容的)



9) 点击“Finish”就可以完成以后名为“led”工程的创建。

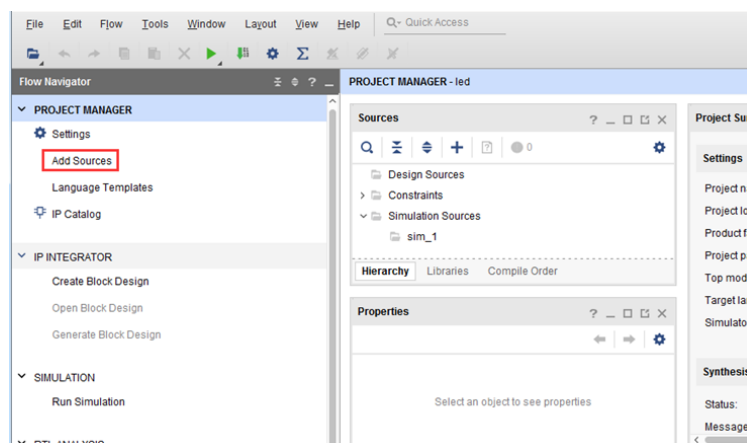


10) Vivado 软件界面

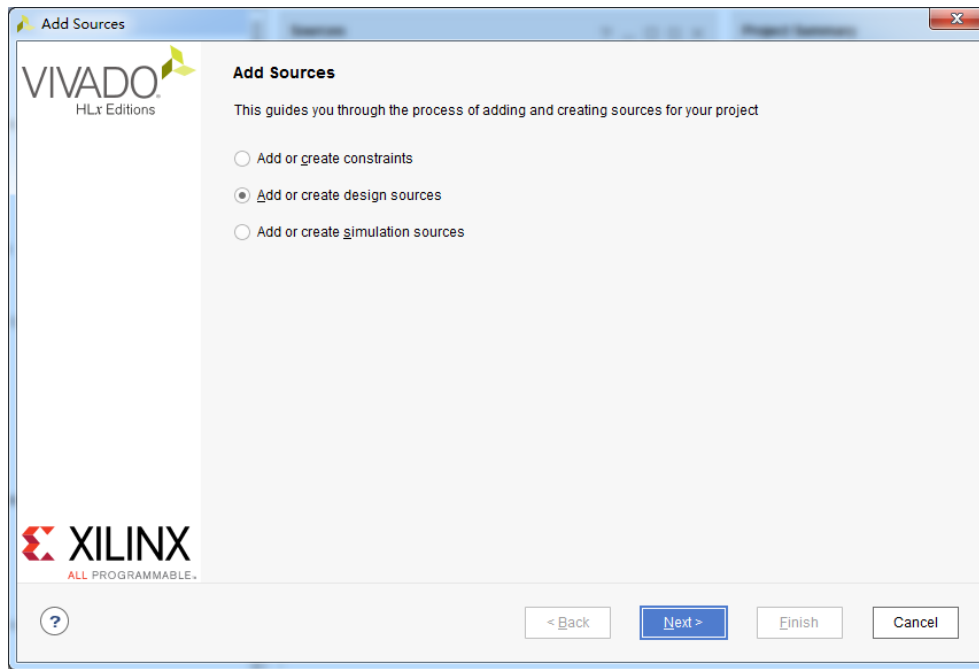


4.3 创建 Verilog HDL 文件点亮 LED

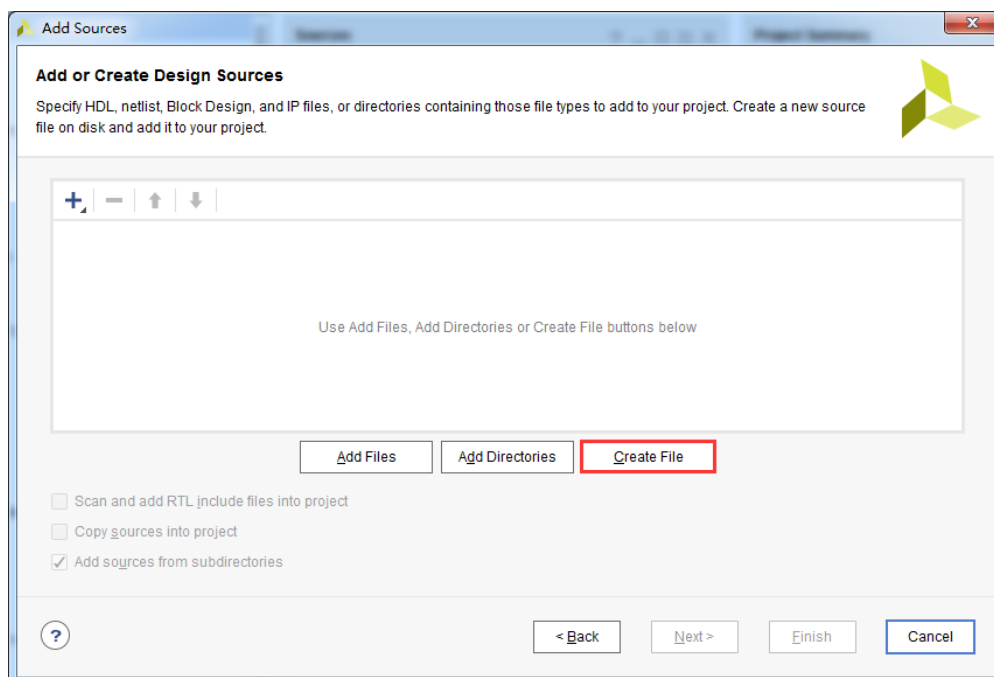
- 1) 点击 Project Manager 下的 Add Sources 图标 (或者使用快捷键 Alt+A)



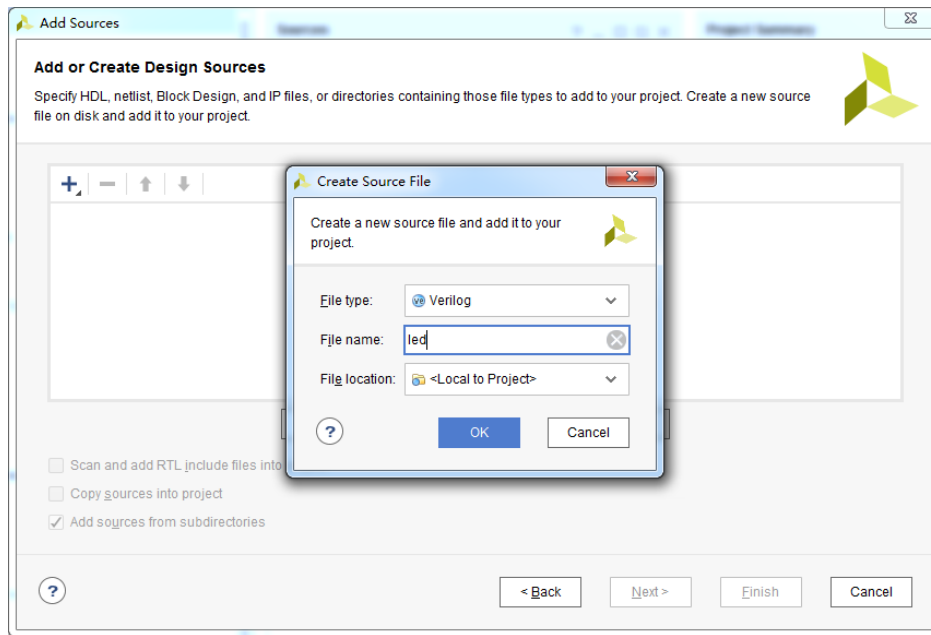
- 2) 选择添加或创建设计源文件 “Add or create design sources” ,点击 “Next”



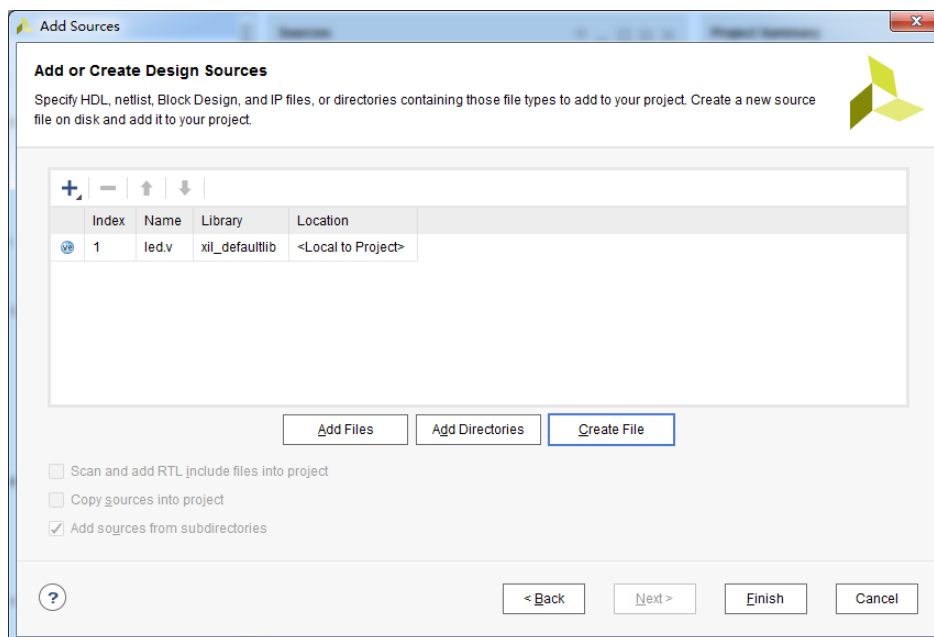
3) 选择创建文件 “Create File”



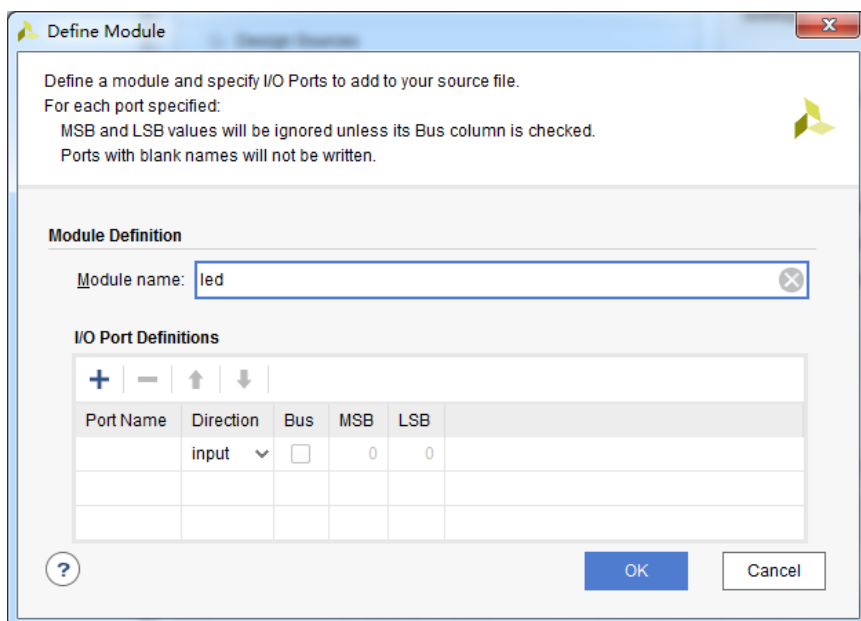
4) 文件名 “File name” 设置为 “led”，点击 “OK”



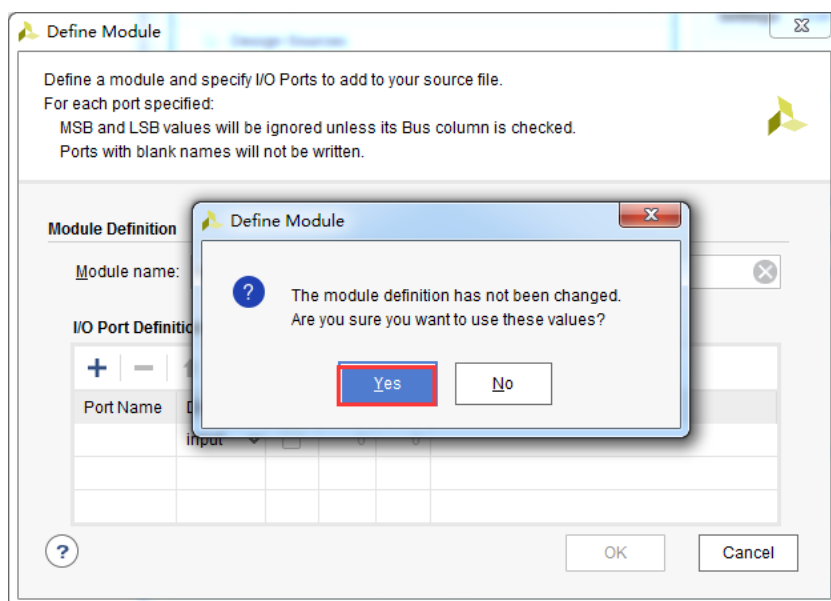
5) 点击“Finish”,完成“led.v”文件添加



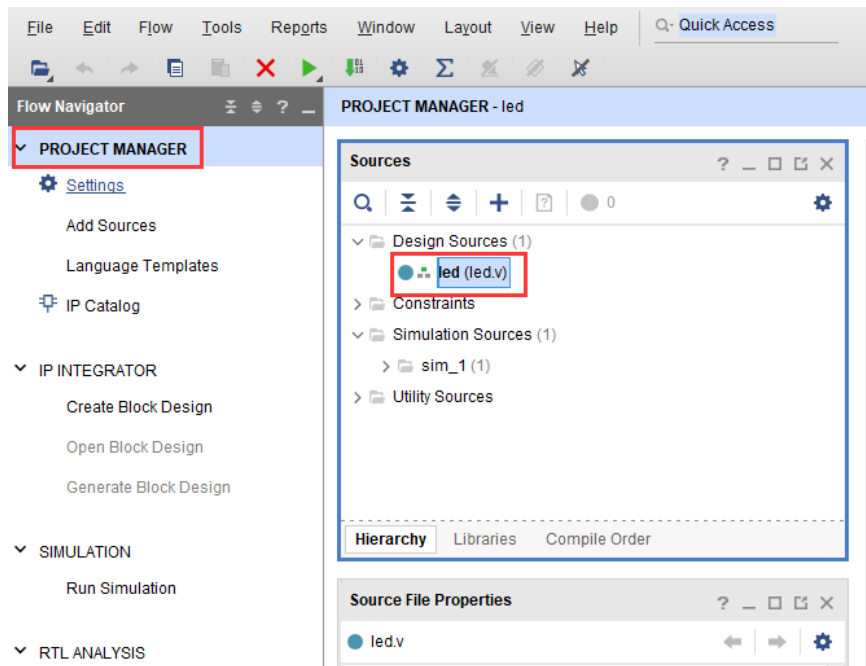
6) 在弹出的模块定义“Define Module”,中可以指定“led.v”文件的模块名称“Module name”,这里默认不变为“led”,还可以指定一些端口,这里暂时不指定,点击“OK”。



7) 在弹出的对话框中选择 “Yes”



8) 双击 “led.v” 可以打开文件，然后编辑



- 9) 编写 “led.v”, 这里定义了一个 32 位的寄存器 timer, 用于循环计数 0~199999999(1 秒钟), 计数到 199999999(1 秒)的时候, 寄存器 timer 变为 0, 并翻转四个 LED。这样原来 LED 是灭的话, 就会点亮, 如果原来 LED 为亮的话, 就会熄灭。由于输入时钟为 200MHz 的差分时钟, 因此需要添加 IBUFDS 原语连接差分信号, 编写好后的代码如下:

```
`timescale 1ns / 1ps
module led(
//Differential system clock
    input sys_clk_p,
    input sys_clk_n,
    input rst_n,
    output reg led
);
reg[31:0] timer_cnt;
wire sys_clk ;

IBUFDS IBUFDS_inst (
    .O(sys_clk), // Buffer output
    .I(sys_clk_p), // Diff_p buffer input (connect directly
to top-level port)
    .IB(sys_clk_n) // Diff_n buffer input (connect directly to
top-level port)
);

always@(posedge sys_clk)
begin
    if (!rst_n)
    begin
        led <= 1'b0 ;
        timer_cnt <= 32'd0 ;
    end
    else if(timer_cnt >= 32'd199_999_999) //1 second counter,
200M-1=199999999
    begin
        led <= ~led;
        timer_cnt <= 32'd0;
    end
end
```

```
end
else
begin
    led <= led;
    timer_cnt <= timer_cnt + 32'd1;
end

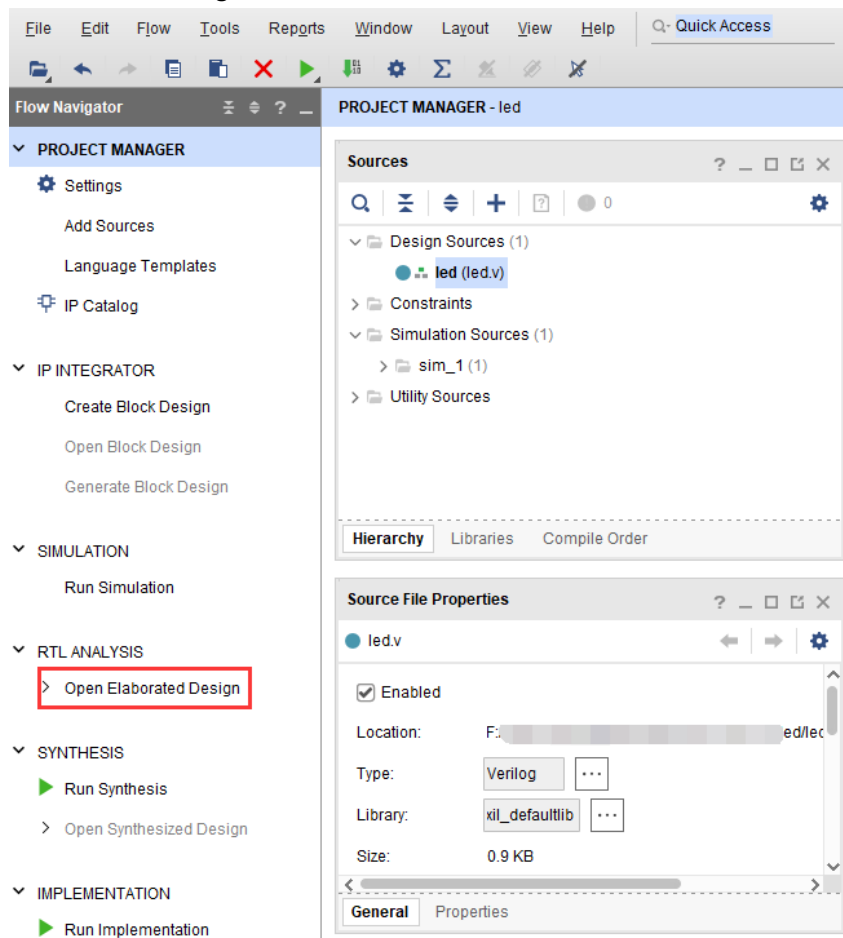
end
endmodule
```

10) 编写好代码后保存

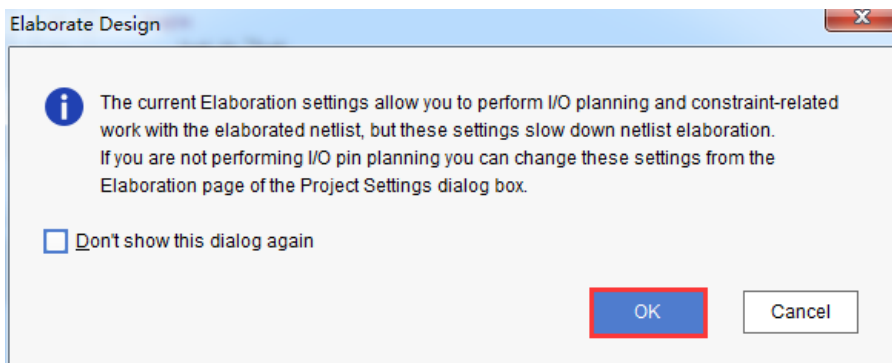
4.4 添加管脚约束

Vivado 使用的约束文件格式为 xdc 文件。xdc 文件里主要是完成管脚的约束,时钟的约束,以及组的约束。这里我们需要对 led.v 程序中的输入输出端口分配到 FPGA 的真实管脚上。

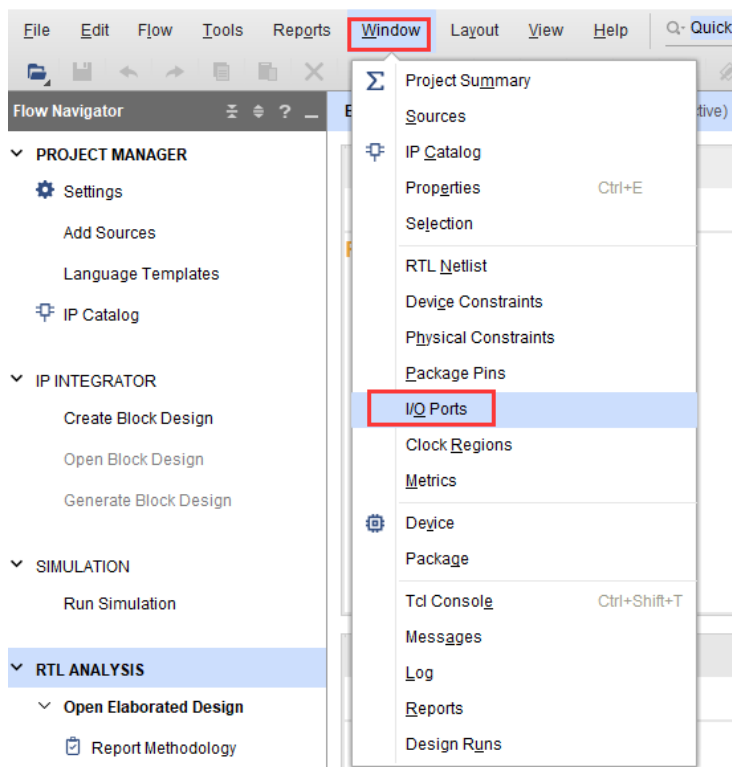
1) 点击 “Open Elaborated Design”



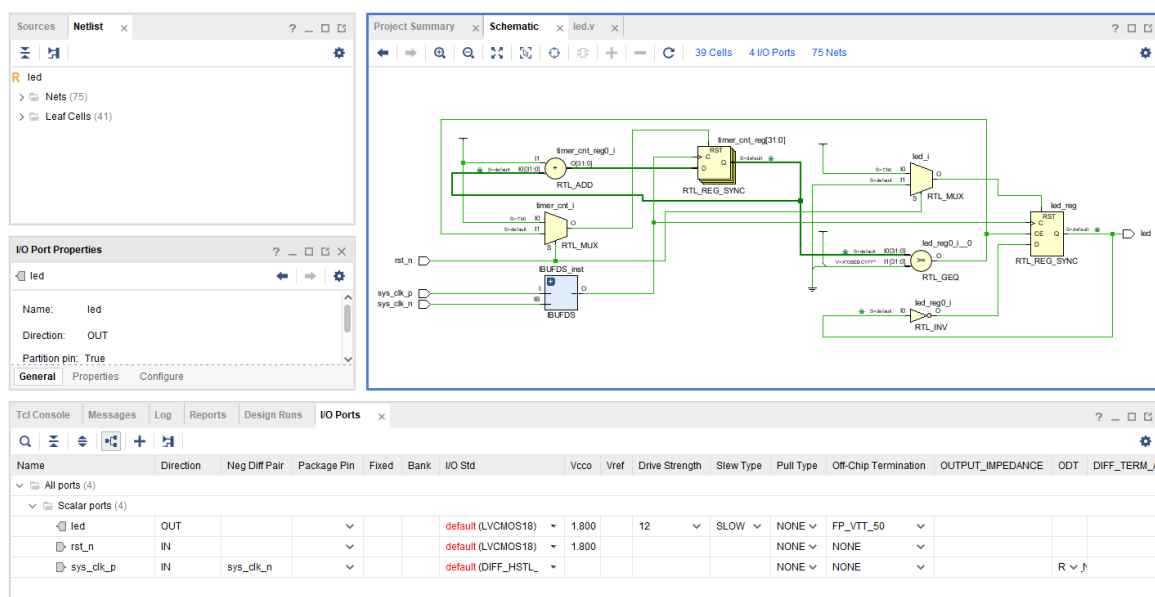
2) 在弹出的窗口中点击 “OK” 按钮



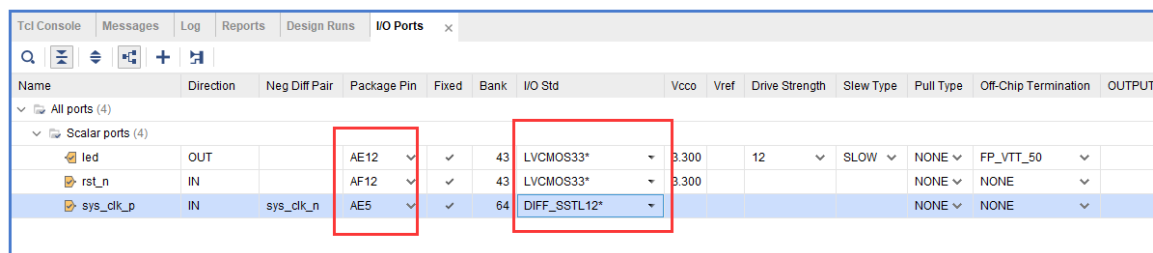
3) 在菜单中选择 “Window -> I/O Ports”



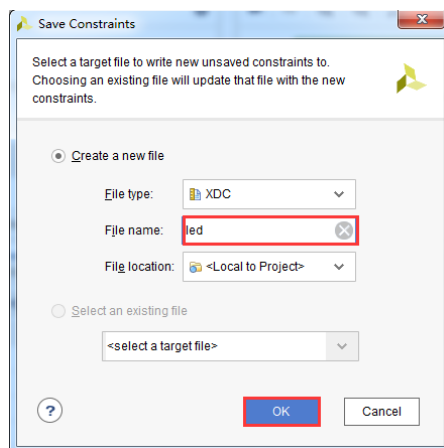
4) 在弹出的 I/O Ports 中可以看到管脚分配情况



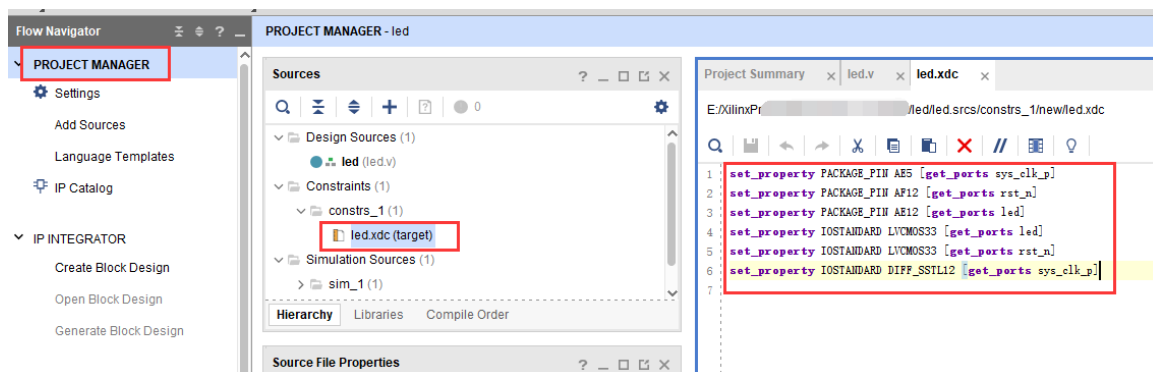
- 5) 将复位信号 `rst_n` 绑定到 PL 端的按键，给 LED 和时钟分配管脚、电平标准，完成后点击保存图标



- 6) 弹出窗口，要求保存约束文件，文件名我们填写“led”，文件类型默认“XDC”，点击“OK”



- 7) 打开刚才生成的“led.xdc”文件，我们可以看到是一个 TCL 脚本，如果我们了解这些语法，完全可以通过自己编写 led.xdc 文件的方式来约束管脚



下面来介绍一下最基本的 XDC 编写的语法，普通 IO 口只需约束引脚号和电压，管脚约束如下：

```
set_property PACKAGE_PIN "引脚编号" [get_ports "端口名称"]
```

电平信号的约束如下：

```
set_property IOSTANDARD "电平标准" [get_ports "端口名称"]
```

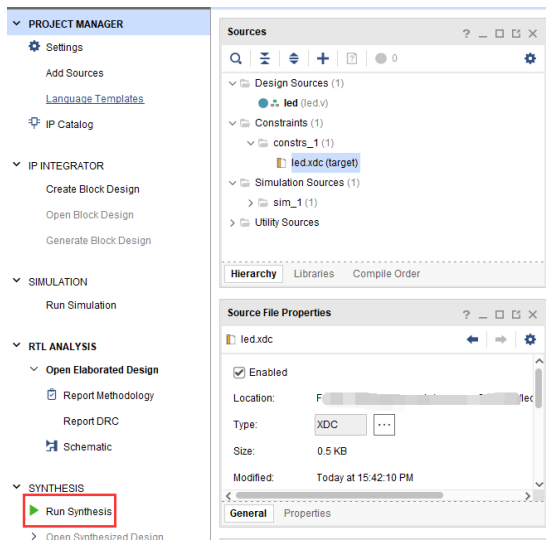
这里需要注意文字的大小写，端口名称是数组的话用{}括起来，端口名称必须和源代码中的名字一致，且端口名字不能和关键字一样。

电平标准中“LVCMOS33”后面的数字指 FPGA 的 BANK 电压，LED 所在 BANK 电压为 3.3 伏，所以电平标准为“LVCMOS33”。Vivado 默认要求为所有 IO 分配正确的电平标准和管脚编号。

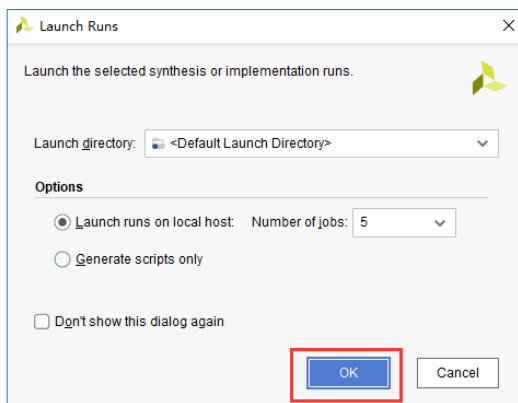
4.5 添加时序约束

一个 FPGA 设计除了管脚分配以外，还有一个重要的约束，那就是时序约束，这里通过向导方式演示如果进行一个时序约束。

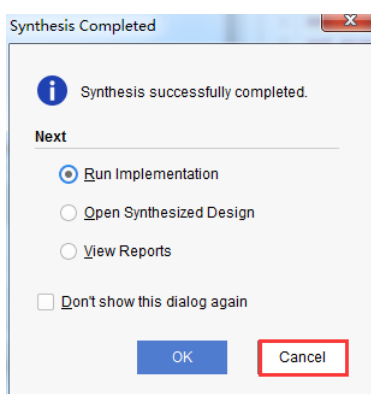
1) 点击“Run Synthesis”开始综合



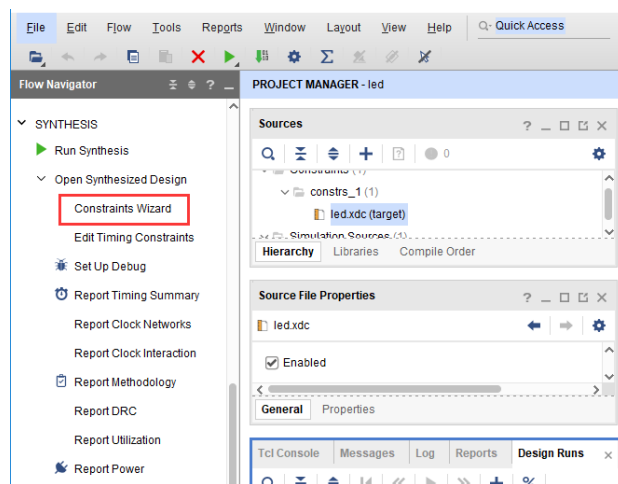
2) 弹出对话框点击“OK”



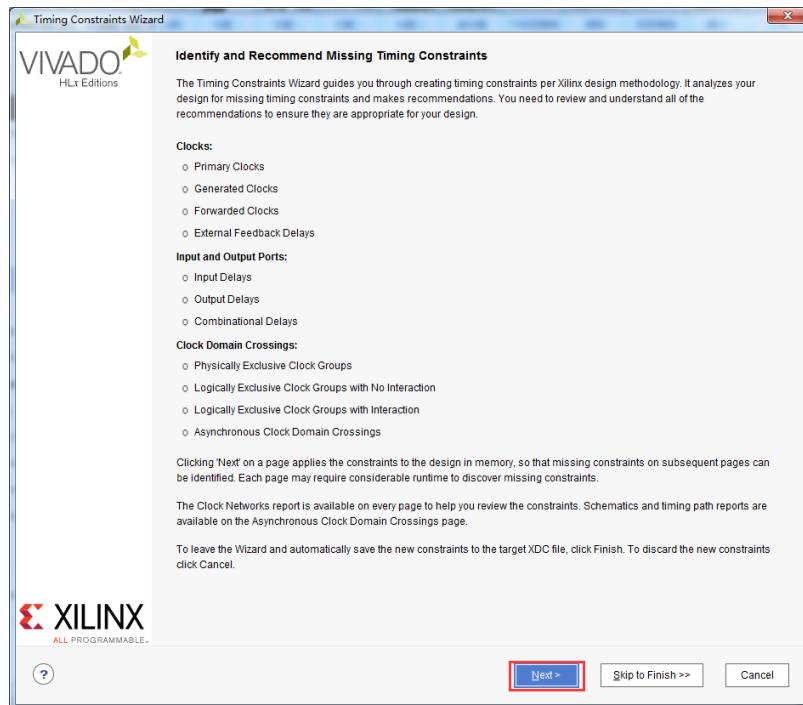
3) 综合完成以后点击 “Cancel”



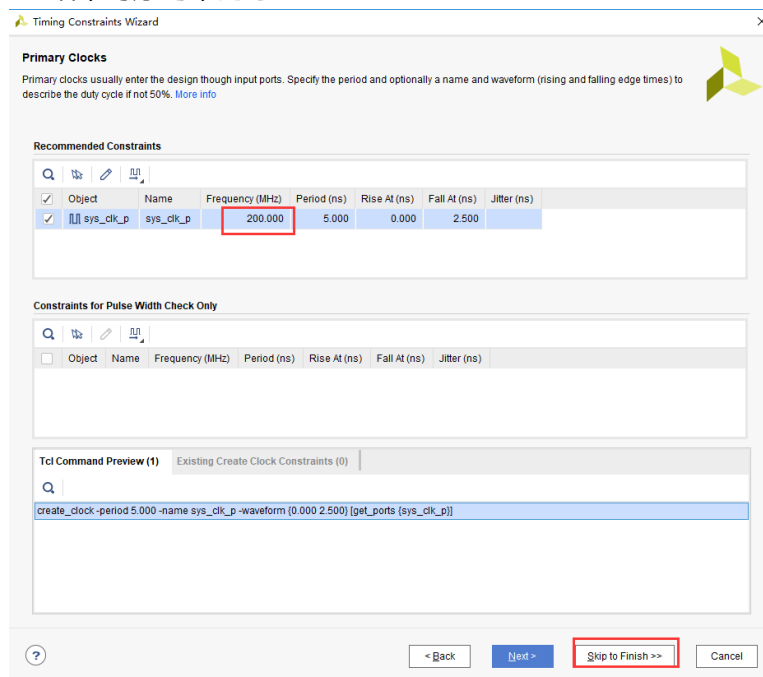
4) 点击 “Constraints Wizard”



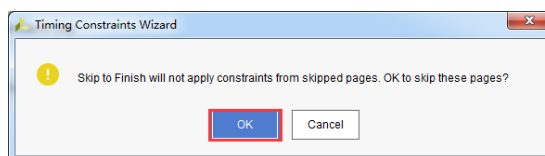
5) 在弹出的窗口中点击 “Next”



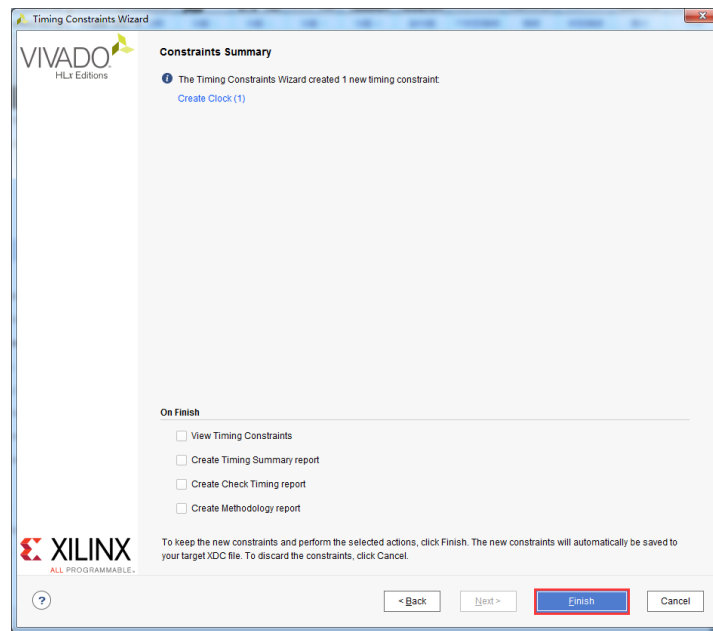
- 6) 时序约束向导分析出设计中的时钟，这里把 “sys_clk_p” 频率设置为 200Mhz，然后点击 “Skip to Finish” 结束时序约束向导。



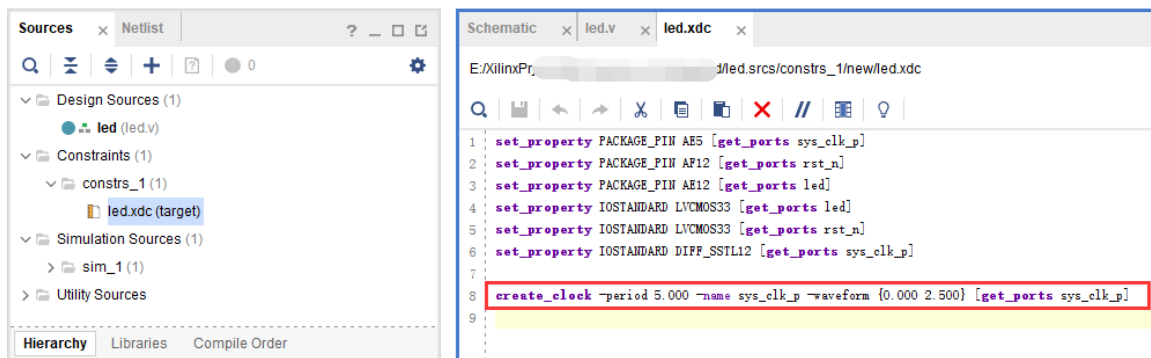
- 7) 弹出的窗口中点击 “OK”



8) 点击 “Finish”

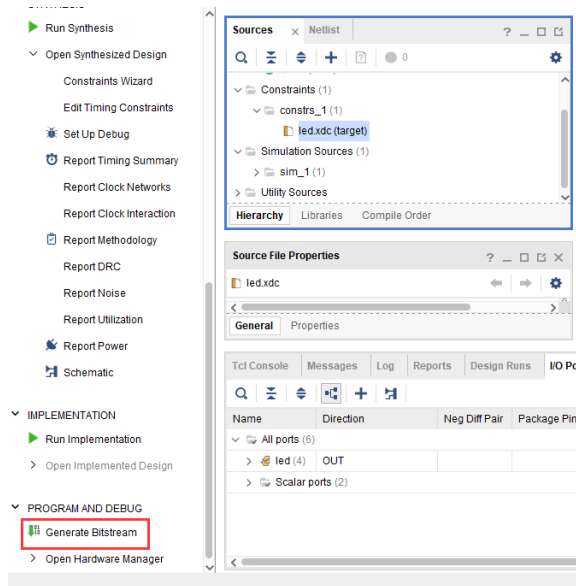


9) 这个时候 led.xdc 文件已经更新，点击 “Reload” 重新加载文件，并保存文件

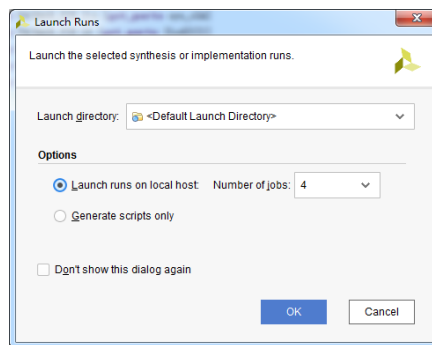


4.6 生成 BIT 文件

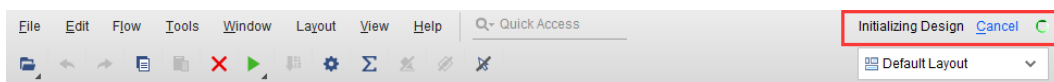
- 1) 编译的过程可以细分为综合、布局布线、生成 bit 文件等，这里我们直接点击 “Generate Bitstream”，直接生成 bit 文件。



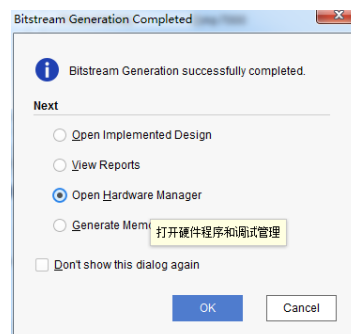
- 2) 在弹出的对话框中可以选择任务数量，这里和 CPU 核心数有关，一般数字越大，编译越快，点击“OK”



- 3) 这个时候开始编译，可以看到右上角有个状态信息，在编译过程中可能会被杀毒软件、电脑管家拦截运行，导致无法编译或很长时间没有编译成功。



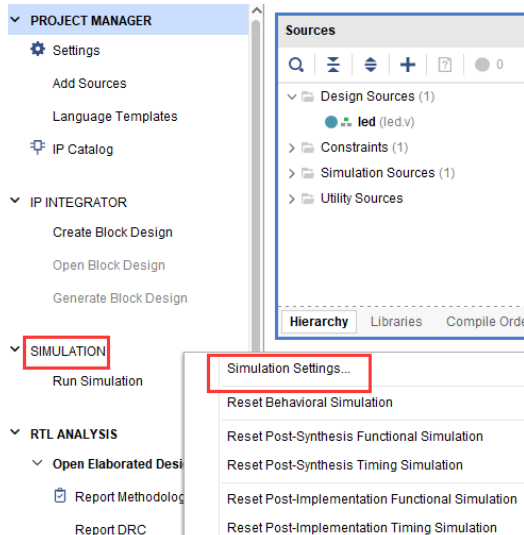
- 4) 编译中没有任何错误，编译完成，弹出一个对话框让我们选择后续操作，可以选择“Open Hardware Manager”，当然，也可以选择“Cancel”，我们这里选择“Cancel”，先不下载。



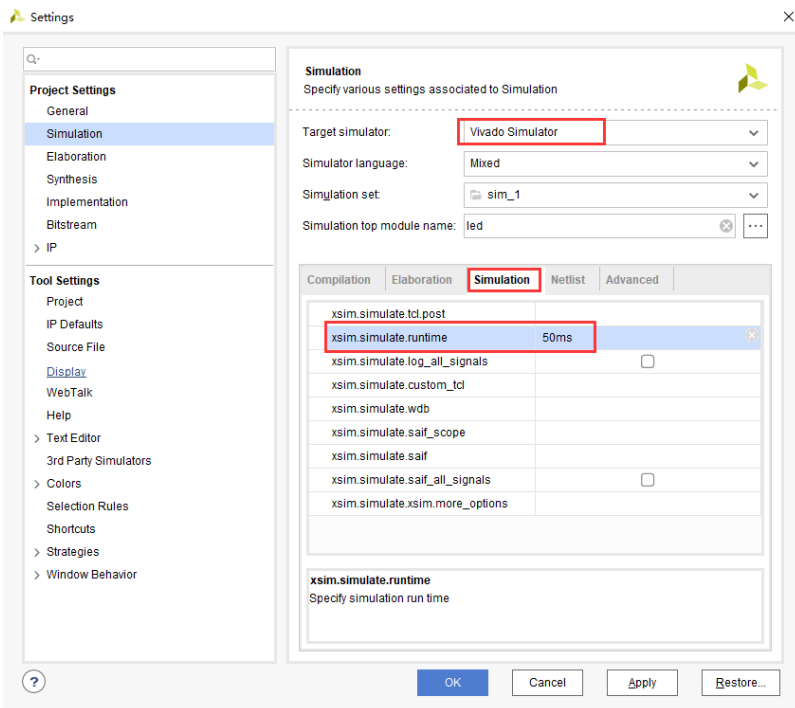
4.7 Vivado 仿真

接下来我们不妨小试牛刀，利用 Vivado 自带的仿真工具来输出波形验证流水灯程序设计结果和我们的预想是否一致（**注意：在生成 bit 文件之前也可以仿真**）。具体步骤如下：

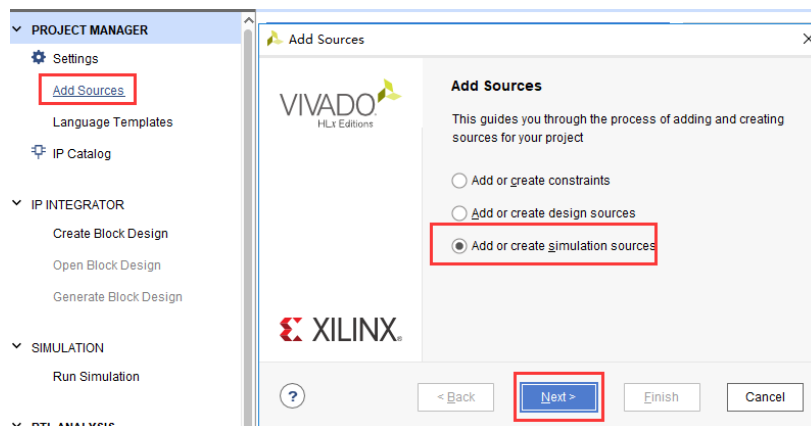
- 1) 设置 Vivado 的仿真配置，右击 SIMULATION 中 Simulation Settings。



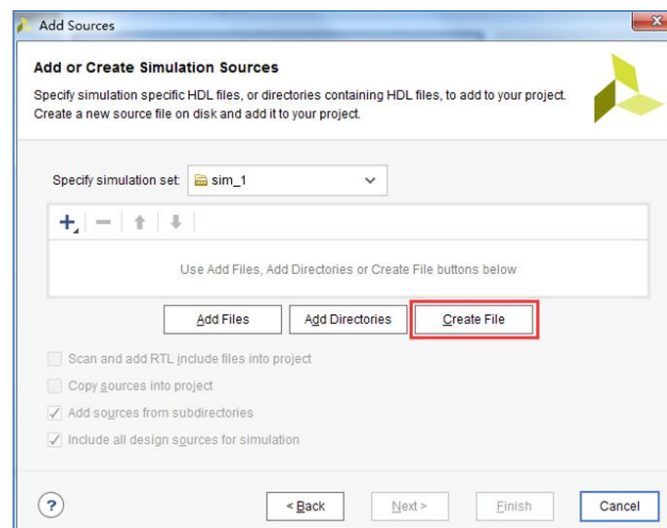
- 2) 在 Simulation Settings 窗口中进行如下图来配置，这里设置成 50ms（根据需要自行设定），其它按默认设置，单击 OK 完成。



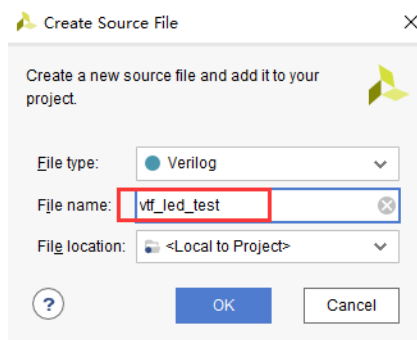
- 3) 添加激励测试文件，点击 Project Manager 下的 Add Sources 图标,按下图设置后单击 Next。



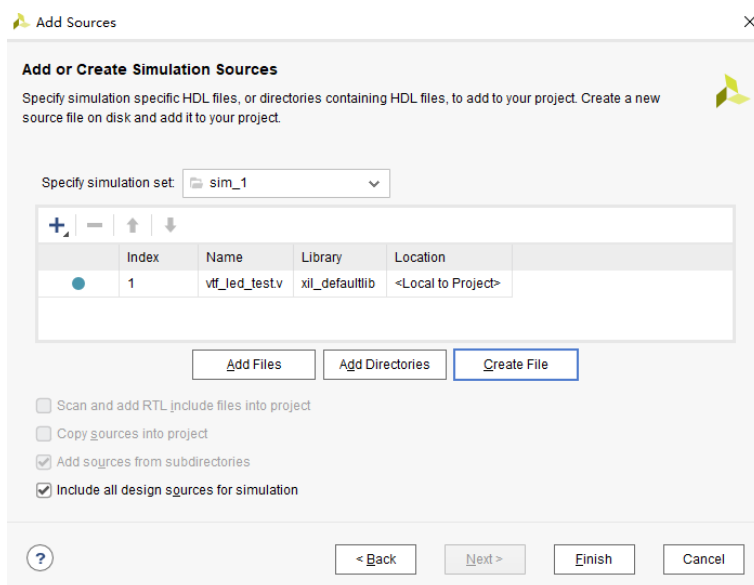
4) 点击 Create File 生成仿真激励文件。



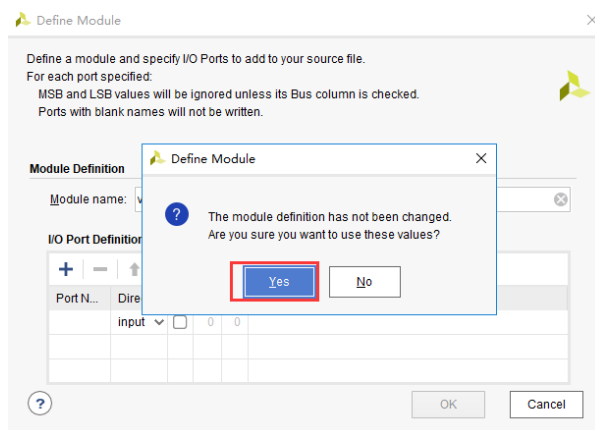
在弹出的对话框中输入激励文件的名字，这里我们输入名为 vtf_led_test。



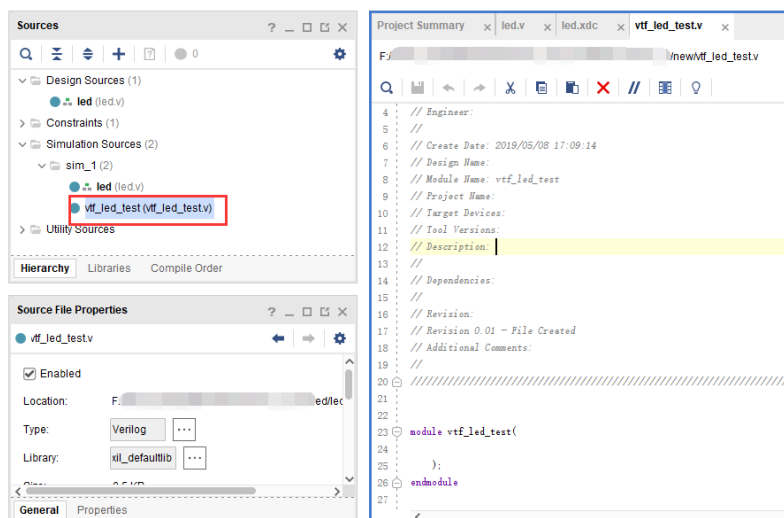
5) 点击 Finish 按钮返回。



这里我们先不添加 IO Ports，点击 OK。



在 Simulation Sources 目录下多了一个刚才添加的 vtf_led_test 文件。双击打开这个文件，可以看到里面只有 module 名的定义，其它都没有。



6) 接下去我们需要编写这个 vtf_led_test.v 文件的内容。首先定义输入和输出信号，然后需要实例化 led_test 模块，让 led_test 程序作为本测试程序的一部分。再添加复位和时钟的激励。

完成后的 vtf_led_test.v 文件如下:

```
`timescale 1ns / 1ps
////////////////////////////////////
////////////////////////////////////
// Module Name: vtf_led_test
////////////////////////////////////
////////////////////////////////////

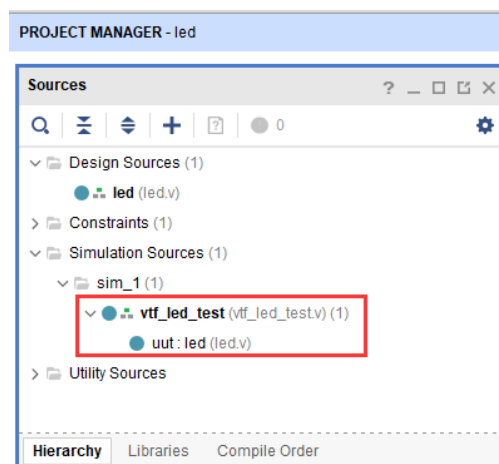
module vtf_led_test;
// Inputs
reg sys_clk_p;
reg rst_n ;
wire sys_clk_n;
// Outputs
wire led;

// Instantiate the Unit Under Test (UUT)
led uut (
    .sys_clk_p(sys_clk_p),
    .sys_clk_n(sys_clk_n),
    .rst_n(rst_n),
    .led(led)
);

initial
begin
// Initialize Inputs
sys_clk_p = 0;
rst_n = 0;
// Wait for global reset to finish
#1000;
rst_n = 1;
end
//Create clock
always #2.5 sys_clk_p = ~ sys_clk_p;
assign sys_clk_n = ~sys_clk_p ;

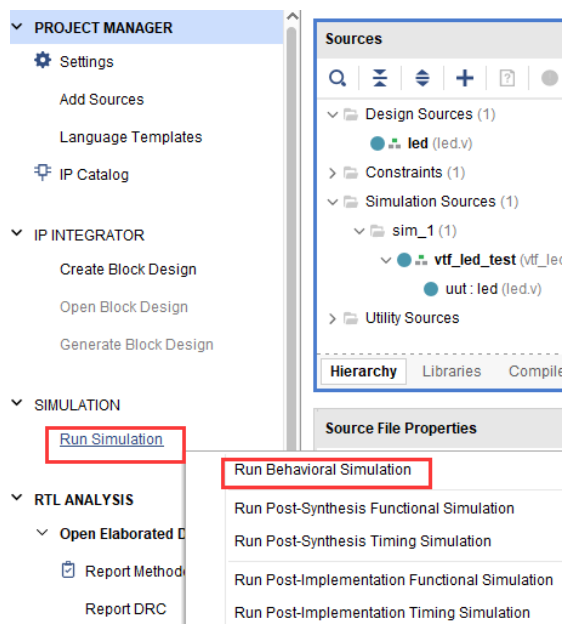
endmodule
```

- 7) 编写好后保存, vtf_led_test.v 自动成了这个仿真 Hierarchy 的顶层了, 它下面是设计文件 led_test.v。



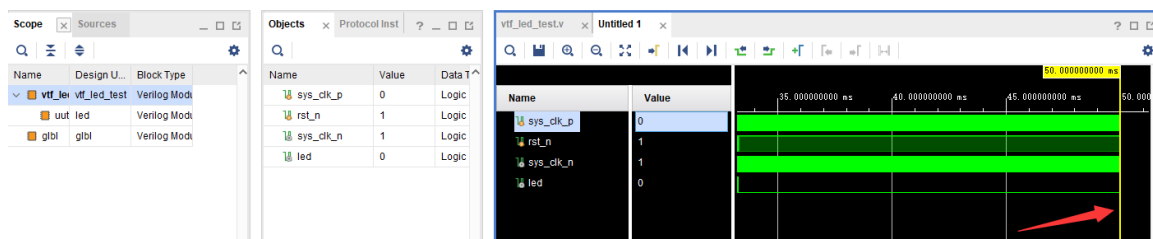
- 8) 点击 Run Simulation 按钮, 再选择 Run Behavioral Simulation。这里我们做一下行为级的仿

真就可以了。

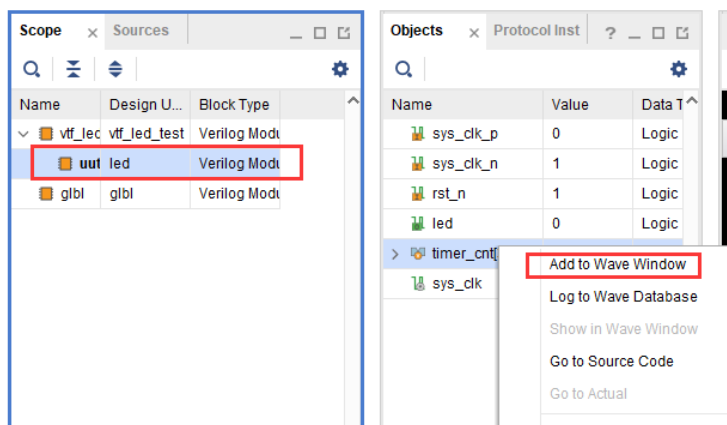


如果没有错误，Vivado 中的仿真软件开始工作了。

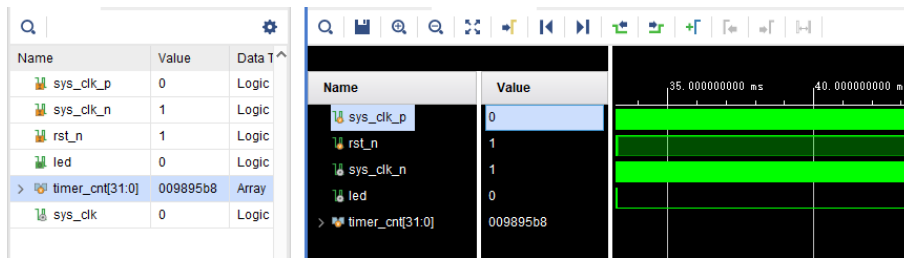
10. 在弹出仿真界面后如下图，界面是仿真软件自动运行到仿真设置的 50ms 的波形。



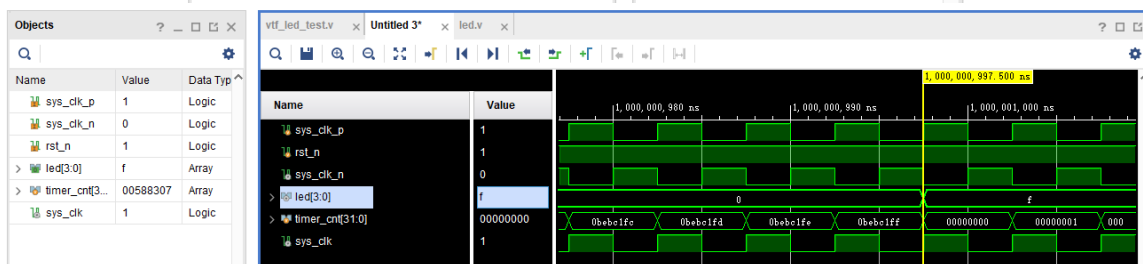
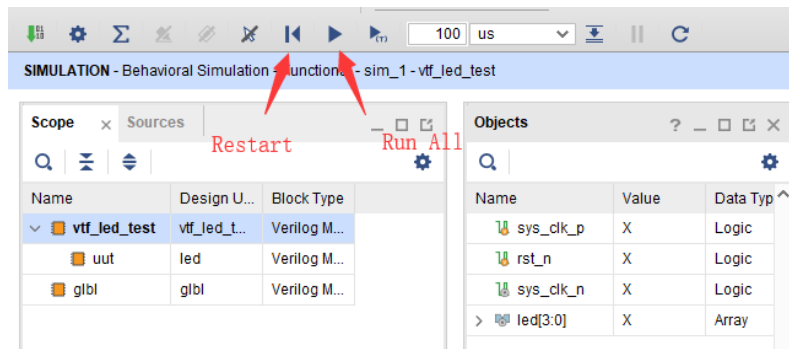
由于 LED[3:0]在程序中设计的状态变化时间长，而仿真又比较耗时，在这里观测 timer[31:0]计数器变化。把它放到 Wave 中观察(点击 Scope 界面下的 uut，再右键选择 Objects 界面下的 timer，在弹出的下拉菜单里选择 Add Wave Window)。



添加后 timer 显示在 Wave 的波形界面上，如下图所示。



11. 点击如下标注的 Restart 按钮复位一下，再点击 Run All 按钮。（需要耐心!!!），可以看到仿真波形与设计相符。（注意：仿真的时间越长，仿真的波形文件占用的磁盘空间越大，波形文件在工程目录的 xx.sim 文件夹）



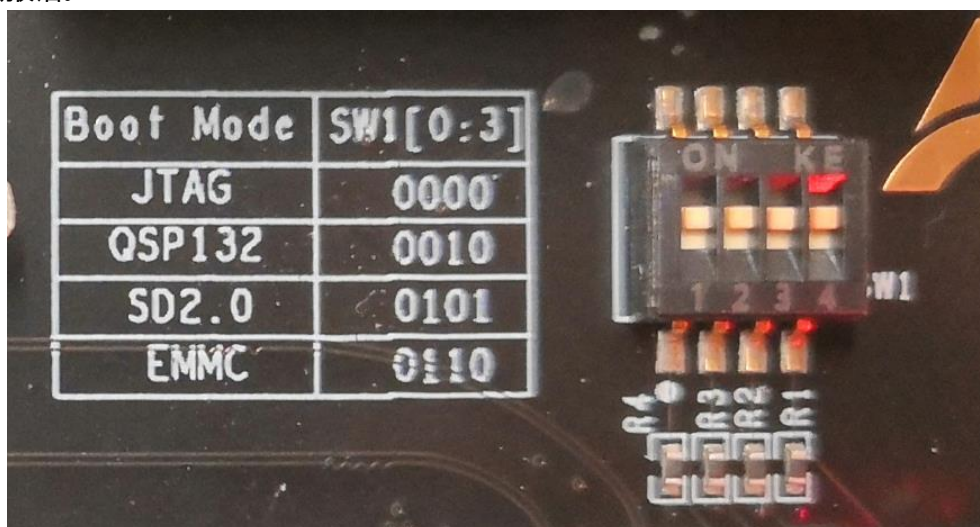
我们可以看到 led 的信号会变成 1，说明 LED 灯会变亮。

4.8 下载

1) 连接好开发板的 JTAG 接口，给开发板上电

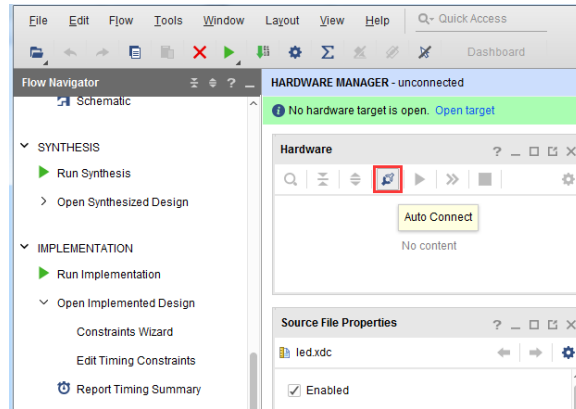


注意拨码开关要选择 JTAG 模式，也就是全部拨到“ON”，“ON”代表的值是 0，不用 JTAG 模式，下载会报错。

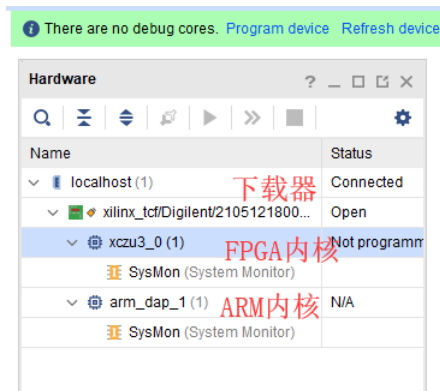


Boot Mode	SW1[0:3]
JTAG	0000
QSP132	0010
SD2.0	0101
EMMC	0110

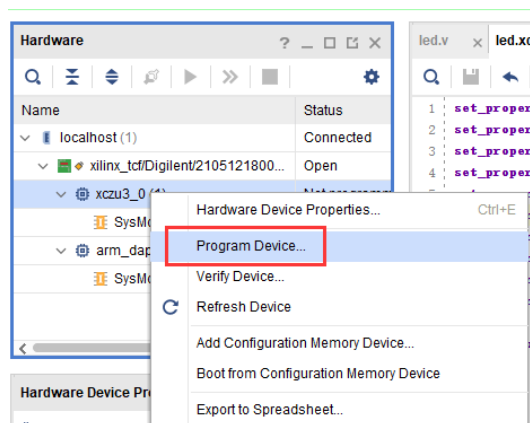
- 2) 在“HARDWARE MANAGER”界面点击“Auto Connect”，自动连接设备



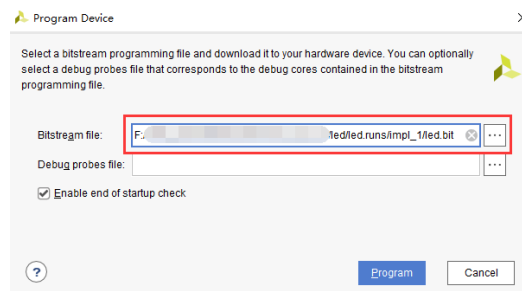
3) 可以看到 JTAG 扫描到 arm 和 FPGA 内核



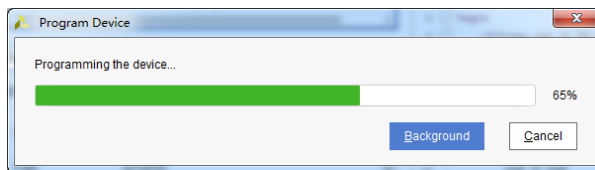
4) 选择芯片，右键 “Program Device...”



5) 在弹出窗口中点击 “Program”



6) 等待下载



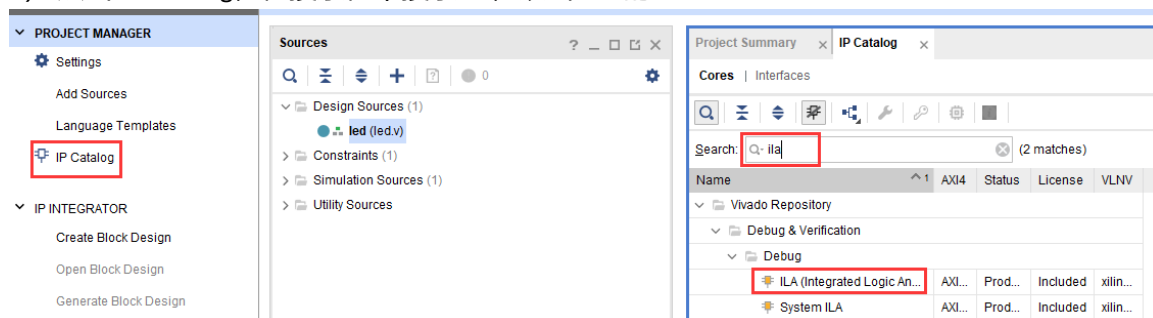
- 7) 下载完成以后，我们可以看到 PL LED 开始每秒变化一次。到此为止 Vivado 简单流程体验完成。后面的章节会介绍如果把程序烧录到 Flash，需要 PS 系统的配合才能完成，只有 PL 的工程不能直接烧写 Flash。在“体验 ARM，裸机输出”Hello World”一章的常见问题中有介绍。

4.9 在线调试

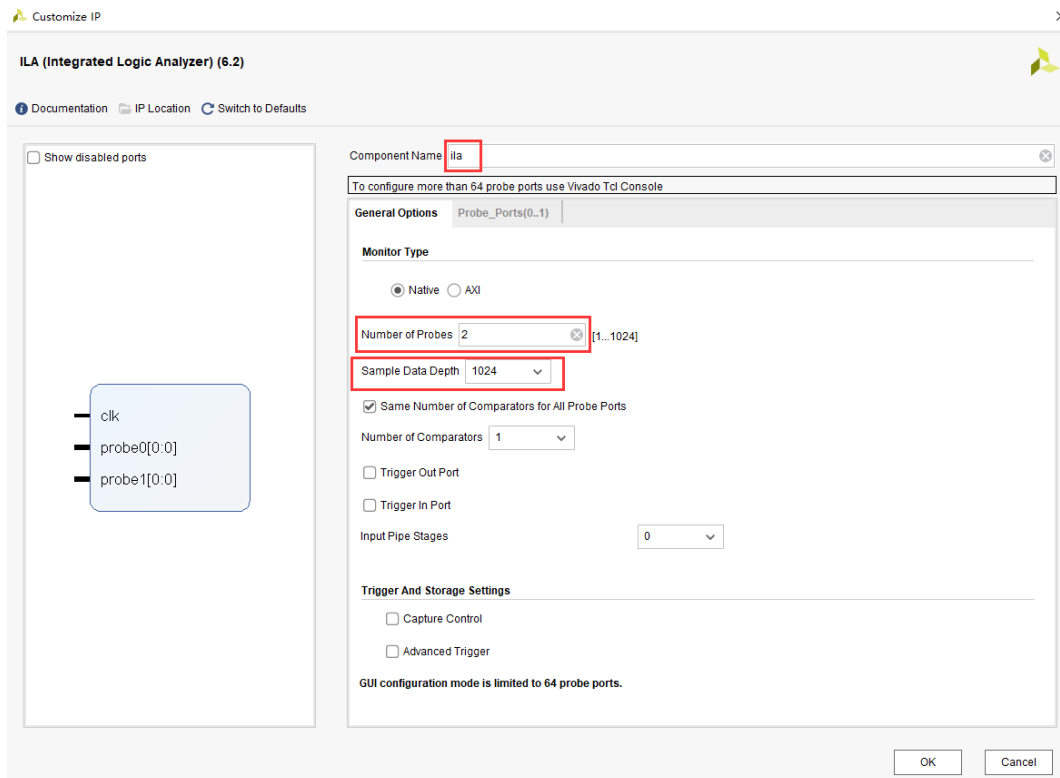
前面介绍了仿真和下载，但仿真并不需要程序烧写到板子，是比较理想化的结果，下面介绍 Vivado 在线调试方法，观察内部信号的变化。Vivado 有内嵌的逻辑分析仪，叫做 ILA，可以用于在线观察内部信号的变化，对于调试有很大帮助。在本实验中我们观察 timer_cnt 和 led 的信号变化。

4.9.1 添加 ILA IP 核

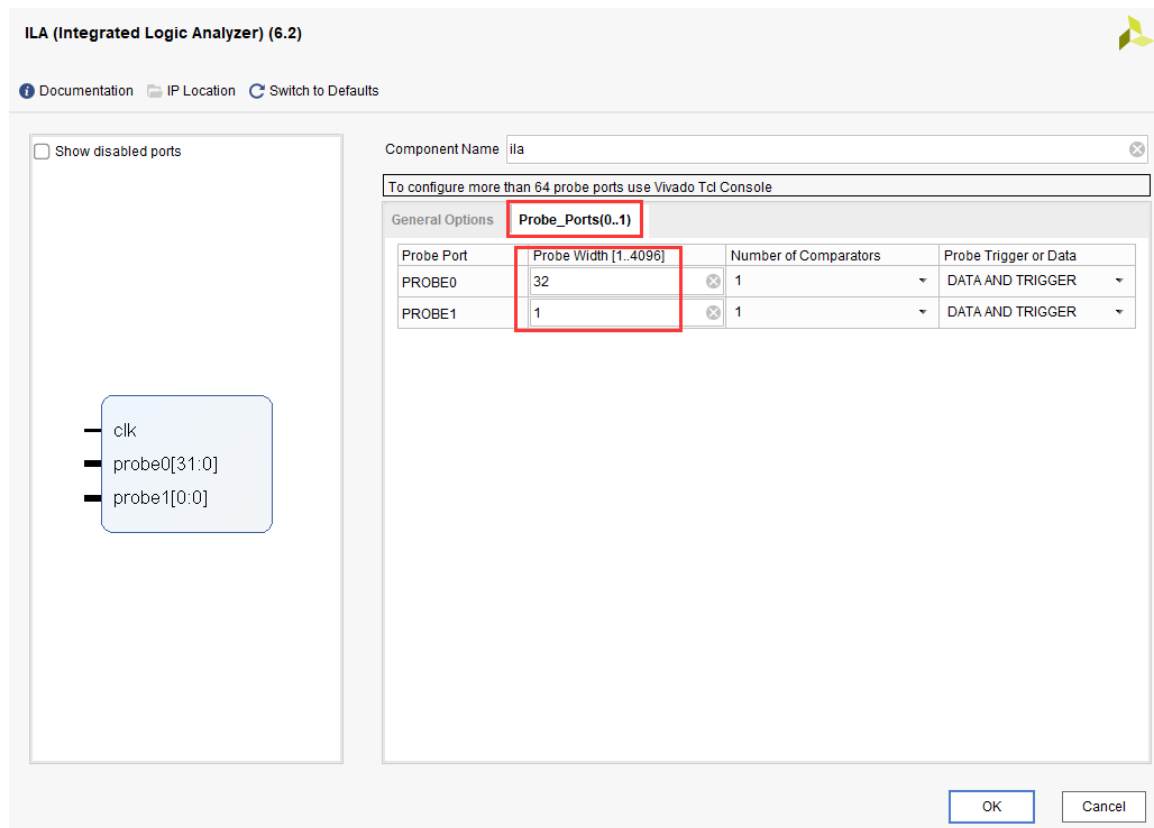
- 1) 点击 IP Catalog，在搜索框中搜索 ila，双击 ILA 的 IP



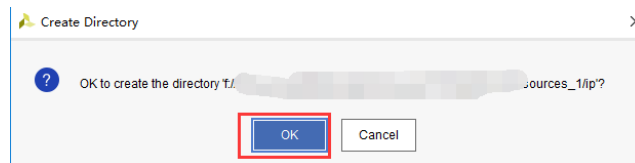
- 2) 修改名称为 ila，由于要采样两个信号，Probes 的数量设置为 2，Sample Data Depth 指的是采样深度，设置的越高，采集的信号越多，同样消耗的资源也会越多。



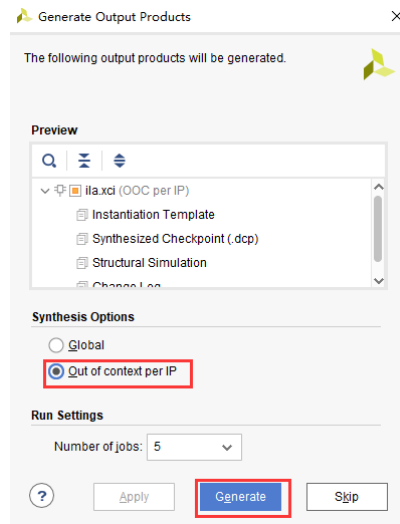
- 3) 在 Probe_Ports 页面，设置 Probe 的宽度，设置 PROBE0 位宽为 32，用于采样 timer_cnt，设置 PROBE1 位宽为 1，用于采样 led。点击 OK



弹出界面，选择 OK



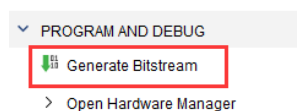
再如下设置，点击 Generate



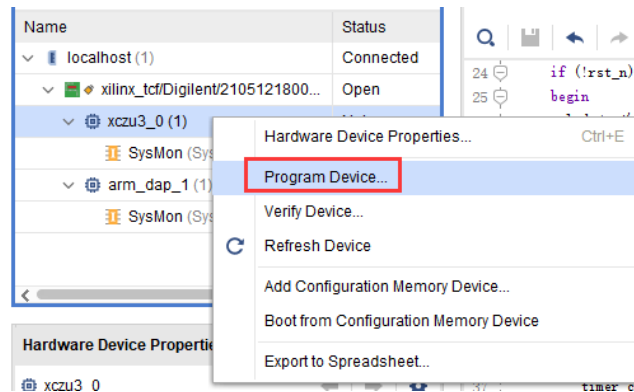
4) 在 led.v 中例化 ila，并保存

```
36 else if(timer_cnt >= 32'd49_999_999)
37 begin
38     led <= ~led;
39     timer_cnt <= 32'd0;
40 end
41 else
42 begin
43     led <= led;
44     timer_cnt <= timer_cnt + 32'd1;
45 end
46 end
47 end
48
49 //Instantiate ila in source file
50 ila ila_inst(
51     .clk(sys_clk),
52     .probe0(timer_cnt),
53     .probe1(led)
54 );
55
56 endmodule
57
```

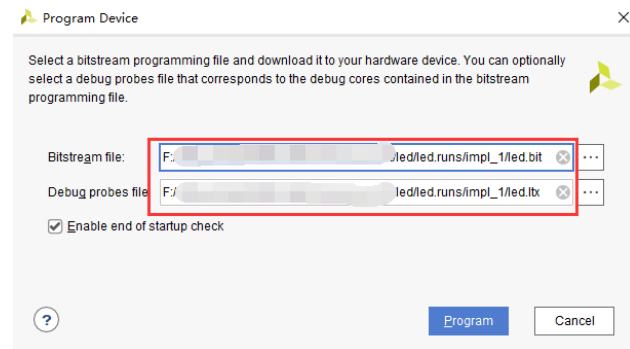
5) 重新生成 Bitstream



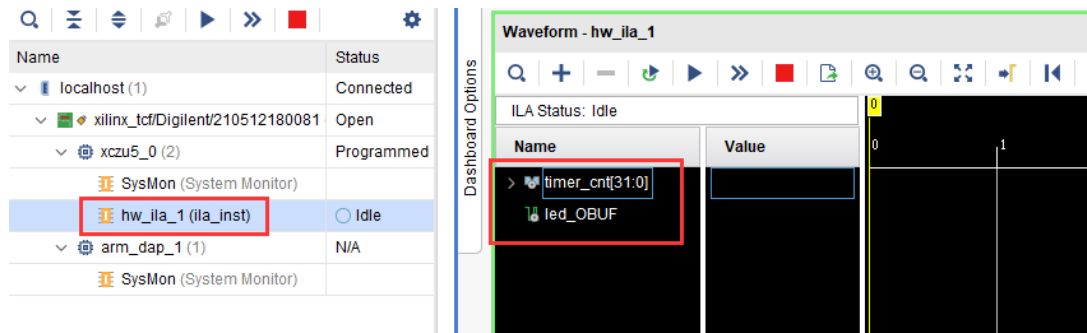
6) 下载程序



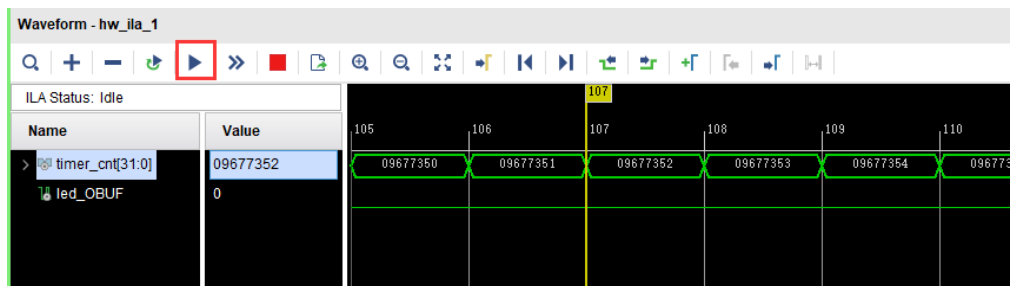
7) 这时候看到有 bit 和 ltx 文件，点击 program



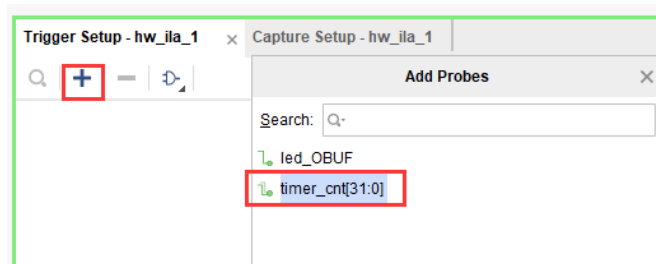
8) 此时弹出在线调试窗口，出现了我们添加的信号



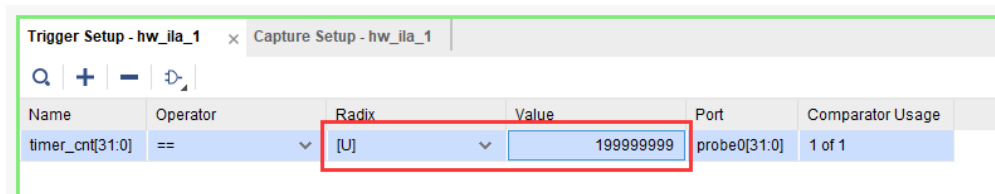
点击运行按钮，出现信号的数据



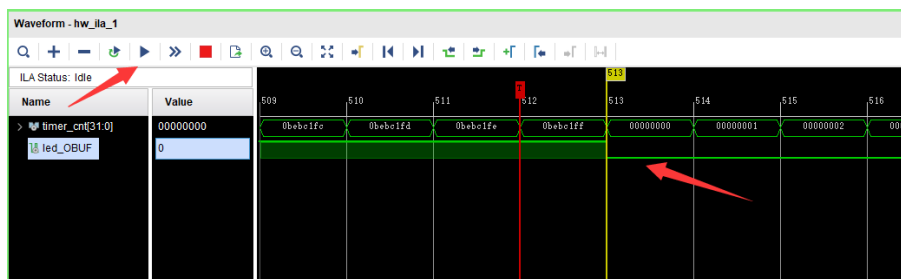
也可以触发采集，在 Trigger Setup 窗口点击“+”，深度选择 timer_cnt 信号



将 Radix 改为 U，也就是十进制，在 Value 中设置为 199999999，也就是 timer_cnt 计数的最大值



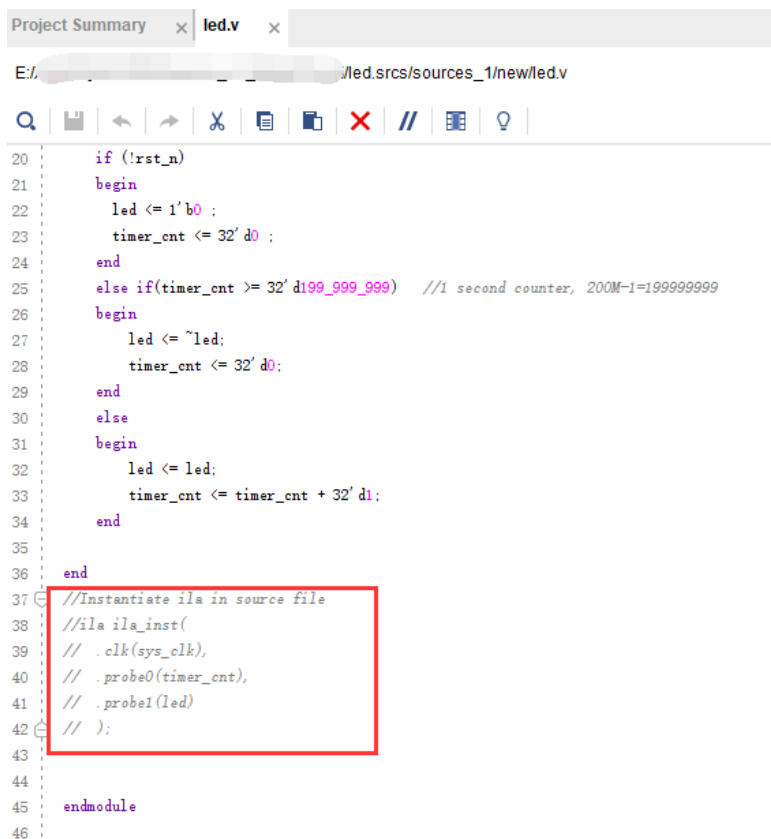
再次点击运行，即可以看到触发成功，此时 timer_cnt 显示为十六进制，而 led 也在此时翻转。



4.9.2 MARK DEBUG

上面介绍了添加 ILA IP 的方式在线调试，下面介绍在代码中添加综合属性，实现在线调试。

- 1) 首先打开 led.v，将 ila 的例化部分注释掉



```

Project Summary x led.v x
E:/.../led.srcs/sources_1/new/led.v

20   if (!rst_n)
21   begin
22       led <= 1'b0 ;
23       timer_cnt <= 32'd0 ;
24   end
25   else if(timer_cnt >= 32'd199_999_999) //1 second counter, 200M-1=199999999
26   begin
27       led <= ~led;
28       timer_cnt <= 32'd0;
29   end
30   else
31   begin
32       led <= led;
33       timer_cnt <= timer_cnt + 32'd1;
34   end
35
36 end
37 //Instantiate ila in source file
38 //ila ila_inst(
39 // .clk(sys_clk),
40 // .probe0(timer_cnt),
41 // .probe1(led)
42 // );
43
44
45 endmodule
46

```

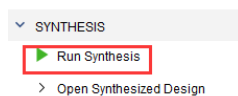
2) 在 led 和 timer_cnt 的定义前面添加(* MARK_DEBUG="true" *), 保存文件。

```

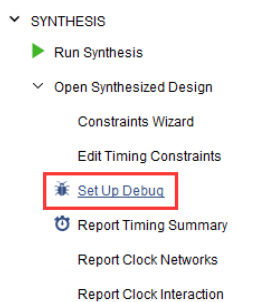
`timescale 1ns / 1ps
module led(
    //Differential system clock
    input sys_clk_p,
    input sys_clk_n,
    input rst_n,
    (* MARK_DEBUG="true" *) output reg led
);
    (* MARK_DEBUG="true" *)reg[31:0] timer_cnt;
    wire sys_clk ;

```

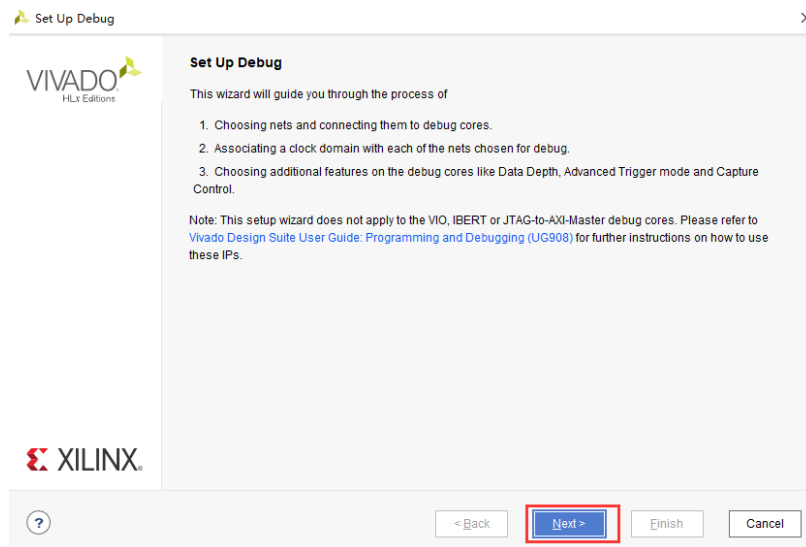
3) 点击综合



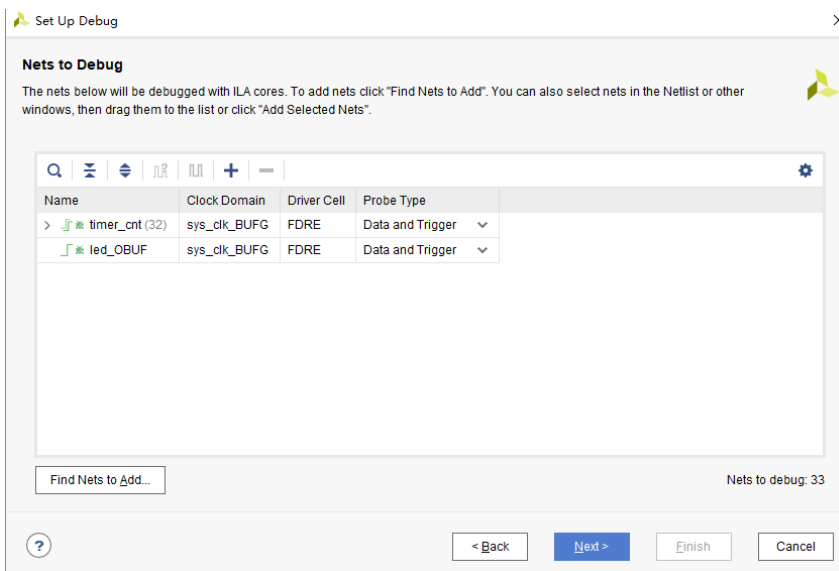
4) 综合结束后, 点击 Set Up Debug



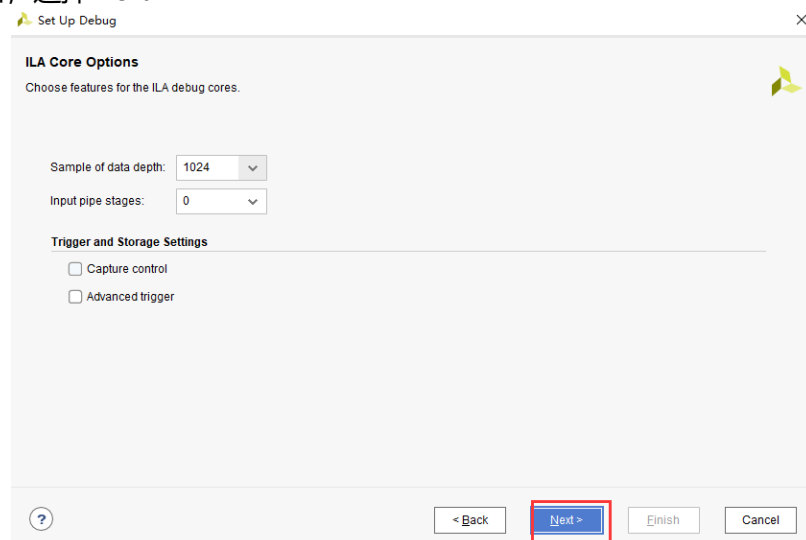
5) 弹出的窗口点击 Next



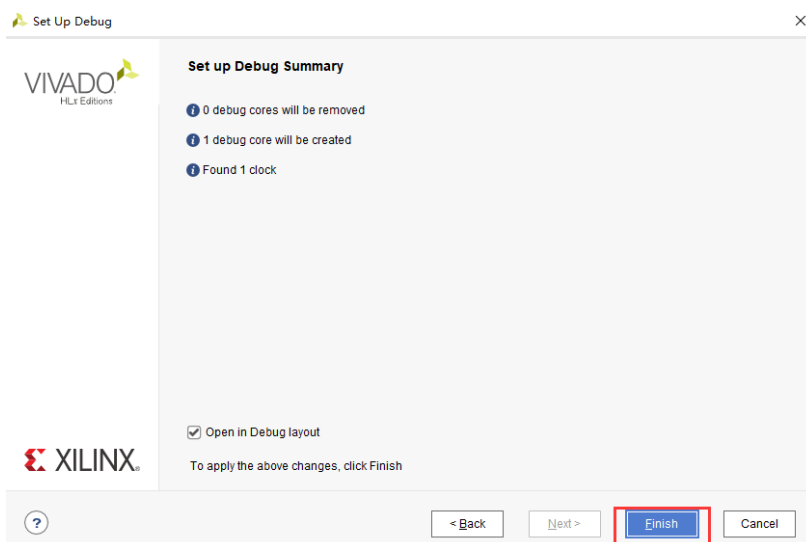
按照默认点击 Next



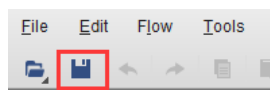
采样深度窗口，选择 Next



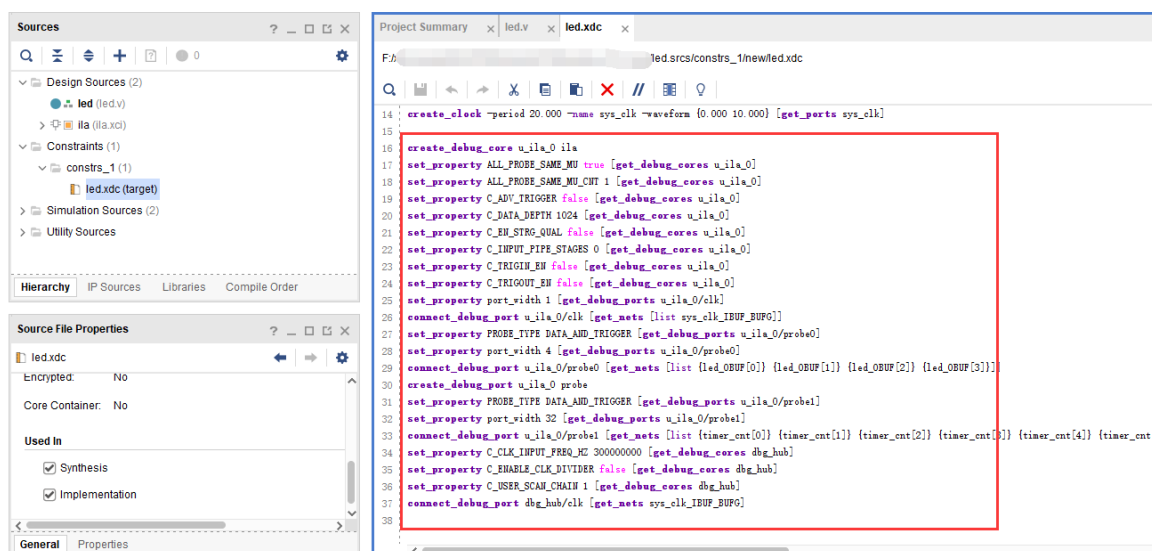
点击 Finish



点击保存



在 xdc 文件中即可看到添加的 ila 核约束



5) 重新生成 bitstream



6) 调试方法与前面一样，不再赘述。

4.10 实验总结

本章节介绍了如何在 PL 端开发程序，包括工程建立，约束，仿真，在线调试等方法，在后续的代码开发方式中皆可参考此方法。

第五章 Vivado 下 PLL 实验

实验 Vivado 工程为 “pll_test”。

很多初学者看到板上只有一个 200Mhz 时钟输入的时候都产生疑惑，时钟怎么是 200Mhz？如果要工作在 100Mhz、150Mhz 怎么办？其实在很多 FPGA 芯片内部都集成了 PLL，其他厂商可能不叫 PLL，但是也有类似的功能模块，通过 PLL 可以倍频分频，产生其他很多时钟。本实验通过调用 PLL IP core 来学习 PLL 的使用、vivado 的 IP core 使用方法。

5.1 实验原理

PLL(phase-locked loop)，即锁相环。是 FPGA 中的重要资源。由于一个复杂的 FPGA 系统往往需要多个不同频率，相位的时钟信号。所以，一个 FPGA 芯片中 PLL 的数量是衡量 FPGA 芯片能力的重要指标。FPGA 的设计中，时钟系统的 FPGA 高速的设计极其重要，一个低抖动，低延迟的系统时钟会增加 FPGA 设计的成功率。

本实验将通过使用 PLL，输出一个方波到开发板上的扩展口，来给大家演示在 Vivado 软件里使用 PLL 的方法。

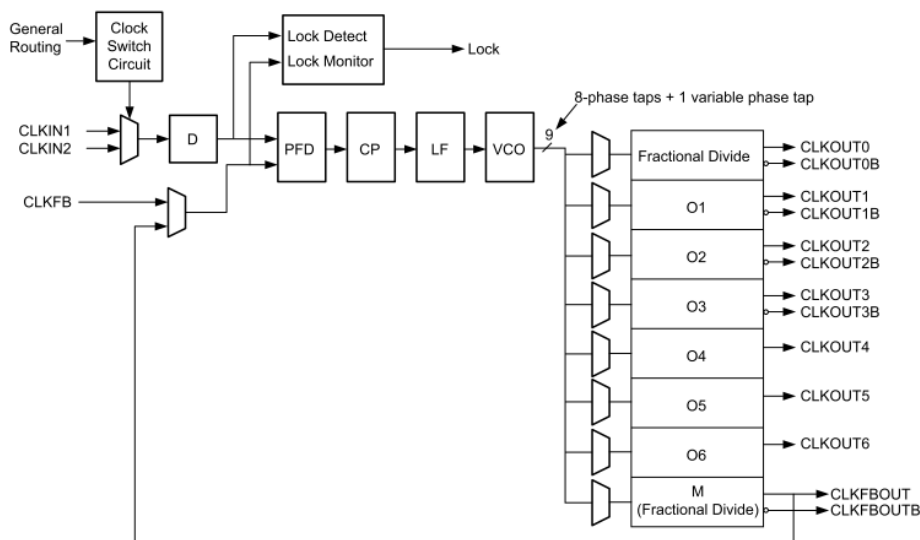
Ultrascale+系列的 FPGA 使用了专用的全局(Global)和区域(Regional)IO 和时钟资源来管理设计中各种的时钟需求。Clock Management Tiles(CMT)提供了时钟合成(Clock frequency synthesis)，倾斜矫正(deskew)，过滤抖动(jitter filtering)功能。

每个 CMTs 包含一个 MMCM(mixed-mode clock manager)和一个 PLL。如下图所示，CMT 的输入可以是 BUFR，IBUFG，BUFG，GT，BUFH，本地布线（不推荐使用），输出需要接到 BUFG 或者 BUFG 后再使用

➤ 混合模式时钟管理器(MMCM)

MMCM 用于在与给定输入时钟有设定的相位和频率关系的情况下，生成不同的时钟信号。MMCM 提供了广泛而强大的时钟管理功能，

MMCM 内部的功能框图如下图所示：



➤ 数字锁相环(PLL)

锁相环 (PLL) 主要用于频率综合。使用一个 PLL 可以从一个输入时钟信号生成多个时钟信号。与 MMCM 相比，不能进行时钟的 deskew，不具备高级相位调整，倍频器和分频器可调范围较小等。

PLL 功能框图如下图所示：

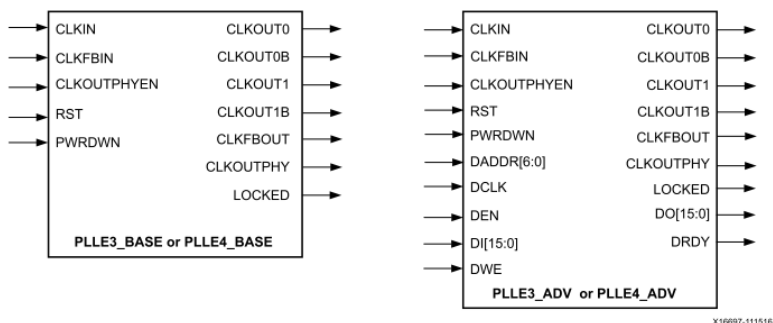


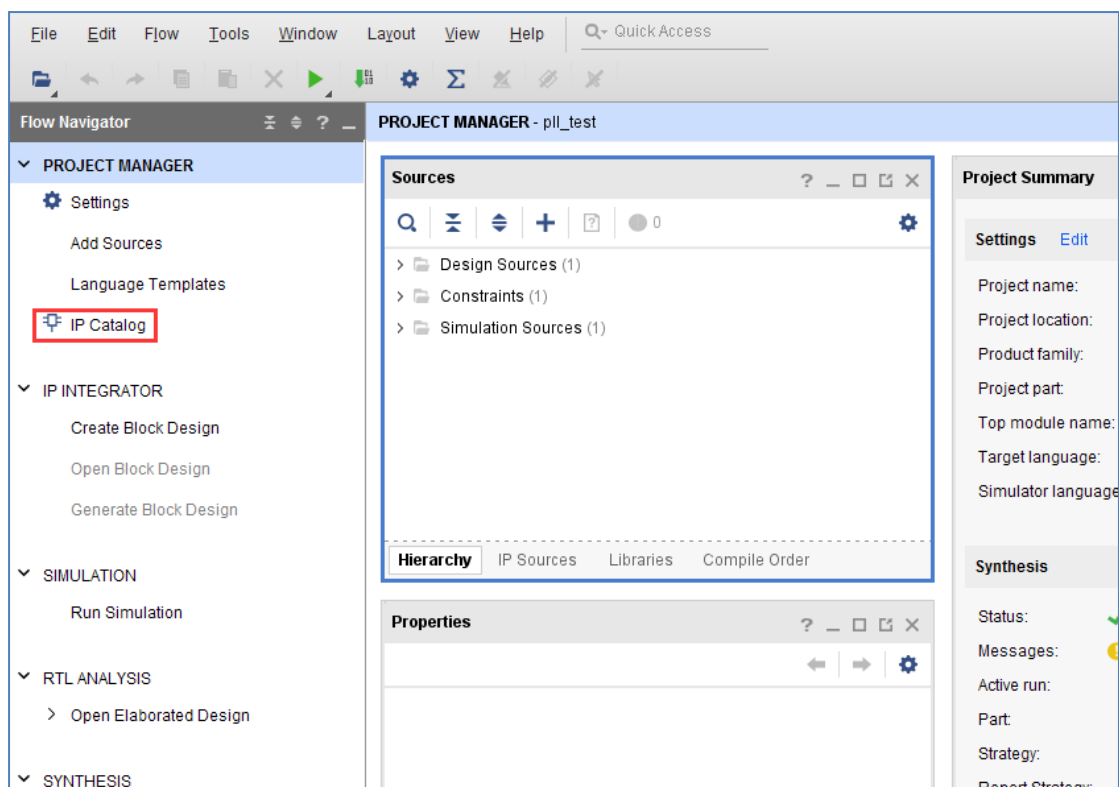
Figure 3-16: PLL Primitives

想了解更多的时钟资源，建议大家看看 Xilinx 提供的文档"7 Series FPGAs Clocking Resources User Guide"。

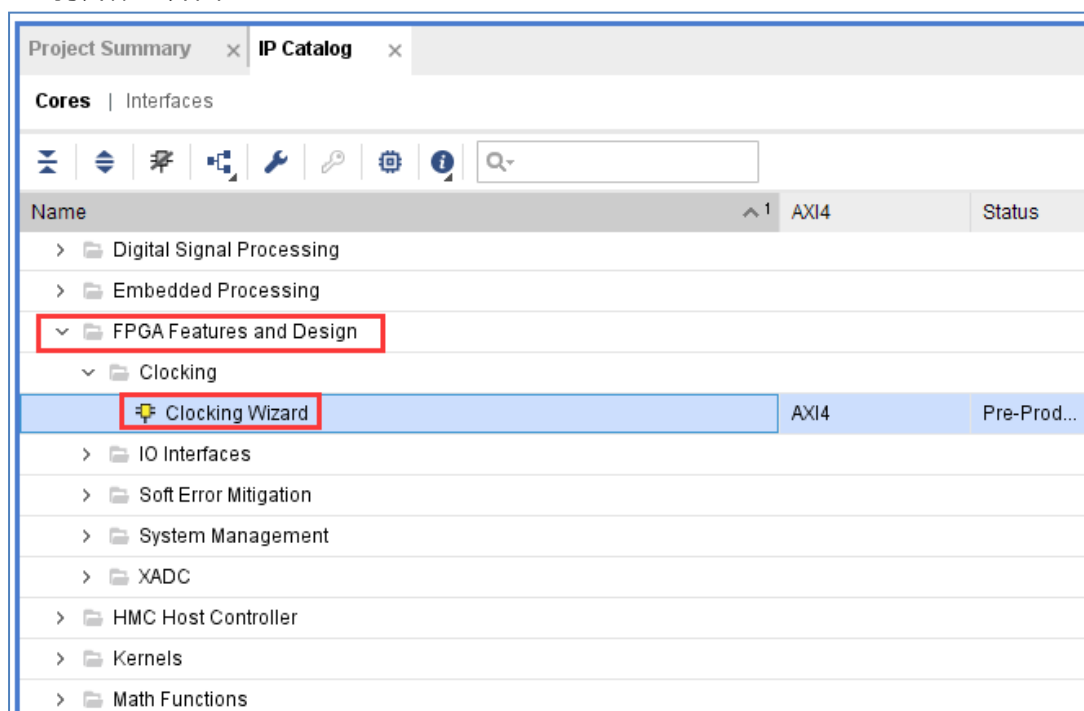
5.2 创建 Vivado 工程

本实验中为大家演示如果调用 Xilinx 提供的 PLL IP 核来产生不同频率的时钟，并把其中的一个时钟输出到 FPGA 外部 IO 上，下面为程序设计的详细步骤。

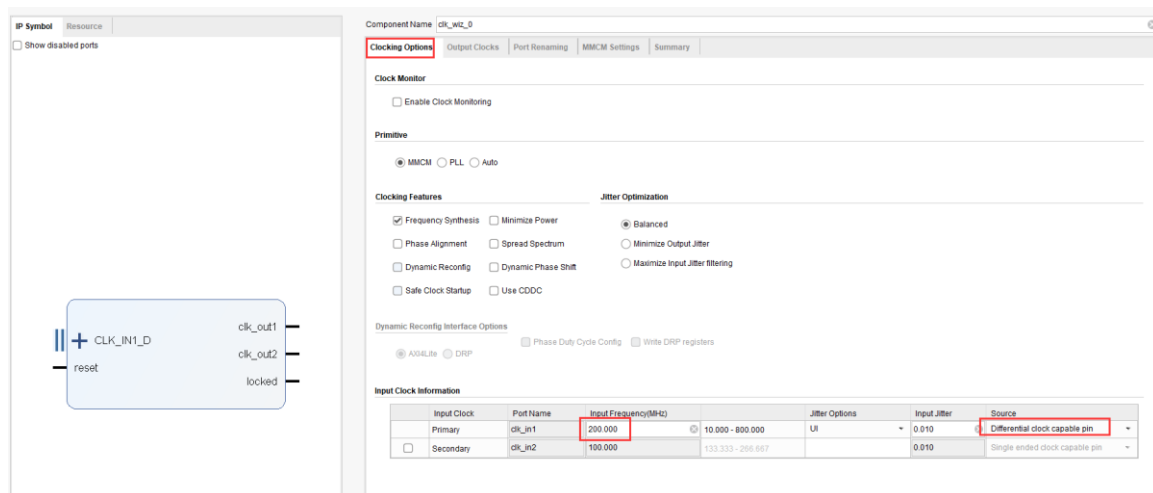
1) 新建一个 pll_test 的工程，点击 Project Manager 界面下的 IP Catalog。



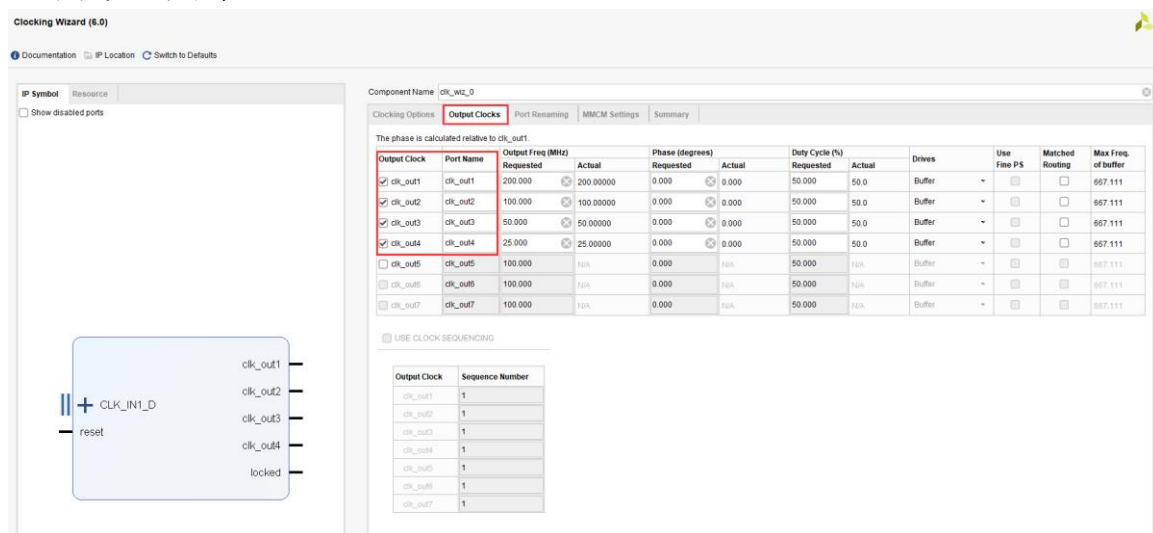
- 2) 再在 IP Catalog 界面里选择 FPGA Features and Design\Clocking 下面的 Clocking Wizard，双击打开配置界面。



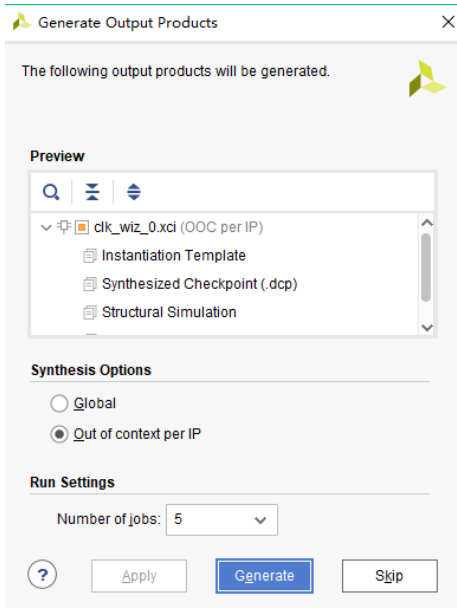
- 3) 默认这个 Clocking Wizard 的名字为 clk_wiz_0，这里我们不做修改。在第一个界面 Clocking Options 里，输入的时钟频率为 200Mhz，并选择 Differential clock capable pin，因为时钟输入是差分的。



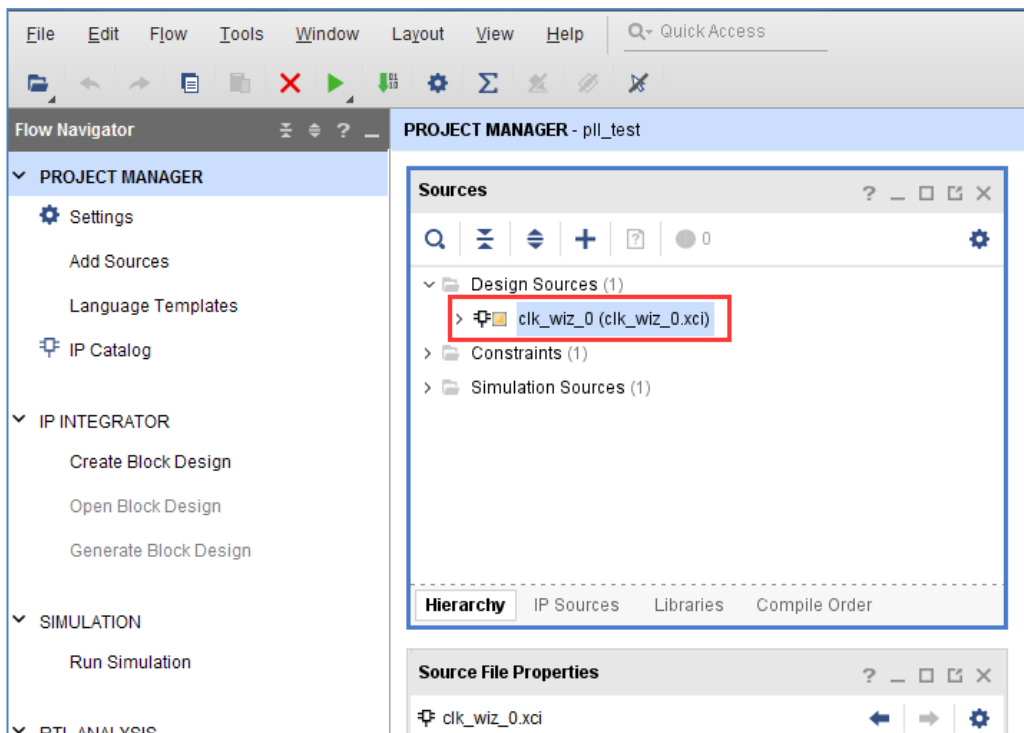
- 4) 在 Output Clocks 界面里选择 clk_out1~clk_out4 四个时钟的输出, 频率分别为 200MHz, 100MHz, 50MHz, 25MHz。这里还可以设置时钟输出的相位, 我们不做设置, 保留默认相位, 点击 OK 完成,



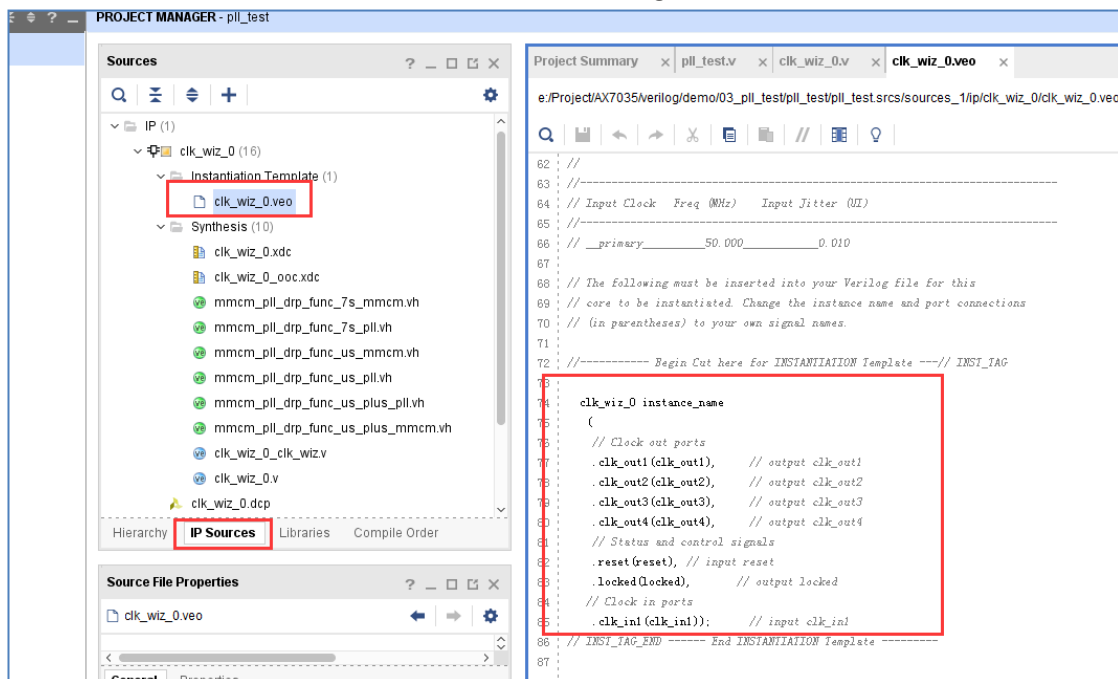
- 5) 在弹出的对话框中点击 Generate 按钮生成 PLL IP 的设计文件。



- 6) 这时一个 clk_wiz_0.xci 的 IP 会自动添加到我们的 pll_test 项目中, 用户可以双击它来修改这个 IP 的配置。



选择 IP Sources 这页, 然后双击打开 clk_wiz_0.veo 文件, 这个文件里提供了这个 IP 的实例化模板。我们只需要把框框的中内容拷贝到我们 verilog 程序中, 对 IP 进行实例化。



- 7) 我们再来编写一个顶层设计文件来实例化这个 PLL IP, 编写 pll_test.v 代码如下。注意 PLL 的复位是高电平有效, 也就是高电平时一直在复位状态, PLL 不会工作, 这一点很多新手会忽略掉。这里我们将 rst_n 绑定到一个按键上, 而按键是低电平复位, 因此需要反向连接到 PLL 的复位。

```

`timescale 1ns / 1ps
module pll_test(
input    sys_clk_p,           //system clock 200Mhz on board
input    sys_clk_n,           //system clock 200Mhz on board
input    rst_n,               //reset ,low active
output    clk_out              //pll clock output

);

wire      locked;

clk_wiz_0 clk_wiz_0_inst
(
    // Clock out ports
    .clk_out1(),              // output clk_out1
    .clk_out2(),              // output clk_out2
    .clk_out3(),              // output clk_out3
    .clk_out4(clk_out),       // output clk_out4
    // Status and control signals
    .reset(~rst_n),           // input reset
    .locked(locked),          // output locked
    // Clock in ports
    .clk_in1_p(sys_clk_p),    // input clk_in1_p
    .clk_in1_n(sys_clk_n));   // input clk_in1_n

endmodule

```

程序中先用实例化 clk_wiz_0, 把差分 200Mhz 时钟信号输入到 clk_wiz_0 的 clk_in1_p 和 clk_in1_n, 把 clk_out4 的输出赋给 clk_out。

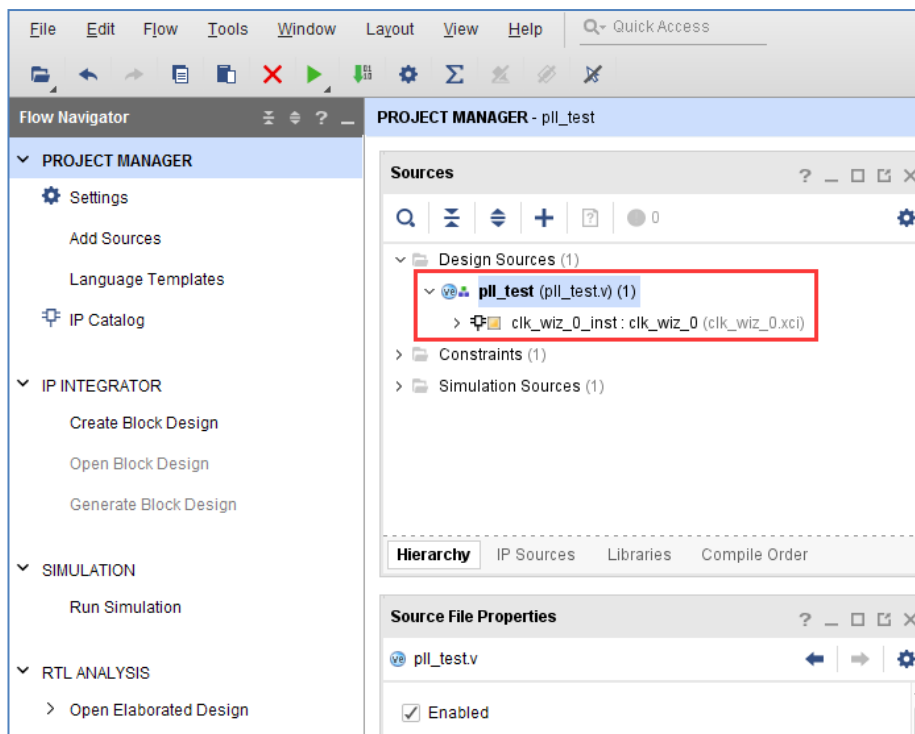
注意：例化的目的是在上一级模块中调用例化的模块完成代码功能，在 Verilog 里例化信号的格式如下：模块名必须和要例化的模块名一致，比如程序中的 clk_wiz_0，包括模块信号名也必须一致，比如 clk_in1, clk_out1, clk_out2.....。连接信号为 TOP 程序跟模块之间传递的信号，模块与模块之间的连接信号不能相互冲突，否则会产生编译错误。

```

模块名 扩展名
(
    .模块信号 1      (连接信号 1),
    .模块信号 2      (连接信号 2),
    .模块信号 3      (连接信号 3),
    .....
    .模块信号 N      (连接信号 N)
);

```

8) 保存工程后，pll_test 自动成为了 top 文件，clk_wiz_0 成为 Pll_test 文件的子模块。



- 9) 再为工程添加 xdc 管脚约束文件 pll.xdc，添加方法参考“PL 的“Hello World”LED 实验”，也可以直接复制以下内容。并编译生成 bitstream。

```
#####Compress Bitstream#####
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property PACKAGE_PIN AE5 [get_ports sys_clk_p]
set_property IOSTANDARD DIFF_SSTL12 [get_ports sys_clk_p]

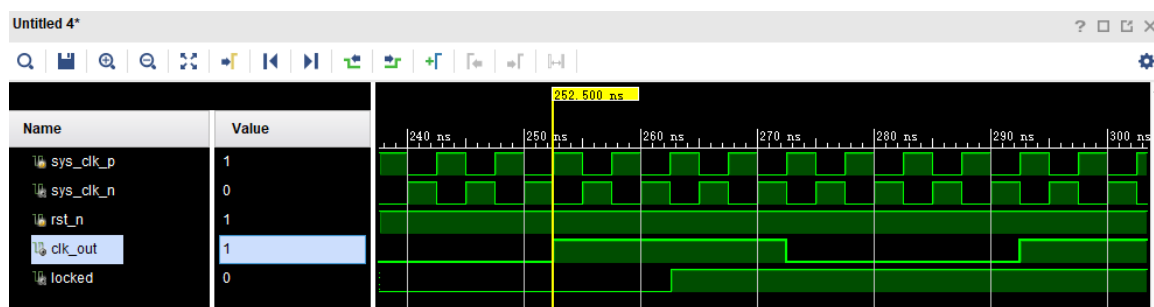
create_clock -period 5.000 -name sys_clk_p -waveform {0.000 2.500}
[get_ports sys_clk_p]

set_property PACKAGE_PIN AF12 [get_ports rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]

set_property PACKAGE_PIN AG11 [get_ports clk_out]
set_property IOSTANDARD LVCMOS33 [get_ports clk_out]
```

5.3 仿真

添加一个 vtf_pll_test 仿真文件，运行后 PLL 的 lock 信号会变高，说明 PLL IP 锁相环已经初始化完成。clk_out 有时钟信号输出，输出的频率为输入时钟频率的 1/8，为 25Mhz。仿真方法可以参考“PL 的“Hello World”LED 实验”。



5.4 板上验证

编译工程并生成 pll_test.bit 文件，再把 bit 文件下载到 FPGA 中，接下去我们就可以用示波器来测量输出时钟波形了。

用示波器探头的地线连接到开发板上的地（开发板 J46 的 PIN1 脚），信号端连接开发板 J46 的 PIN3 脚（测量的时候需要注意，避免示波器表头碰到其它管脚而导致电源和地短路）。

这时我们可以在示波器里看到 25MHz 的时钟波形，波形的幅度为 3.3V，占空比为 1:1，波形显示如下图所示：



如果您想输出其它频率的波形，可以修改时钟的输出为 clk_wiz_0 的 clk_out2 或 clk_out3 或 clk_out4。也可以修改 clk_wiz_0 的 clk_out4 为您想要的频率，这里也需要注意一下，因为时钟的输出是通过 PLL 对输入时钟信号的倍频和分频系数来得到的，所以并不是所有的时钟频率都可以用 PLL 能够精确产生的，不过 PLL 也会自动为您计算实际输出接近的时钟频率。

另外需要注意的是，有些用户的示波器的带宽和采样率太低，会导致测量高频时钟信号的时候，高频部分衰减太大，测量波形的幅度会变低。

第六章 FPGA 片内 RAM 读写测试实验

实验 Vivado 工程为 “ram_test”。

RAM 是 FPGA 中常用的基础模块，可广泛用于缓存数据的情况，同样它也是 ROM，FIFO 的基础。本实验将为大家介绍如何使用 FPGA 内部的 RAM 以及程序对该 RAM 的数据读写操作。

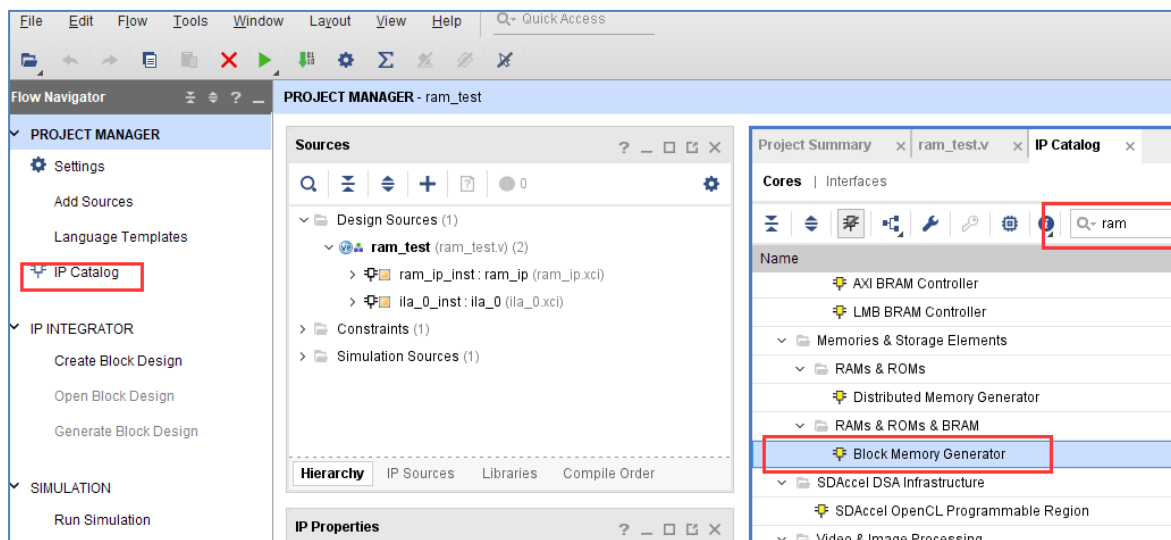
6.1 实验原理

Xilinx 在 VIVADO 里为我们已经提供了 RAM 的 IP 核，我们只需通过 IP 核例化一个 RAM，根据 RAM 的读写时序来写入和读取 RAM 中存储的数据。实验中会通过 VIVADO 集成的在线逻辑分析仪 ila，我们可以观察 RAM 的读写时序和从 RAM 中读取的数据。

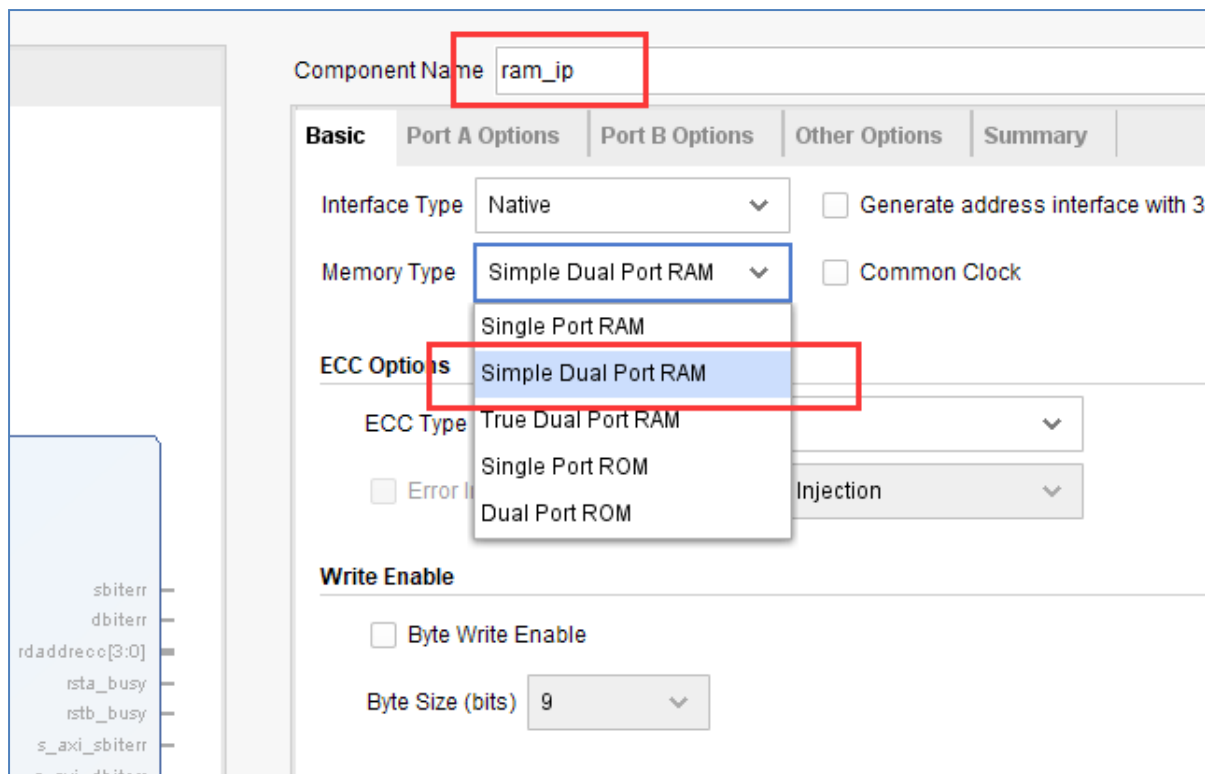
6.2 创建 Vivado 工程

在添加 RAM IP 之前先新建一个 ram_test 的工程，然后在工程中添加 RAM IP，方法如下：

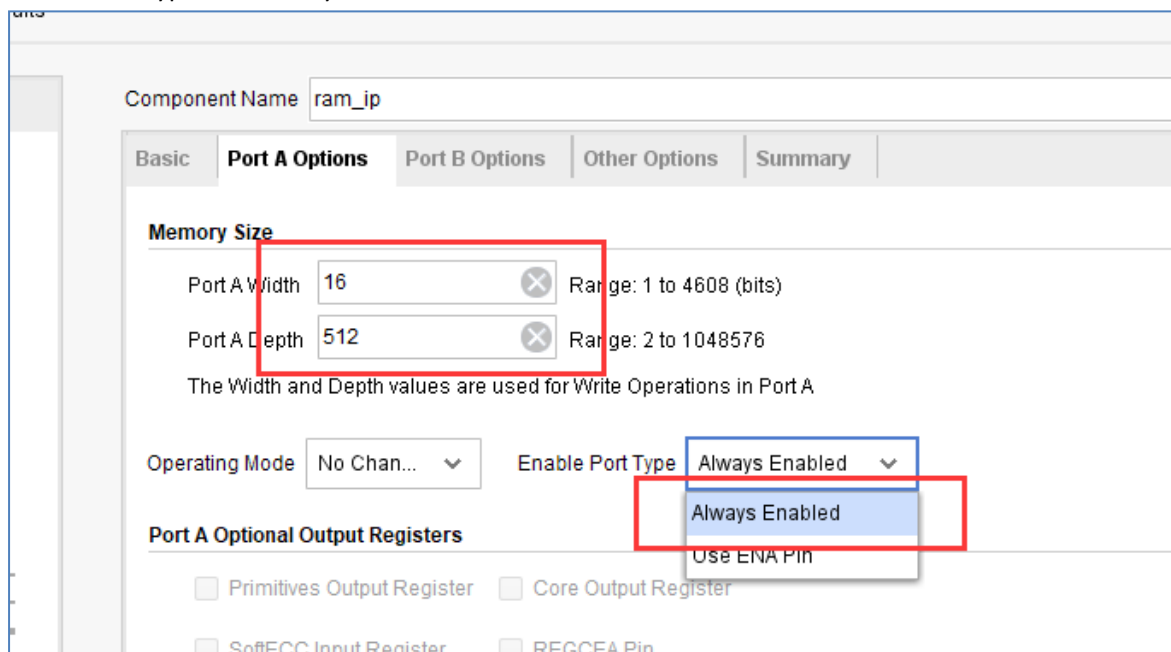
- 1) 点击下图中 IP Catalog，在右侧弹出的界面中搜索 ram，找到 Block Memory Generator, 双击打开。



- 2) 将 Component Name 改为 ram_ip, 在 Basic 栏目下，将 Memory Type 改为 Simple Dual Port RAM，也就是伪双口 RAM。一般来讲“Simple Dual Port RAM”是最常用的，因为它是两个端口，输入和输出信号独立。



- 3) 切换到 Port A Options 栏目下，将 RAM 位宽 Port A Width 改为 16，也就是数据宽度。将 RAM 深度 Port A Depth 改为 512，深度指的是 RAM 里可以存放多少个数据。使能管脚 Enable Port Type 改为 Always Enable。



- 4) 切换到 Port B Options 栏目下，将 RAM 位宽 Port B Width 改为 16，使能管脚 Enable Port Type 改为 Always Enable，当然也可以使用 Use ENB Pin，相当于读使能信号。而 Primitives Output Register 取消勾选，其功能是在输出数据加上寄存器，可以有效改善时序，但读出的数据会落后地址两个周期。很多情况下，不使能这项功能，保持数据落后地址一个周期。

Component Name

Basic | Port A Options | **Port B Options** | Other Options | Summary

Memory Size

Port B Width

Port B Depth : 512

The Width and Depth values are used for Read Operation in Port B

Operating Mode Enable Port Type

Port B Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Output Register ☐ REGCEB Pin

- 5) 在 Other Options 栏目中，这里不像 ROM 那样需要初始化 RAM 的数据，我们可以在程序中写入，所以配置默认即可，直接点击 OK。

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

Component Name

Basic | Port A Options | Port B Options | **Other Options** | Summary

Pipeline Stages within Mux Mux Size: 1x1

Memory Initialization

☐ Load Init File

Coe File

☐ Fill Remaining Memory Locations

Remaining Memory Locations (Hex)

Structural/UniSim Simulation Model Options

Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.

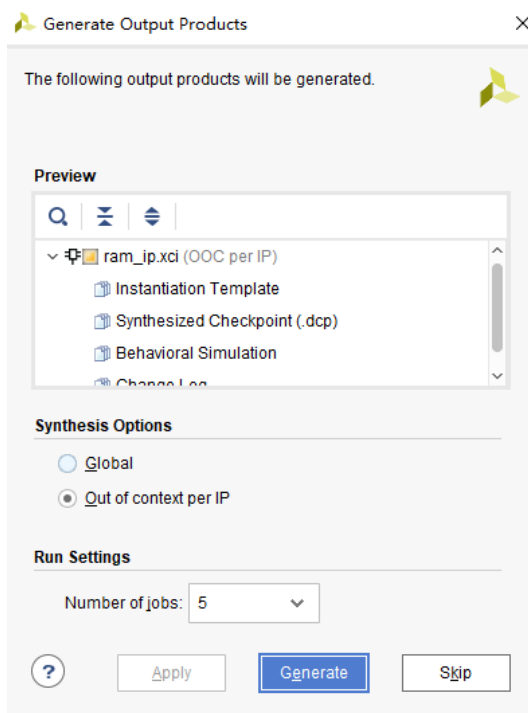
Collision Warnings

Behavioral Simulation Model Options

☐ Disable Collision Warnings ☐ Disable Out of Range Warnings

OK Cancel

- 6) 点击“Generate”生成 RAM IP。

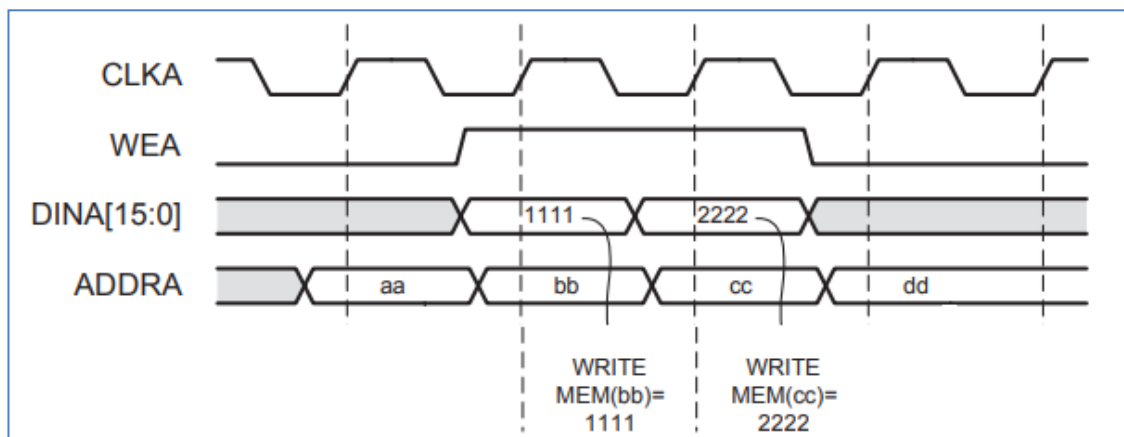


6.3 RAM 的端口定义和时序

Simple Dual Port RAM 模块端口的说明如下：

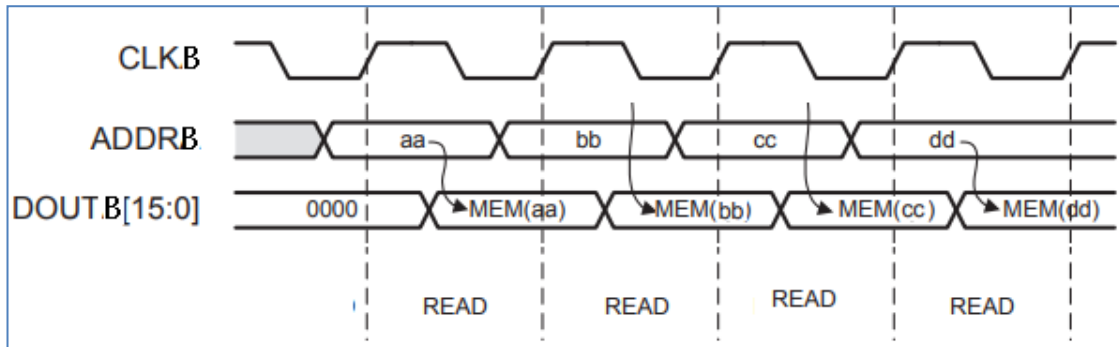
信号名称	方向	说明
clka	in	端口 A 时钟输入
wea	in	端口 A 使能
addra	in	端口 A 地址输入
dina	in	端口 A 数据输入
clkb	in	端口 B 时钟输入
addrb	in	端口 B 地址输入
doutb	out	端口 B 数据输出

RAM 的数据写入和读出都是按时钟的上升沿操作的，端口 A 数据写入的时候需要置高 wea 信号，同时提供地址和要写入的数据。下图为输入写入到 RAM 的时序图。



RAM 写时序

而端口 B 是不能写入数据的，只能从 RAM 中读出数据，只要提供地址就可以了，一般情况下可以在下一个周期采集到有效的数据。



RAM 读时序

6.4 测试程序编写

下面进行 RAM 的测试程序的编写，由于测试 RAM 的功能，我们向 RAM 的端口 A 写入一串连续的数据，只写一次，并从端口 B 中读出，使用逻辑分析仪查看数据。代码如下

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
module ram_test(
    input      sys_clk_p,          //system clock 200Mhz on
board
    input      sys_clk_n,          //system clock 200Mhz on board
    input      rst_n               //复位信号，低电平有效
);

//-----
reg    [8:0]    w_addr;           //RAM PORTA写地址
reg    [15:0]   w_data;           //RAM PORTA写数据
reg     wea;        //RAM PORTA使能
reg    [8:0]    r_addr;           //RAM PORTB读地址
wire    [15:0]   r_data;          //RAM PORTB读数据

wire clk ;

IBUFDS IBUFDS_inst (
    .O(clk), // Buffer output
    .I(sys_clk_p), // Diff_p buffer input (connect directly to top-
level port)
    .IB(sys_clk_n) // Diff_n buffer input (connect directly to top-
level port)
);

//产生RAM PORTB读地址
always @(posedge clk or negedge rst_n)
begin
    if(!rst_n)
```

```

    r_addr <= 9'd0;
else if (!w_addr)           //w_addr位或，不等于0
    r_addr <= r_addr+1'b1;
else
    r_addr <= 9'd0;
end

//产生RAM PORTA写使能信号
always@(posedge clk or negedge rst_n)
begin
    if(!rst_n)
        wea <= 1'b0;
    else
        begin
            if(&w_addr)           //w_addr的bit位全为1，共写入512个数据，写入完成
                wea <= 1'b0;
            else
                wea <= 1'b1;       //ram写使能
        end
    end
end

//产生RAM PORTA写入的地址及数据
always@(posedge clk or negedge rst_n)
begin
    if(!rst_n)
        begin
            w_addr <= 9'd0;
            w_data <= 16'd1;
        end
    else
        begin
            if(wea)               //ram写使能有效
                begin
                    if (&w_addr)           //w_addr的bit位全为1，共写入512个数据，写入完成
                        begin
                            w_addr <= w_addr ;    //将地址和数据的值保持住，只写一次RAM
                            w_data <= w_data ;
                        end
                    else
                        begin
                            w_addr <= w_addr + 1'b1;
                            w_data <= w_data + 1'b1;
                        end
                    end
                end
        end
    end
end

//-----
//实例化RAM
ram_ip ram_ip_inst (
    .clka      (clk           ),    // input clka
    .wea       (wea           ),    // input [0 : 0] wea
    .addra     (w_addr        ),    // input [8 : 0] addra
    .dina      (w_data        ),    // input [15 : 0] dina
    .clkb      (clk           ),    // input clkb
    .addrb     (r_addr        ),    // input [8 : 0] addrb
    .doutb     (r_data        )    // output [15 : 0] doutb
);

```



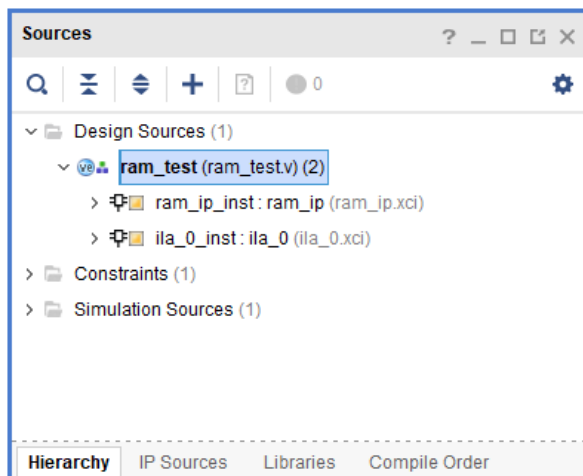
```
//实例化ila逻辑分析仪
ila_0 ila_0_inst (
    .clk      (clk      ),
    .probe0    (r_data    ),
    .probe1    (r_addr    )
);

endmodule
```

为了能实时看到 RAM 中读取的数据值，我们这里添加了 ila 工具来观察 RAM PORTB 的数据信号和地址信号。关于如何生成 ila 大家请参考“PL 的“Hello World”LED 实验”。

```
//实例化ila逻辑分析仪
ila_0 ila_0_inst (
    .clk      (clk      ),
    .probe0    (r_data    ),
    .probe1    (r_addr    )
);
```

程序结构如下：



绑定引脚

```
#####Compress Bitstream#####
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property PACKAGE_PIN AE5 [get_ports sys_clk_p]
set_property IOSTANDARD DIFF_SSTL12 [get_ports sys_clk_p]

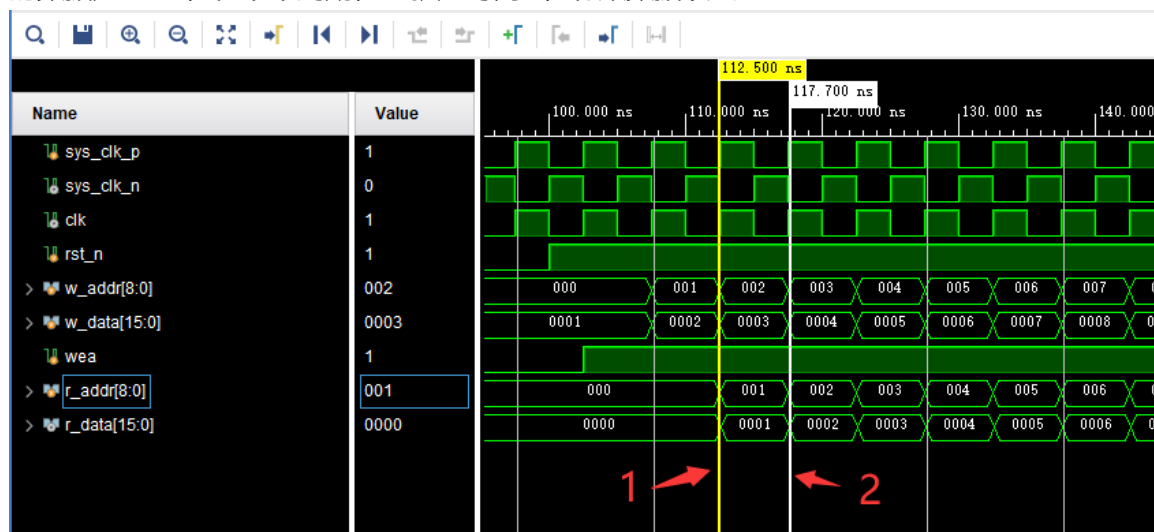
create_clock -period 5.000 -name sys_clk_p -waveform {0.000 2.500}
[get_ports sys_clk_p]

set_property PACKAGE_PIN AF12 [get_ports rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]
```

6.5 仿真

仿真方法参考“PL 的“Hello World”LED 实验”，仿真结果如下，从图中可以看出地址 1 写入

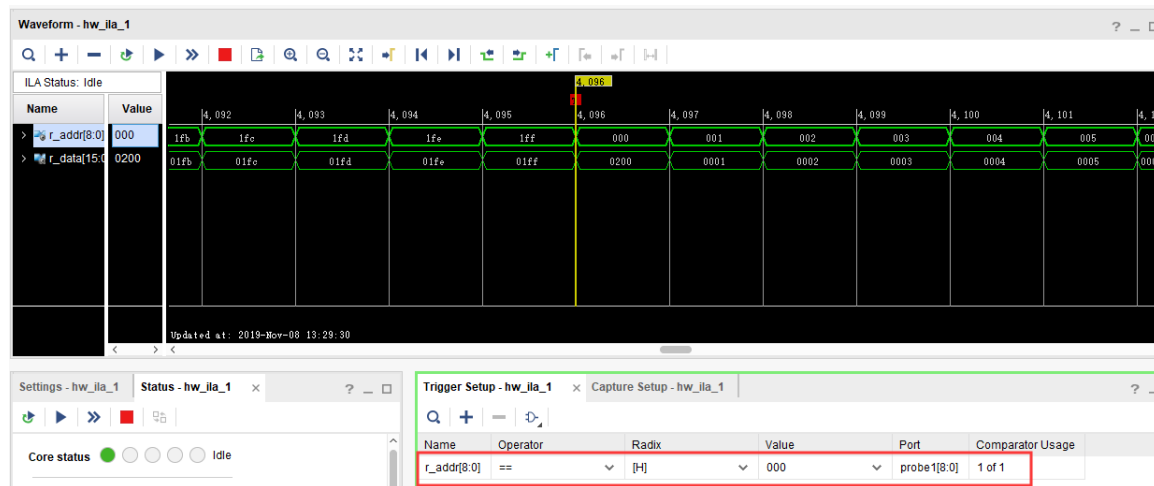
的数据是 0002，在下个周期，也就是时刻 2，有效数据读出。



6.6 板上验证

生成 bitstream，并下载 bit 文件到 FPGA。接下来我们通过 ila 来观察一下从 RAM 中读出的数据是否为我们初始化的数据。

在 Waveform 的窗口设置 r_addr 地址为 0 作为触发条件，我们可以看到 r_addr 在不断的从 0 累加到 1ff，随着 r_addr 的变化，r_data 也在变化，r_data 的数据正是我们写入到 RAM 中的 512 个数据，这里需要注意，r_addr 出现新地址时，r_data 对应的数据要延时两个时钟周期才会出现，数据比地址出现晚两个时钟周期，与仿真结果一致。



第七章 FPGA 片内 ROM 测试实验

实验 Vivado 工程为 “rom_test”。

FPGA 本身是 SRAM 架构的, 断电之后, 程序就消失, 那么如何利用 FPGA 实现一个 ROM 呢, 我们可以利用 FPGA 内部的 RAM 资源实现 ROM, 但不是真正意义上的 ROM, 而是每次上电都会把初始化的值先写入 RAM。本实验将为大家介绍如何使用 FPGA 内部的 ROM 以及程序对该 ROM 的数据读操作。

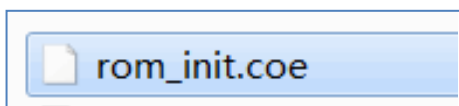
7.1 实验原理

Xilinx 在 VIVADO 里为我们已经提供了 ROM 的 IP 核, 我们只需通过 IP 核例化一个 ROM, 根据 ROM 的读时序来读取 ROM 中存储的数据。实验中会通过 VIVADO 集成的在线逻辑分析仪 ila, 我们可以观察 ROM 的读时序和从 ROM 中读取的数据。

7.2 程序设计

7.2.1 创建 ROM 初始化文件

既然是 ROM, 那么我们就必须提前给它准备好数据, 然后在 FPGA 实际运行时, 我们直接读取这些 ROM 中预存储好的数据就行。Xilinx FPGA 的片内 ROM 支持初始化数据配置。如下图所示, 我们可以创建一个名为 rom_init.coe 的文件, 注意后缀一定是“.coe”, 前面的名称当然可以随意起。



ROM 初始化文件的内容格式很简单, 如下图所示。第一行为定义数据格式, 16 代表 ROM 的数据格式为 16 进制。从第 3 行开始到第 34 行, 是这个 32*8bit 大小 ROM 的初始化数据。每行数字后面用逗号, 最后一行数字结束用分号。

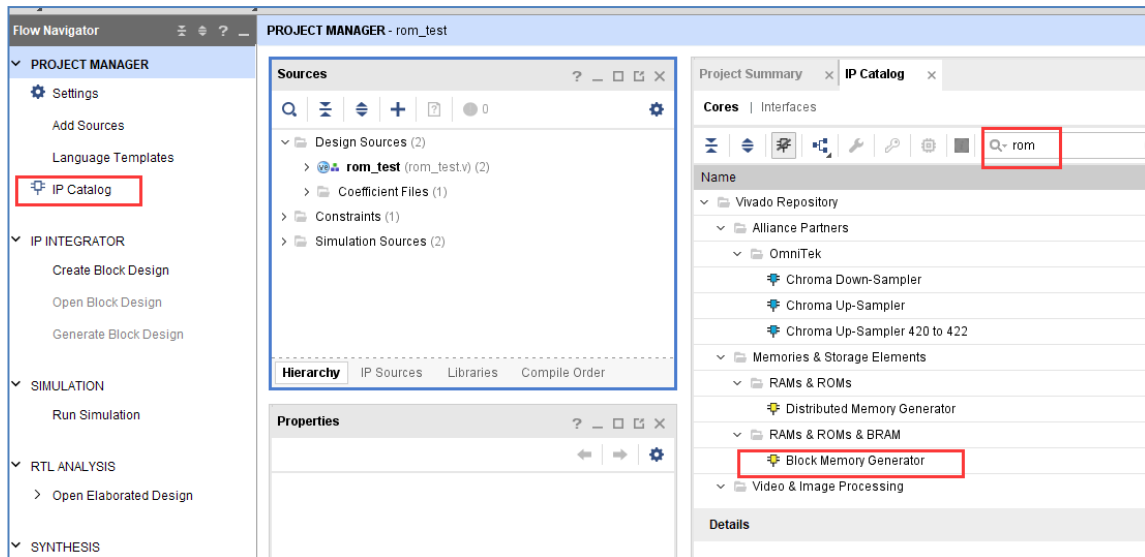
```
rom_test.v  rom_init.coe
1 MEMORY_INITIALIZATION_RADIX=16; //表示ROM内容的数据格式是16进制
2 MEMORY_INITIALIZATION_VECTOR=
3 11,
4 22,
5 33,
6 44,
7 55,
8 66,
9 77,
10 88,
11 99,
12 aa,
13 bb,
14 cc,
15 dd,
16 ee,
17 ff,
18 00,
19 a1,
20 a2,
21 a3,
22 a4,
23 a5,
24 a6,
25 a7,
26 a8,
27 b1,
28 b2,
29 b3,
30 b4,
31 b5,
32 b6,
33 b7,
34 b8; //每个数据后面用逗号或者空格或者换行符隔开，最后一个数据后面加分号
35
```

rom_init.coe 编写完成后保存一下，接下去我们开始设计和配置 ROM IP 核。

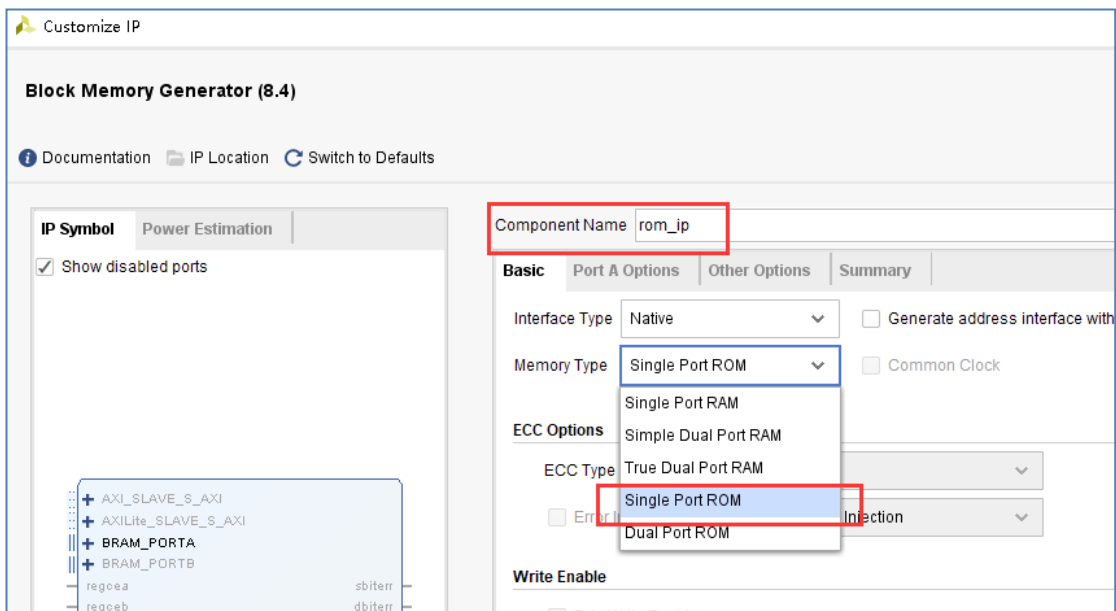
7.2.2 添加 ROM IP 核

在添加 ROM IP 之前先新建一个 rom_test 的工程，然后在工程中添加 ROM IP，方法如下：

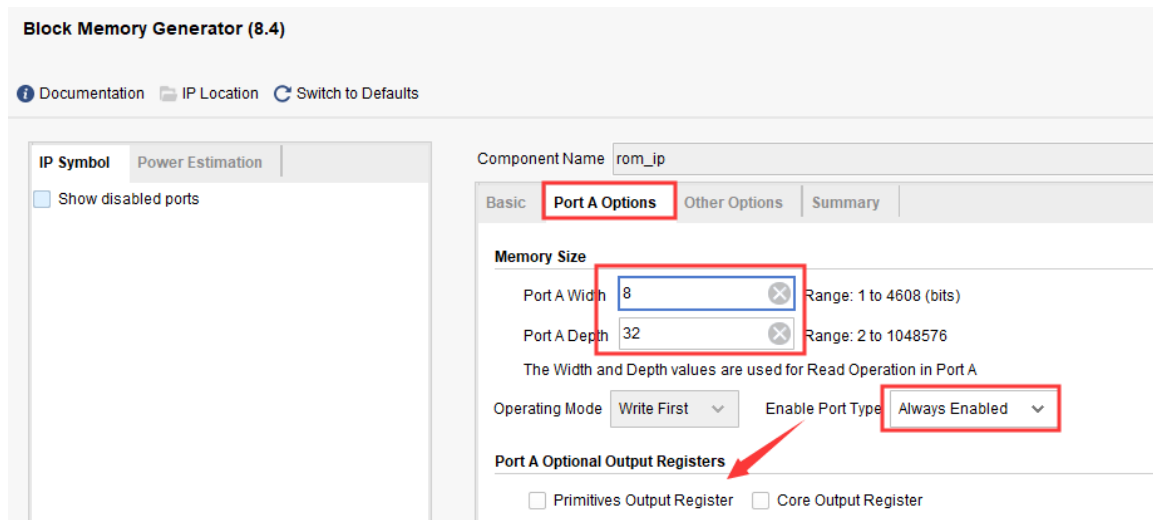
- 1) 点击下图中 IP Catalog，在右侧弹出的界面中搜索 rom，找到 Block Memory Generator, 双击打开。



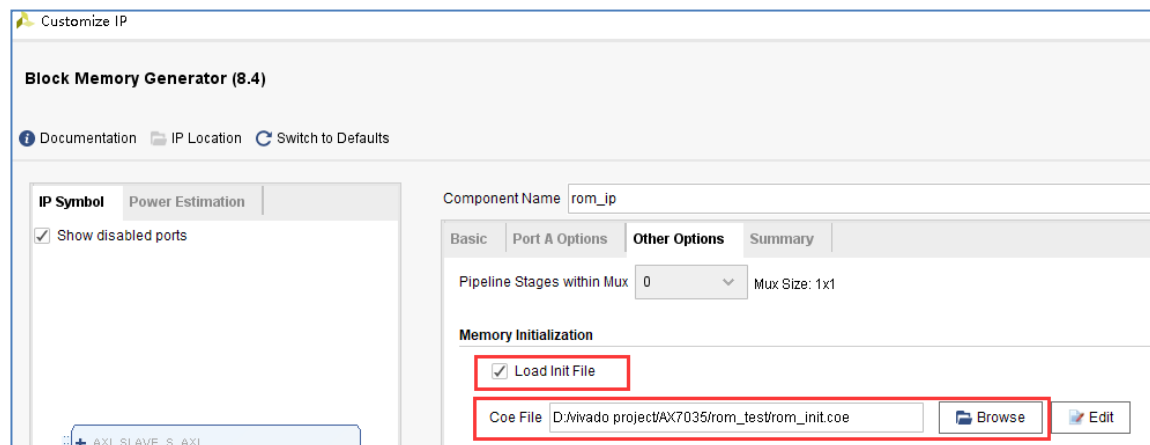
2) 将 Component Name 改为 rom_ip,在 Basic 栏目下, 将 Memory Type 改为 Single Port ROM。



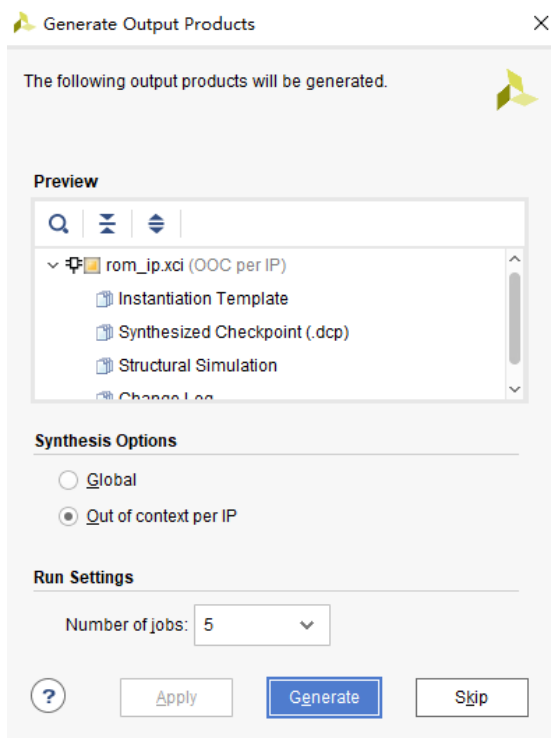
3) 切换到 Port A Options 栏目下, 将 ROM 位宽 Port A Width 改为 8, 将 ROM 深度 Port A Depth 改为 32, 使能管脚 Enable Port Type 改为 Always, 并取消 Primitives Output Register



- 4) 切换到 Other Options 栏目下, 勾选 Load Init File, 点击 Browse, 选中之前制作好的.coe 文件。



- 5) 点击 ok, 点击 Generate 生成 ip 核。



7.3 ROM 测试程序编写

ROM 的程序设计非常简单，在程序中我们只要每个时钟改变 ROM 的地址，ROM 就会输出当前地址的内部存储数据，例化 ila，用于观察地址和数据的变化。ROM IP 的实例化及程序设计如下：

```
`timescale 1ns / 1ps

module rom_test(
    input    sys_clk_p,           //system clock 200Mhz postive pin
    input    sys_clk_n,           //system clock 200Mhz negetive pin
    input    rst_n                //复位，低电平有效
);

wire [7:0] rom_data; //ROM读出数据
reg  [4:0] rom_addr;  //ROM输入地址

wire sys_clk ;

IBUFDS IBUFDS_inst (
    .O(sys_clk), // Buffer output
    .I(sys_clk_p), // Diff_p buffer input (connect directly to top-
level port)
    .IB(sys_clk_n) // Diff_n buffer input (connect directly to top-
level port)
);

//产生ROM地址读取数据
always @ (posedge sys_clk or negedge rst_n)
begin
```

```

    if(!rst_n)
        rom_addr <= 10'd0;
    else
        rom_addr <= rom_addr+1'b1;
end
//实例化ROM
rom_ip rom_ip_inst
(
    .clka    (sys_clk    ),      //input clka
    .addra   (rom_addr   ),      //input [4:0] addra
    .douta   (rom_data   )      //output [7:0] douta
);
//实例化逻辑分析仪
ila_0 ila_m0
(
    .clk      (sys_clk),
    .probe0   (rom_addr),
    .probe1   (rom_data)
);
endmodule

```

绑定引脚

```

#####Compress Bitstream#####
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]

set_property PACKAGE_PIN AE5 [get_ports sys_clk_p]
set_property IOSTANDARD DIFF_SSTL12 [get_ports sys_clk_p]

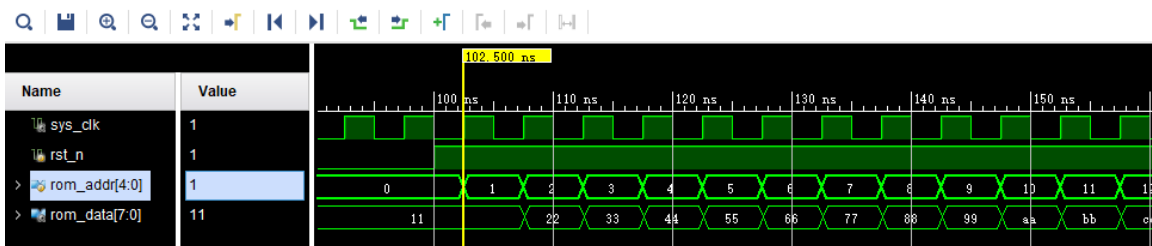
create_clock -period 5.000 -name sys_clk_p -waveform {0.000 2.500}
[get_ports sys_clk_p]

set_property PACKAGE_PIN AF12 [get_ports rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]

```

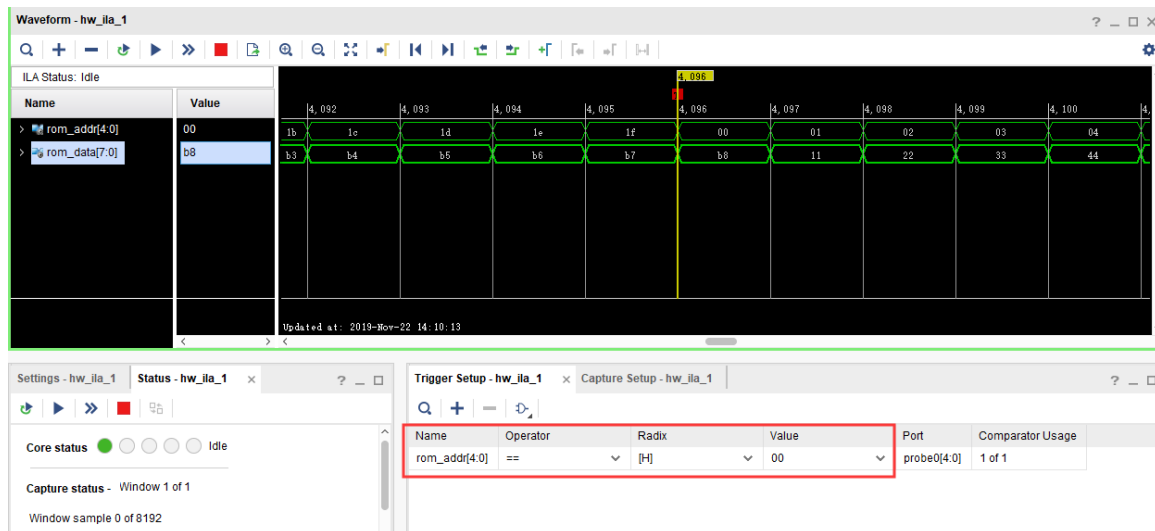
7.4 仿真

仿真结果如下，符合预期，与 RAM 的读取数据一样，数据也是滞后于地址一个周期。



7.5 板上验证

以地址 0 为触发条件，可以看到读取的数据与仿真一致。



第八章 FPGA 片内 FIFO 读写测试实验

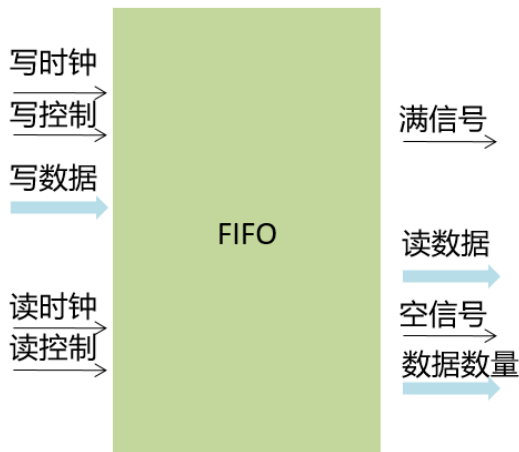
实验 Vivado 工程为 “fifo_test”。

FIFO 是 FPGA 应用当中非常重要的模块，广泛用于数据的缓存，跨时钟域数据处理等。学好 FIFO 是 FPGA 的关键，灵活运用好 FIFO 是一个 FPGA 工程师必备的技能。本章主要介绍利用 XILINX 提供的 FIFO IP 进行读写测试。

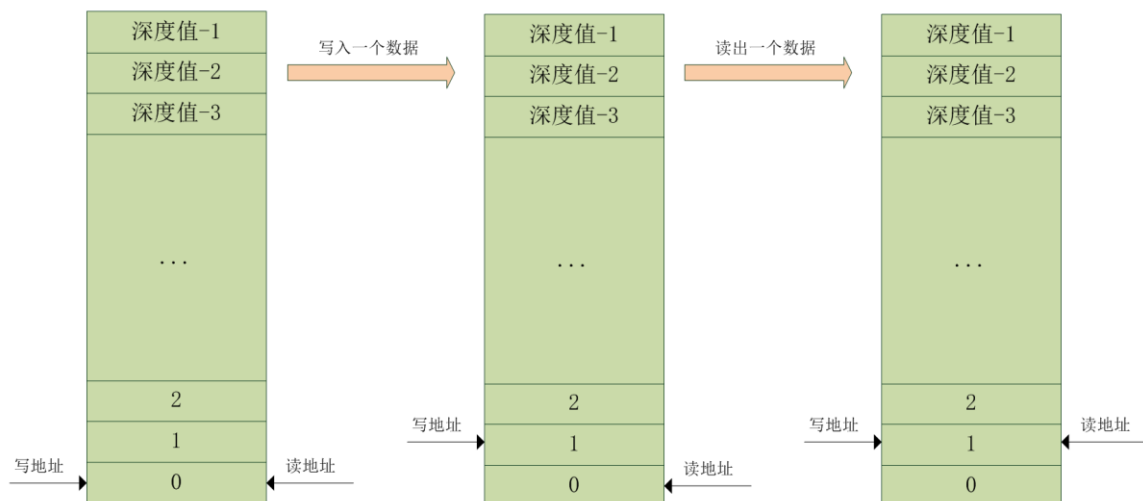
8.1 实验原理

FIFO: First in, First out 代表先进的数据先出，后进的数据后出。Xilinx 在 VIVADO 里为我们已经提供了 FIFO 的 IP 核，我们只需通过 IP 核例化一个 FIFO，根据 FIFO 的读写时序来写入和读取 FIFO 中存储的数据。

其实 FIFO 是也是在 RAM 的基础上增加了许多功能，FIFO 的典型结构如下，主要分为读和写两部分，另外就是状态信号，空和满信号，同时还有数据的数量状态信号，与 RAM 最大的不同是 FIFO 没有地址线，不能进行随机地址读取数据，什么是随机读取数据呢，也就是可以任意读取某个地址的数据。而 FIFO 则不同，不能进行随机读取，这样的好处是不用频繁地控制地址线。



虽然用户看不到地址线，但是在 FIFO 内部还是有地址的操作的，用来控制 RAM 的读写接口。其地址在读写操作时如下图所示，其中深度值也就是一个 FIFO 里最大可以存放多少个数据。初始状态下，读写地址都为 0，在向 FIFO 中写入一个数据后，写地址加 1，从 FIFO 中读出一个数据后，读地址加 1。此时 FIFO 的状态即为空，因为写了一个数据，又读出了一个数据。



可以把 FIFO 想象成一个水池，写通道即为加水，读通道即为放水，假如不间断的加水和放水，如果加水速度比放水速度快，那么 FIFO 就会有满的时候，如果满了还继续加水就会溢出 overflow，如果放水速度比加水速度快，那么 FIFO 就会有空的时候，所以把握好加水与放水的时机和速度，保证水池一直有水是一项很艰巨的任务。也就是判断空与满的状态，择机写数据或读数据。

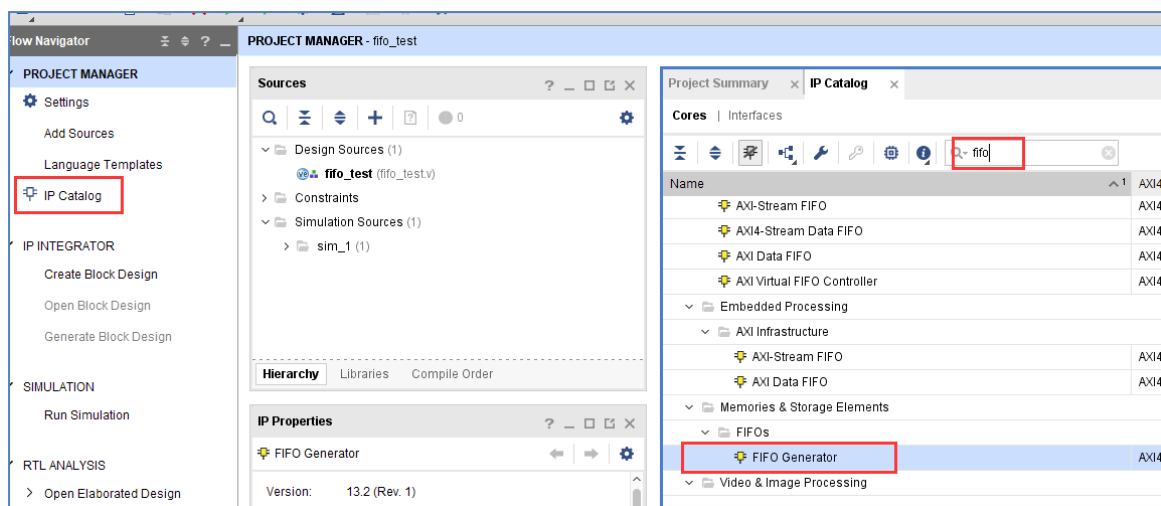
根据读写时钟，可以分为同步 FIFO（读写时钟相同）和异步 FIFO（读写时钟不同）。同步 FIFO 控制比较简单，不再介绍，本节实验主要介绍异步 FIFO 的控制，其中读时钟为 75MHz，写时钟为 100MHz。实验中会通过 VIVADO 集成的在逻辑分析仪 ila，我们可以观察 FIFO 的读写时序和从 FIFO 中读取的数据。

8.2 创建 Vivado 工程

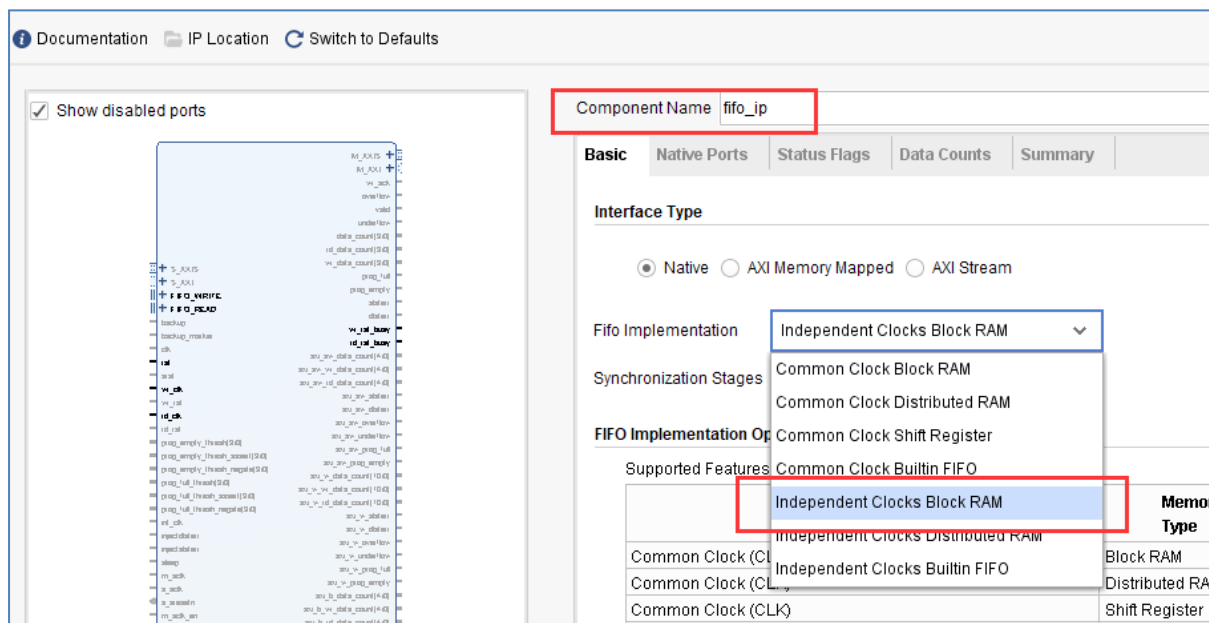
8.2.1 添加 FIFO IP 核

在添加 FIFO IP 之前先新建一个 fifo_test 的工程，然后在工程中添加 FIFO IP，方法如下：

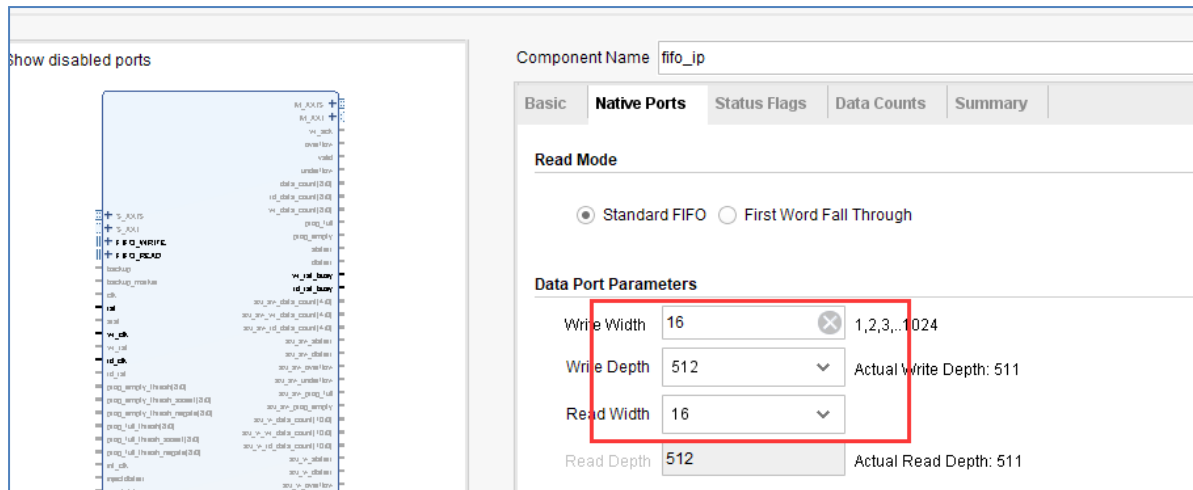
- 1) 点击下图中 IP Catalog，在右侧弹出的界面中搜索 fifo，找到 FIFO Generator,双击打开。



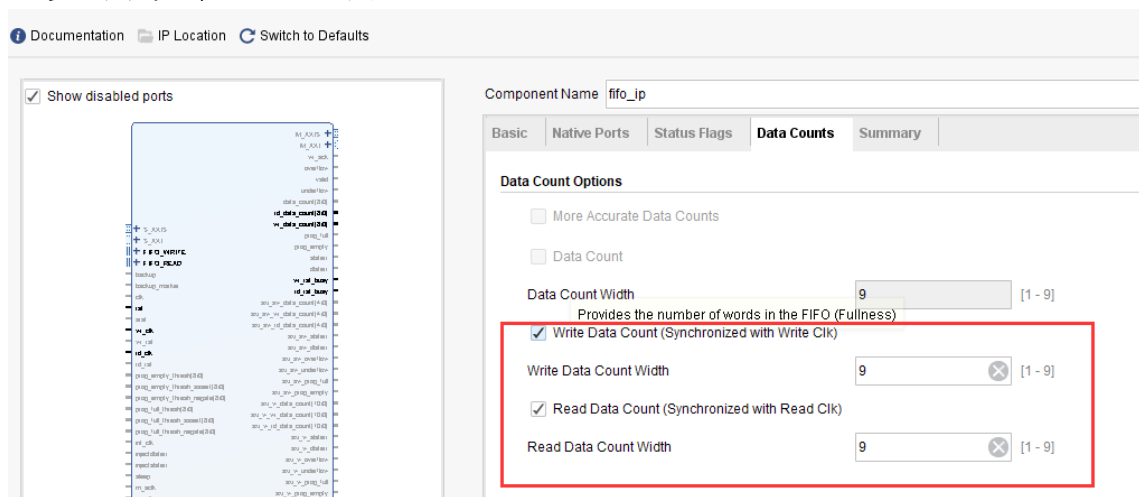
- 2) 弹出的配置页面中，这里可以选择读写时钟分开还是用同一个，一般来讲我们使用 FIFO 为了缓存数据，通常两边的时钟速度是不一样的。所以独立时钟是最常用的，我们这里选择“Independent Clocks Block RAM”，然后点击“Next”到下一个配置页面。



- 3) 切换到 Native Ports 栏目下，选择数据位宽 16；FIFO 深选择 512，实际使用大家根据需要自行设置就可以。Read Mode 有两种方式，一个 Standard FIFO，也就是平时常见的 FIFO，数据滞后于读信号一个周期，还有一种方式为 First Word Fall Through，数据预取模式，简称 FWFT 模式。也就是 FIFO 会预先取出一个数据，当读信号有效时，相应的数据也有效。我们首先做标准 FIFO 的实验。



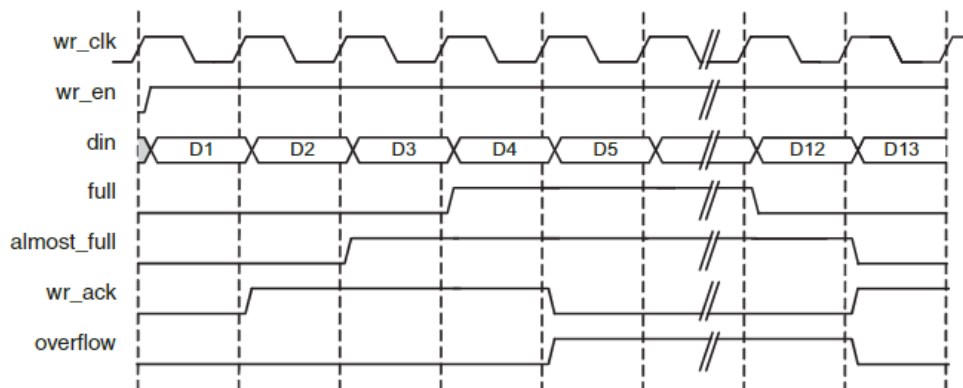
- 4) 切换到 Data Counts 栏目下，使能 Write Data Count（已经 FIFO 写入多少数据）和 Read Data Count（FIFO 中有多少数据可以读），这样我们可以通过这两个值来看 FIFO 内部的数据多少。点击 OK,Generate 生成 FIFO IP。



8.2.2 FIFO 的端口定义与时序

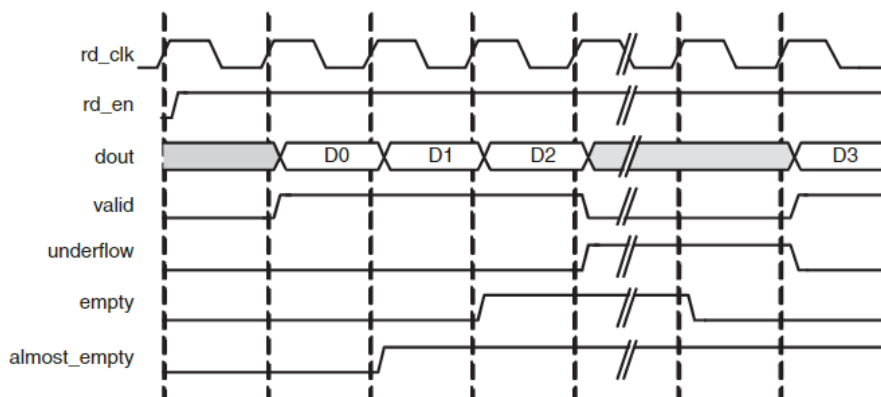
信号名称	方向	说明
rst	in	复位信号，高有效
wr_clk	in	写时钟输入
rd_clk	in	读时钟输入
din	in	写数据
wr_en	in	写使能，高有效
rd_en	in	读使能，高有效
dout	out	读数据
full	out	满信号
empty	out	空信号
rd_data_count	out	可读数据数量
wr_data_count	out	已写入的数据数量

FIFO 的数据写入和读出都是按时钟的上升沿操作的, 当 `wr_en` 信号为高时写入 FIFO 数据, 当 `almost_full` 信号有效时, 表示 FIFO 只能再写入一个数据, 一旦写入一个数据了, `full` 信号就会拉高, 如果在 `full` 的情况下 `wr_en` 仍然有效, 也就是继续向 FIFO 写数据, 则 FIFO 的 `overflow` 就会有效, 表示溢出。



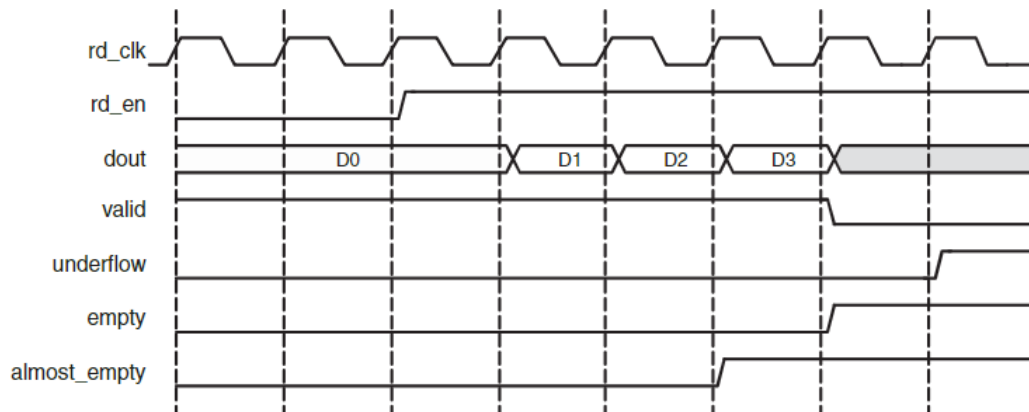
标准 FIFO 写时序

当 `rd_en` 信号为高时读 FIFO 数据, 数据在下一个周期有效。 `valid` 为数据有效信号, `almost_empty` 表示还有一个数据读, 当再读一个数据, `empty` 信号有效, 如果继续读, 则 `underflow` 有效, 表示下溢, 此时读出的数据无效。



标准 FIFO 读时序

而从 FWFT 模式读数据时序图可以看出, `rd_en` 信号有效时, 有效数据 D0 已经在数据线上准备好有效了, 不会再延后一个周期。这就是与标准 FIFO 的不同之处。



FWFT FIFO 读时序

关于 FIFO 的详细内容可参考 pg057 文档，可在 xilinx 官网下载。

8.3 FIFO 测试程序编写

我们按照异步 FIFO 进行设计，用 PLL 产生出两路时钟，分别是 100MHz 和 75MHz，用于写时钟和读时钟，也就是写时钟频率高于读时钟频率。

```
`timescale 1ns / 1ps
module fifo_test
(
    input sys_clk_p,           //system clock 200Mhz postive pin
    input sys_clk_n,           //system clock 200Mhz negative
    pin
    input rst_n                 //复位信号，低电平有效
);

reg [15:0] w_data ;           //FIFO写数据
wire wr_en ;                 //FIFO写使能
wire rd_en ;                 //FIFO读使能
wire [15:0] r_data ;          //FIFO读数据
wire full ;                  //FIFO满信号
wire empty ;                 //FIFO空信号
wire [8:0] rd_data_count ;    //可读数据数量
wire [8:0] wr_data_count ;    //已写入数据数量

wire clk_100M ;              //PLL产生100MHz时钟
wire clk_75M ;               //PLL产生100MHz时钟
wire locked ;                //PLL lock信号，可作为系统复位
信号，高电平表示lock住
wire fifo_rst_n ;            //fifo复位信号，低电平有效

wire wr_clk ;                //写FIFO时钟
wire rd_clk ;                //读FIFO时钟
reg [7:0] wcnt ;              //写FIFO复位后等待计数器
reg [7:0] rcnt ;              //读FIFO复位后等待计数器
```

```

//例化PLL, 产生100MHz和75MHz时钟
clk_wiz_0 fifo_pll
(
    // Clock out ports
    .clk_out1(clk_100M),           // output clk_out1
    .clk_out2(clk_75M),           // output clk_out2
    // Status and control signals
    .reset(~rst_n),               // input reset
    .locked(locked),              // output locked
    // Clock in ports
    .clk_in1_p(sys_clk_p),        // input clk_in1
    .clk_in1_n(sys_clk_n)        // input clk_in1
);

assign fifo_rst_n    = locked    ; //将PLL的LOCK信号赋值给fifo的复位信号
assign wr_clk        = clk_100M  ; //将100MHz时钟赋值给写时钟
assign rd_clk        = clk_75M   ; //将75MHz时钟赋值给读时钟

/* 写FIFO状态机 */
localparam W_IDLE    = 1 ;
localparam W_FIFO    = 2 ;

reg[2:0] write_state;
reg[2:0] next_write_state;

always@(posedge wr_clk or negedge fifo_rst_n)
begin
    if(!fifo_rst_n)
        write_state <= W_IDLE;
    else
        write_state <= next_write_state;
end

always@(*)
begin
    case(write_state)
        W_IDLE:
            begin
                if(wcnt == 8'd79) //复位后等待一定时间, safety
circuit模式下的最慢时钟60个周期
                    next_write_state <= W_FIFO;
                else
                    next_write_state <= W_IDLE;
            end
        W_FIFO:
            next_write_state <= W_FIFO; //一直在写FIFO状态
        default:
            next_write_state <= W_IDLE;
    endcase
end

//在IDLE状态下, 也就是复位之后, 计数器计数
always@(posedge wr_clk or negedge fifo_rst_n)
begin
    if(!fifo_rst_n)
        wcnt <= 8'd0;
    else if (write_state == W_IDLE)
        wcnt <= wcnt + 1'b1 ;
    else
        wcnt <= 8'd0;
end

```



```

end
//在写FIFO状态下, 如果不满就向FIFO中写数据
assign wr_en = (write_state == W_FIFO) ? ~full : 1'b0;
//在写使能有效情况下, 写数据值加1
always@(posedge wr_clk or negedge fifo_rst_n)
begin
    if(!fifo_rst_n)
        w_data <= 16'd1;
    else if (wr_en)
        w_data <= w_data + 1'b1;
end

/* 读FIFO状态机 */

localparam    R_IDLE      = 1 ;
localparam    R_FIFO      = 2 ;
reg[2:0] read_state;
reg[2:0] next_read_state;

///产生FIFO读的数据
always@(posedge rd_clk or negedge fifo_rst_n)
begin
    if(!fifo_rst_n)
        read_state <= R_IDLE;
    else
        read_state <= next_read_state;
end

always@(*)
begin
    case(read_state)
        R_IDLE:
            begin
                if (rcnt == 8'd59)                //复位后等待一定时间,
safety circuit模式下的最慢时钟60个周期
                    next_read_state <= R_FIFO;
                else
                    next_read_state <= R_IDLE;
            end
        R_FIFO:
            next_read_state <= R_FIFO ;           //一直在读FIFO状态
        default:
            next_read_state <= R_IDLE;
    endcase
end

//在IDLE状态下, 也就是复位之后, 计数器计数
always@(posedge rd_clk or negedge fifo_rst_n)
begin
    if(!fifo_rst_n)
        rcnt <= 8'd0;
    else if (write_state == W_IDLE)
        rcnt <= rcnt + 1'b1 ;
    else
        rcnt <= 8'd0;
end

//在读FIFO状态下, 如果不空就从FIFO中读数据
assign rd_en = (read_state == R_FIFO) ? ~empty : 1'b0;

//-----

```

```

//实例化FIFO
fifo_ip fifo_ip_inst
(
    .rst            (~fifo_rst_n    ), // input rst
    .wr_clk         (wr_clk         ), // input wr_clk
    .rd_clk         (rd_clk         ), // input rd_clk
    .din            (w_data         ), // input [15 : 0] din
    .wr_en          (wr_en          ), // input wr_en
    .rd_en          (rd_en          ), // input rd_en
    .dout           (r_data         ), // output [15 : 0] dout
    .full           (full           ), // output full
    .empty          (empty          ), // output empty
    .rd_data_count  (rd_data_count), // output [8 : 0] rd_data_count
    .wr_data_count  (wr_data_count), // output [8 : 0] wr_data_count
);

//写通道逻辑分析仪
ila_m0 ila_wfifo (
    .clk(wr_clk),
    .probe0(w_data),
    .probe1(wr_en),
    .probe2(full),
    .probe3(wr_data_count)
);

//读通道逻辑分析仪
ila_m0 ila_rfifo (
    .clk(rd_clk),
    .probe0(r_data),
    .probe1(rd_en),
    .probe2(empty),
    .probe3(rd_data_count)
);

endmodule

```

在程序中采用 PLL 的 lock 信号作为 fifo 的复位，同时将 100MHz 时钟赋值给写时钟，75MHz 时钟赋值给读时钟。

```

assign fifo_rst_n = locked ; //将PLL的LOCK信号赋值给fifo的复位信号
assign wr_clk     = clk_100M ; //将100MHz时钟赋值给写时钟
assign rd_clk     = clk_75M  ; //将75MHz时钟赋值给读时钟

```

有一点需要注意的是，FIFO 设置默认为采用 safety circuit，此功能是保证到达内部 RAM 的输入信号是同步的，在这种情况下，如果异步复位后，则需要等待 60 个最慢时钟周期，在本实验也就是 75MHz 的 60 个周期，那么 100MHz 时钟大概需要 $(100/75) \times 60 = 80$ 个周期。

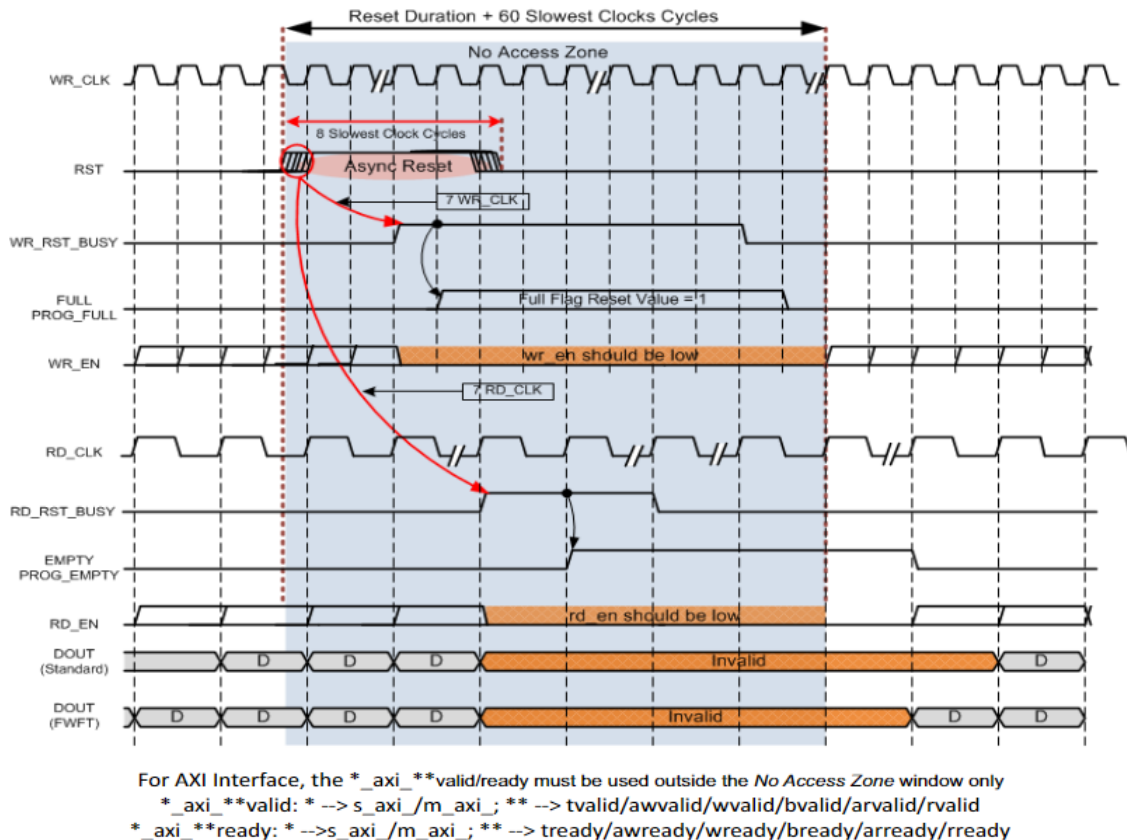


Figure 3-29: FIFO Asynchronous Reset Timing With Safety Circuit

因此在写状态机中，等待 80 个周期进入写 FIFO 状态

```
always@(*)
begin
    case(write_state)
        W_IDLE:
            begin
                if(wcnt == 8'd79) //复位后等待一定时间, safety circuit模式下的最慢时钟60个周期
                    next_write_state <= W_FIFO;
                else
                    next_write_state <= W_IDLE;
            end
        W_FIFO:
            next_write_state <= W_FIFO; //一直在写FIFO状态
        default:
            next_write_state <= W_IDLE;
    endcase
end
```

在读状态机中，等待 60 个周期进入读状态

```
always@(*)
begin
    case(read_state)
        R_IDLE:
            begin
                if(rcnt == 8'd59) //复位后等待一定时间, safety circuit模式下的最慢时钟60个周期
                    next_read_state <= R_FIFO;
                else
                    next_read_state <= R_IDLE;
            end
        R_FIFO:
            next_read_state <= R_FIFO ; //一直在读FIFO状态
        default:
            next_read_state <= R_IDLE;
    endcase
end
```

如果 FIFO 不满，就一直向 FIFO 写数据

```
//在写FIFO状态下, 如果不满就向FIFO中写数据
```

```
assign wr_en = (write_state == W_FIFO) ? ~full : 1'b0;
```

如果 FIFO 不空, 就一直从 FIFO 读数据

```
//在读FIFO状态下, 如果不空就从FIFO中读数据
```

```
assign rd_en = (read_state == R_FIFO) ? ~empty : 1'b0;
```

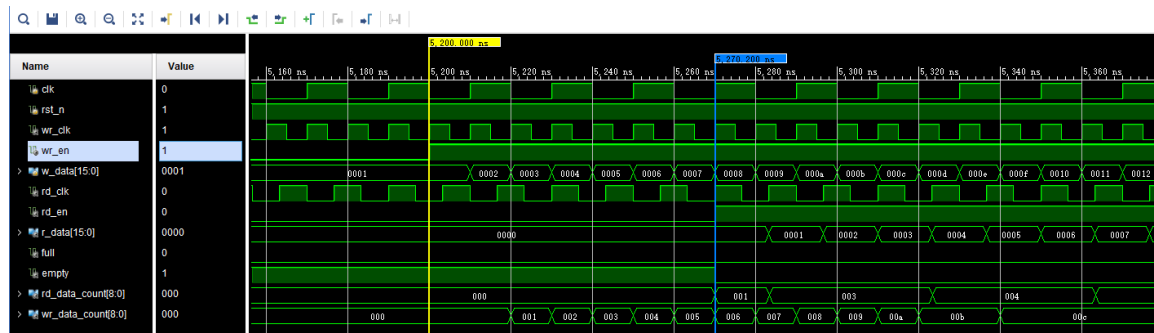
例化两个逻辑分析仪, 分别连接写通道和读通道的信号

```
//写通道逻辑分析仪
ila_m0 ila_wfifo (
    .clk      (wr_clk      ),
    .probe0   (w_data     ),
    .probe1   (wr_en      ),
    .probe2   (full       ),
    .probe3   (wr_data_count)
);

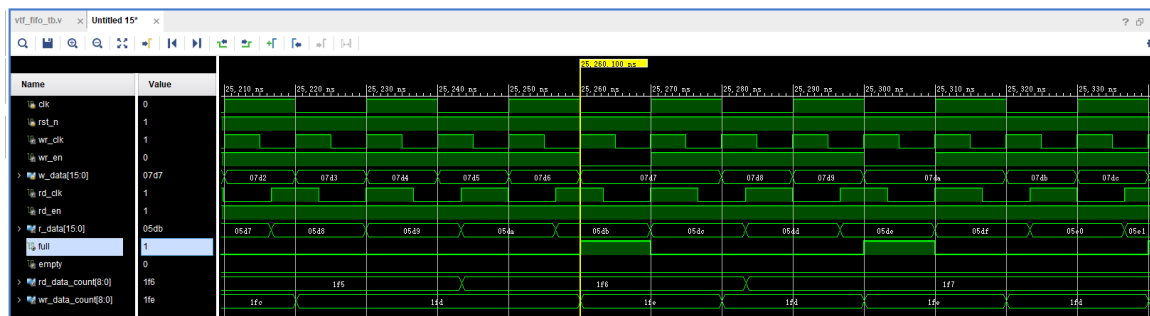
//读通道逻辑分析仪
ila_m0 ila_rfifo (
    .clk      (rd_clk      ),
    .probe0   (r_data     ),
    .probe1   (rd_en      ),
    .probe2   (empty      ),
    .probe3   (rd_data_count)
);
```

8.4 仿真

以下为仿真结果, 可以看到写使能 `wr_en` 有效后开始写数据, 初始值为 0001, 从开始写到 `empty` 不空, 是需要一定周期的, 因为内部还要做同步处理。在不空后, 开始读数据, 读出的数据相对于 `rd_en` 滞后一个周期。



在后面可以看到如果 FIFO 满了, 根据程序的设计, 满了就不向 FIFO 写数据了, `wr_en` 也就拉低了。为什么会满呢, 就是因为写时钟比读时钟快。如果将写时钟与读时钟调换, 也就是读时钟快, 就会出现读空的情况, 大家可以试一下。



如果将 FIFO 的 Read Mode 改成 First Word Fall Through

FIFO Generator (13.2)

Documentation IP Location Switch to Defaults

Component Name: `fifo_ip`

Basic Native Ports Status Flags Data Counts Summary

Read Mode

☐ Standard FIFO ☒ First Word Fall Through

Data Port Parameters

Write Width: 16 1,2,3...1024

Write Depth: 512 Actual Write Depth: 513

Read Width: 16

Read Depth: 512 Actual Read Depth: 513

ECC, Output Register and Power Gating Options

☐ ECC Hard ECC ☐ Single Bit Error Injection ☐ Double Bit Error Injection

☐ ECC Pipeline Reg ☐ Dynamic Power Gating

☐ Output Registers Embedded Registers

Initialization

☒ Reset Pin ☒ Enable Reset Synchronization ☒ Enable Safety Circuit

Reset Type: Asynchronous Reset

Full Flags Reset Value: 1

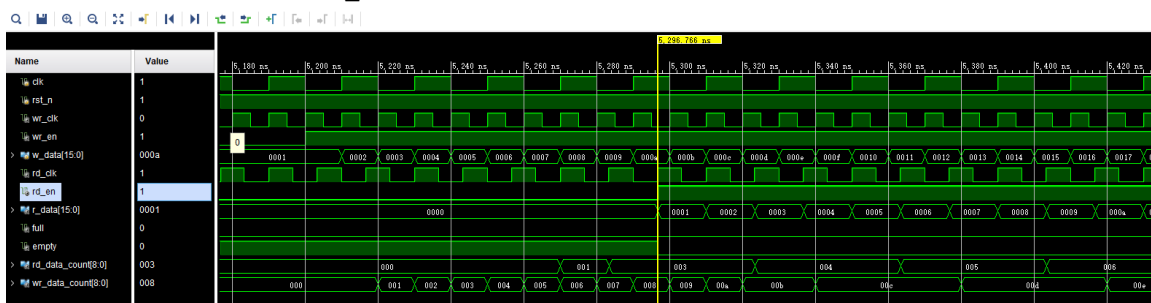
☒ Dout Reset Value: 0 (Hex)

Read Latency: 0

Ports:

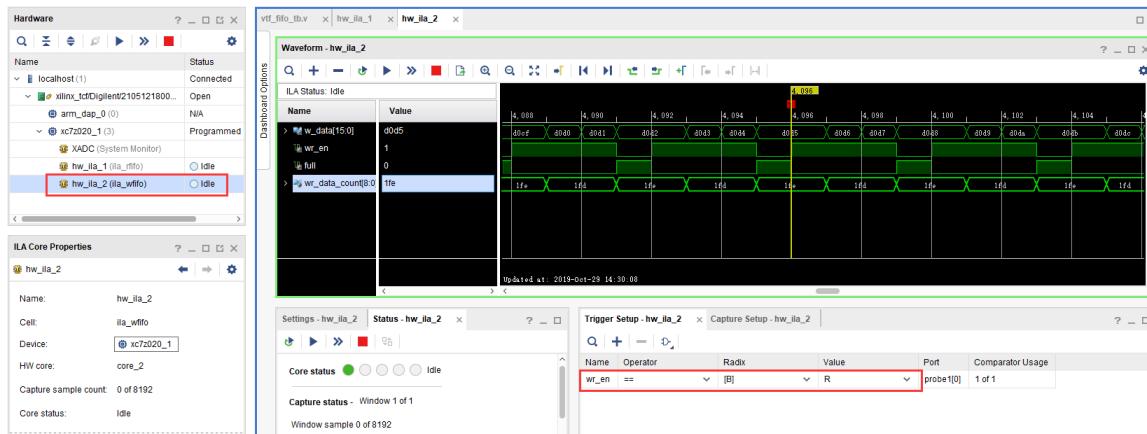
- FIFO_WRITE: rd_data_count[8:0]
- FIFO_READ: wr_data_count[8:0]
- rst: wr_rst_busy
- wr_clk: rd_rst_busy
- rd_clk: rd_rst_busy

仿真结果如下，可以看到 rd_en 有效的时候数据也有效，没有相差一个周期

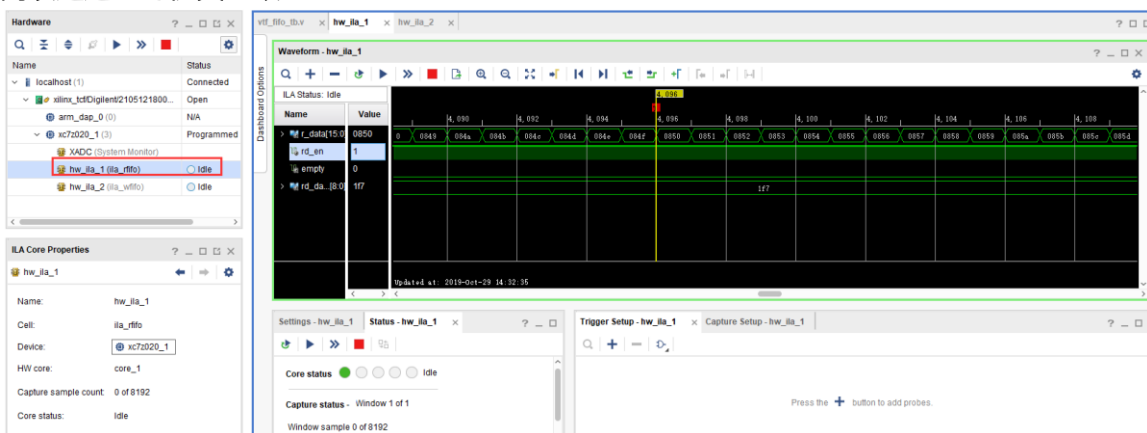


8.5 板上验证

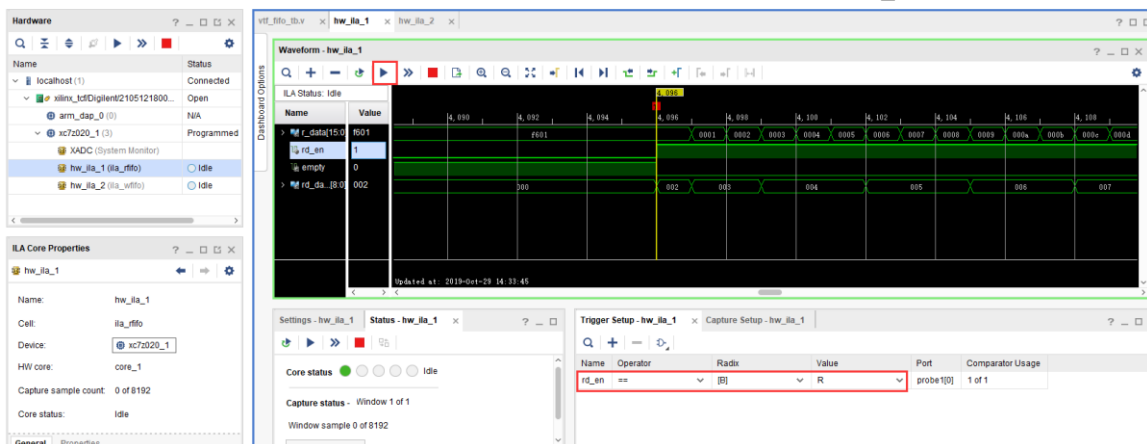
生成好 bit 文件，下载 bit 文件，会出现两个 ila，先来看写通道的，可以看到 full 信号为高电平时，wr_en 为低电平，不再向里面写数据。



而读通道也与仿真一致



如果以 rd_en 上升沿作为触发条件，点击运行，然后按下复位，也就是我们绑定的 PL KEY1，会出现下面的结果，与仿真一致，标准 FIFO 模式下，数据滞后 rd_en 一个周期。

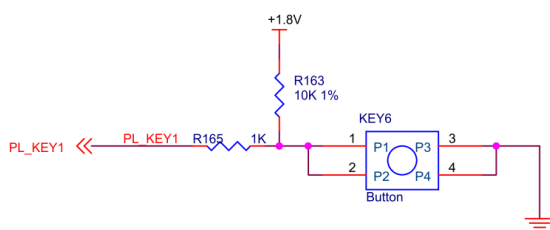


第九章 Vivado 下按键实验

实验 Vivado 工程为 “key_test”。

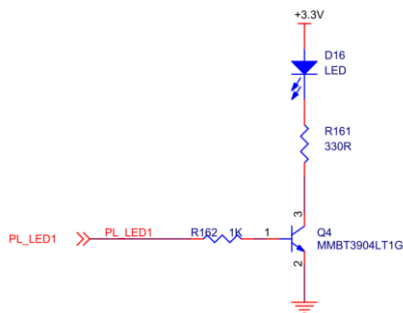
按键是 FPGA 设计当中最常用也是最简单的外设，本章通过按键检测实验，检测开发板的按键功能是否正常，并了解硬件描述语言和 FPGA 的具体关系，学习 Vivado RTL ANALYSIS 的使用。

9.1 按键硬件电路



AXU3EG/AXU4EV/AXU5EV 开发板按键部分电路

从图中可以看到，电路的按键松开时是高电平，按下时是低电平。

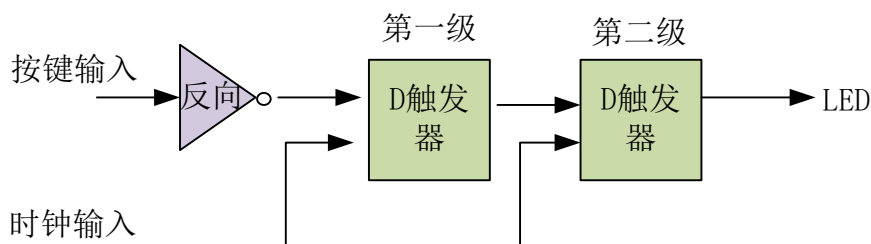


AXU3EG/AXU4EV/AXU5EV 开发板 LED 部分电路

而 LED 部分，高电平亮，低电平灭

9.2 程序设计

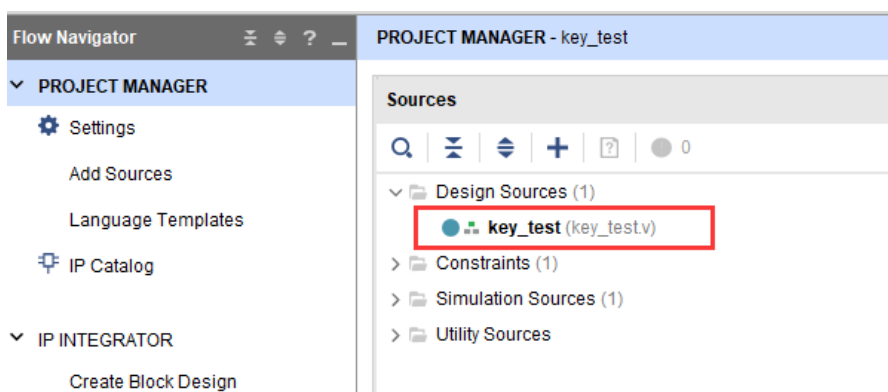
这个程序没有设计的很复杂，通过简单的硬件描述语言看透硬件描述语言和 FPGA 硬件的联系。首先我们将按键输入经过一个非门后再经过 2 组 D 触发器。经过 D 触发器的信号，会在 D 触发器时钟输入的上升沿锁存然后再送到输出。



在进行硬件描述语言编码之前，我们已经把硬件构建完成，这是一个正常的开发流程。有了硬件设计思路无论是通过画图还是通过 Verilog HDL、VHDL 都能完成设计，根据设计的复杂程序和对某种语言的熟悉程序来选择工具。

9.3 创建 Vivado 工程

- 1) 首先建立按键的测试工程，添加 verilog 测试代码，完成编译分配管脚等流程。



```

`timescale 1ns / 1ps
module key_test
(
    input                sys_clk_p,           //system clock 200Mhz
    postive pin          sys_clk_n,           //system clock 200Mhz
    negative pin         key,                 //input four key signal,when the
    keydown,the value is 0
    output               led                  //LED display ,when the siganl low,LED
    lighten
);

reg led_r;                //define the first stage register , generate four
D Flip-flop
reg led_r1;               //define the second stage register ,generate four
D Flip-flop

wire clk ;

IBUFDS IBUFDS_inst (
    .O(clk), // Buffer output
    .I(sys_clk_p), // Diff_p buffer input (connect directly to top-

```



```
level port)
    .IB(sys_clk_n) // Diff_n buffer input (connect directly to top-
level port)
);

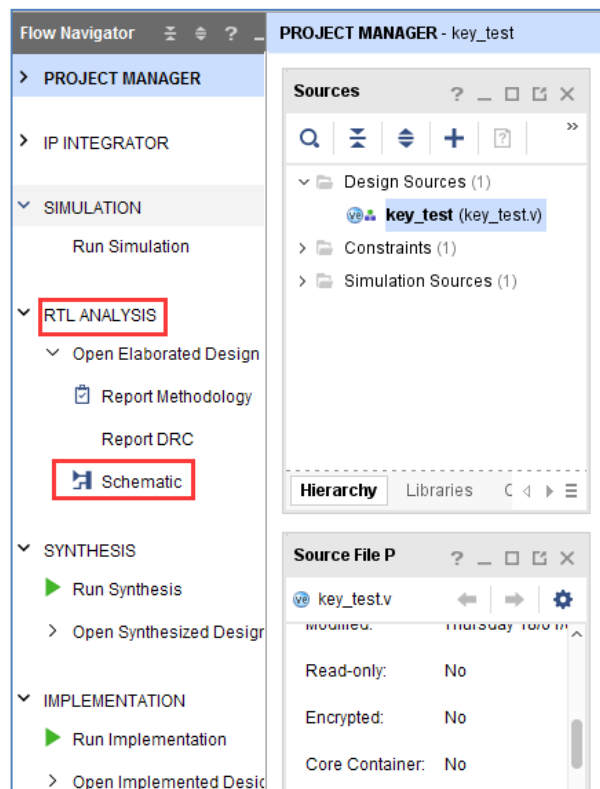
always@(posedge clk)
begin
    led_r <= ~key;      //first stage latched data
end

always@(posedge clk)
begin
    led_r1 <= led_r;    //second stage latched data
end

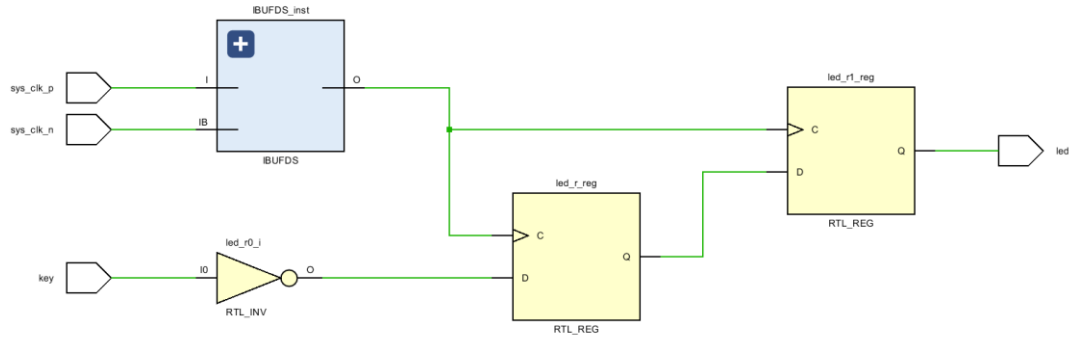
assign led = led_r1;

endmodule
```

2) 我们可以使用 RTL ANALYSIS 工具查看设计



3) 分析 RTL 图，可以看出第一级 D 触发器经过取反后输入，第二级直接输入，和预期设计一致。



9.4 板上验证

Bit 文件下载到开发板以后，开发板上的" PL LED"处于灭状态，按键 "PL KEY" 按下 "PL LED" 亮。

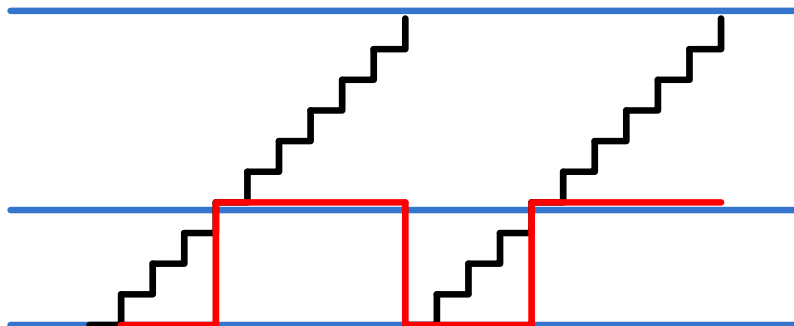
第十章 PWM 呼吸灯实验

实验 Vivado 工程为 “pwm_led”。

本文主要讲解使用 PWM 控制 LED，实现呼吸灯的效果。

10.1 实验原理

如下图所示，用一个 N 比特的计数器，最大值可以表示为 2 的 N 次方，最小值 0，计数器以 “period” 为步进值累加，加到最大值后会溢出，进入下一个累加周期。当计数器值大于 “duty” 时，脉冲输出高，否则输出低，这样就可以完成图中红色线所示的脉冲占空比可调的脉冲输出，同时 “period” 可以调节脉冲频率，可以理解为计数器的步进值。



PWM 脉宽调制示意图

不同的脉冲占空比的方波输出后加在 LED 上，LED 灯就会显示不同的亮度，通过不断地调节方波的占空比，从而实现 LED 灯亮度的调节。

10.2 实验设计

PWM 模块设计非常简单，在上面的原理中已经讲到，这里不再说原理。

信号名称	方向	说明
clk	in	时钟输入
rst	in	异步复位输入，高复位
period	in	PWM 脉宽周期（频率）控制。period = PWM 输出频率 * (2 的 N 次方) / 系统时钟频率。显然 N 越大，频率精度越高。
duty	in	占空比控制，占空比 = duty / (2 的 N 次方) * 100%

PWM 模块 (ax_pwm) 端口

```
`timescale 1ns / 1ps
module ax_pwm
#(
    parameter N = 16 //pwm bit width
)
(
    input      clk,
    input      rst,
    input[N - 1:0]period, //pwm step value
    input[N - 1:0]duty,    //duty value
    output     pwm_out //pwm output
);

reg[N - 1:0] period_r;    //period register
reg[N - 1:0] duty_r;      //duty register
reg[N - 1:0] period_cnt; //period counter
reg pwm_r;
assign pwm_out = pwm_r;
always@(posedge clk or posedge rst)
begin
    if(rst==1)
    begin
        period_r <= { N {1'b0} };
        duty_r <= { N {1'b0} };
    end
    else
    begin
        period_r <= period;
        duty_r <= duty;
    end
end
//period counter, step is period value
always@(posedge clk or posedge rst)
begin
    if(rst==1)
        period_cnt <= { N {1'b0} };
    else
        period_cnt <= period_cnt + period_r;
end

always@(posedge clk or posedge rst)
begin
    if(rst==1)
    begin
        pwm_r <= 1'b0;
    end
    else
    begin
        if(period_cnt >= duty_r) //if period counter is bigger or
//equals to duty value, then set pwm value to high
            pwm_r <= 1'b1;
        else
            pwm_r <= 1'b0;
    end
end
end
```

那么如何实现呼吸灯的效果呢？我们知道呼吸灯效果是由暗不断的变亮，再由亮不断的变

暗的过程，而亮暗效果是由占空比来调节的，因此我们主要来控制占空比，也就是控制 duty 的值。

在下面的测试代码中，通过设置 period 的值，设定 PWM 的频率为 200Hz，PWM_PLUS 状态即是增加 duty 值，如果增加到最大值，将 pwm_flag 置 1，并开始将 duty 值减少，待减少到最小的值，则开始增加 duty 值，不断循环。其中 PWM_GAP 状态为调整间隔，时间为 100us。

```
`timescale 1ns / 1ps
module pwm_test(
    input                sys_clk_p,                //system clock
    200Mhz postive pin   input                sys_clk_n,                //system
    clock 200Mhz negetive pin
    input                rst_n,                    //low active
    output               led                      //high-on, low-off
);
localparam CLK_FREQ = 200 ;                    //200MHz
localparam US_COUNT = CLK_FREQ ;                //1 us counter
localparam MS_COUNT = CLK_FREQ*1000 ;          //1 ms counter

localparam DUTY_STEP = 32'd100000 ; //duty step
localparam DUTY_MIN_VALUE = 32'h6fffffff ; //duty minimum value
localparam DUTY_MAX_VALUE = 32'hffffffff ; //duty maximum value

localparam IDLE = 0; //IDLE state
localparam PWM_PLUS = 1; //PWM duty plus state
localparam PWM_MINUS = 2; //PWM duty minus state
localparam PWM_GAP = 3; //PWM duty adjustment gap

wire pwm_out; //pwm output
reg[31:0] period; //pwm step value
reg[31:0] duty; //duty value
reg pwm_flag ; //duty value plus and minus flag, 0: plus; 1: minus

reg[3:0] state;
reg[31:0] timer; //duty adjustment counter

assign led = pwm_out ; //led high active

wire clk ;

IBUFDS IBUFDS_inst (
    .O(clk), // Buffer output
    .I(sys_clk_p), // Diff_p buffer input (connect directly to top-level port)
    .IB(sys_clk_n) // Diff_n buffer input (connect directly to top-level port)
);

always@(posedge clk or negedge rst_n)
begin
    if(rst_n == 1'b0)
    begin
        period    <= 32'd0;
        timer      <= 32'd0;
        duty       <= 32'd0;
        pwm_flag   <= 1'b0 ;
        state      <= IDLE;
    end
end
```

```

else
    case(state)
        IDLE:
            begin
                period      <= 32'd17179;    //The pwm step value, pwm
200Hz(period = 200*2^32/500000000)
                state      <= PWM_PLUS;
                duty        <= DUTY_MIN_VALUE;
            end
            PWM_PLUS :
            begin
                if (duty > DUTY_MAX_VALUE - DUTY_STEP) //if duty is
bigger than DUTY MAX VALUE minus DUTY_STEP , begin to minus duty value
                begin
                    pwm_flag    <= 1'b1 ;
                    duty        <= duty - DUTY_STEP ;
                end
                else
                begin
                    pwm_flag    <= 1'b0 ;
                    duty        <= duty + DUTY_STEP ;
                end

                state          <= PWM_GAP ;
            end
            PWM_MINUS :
            begin
                if (duty < DUTY_MIN_VALUE + DUTY_STEP) //if duty is
little than DUTY MIN VALUE plus duty step, begin to add duty value
                begin
                    pwm_flag    <= 1'b0 ;
                    duty        <= duty + DUTY_STEP ;
                end
                else
                begin
                    pwm_flag    <= 1'b1 ;
                    duty        <= duty - DUTY_STEP ;
                end
                state          <= PWM_GAP ;
            end
            PWM_GAP:
            begin
                if(timer >= US_COUNT*100)          //adjustment gap is 100us
                begin
                    if (pwm_flag)
                        state <= PWM_MINUS ;
                    else
                        state <= PWM_PLUS ;

                    timer <= 32'd0;
                end
                else
                begin
                    timer <= timer + 32'd1;
                end
            end
            default:
            begin
                state <= IDLE;
            end
        endcase
    end
endcase

```

```
end

//Instantiate pwm module
ax_pwm
#(
    .N(32)
)
ax_pwm_m0(
    .clk      (clk),
    .rst      (~rst_n),
    .period   (period),
    .duty      (duty),
    .pwm_out  (pwm_out)
);
endmodule
```

10.3 下载验证

生成 bitstream, 并下载 bit 文件, 可以看到 PL LED 灯产生呼吸灯效果。PWM 是比较常用的模块, 比如风扇转速控制, 电机转速控制等等。

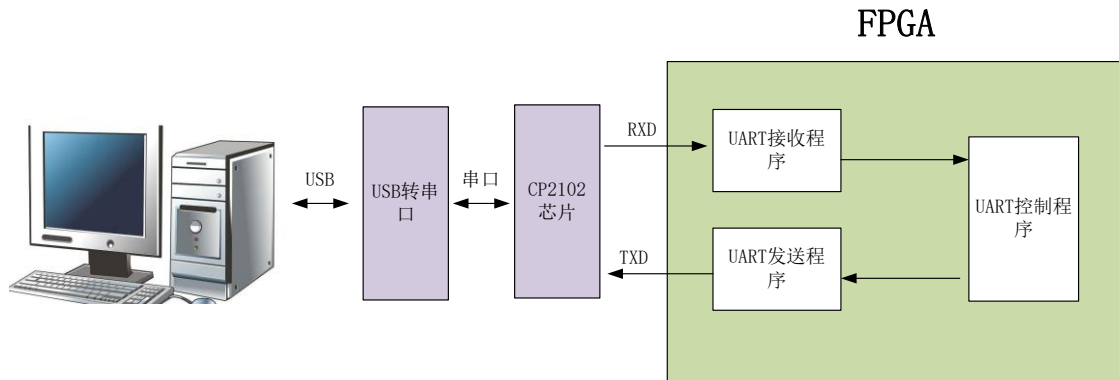
第十一章 UART 实验

实验 Vivado 工程为 “rs232_test”。

本章利用开发板上 PL 端的 UART 接口电路实现 UART 数据传输。

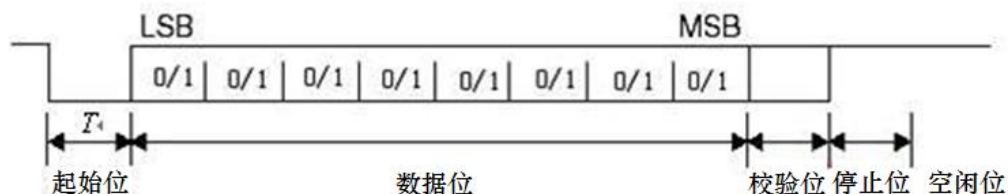
11.1 程序设计

本文所述的串口指异步串行通信，异步串行是指 UART (Universal Asynchronous Receiver/Transmitter)，通用异步接收/发送。本实验程序设计为每秒钟向串口发送“HELLO ALINX”，如果收到 RXD 接收的数据，再把接收的数据发送出去，实现回环的功能。



11.1.1 异步串口通信协议

消息帧从一个低位起始位开始，后面是 7 个或 8 个数据位，一个可用的奇偶位和一个或几个高位停止位。接收器发现开始位时它就知道数据准备发送，并尝试与发送器时钟频率同步。如果选择了奇偶校验，UART 就在数据位后面加上奇偶位。奇偶位可用来帮助错误校验。在接收过程中，UART 从消息帧中去掉起始位和结束位，对进来的字节进行奇偶校验，并将数据字节从串行转换成并行。UART 传输时序如下图所示：



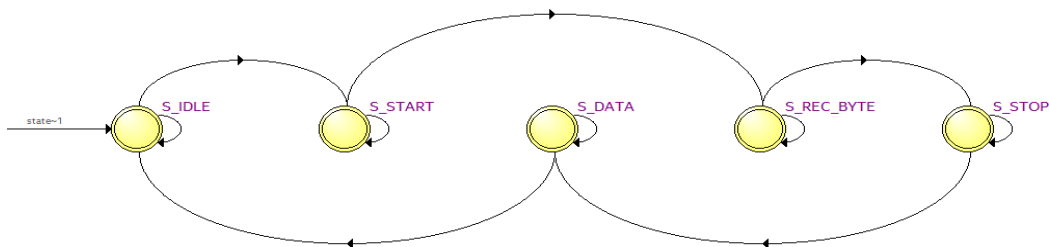
从波形上可以看出起始位是低电平，停止位和空闲位都是高电平，也就是说没有数据传输时是高电平，利用这个特点我们可以准确接收数据，当一个下降沿事件发生时，我们认为将进行一次数据传输。

11.1.2 波特率

常见的串口通信波特率有 2400、9600、115200 等，发送和接收波特率必须保持一致才能正确通信。波特率是指 1 秒最大传输的数据位数，包括起始位、数据位、校验位、停止位。假如通信波特率设定为 9600，那么一个数据位的时间长度是 $1/9600$ 秒，本实验中的波特率由 50MHz 时钟产生。

11.1.3 接收模块设计

串口接收模块 `uart_rx` 是个参数化可配置模块，参数 “CLK_FRE” 定义接收模块的系统时钟频率，单位是 Mhz，参数 “BAUD_RATE” 是波特率。接收状态机状态转换图如下：



“S_IDLE” 状态为空闲状态，上电后进入 “S_IDLE”，如果信号 “rx_pin” 有下降沿，我们认为是串口的起始位，进入状态 “S_START”，等一个 BIT 时间起始位结束后进入数据位接收状态 “S_REC_BYTE”，本实验中数据位设计是 8 位，接收完成以后进入 “S_STOP” 状态，在 “S_STOP” 没有等待一个 BIT 周期，**只等待了半个 BIT 时间**，这是因为如果等待了一个周期，有可能会错过下一个数据的起始位判断，最后进入 “S_DATA” 状态，将接收到的数据送到其他模块。在这个模块我们提一点：为了满足采样定理，在接受数据时每个数据都在波特率计数器的时间中点进行采样，以避免数据出错的情况：

```
//receive serial data bit data
always@(posedge clk or negedge rst_n)
begin
    if(rst_n == 1'b0)
        rx_bits <= 8'd0;
    else if(state == S_REC_BYTE && cycle_cnt == CYCLE/2 - 1)
        rx_bits[bit_cnt] <= rx_pin;
    else
        rx_bits <= rx_bits;
end
```

注意：**本实验没有设计奇偶校验位。**

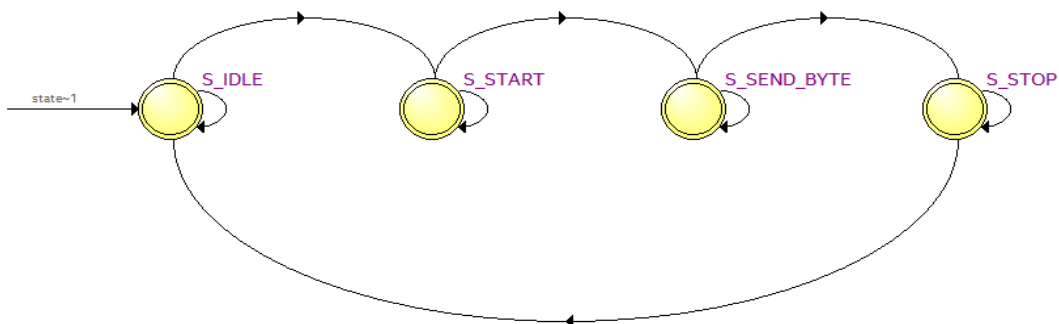
信号名称	方向	宽度 (bit)	说明
clk	in	1	系统时钟
rst_n	in	1	异步复位，低电平复位
rx_data	out	8	接收到的串口数据（8 位数据）
rx_data_valid	out	1	接收到的串口数据有效（高有效）

rx_data_ready	in	1	表示用户可以从接收模块接收数据，当 rx_data_ready 和 rx_data_valid 都为高时数据送出
rx_pin	in	1	串口接收数据输入

串口接收模块 uart_rx 端口

11.1.4 发送模块设计

发送模块 uart_tx 设计和接收模块相似，也是使用状态机，状态转换图如下：



上电后进入“S_IDLE”空闲状态，如果有发送请求，进入发送起始位状态“S_START”，起始位发送完成后进入发送数据位状态“S_SEND_BYTE”，数据位发送完成后进入发送停止位状态“S_STOP”，停止位发送完成后又进入空闲状态。在数据发送模块中，从顶层模块写入的数据直接传递给寄存器‘tx_reg’，并通过‘tx_reg’寄存器模拟串口传输协议在状态机的条件转换下进行数据传送：

```

always@(posedge clk or negedge rst_n)
begin
    if(rst_n == 1'b0)
        tx_reg <= 1'b1;
    else
        case(state)
            S_IDLE,S_STOP:
                tx_reg <= 1'b1;
            S_START:
                tx_reg <= 1'b0;
            S_SEND_BYTE:
                tx_reg <= tx_data_latch[bit_cnt];
            default:
                tx_reg <= 1'b1;
        endcase
    end
end
  
```

信号名称	方向	宽度 (bit)	说明
clk	in	1	系统时钟
rst_n	in	1	异步复位，低电平复位
tx_data	in	8	要发送的串口数据(8 位数据)
tx_data_valid	in	1	发送的串口数据有效（高有效）

tx_data_ready	out	1	发送模块已准备好发送数据，用户可将 tx_data_valid 信号拉高发送数据给发送模块。当 tx_data_ready 和 tx_data_valid 都为高时数据被发送
tx_pin	out	1	串口发送数据发送

串口发送模块 uart_tx 端口

11.1.5 波特率的产生

在发送和接收模块中，声明了参数 CYCLE，也就是 UART 一个周期的计数值，当然计数是在 50MHz 时钟下进行的。用户只要设定好 CLK_FRE 和 BAUD_RATE 这两个参数即可。

```

module uart_rx
#(
    parameter CLK_FRE = 50,          //clock frequency(Mhz)
    parameter BAUD_RATE = 115200    //serial baud rate
)
(
    input                clk,          //clock input
    input                rst_n,        //asynchronous reset input, low active
    output reg[7:0]      rx_data,      //received serial data
    output reg           rx_data_valid, //received serial data is valid
    input                rx_data_ready, //data receiver module ready
    input                rx_pin        //serial data input
);
//calculates the clock cycle for baud rate
localparam CYCLE = CLK_FRE * 1000000 / BAUD_RATE;
//state machine code

```

11.1.6 测试程序

测试程序设计 FPGA 为 1 秒向串口发送一次 “HELLO ALINX\r\n”，不发送期间，如果接受到串口数据，直接把接收到的数据送到发送模块再返回。“\r\n”，在这里和 C 语言中表示一致，都是回车换行。

测试程序分别例化了发送模块和接收模块，同时将参数传递进去，波特率设置为 115200。

```

always@(posedge sys_clk or negedge rst_n)
begin
    if(rst_n == 1'b0)
    begin
        wait_cnt <= 32'd0;
        tx_data <= 8'd0;
        state <= IDLE;
        tx_cnt <= 8'd0;
        tx_data_valid <= 1'b0;
    end
    else
    case(state)
        IDLE:
            state <= SEND;
        SEND:
            begin

```

```

wait_cnt <= 32'd0;
tx_data <= tx_str;

data
if(tx_data_valid == 1'b1 && tx_data_ready == 1'b1 && tx_cnt < 8'd12)//Send 12 bytes
begin
    tx_cnt <= tx_cnt + 8'd1; //Send data counter
end
else if(tx_data_valid && tx_data_ready)//last byte sent is complete
begin
    tx_cnt <= 8'd0;
    tx_data_valid <= 1'b0;
    state <= WAIT;
end
else if(~tx_data_valid)
begin
    tx_data_valid <= 1'b1;
end
end
WAIT:
begin
    wait_cnt <= wait_cnt + 32'd1;

    if(rx_data_valid == 1'b1)
    begin
        tx_data_valid <= 1'b1;
        tx_data <= rx_data; // send uart received data
    end
    else if(tx_data_valid && tx_data_ready)
    begin
        tx_data_valid <= 1'b0;
    end
    else if(wait_cnt >= CLK_FRE * 1000000) // wait for 1 second
    state <= SEND;

end
default:
    state <= IDLE;
endcase
end

//combinational logic
//Send "HELLO ALINX\r\n"
always@(*)
begin
    case(tx_cnt)
        8'd0: tx_str <= "H";
        8'd1: tx_str <= "E";
        8'd2: tx_str <= "L";
        8'd3: tx_str <= "L";
        8'd4: tx_str <= "O";
        8'd5: tx_str <= " ";
        8'd6: tx_str <= "A";
        8'd7: tx_str <= "L";
        8'd8: tx_str <= "I";
        8'd9: tx_str <= "N";
        8'd10: tx_str <= "X";
        8'd11: tx_str <= "\r";
        8'd12: tx_str <= "\n";
        default:tx_str <= 8'd0;
    endcase
end
uart_rx#
(
    .CLK_FRE (CLK_FRE) ,
    .BAUD_RATE (115200)
) uart_rx_inst

```

```

(
    .clk                (sys_clk                ),
    .rst_n              (rst_n                  ),
    .rx_data            (rx_data                ),
    .rx_data_valid      (rx_data_valid          ),
    .rx_data_ready      (rx_data_ready          ),
    .rx_pin             (uart_rx                )
);

uart_tx#
(
    .CLK_FRE(CLK_FRE),
    .BAUD_RATE(115200)
) uart_tx_inst
(
    .clk                (sys_clk                ),
    .rst_n              (rst_n                  ),
    .tx_data            (tx_data                ),
    .tx_data_valid      (tx_data_valid          ),
    .tx_data_ready      (tx_data_ready          ),
    .tx_pin             (uart_tx                )
);

```

11.2 仿真

这里我们添加了一个串口接收的激励程序 vtf_uart_test.v 文件，用来仿真 uart 串口接收。这里向串口模块的 uart_rx 发送 0xa3 的数据，每位的数据按 115200 的波特率发送，1 位起始位，8 位数据位和 1 位停止位。

```

always #10 sys_clk = ~ sys_clk; //20ns一个周期，产生50MHz时钟源

parameter BPS_115200 = 8680; //每个比特的时间
parameter SEND_DATA = 8'b1010_0011; //要发送的数据

integer i = 0;

initial begin
    uart_rx = 1'b1; //bus idle
    #1000 uart_rx = 1'b0; //transmit start bit

    for (i=0;i<8;i=i+1)
        #BPS_115200 uart_rx = SEND_DATA[i]; //transmit data bit

    #BPS_115200 uart_rx = 1'b0; //transmit stop bit
    #BPS_115200 uart_rx = 1'b1; //bus idle

end

endmodule

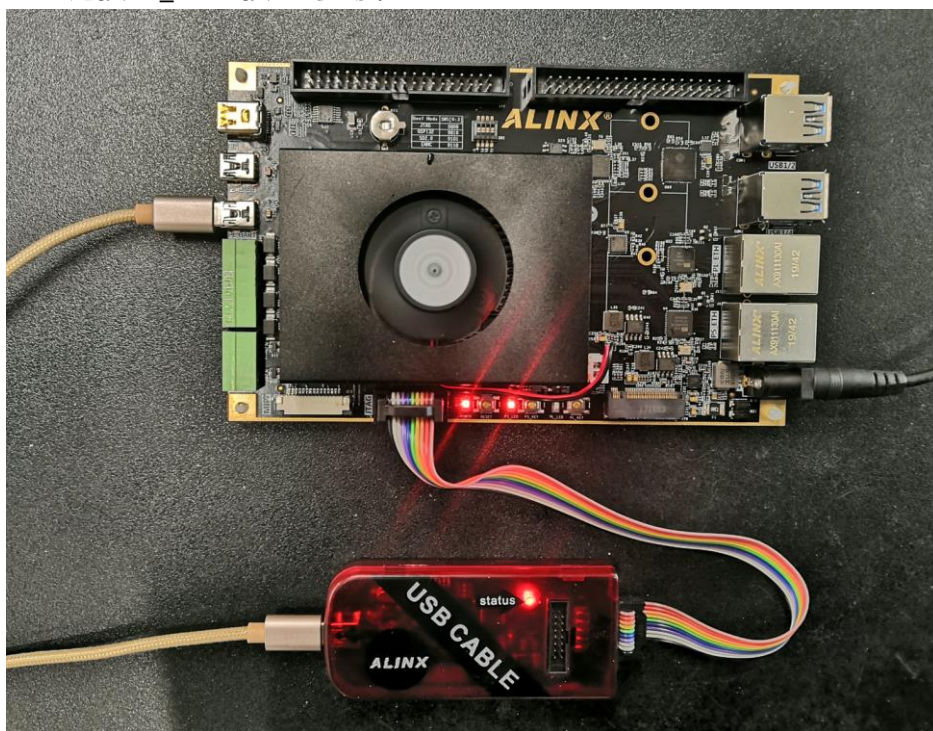
```

仿真的结果如下，当程序接收到 8 位数据的时候，rx_data_valid 有效，rx_data[7:0] 的数据位 a3。

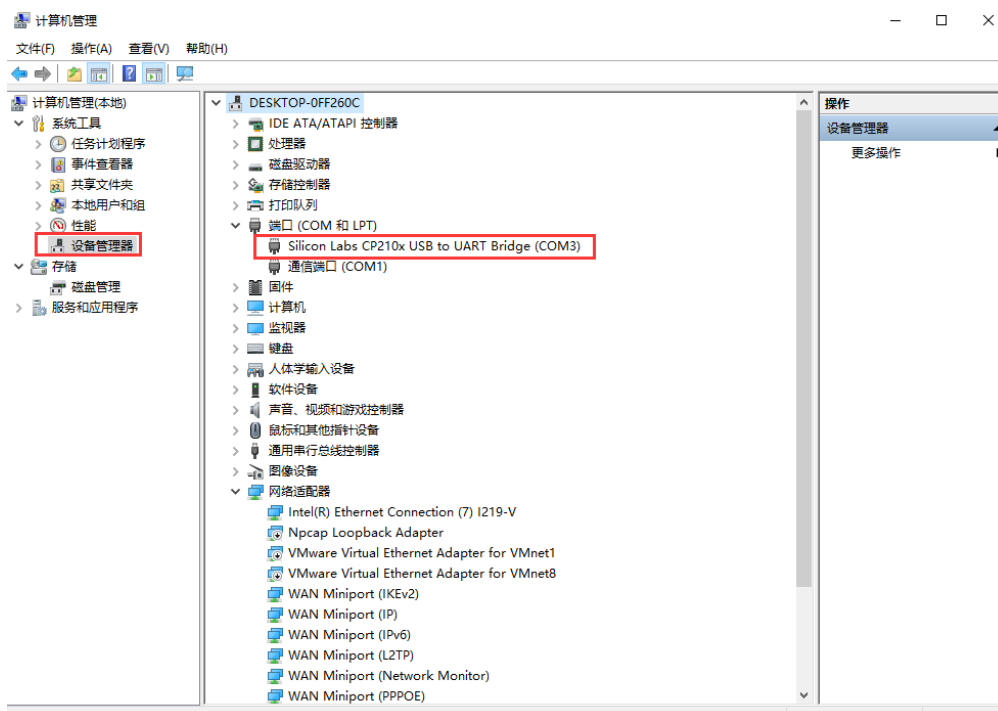


11.3 实验测试

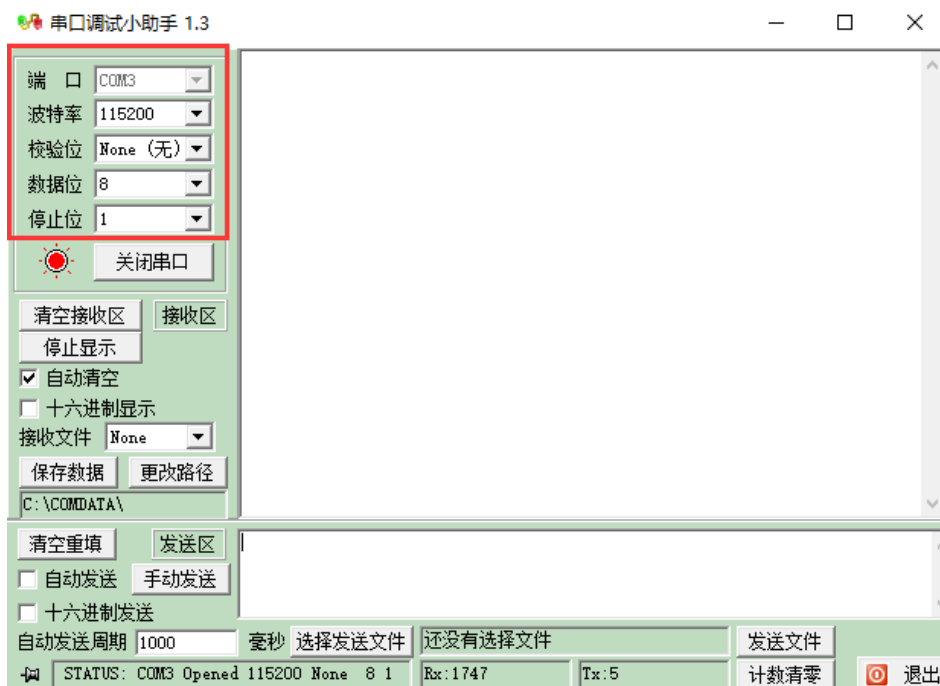
通过 USB 连接 PL_UART 接口到电脑上



在设备管理器中找到串口号“COM5”



打开串口调试，端口选择“COM79”（根据自己情况选择），波特率设置 115200，检验位选 None，数据位选 8，停止位选 1，然后点击“打开串口”。此软件在例程文件夹下。



打开串口以后，每秒可收到“HELLO ALINX”，在发送区输入框输入要发送的文字，点击“手动发送”，可以看到接收到自己发送的字符。



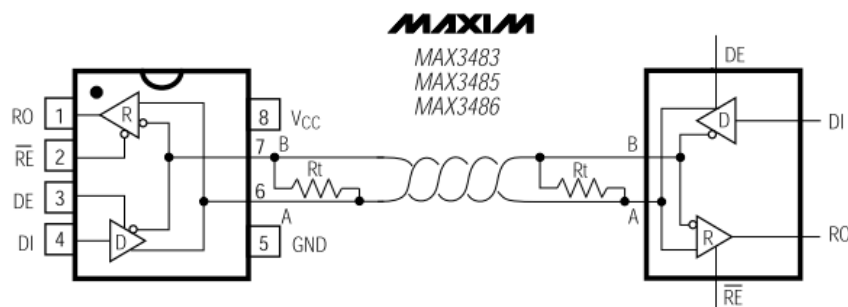
第十二章 RS485 实验

实验 Vivado 工程为 “rs485_test”。

本章以 AN3485 模块介绍 RS485 的数据传输。

12.1 实验原理

前面介绍过 UART 的实验，RS485 是采用差分信号传输，但 RS485 是半双工传输，也就是说，同一时刻只能有一个方向的数据传输。只有差分信号 A 和 B，而与 ARM 或 FPGA 相连的信号为 DE（方向选择），DI（输入信号 TXD），RO（输出信号 RXD）。



从 MAX3485 文档中，发送方向，如果 DE 为 1 时，也就是输出使能，DI 值为 1 时，对于差分信号 A 和 B 值为 1 和 0，否则为 0 和 1。

Table 1. Transmitting

INPUTS			OUTPUTS		MODE
RE	DE	DI	B*	A*	
X	1	1	0	1	Normal
X	1	0	1	0	Normal
0	0	X	High-Z	High-Z	Normal
1	0	X	High-Z	High-Z	Shutdown

从接收来看，如果 DE 为 0，A 和 B 之间差值大于等于 +0.2V，则 RO 值为 1，否则为 0。

Table 2. Receiving

INPUTS			OUTPUTS	MODE
RE	DE	A, B	RO	
0	0*	$\geq +0.2V$	1	Normal
0	0*	$\leq -0.2V$	0	Normal
0	0*	Inputs Open	1	Normal
1	0	X	High-Z	Shutdown

12.2 程序设计

由于 RS485 是半双工传输，那么我们需要制定传输协议进行握手，设定第一个字节为 8'h55，表示一帧数据的开始，接下来是传输的数据长度信息，由于 FIFO 大小限制（256），范围为 1~255，接下来是数据。格式即为：起始 8'h55+数据长度+数据。

其中 uart_tx 和 uart_rx 跟 UART 实验一样，在这里只修改 uart_test 即可。我们设计的功能为初始状态下将 DE 设为 0，也就是输入，等待接收上位机发来的数据，并缓存到 FIFO 中，FIFO 大小设置为 256，然后切换 DE 为 1，也就是输出，把接收到的数据从 FIFO 中读出并发送出去。注意缓存的数据是除去起始 8'h55 和数量信息的。

在 RCV_HEAD 状态时，判断接收到的数据是否是“S”。

```
RCV_HEAD:
begin
    if (rx_data_valid == 1'b1 && rx_data == "s")    //when
        state <= RCV_COUNT ;
    end
```

在 RCV_COUNT 状态时，如果数据长度小于 0，则跳转到 IDLE 状态，如果大于 0，则进入接收数据状态。

```
RCV_COUNT:
begin
    if (rx_data_valid == 1'b1)    //recor
    begin
        if (rx_data > 0)    //if da
            state <= RCV_DATA ;
        else
            state <= IDLE ;
        data_count <= rx_data ;
    end
end
```

在 RCV_DATA 状态下，把数据写入 FIFO，并且检查数据长度，切换 RS485 的方向为输出，并跳转状态。

```
RCV_DATA:
begin
    fifo_wren <= rx_data_valid ;
    fifo_wdata <= rx_data ;

    if (rx_data_valid == 1'b1)
    begin
        if (rx_cnt == data_count - 1)    //the last received data
        begin
            rx_cnt <= 8'd0 ;
            rs485_de <= 1'b1 ;
            state <= WAIT ;
        end
        else
            rx_cnt <= rx_cnt + 1'b1 ;
        end
    end
end
```

在切换总线状态时，为了可靠工作，在 WAIT 状态下，延时 1ms 进行方向切换。

```
WAIT:
begin
    fifo_wren <= 1'b0 ;
    if (wait_cnt >= CLK_FRE * 1000)    // wait for 1 ms for direction change
    begin
        wait_cnt <= 32'd0 ;
        state <= SEND_WAIT;
    end
    else
    begin
        wait_cnt <= wait_cnt + 32'd1;
    end
end
```

再然后是发送 FIFO 中的数据，SEND_WAIT 状态是控制读使能信号 fifo_rden，并且判断数据是否发送完，发送完后进入 IDLE 状态。

```

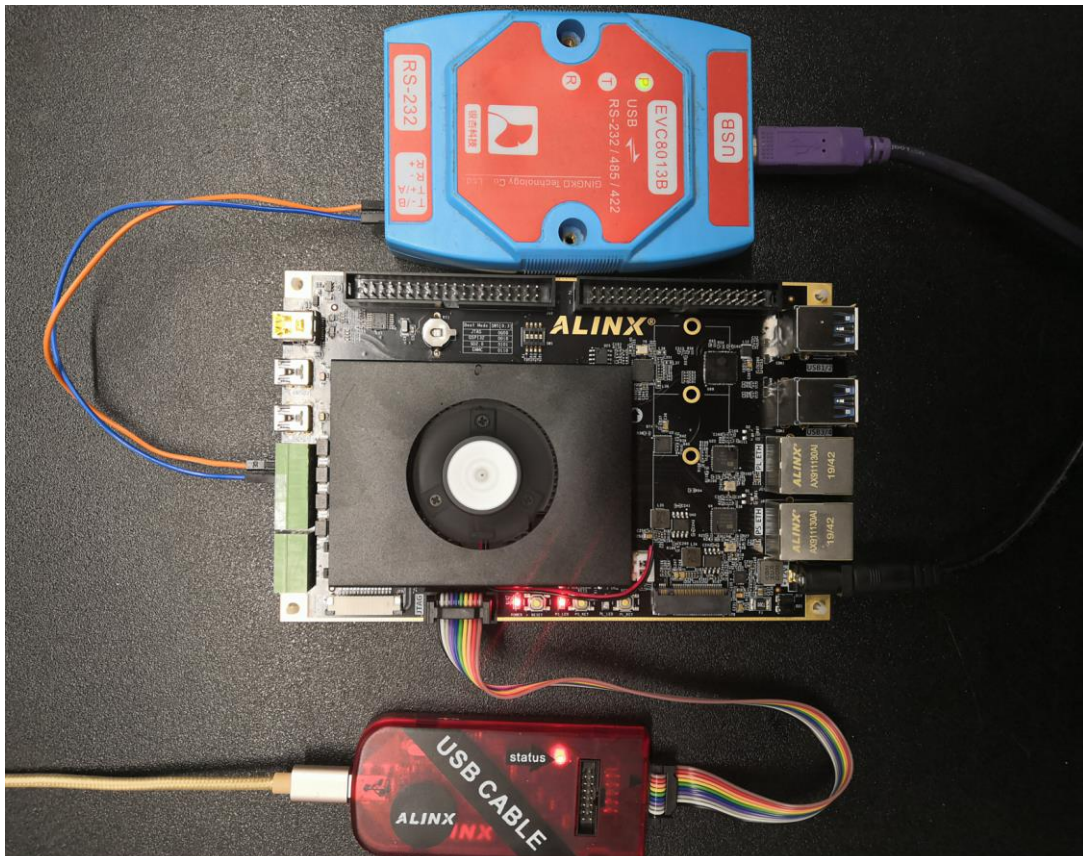
SEND_WAIT:
begin
  if (tx_data_ready == 1'b1)
  begin
    if (tx_cnt == data_count)           //the last data has transferred
    begin
      tx_cnt      <= 8'd0 ;
      fifo_rden   <= 1'b0 ;
      state       <= IDLE ;
    end
    else
    begin
      fifo_rden   <= 1'b1 ;           //read data from fifo
      state       <= SEND ;
    end
  end
  tx_data_valid  <= 1'b0 ;
end
SEND:
begin
  fifo_rden      <= 1'b0 ;
  tx_data_valid  <= 1'b1 ;

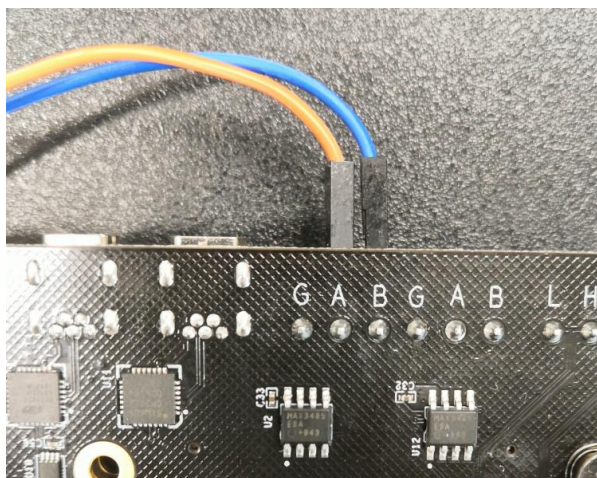
  if(tx_data_valid == 1'b1 && tx_data_ready == 1'b1 && tx_cnt < data_count)
  begin
    tx_cnt <= tx_cnt + 8'd1; //Send data counter
    state <= SEND_WAIT ;
  end
end
end

```

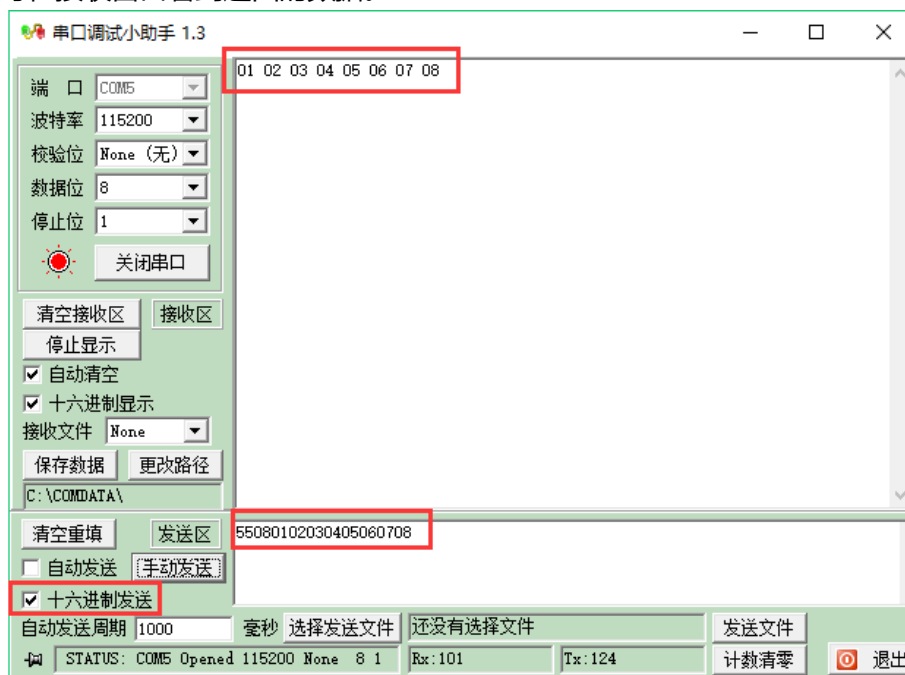
12.3 实验测试

我们使用 USB 转串口设备，通过杜邦线将 RS485_1 的 A 和 B 分别与设备的 A 和 B 连接。





打开串口工具，设置好串口号波特率，选择 16 进制发送，发送数据以 8'h55 开头，点击发送，即可在接收窗口看到返回的数据。



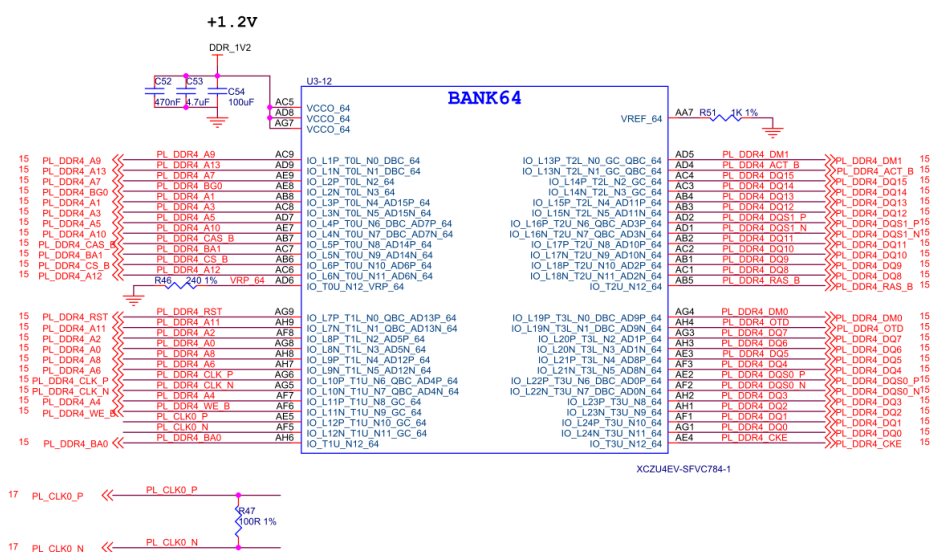
可以在此基础上，测试另外一路接口。

第十三章 PL 端 DDR4 读写测试实验

实验 Vivado 工程为 “pl_ddr4_test”。

13.1 硬件介绍

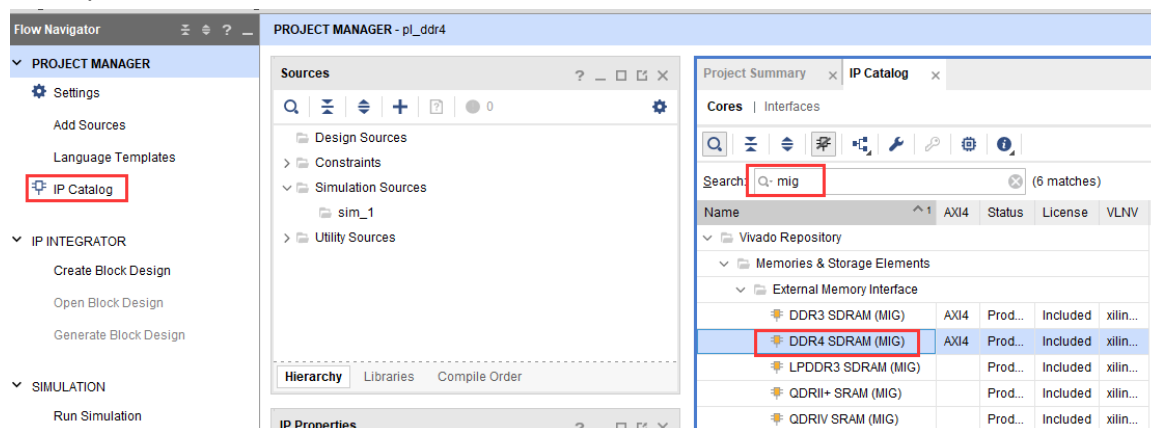
开发板的 PL 端有一颗 16bit ddr4，这很大程度方便我们移植以前的 FPGA 工程到 ZYNQ 系统中，同时也提供了更大的带宽。



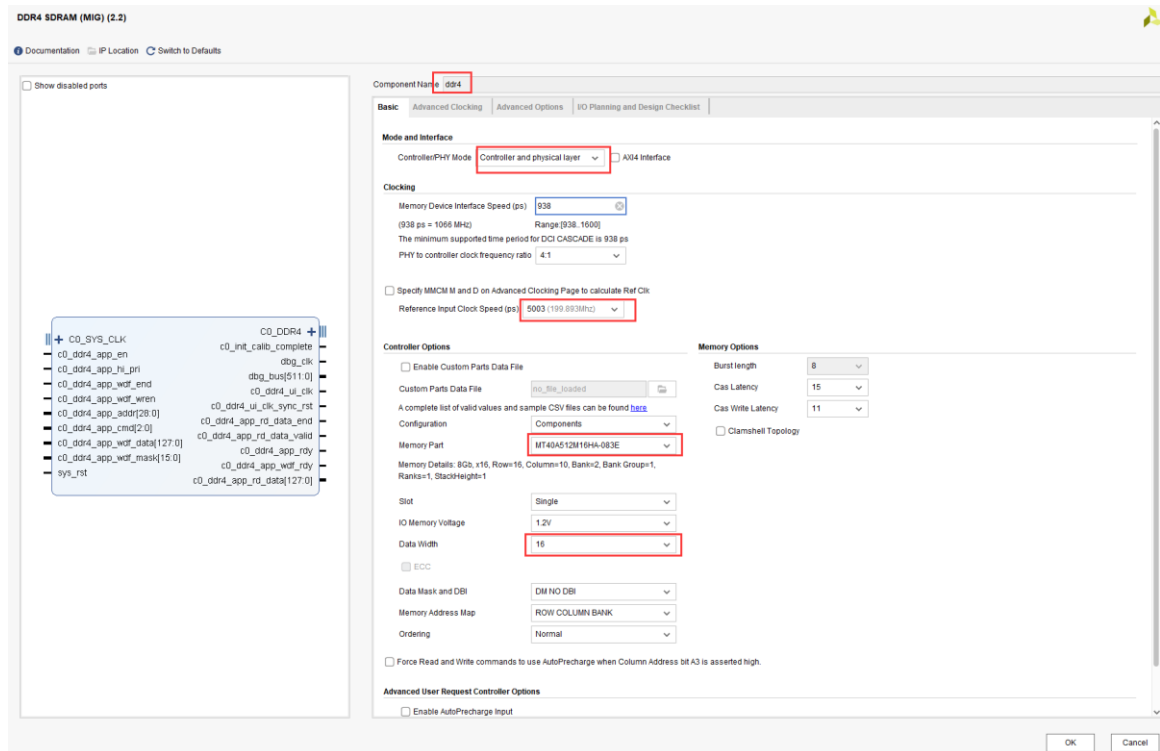
13.2 Vivado 工程建立

13.2.1 创建一个 PL 端 ddr4 测试工程并配置 ddr4 IP

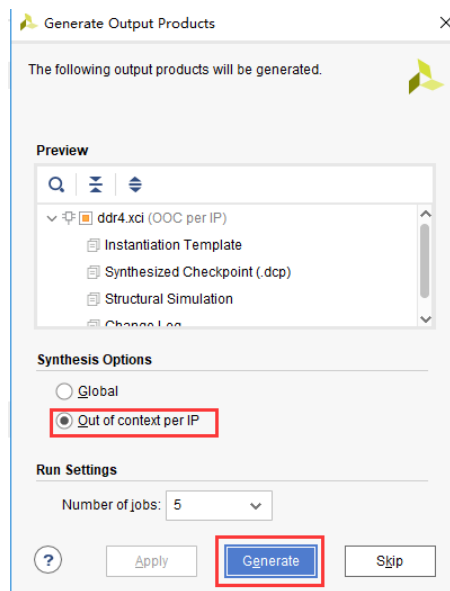
- 1) 在 “IP Catalog” 的搜索框搜索 “mig”，快速找到 “Memory Interface Generator”，双击



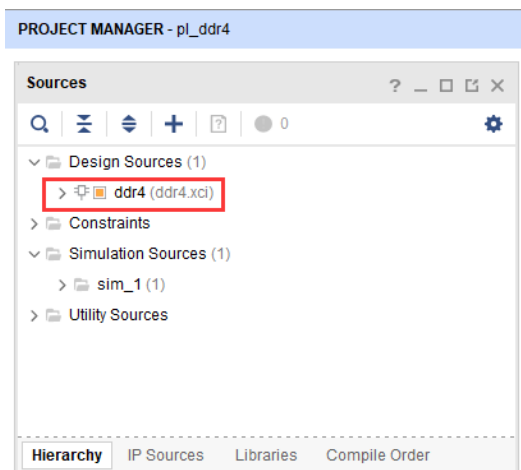
- 2) Component Name 可以修改，Controller/PHY Mode 选择“Controller and physical layer”，参考时钟选择 200MHz，即 5003ps，Memory Part 选择“MT40A512M16HA-083E”，Data Width 选择 16，其他设置保持默认，点击 OK



- 3) Generate

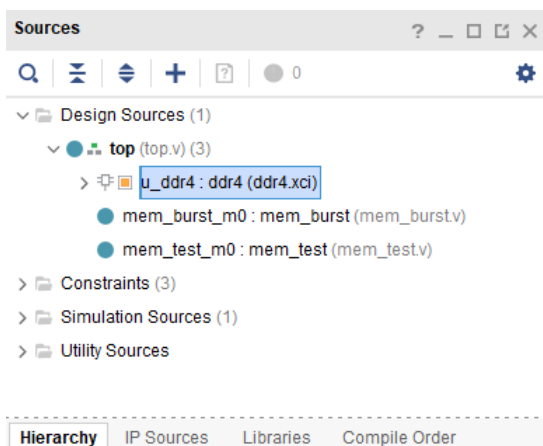


- 4) 生成结果如下

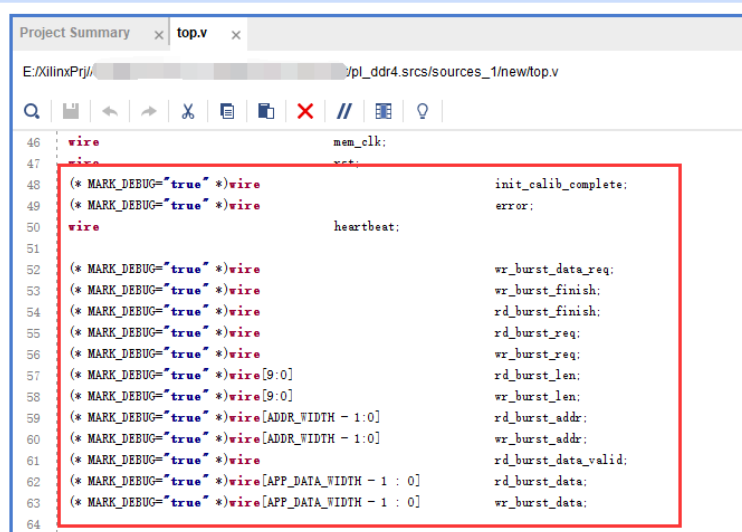


13.2.2 添加其他测试代码

其他代码主要功能是读写 ddr3 并比较数据是否一致，这里不做详细介绍，可参考工程代码。



在 mem_test.v 中添加 mark_debug 调试，具体操作流程请参考 PL 的“Hello World”LED 实验



13.3 下载调试

生成 bit 文件以后，使用 JTAG 下载到开发板，在 MIG_1 窗口会显示 DDR4 校准等信息

The screenshot shows the Xilinx IDE interface. On the left, the 'Hardware' pane displays the device hierarchy with 'MIG_1 (u_d04)' selected. The 'Hardware Device Properties' pane shows details for 'xczu3_0'. The main window displays the 'MIG_1' properties, including the 'Calibration and Margins' table.

Name	Left Margin (ps)	Center Point (ps)	Right Margin (ps)
Rank 0			
Byte 0			
Nibble 0		238	299
Nibble 1		238	270
Byte 1			
Nibble 0	238	295	238
Nibble 1	245	282	245

在 hw_ila_1 中可以查看调试信号

The screenshot shows the Xilinx IDE interface. On the left, the 'Hardware' pane displays the device hierarchy with 'hw_ila_1 (u_ila_0)' selected. The 'Hardware Device Properties' pane shows details for 'hw_ila_1'. The main window displays the 'Waveform - hw_ila_1' view, showing various debug signals.

Name	Value
mem_test_m0memor	0
mem_test_m0test_cntr[15:0]	0046
mem_test_m0state[2:0]	MEM_READ
mem_test_m0wr_pre_add[15:0]	0046
mem_test_m0wr_data[127:0]	00450045004500450045004500450045
mem_test_m0wr_burst_len[9:0]	080
mem_test_m0wr_cntr[9:0]	000
mem_test_m0wr_burst_addr[28:0]	00000000
mem_test_m0wr_burst_data_req	0
mem_test_m0wr_burst_finish	0
mem_test_m0wr_burst_req	0
mem_test_m0rd_data[127:0]	19a519a519
mem_test_m0rd_burst_len[9:0]	080
mem_test_m0rd_burst_addr[26:0]	3cd1980
mem_test_m0rd_pre_add[15:0]	19c6
mem_test_m0rd_cntr[9:0]	000

13.4 实验总结

本实验通过 PL 端 Verilog 代码直接读写 ddr4，我们也可以把 ddr4 配置成 AXI 接口，这样方便和 ARM 系统完成数据交互。

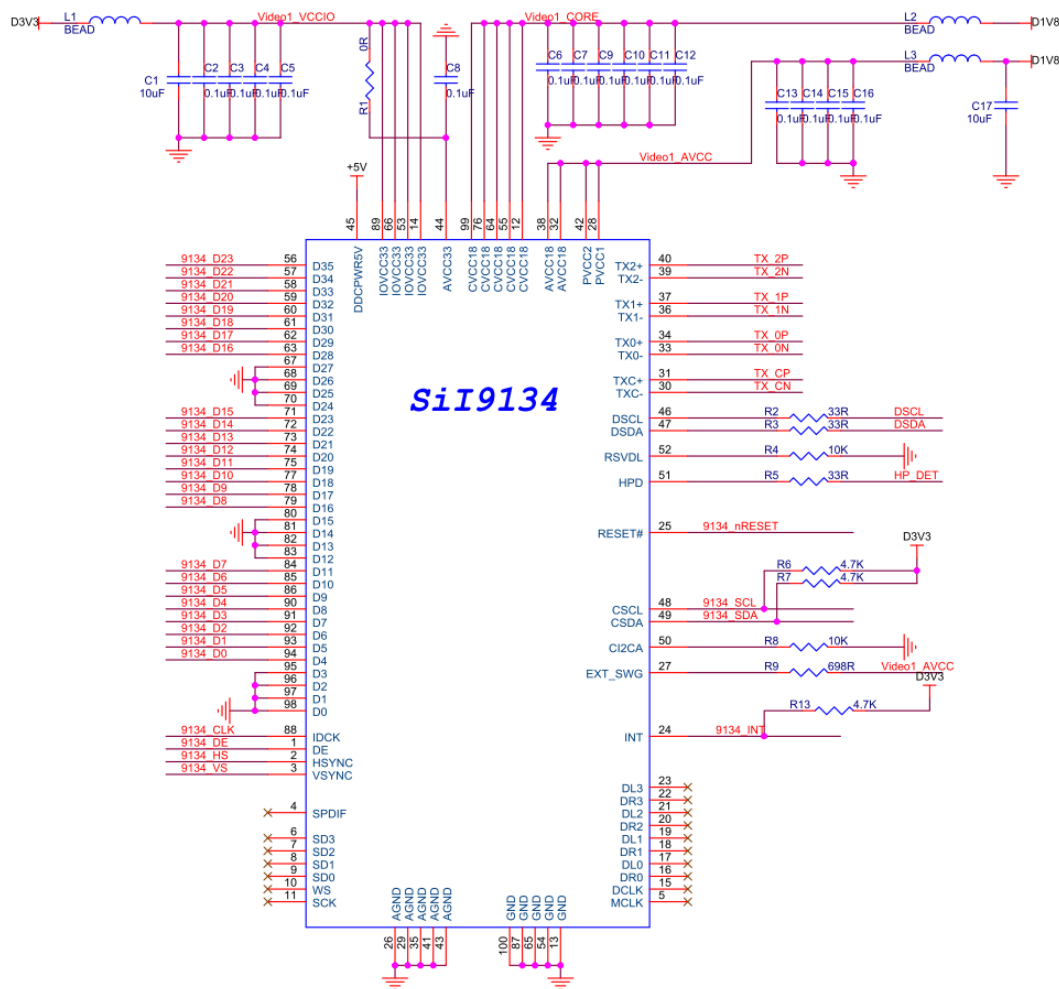
第十四章 HDMI 输出实验

实验 Vivado 工程为 “hdmi_out_test”。

前面我们介绍了 led 闪灯实验，只是为了了解 Vivado 的基本开发流程，本章这个实验相对 LED 闪灯实验复杂点，做一个 HDMI 输出的彩条，这也是我们后面学习显示、视频处理的基础。实验还不涉及到 PS 系统，从实验设计可以看出如果要非常好的使用 ZYNQ 芯片，需要良好的 FPGA 基础知识。

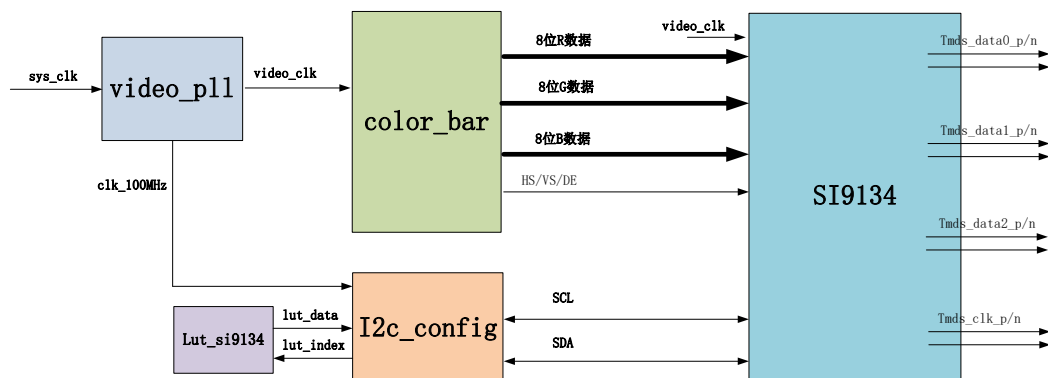
14.1 硬件介绍

由于开发板上只有 DP 可以显示，但却是 PS 端的，PL 端没有 HDMI 的接口，因此我们采用 AN9134 的 HDMI 扩展模块实现 HDMI 显示。将 24 位 RGB 编码输出 TMDS 差分信号。SiI9134 功能强大，本实验只使用其中一小部分，将 RGB24 视频数据显示出来即可。



SI9134 芯片需要通过 I2C 总线配置寄存器才能正常工作，从原理图中可以看出 I2C 总线连接到 PL 端的 IO，可以通过 PL 直接配置。

14.2 程序设计



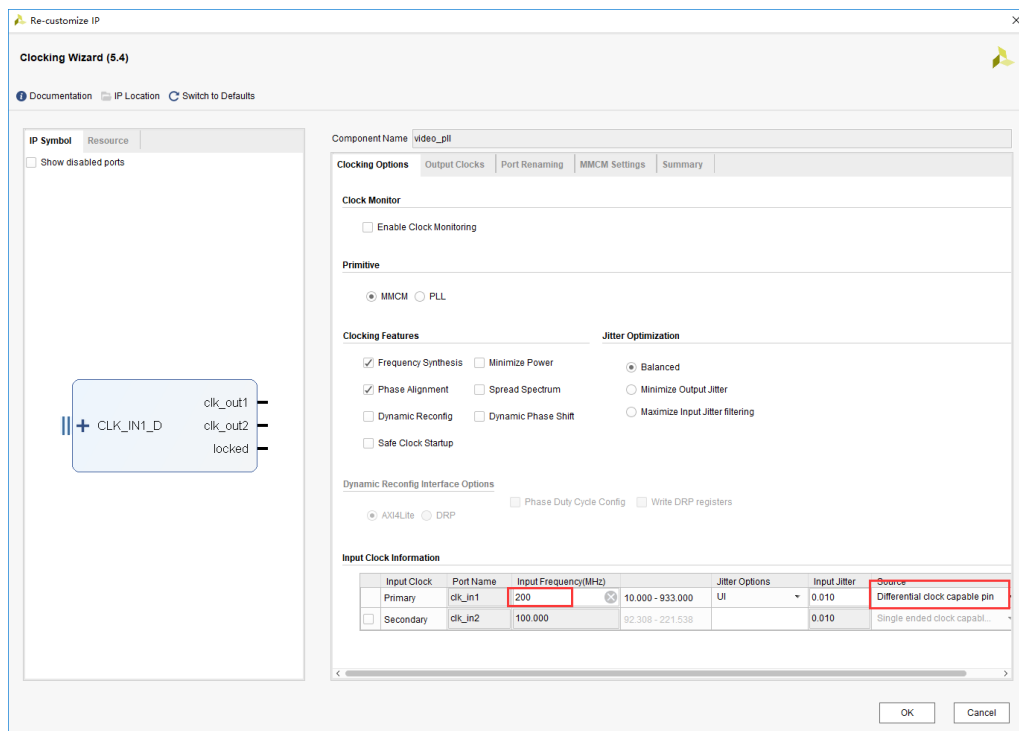
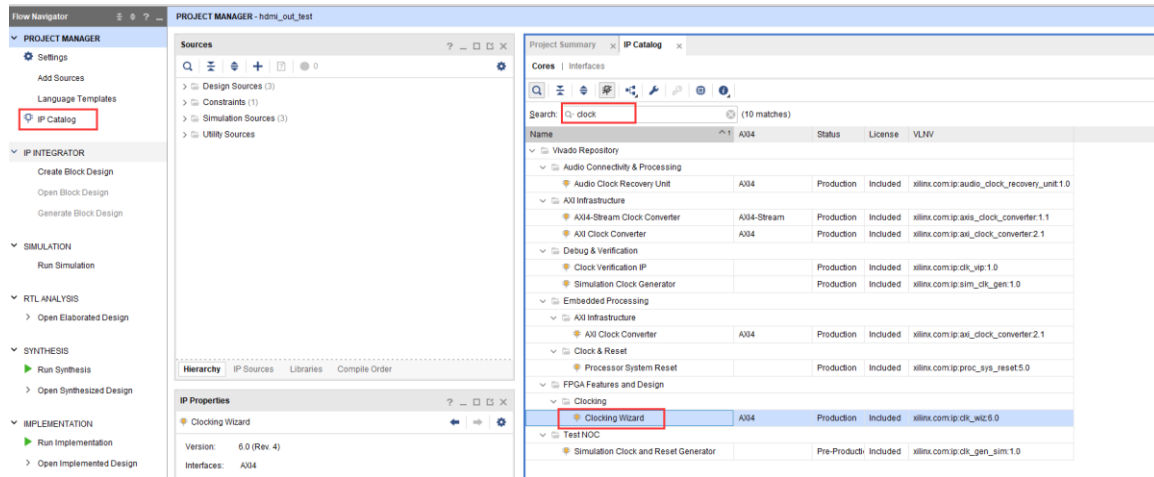
本实验实现通过 HDMI 显示彩条，实验中设计了视频时序发生和彩条发生模块“color_bar.v”，I2C Master 寄存器配置模块“i2c_config.v”，配置数据查找表模块“lut_si9134.v”。

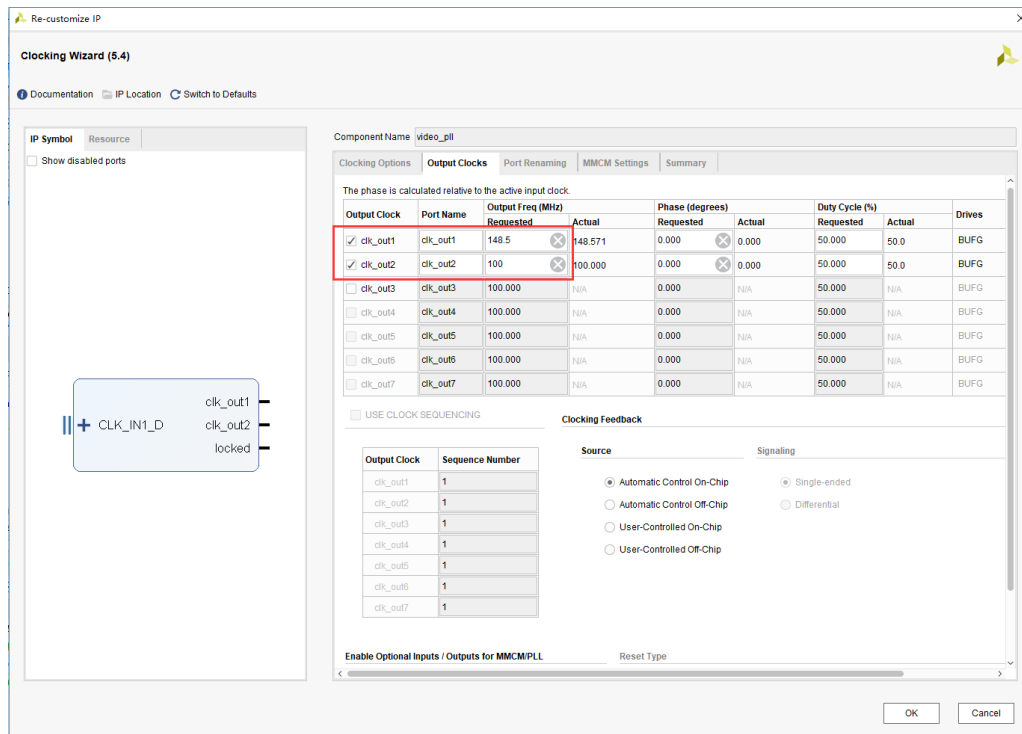
具体代码在这里不再一一介绍了，大家自己去看。下面针对每个模块实现的功能给大家做一下简介：

顶层模块 top.v 是项目的顶层文件，主要是实例化 4 个子模块（时钟模块 video_pll，彩条生成模块 color_bar 和 I2C 配置模块 i2c_config 和配置查找表模块 lut_si9134。

彩条产生模块 color_bar.v 是产生 8 种颜色的 VGA 格式的彩条，彩条分别为白、黄、青、绿、紫、红、蓝和黑。产生分辨率为 1920x1080 刷新率为 60Hz 的彩条,也就是所谓的 1080P 的高清视频图像。所以这个模块会输出 R（8 位）G（8 位）B（8 位）图像信号、行同步、列同步和数据有效信号。

时钟模块 video_pll 调用的是一个 Xilinx 提供的时钟 IP，通过输入的系统时钟产生一个 100Mhz 时钟和一个 1080P 的像素时钟 148.5Mhz。生成时钟 IP 的方法是点击 Project Manager 目录下的 IP Catalog,再选择 FPGA Features and Design->Clocking->Clocking Wizard 图标。





14.3 添加 XDC 约束文件

添加以下的 xdc 约束文件到项目中，在约束文件里添加了时钟和 HDMI 相关的管脚。

```
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
##### clock define#####
create_clock -period 5.000 [get_ports sys_clk_p]
set_property PACKAGE_PIN AE5 [get_ports sys_clk_p]
set_property IOSTANDARD DIFF_SSTL12 [get_ports sys_clk_p]

set_property PACKAGE_PIN H12 [get_ports hdmi_clk]
set_property PACKAGE_PIN G13 [get_ports {hdmi_d[0]}]
set_property PACKAGE_PIN H13 [get_ports {hdmi_d[1]}]
set_property PACKAGE_PIN H14 [get_ports {hdmi_d[2]}]
set_property PACKAGE_PIN J14 [get_ports {hdmi_d[3]}]
set_property PACKAGE_PIN K14 [get_ports {hdmi_d[4]}]
set_property PACKAGE_PIN J12 [get_ports {hdmi_d[5]}]
set_property PACKAGE_PIN L13 [get_ports {hdmi_d[6]}]
set_property PACKAGE_PIN L14 [get_ports {hdmi_d[7]}]
set_property PACKAGE_PIN C13 [get_ports {hdmi_d[8]}]
set_property PACKAGE_PIN C14 [get_ports {hdmi_d[9]}]
set_property PACKAGE_PIN A14 [get_ports {hdmi_d[10]}]
set_property PACKAGE_PIN B14 [get_ports {hdmi_d[11]}]
set_property PACKAGE_PIN A13 [get_ports {hdmi_d[12]}]
set_property PACKAGE_PIN B13 [get_ports {hdmi_d[13]}]
set_property PACKAGE_PIN E13 [get_ports {hdmi_d[14]}]
set_property PACKAGE_PIN E14 [get_ports {hdmi_d[15]}]
set_property PACKAGE_PIN F11 [get_ports {hdmi_d[16]}]
set_property PACKAGE_PIN F12 [get_ports {hdmi_d[17]}]
set_property PACKAGE_PIN A11 [get_ports {hdmi_d[18]}]
```

```
set_property PACKAGE_PIN A12 [get_ports {hdmi_d[19]]}
set_property PACKAGE_PIN C12 [get_ports {hdmi_d[20]]}
set_property PACKAGE_PIN D12 [get_ports {hdmi_d[21]]}
set_property PACKAGE_PIN F10 [get_ports {hdmi_d[22]]}
set_property PACKAGE_PIN G11 [get_ports {hdmi_d[23]]}
set_property PACKAGE_PIN F13 [get_ports hdmi_de]
set_property PACKAGE_PIN G15 [get_ports hdmi_hs]
set_property PACKAGE_PIN G14 [get_ports hdmi_vs]
set_property PACKAGE_PIN B10 [get_ports hdmi_scl]
set_property PACKAGE_PIN C11 [get_ports hdmi_sda]
set_property PACKAGE_PIN D14 [get_ports hdmi_nreset]

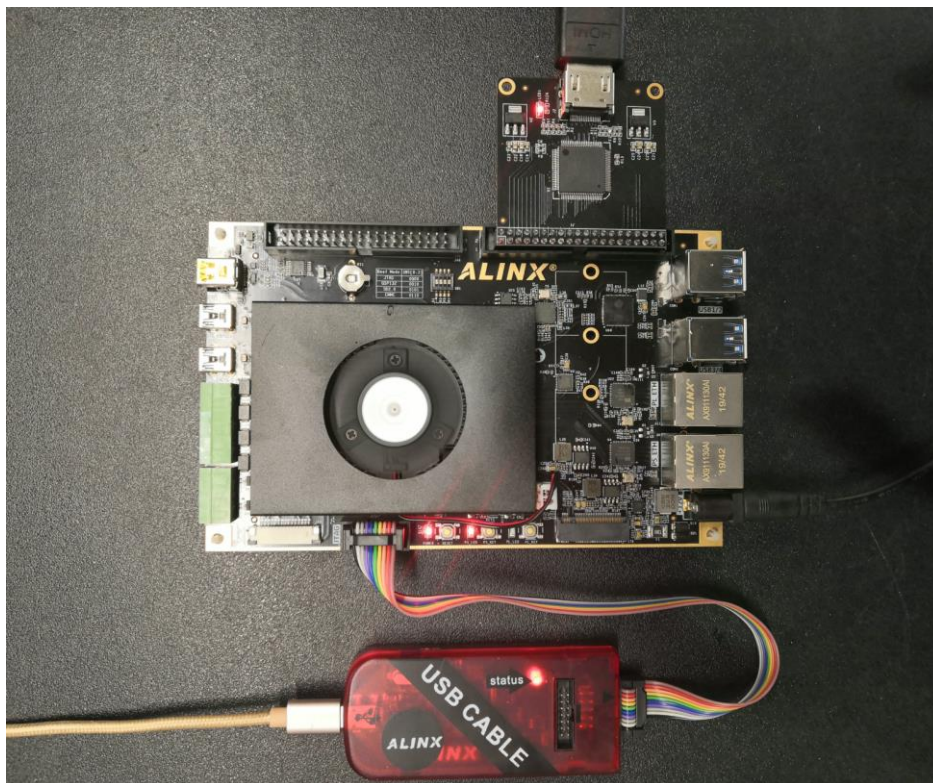
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_scl]
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_sda]
set_property IOSTANDARD LVCMOS33 [get_ports {hdmi_d[*]}]
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_clk]
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_de]
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_vs]
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_hs]
set_property IOSTANDARD LVCMOS33 [get_ports hdmi_nreset]

set_property PULLUP true [get_ports hdmi_scl]
set_property PULLUP true [get_ports hdmi_sda]

set_property SLEW FAST [get_ports {hdmi_d[*]}]
set_property DRIVE 8 [get_ports {hdmi_d[*]}]
set_property SLEW FAST [get_ports hdmi_clk]
set_property SLEW FAST [get_ports hdmi_de]
set_property SLEW FAST [get_ports hdmi_hs]
set_property SLEW FAST [get_ports hdmi_scl]
set_property SLEW FAST [get_ports hdmi_sda]
set_property SLEW FAST [get_ports hdmi_vs]
```

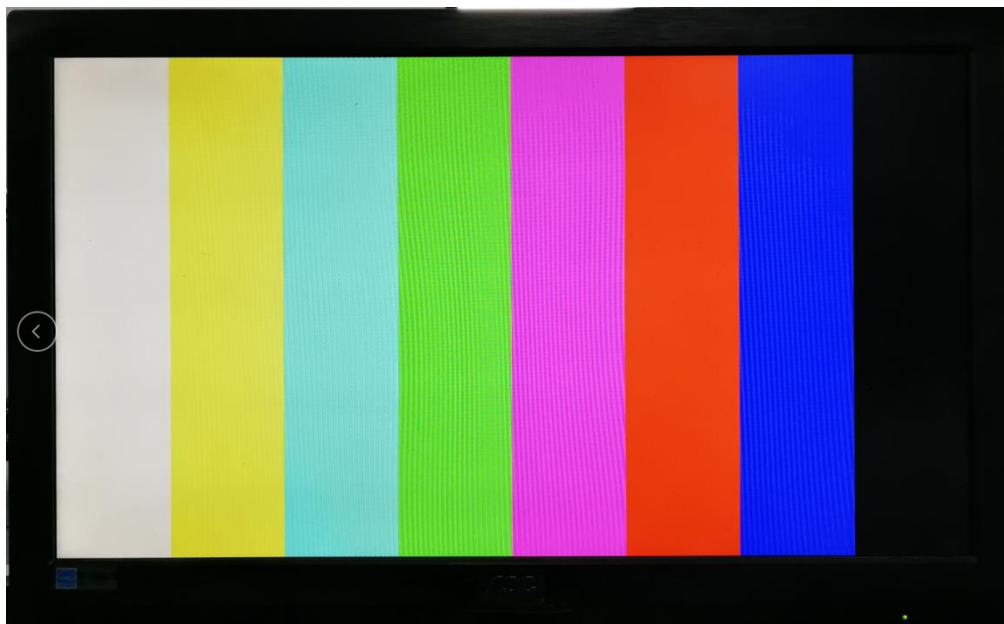
14.4 下载调试

保存工程并编译生成 bit 文件，连接 HDMI 模块到 J45 扩展口，连接 HDMI 接口到 HDMI 显示器，需要注意，这里使用 1920x1080@60Hz，请确保自己的显示器支持这个分辨率。



硬件连接图 (J45 扩展口)

下载后显示器显示如下图像



14.5 实验总结

本实验初步接触到视频显示，涉及到视频知识，这不是 zynq 学习的重点，所以没有详细介绍，但 zynq 在视频处理领域用途广泛，需要学习者有良好的基础知识。实验中仅仅使用 PL 来驱动 HDMI 芯片，包括 I2C 寄存器配置，当然 I2C 的配置还是使用 PS 来配置比较合适。

第十五章 HDMI 字符显示实验

实验 Vivado 工程为 “hdmi_char”。

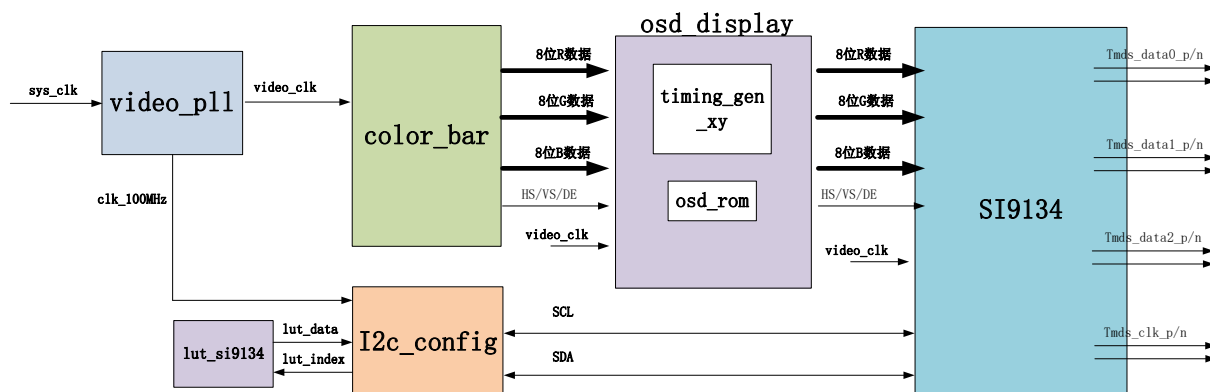
在 HDMI 输出实验中讲解了 HDMI 显示原理和显示方式，本实验介绍如何使用 FPGA 实现字符显示，通过这个实验更加深入的了解 HDMI 的显示方式。

15.1 实验原理

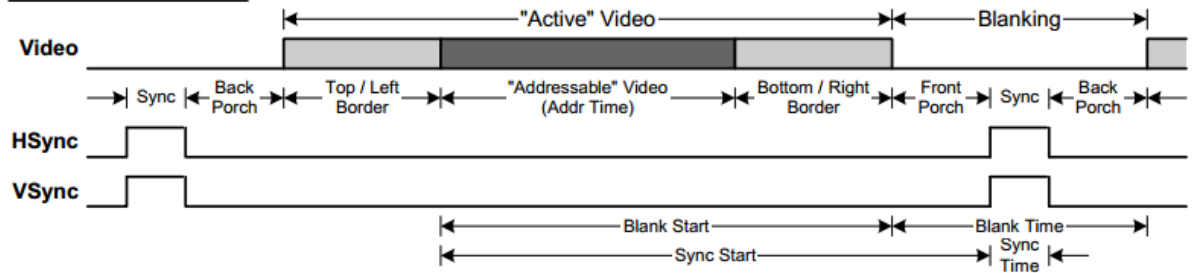
实验通过字符转换工具将字符转换为 16 进制 coe 文件存放到单端口的 ROM IP 核中，再从 ROM 中把转换后的数据读取出来显示到 HDMI 上。

15.2 程序设计

字符显示例程是在 HDMI 显示的基础上增加了一个 osd_display 的模块，“osd_display” 模块是用来读取存储在 Rom ip 核里转换后的字符信息，并在指定区域显示。程序框图如下图所示：



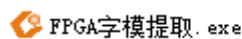
- 1) 在 “timing_gen_xy” 模块是根据 HDMI 时序标准定义了 “x_cnt” 和 “y_cnt” 两个计数器并由这两个计数器产生了 HDMI 显示的 “x” 坐标和 “y” 坐标。程序中用 “vs_edge” 和 “de_falling” 分别表示场同步开始信号和数据有效结束信号。其原理如下图所示：

Definition of Terms

信号名称	方向	说明
rst_n	in	异步复位输入,低复位
clk	in	外部时钟输入
i_hs	in	行同步信号
i_vs	in	场同步信号
i_de	in	数据有效信号
i_data	in	color_bar 数据
o_hs	out	输出行同步信号
o_vs	out	输出场同步信号
o_de	out	输出数据有效信号
o_data	out	输出数据
x	out	生成 x 坐标
y	out	生成 y 坐标

timing_gen_xy 模块端口

- 2) 下面介绍如何存储文字信息的 ROM IP，首先需要生成能够被 XILINX FPGA 识别的 .coe 文件。首先在工程文件夹下找到“FPGA 字模提取”工具。



FPGA字模提取.exe

双击.exe 文件打开工具



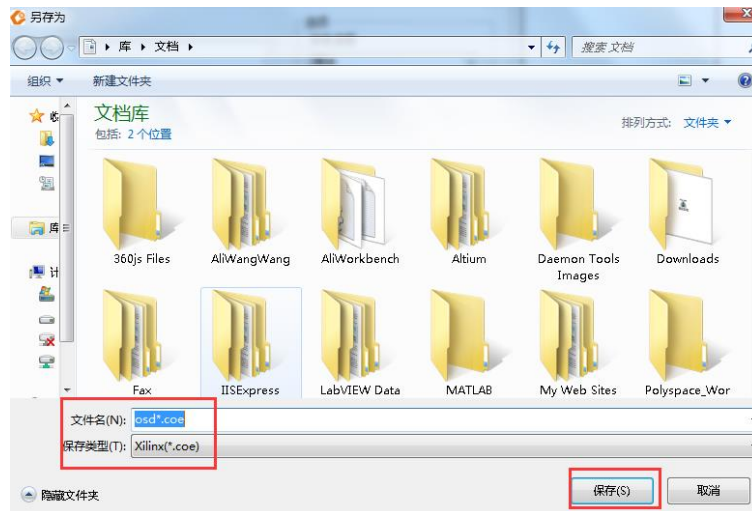
在提取工具的“字符输入”框中输入需要显示的字符，字体和字符高度可以自定义选择。设置完成后点击“转换”按钮，在界面左下角可以看到转换后的字符点阵大小，点阵的宽和高在程序中是需要用到



点阵的宽和高这里位 144x32,需要跟 osd_display 程序中定义的一致:

```
parameter OSD_WIDTH  = 12' d144;  
parameter OSD_HEIGHT = 12' d32;
```

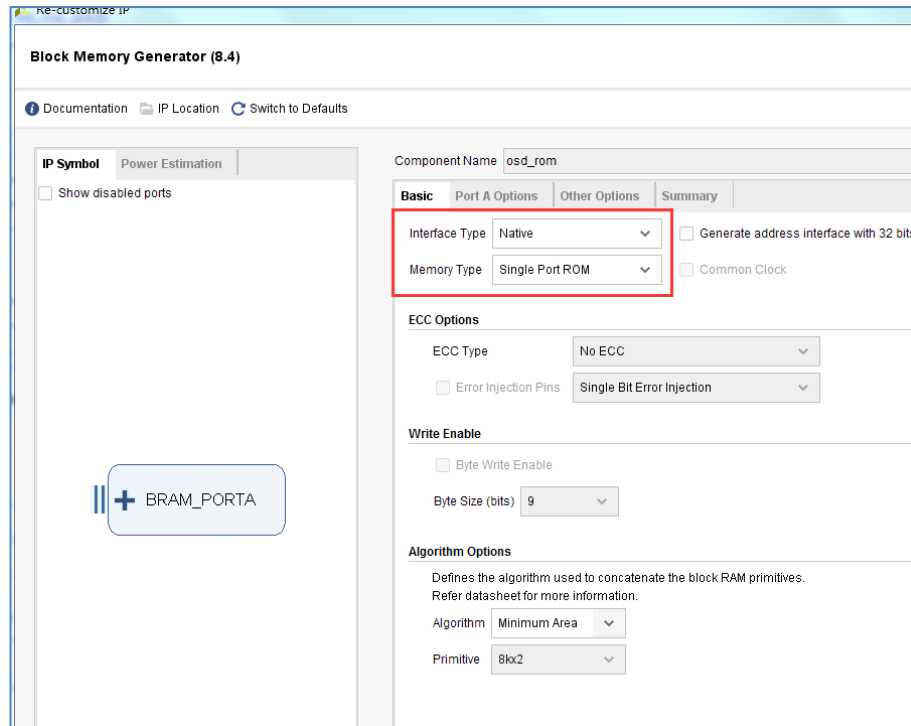
点击“保存”按钮，将文件保存到本例程源文件目录下，需要注意的是在保存类型下应该选择 Xilinx (*.coe) ,点击“保存”按钮。



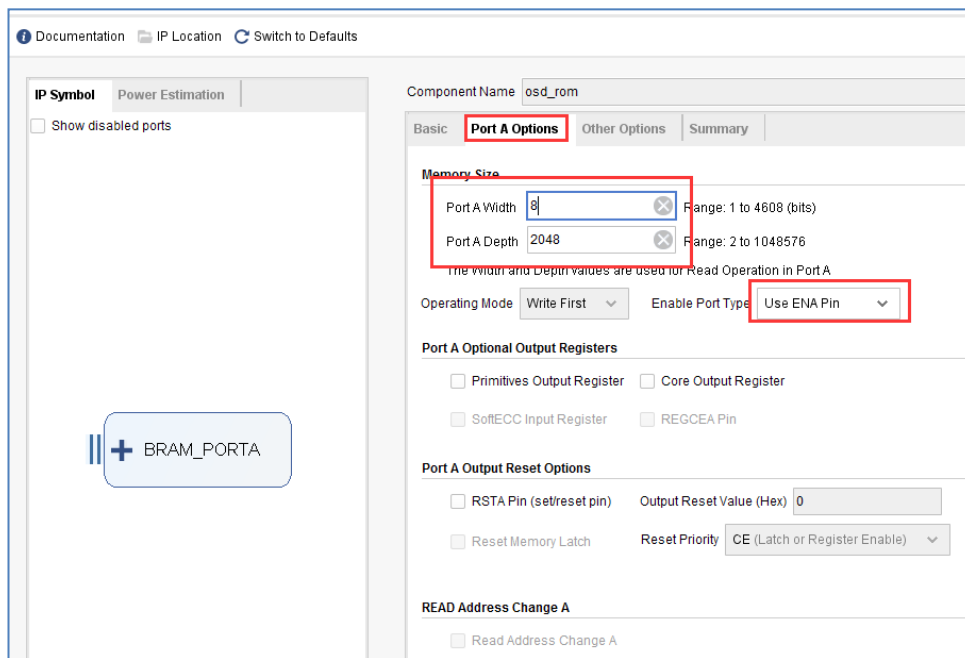
找到生成的.coe 文件打开后可以看到如下：

```
1 MEMORY_INITIALIZATION_RADIX=16;
2 MEMORY_INITIALIZATION_VECTOR=
3 00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,
4 00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,
5 00,00,00,00,00,00,00,00,00,00,00,00,00,00,04,
6 00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,00,
7 00,1C,38,00,00,00,00,03,00,00,00,00,00,00,00,00,
8 00,00,00,0C,38,00,00,E6,FF,07,80,01,00,00,00,00,
9 00,00,00,00,00,0C,18,08,FC,0F,01,03,C0,01,7E,00,
10 F8,1F,0F,FC,3E,7E,00,0C,18,1C,00,06,82,01,C0,01,
11 18,00,80,01,1C,10,1C,18,FC,FF,FF,3F,30,06,C6,00,
12 C0,01,18,00,80,01,1C,10,18,08,00,0C,18,00,30,06,
13 EC,00,C0,03,18,00,80,01,3C,10,18,0C,00,0C,18,00,
14 10,06,78,00,60,03,18,00,80,01,34,10,30,04,00,0C,
15 18,00,18,02,38,00,20,03,18,00,80,01,74,10,30,06,
16 00,04,08,00,18,03,FC,00,20,03,18,00,80,01,64,10,
17 60,02,00,60,00,00,18,03,C6,03,20,03,18,00,80,01,
18 E4,10,60,03,00,C0,00,00,18,83,01,3F,30,06,18,00,
19 80,01,C4,10,C0,01,00,82,03,00,18,63,38,0C,10,06,
```

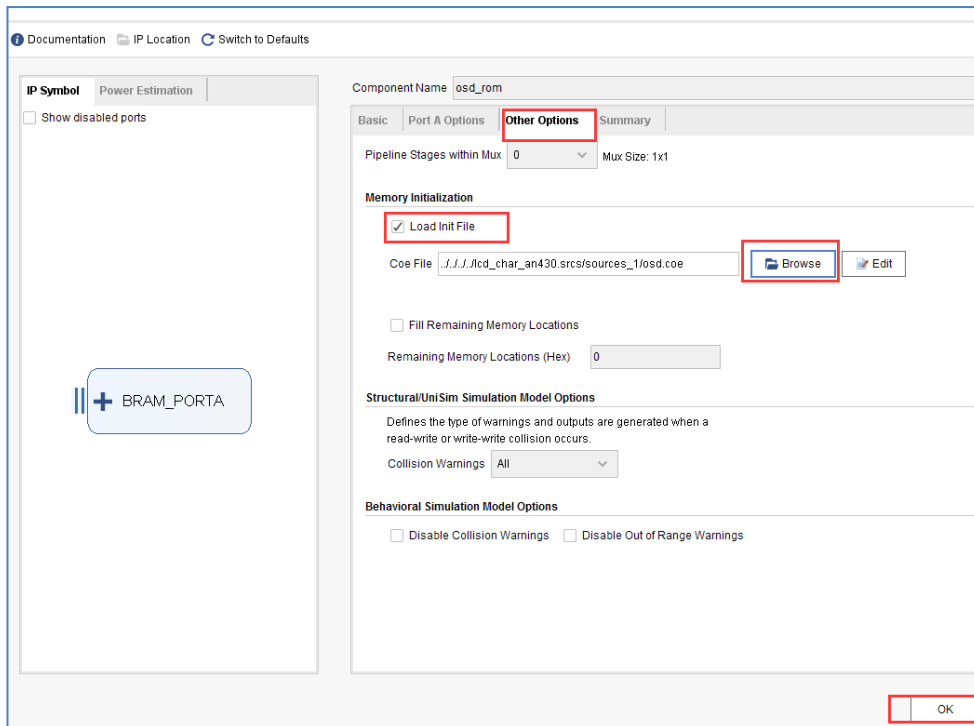
调用单端口 Rom IP 核的过程在前面 ROM 的使用中已经介绍过，设置为 Single Port ROM



在 PortA Options 栏中设置如下：



按如下图添加 osd.coe 文件（找到前面生成的 coe 文件），完成后点击“OK”按钮：



- 3) osd_display 模块包含 timing_gen_xy 模块和 osd_rom 模块。osd_rom 里存储的字符数据，如果数据为 1，OSD 的区域显示 ROM 中的前景红色（显示 ALINX 芯驿），如果数据是 0，OSD 的区域显示数据为背景色（彩条）。

```
always@(posedge pclk)
begin
    if(region_active_d0 == 1'b1)
        if(q[osd_x[2:0]] == 1'b1)
            v_data <= 24'hff0000;
        else
            v_data <= pos_data;
    else
        v_data <= pos_data;
end
```

设置区域有效信号，也就是字符显示在此区域中，起始坐标设置成 (9, 9)，区域大小可以根据字符生成工具设置的区域设置。

```
always@(posedge pclk)
begin
    if(pos_y >= 12'd9 && pos_y <= 12'd9 + OSD_HEGHT - 12'd1 && pos_x >= 12'd9 && pos_x <= 12'd9 + OSD_WIDTH - 12'd1)
        region_active <= 1'b1;
    else
        region_active <= 1'b0;
end
```

在 ROM 的读地址部分可能很多人不理解，为什么是[15:3]，也就是八个时钟周期才读出一个数据，这是因为字符的一个点只表示 1bit，而 ROM 的存储数据宽度是 8 位，因此需要八个周期取出一个数据，并比较每个 bit 位的值，将字符一个点转换成图像上的一个像素。

```
osd_rom osd_rom_m0 (
    .clka                                (pclk),
    .ena                                (1'b1),
    .addra                              (osd_ram_addr[15:3]),
    .douta                              (q)
);
```

信号名称	方向	说明
rst_n	in	异步复位输入,低复位

pclk	in	外部时钟输入
i_hs	in	行同步信号
i_vs	in	场同步信号
i_de	in	数据有效信号
i_data	in	color_bar 数据
o_hs	out	输出行同步信号
o_vs	out	输出场同步信号
o_de	out	输出数据有效信号
o_data	out	输出数据

osd_display 模块端口

15.3 实验现象

连接好开发板和显示器，连接方式参考《HDMI 输出实验》教程，需要注意，开发板的各个连接器不要带电热插拔，下载好实验程序，可以看到显示器显示以彩条为背景的字符。开发板作为 HDMI 输出设备，只能通过 HDMI 显示设备来显示，不要试图通过笔记本电脑的 HDMI 接口来显示，因为笔记本也是输出设备。



默认字符显示的位置在坐标为 (9, 9)，另外用户可以修改下面的 pos_y 和 pos_x 的判断条件将字符显示在显示屏的任意位置：

```

73 always@(posedge pclk)
74 begin
75     if(pos_y >= 12'd9 && pos_y <= 12'd9 + OSD_HREGINT - 12'd1 && pos_x >= 12'd9 && pos_x <= 12'd9 + OSD_WIDTH - 12'd1)
76         region_active <= 1'b1;
77     else
78         region_active <= 1'b0;
79 end

```

第十六章 7 寸液晶屏显示实验

实验 Vivado 工程为 “lcd7_test”。

基于 HDMI 输出实验，本章介绍 7 寸液晶屏的显示。

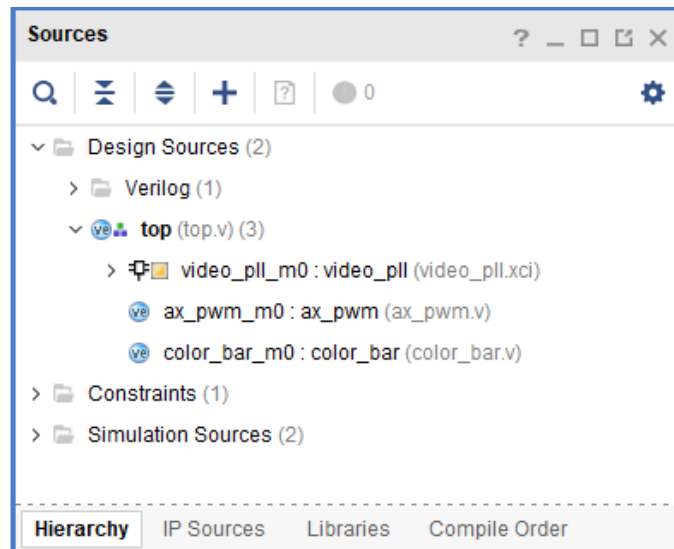
16.1 硬件介绍

AN970 LCD 触摸屏模块由 TFT 液晶屏，电容触摸屏和驱动板组成，详细信息查看 AN970 用户手册。AN970 实物照片如下：

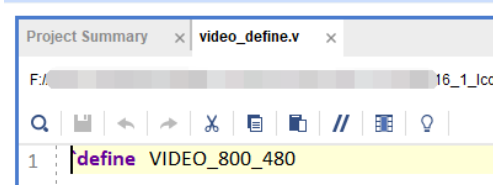


16.2 程序设计

本章实验其实很简单，与 HDMI 显示最大的不同是不需要 i2c 配置，输出按照 RGB 即可。以下是文件结构。



同时因为液晶屏的分辨率是 800x480，需要修改 video_define.v 的宏定义。



同时将 PLL 的输出时钟频率修改为 33MHz，即 800x480 的像素时钟。

Clocking Wizard (5.4)

Documentation IP Location Switch to Defaults

IP Symbol Resource

Show disabled ports

Component Name video_pll

Clocking Options Output Clocks Port Renaming MMCM Settings Summary

The phase is calculated relative to the active input clock

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)	
		Requested	Actual	Requested	Actual	Requested	Actual
☒ clk_out1	clk_out1	33	33.000	0.000	0.000	50.000	50.0
☐ clk_out2	clk_out2	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A

☐ USE CLOCK SEQUENCING

Clocking Feedback

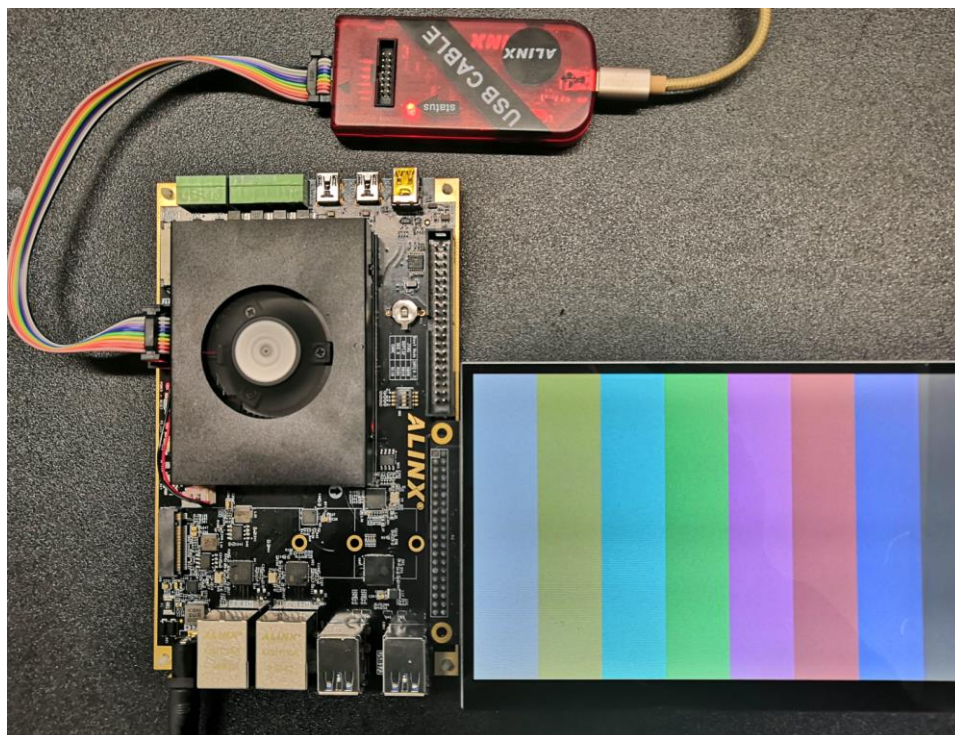
Output Clock	Sequence Number	Source	Signaling

同时在 top.v 中例化了 ax_pwm，用于调节液晶屏的亮度，设置为 200Hz，30%点空比。

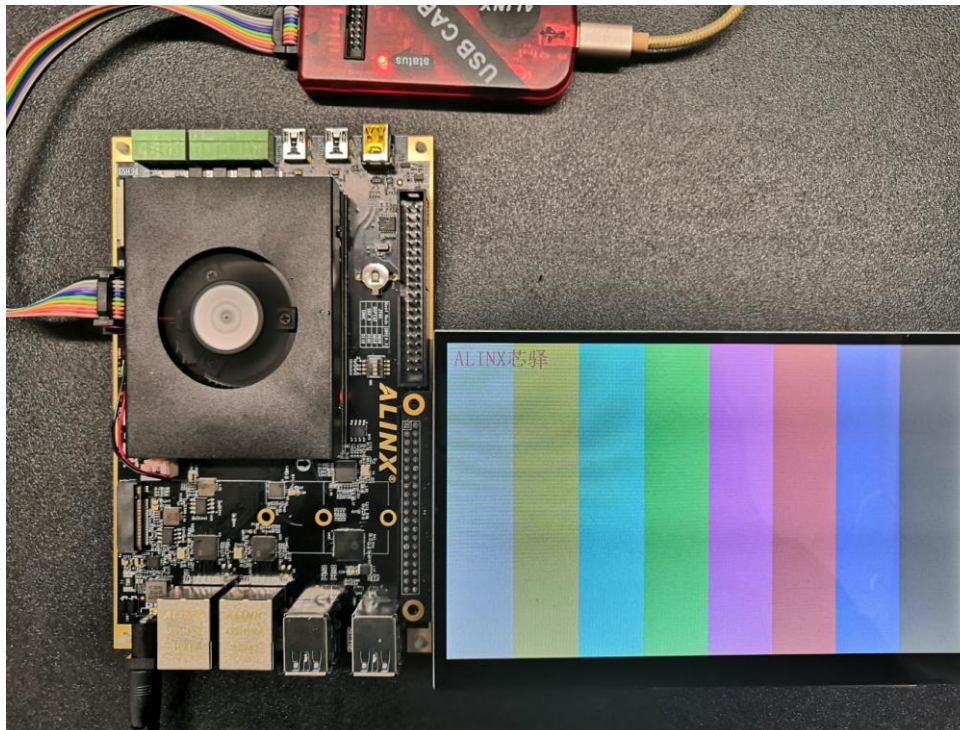
```
//200hz 30% duty
ax_pwm#(.N(24)) //pass new parameters
ax_pwm_m0(
    .clk          (sys_clk),
    .rst          (~rst_n),
    .period       (24'd67), // (2^24)*200Hz/50MHz
    .duty         (24'd5033164), // (2^24)*30%
    .pwm_out      (lcd_pwm)
);
```


16.3 实验现象

连接液晶屏到 J45 扩展口，下载程序，即可看到彩条显示。



同时也准备了字符显示的例程：

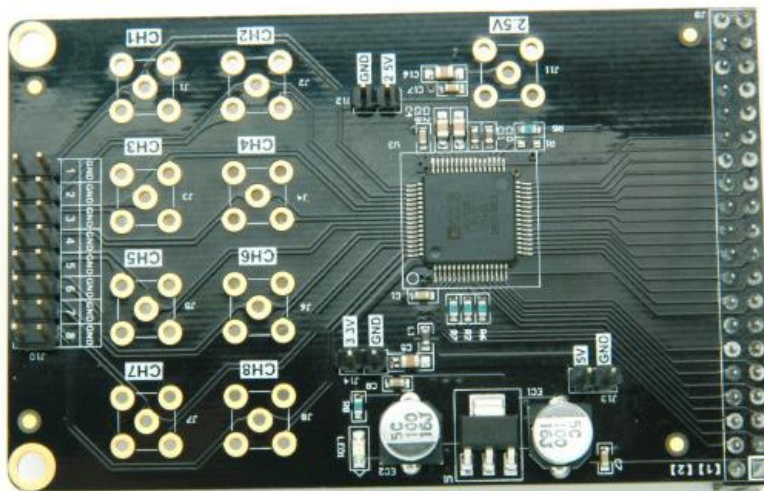


字符显示

第十七章 AD7606 多通道波形显示实验

实验 Vivado 工程为 “ad7606_hdmi_test”。

本实验练习使用 ADC，实验中使用的 ADC 模块型号为 AN706，最大采样率 200Khz，精度为 16 位。实验中把 AN706 的 2 路输入以波形方式在 HDMI 上显示出来，我们可以用更加直观的方式观察波形，是一个数字示波器雏形。



8 路 200K 采样 16 位 ADC 模块



实验预期结果

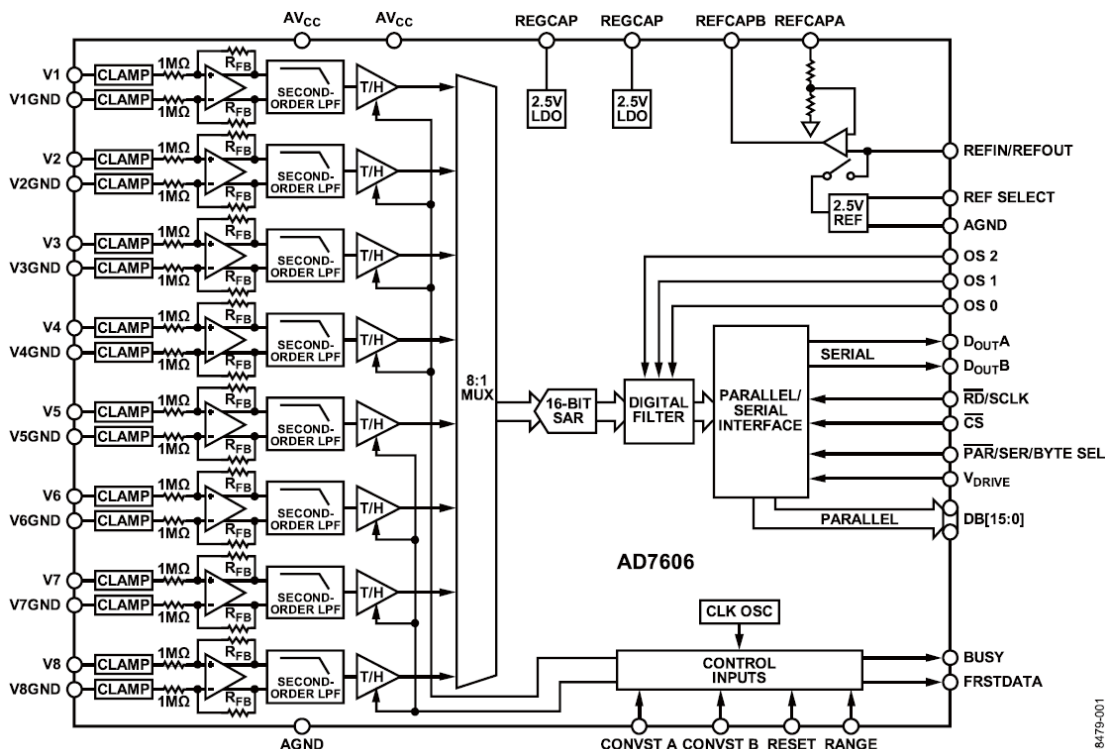
17.1 实验原理

AD7606 是一款集成式 8 通道同步采样数据采集系统, 片内集成输入放大器、过压保护电路、二阶模拟抗混叠滤波器、模拟多路复用器、16 位 200 kSPS SAR ADC 和一个数字滤波器, 2.5 V 基准电压源、基准电压缓冲以及高速串行和并行接口。

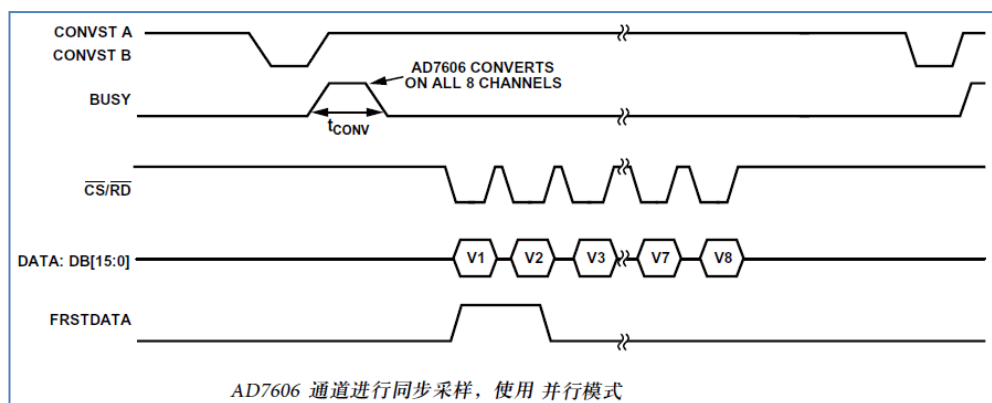
AD7606 采用+5V 单电源供电, 可以处理 $\pm 10V$ 和 $\pm 5V$ 真双极性输入信号, 同时所有通道均以高达 200KSPS 的吞吐速率采样。输入钳位保护电路可以耐受最高达 $\pm 16.5V$ 的电压。

无论以何种采样频率工作, AD7606 的模拟输入阻抗均为 1M 欧姆。它采用单电源工作方式, 具有片内滤波和高输入阻抗, 因此无需驱动运算放大器和外部双极性电源。

AD7606 抗混叠滤波器的 3dB 截至频率为 22kHz; 当采样速率为 200kSPS 时, 它具有 40dB 抗混叠抑制特性。灵活的数字滤波器采用引脚驱动, 可以改善信噪比(SNR), 并降低 3dB 带宽。



17.1.1 AD7606 时序



AD7606 可以对所有 8 路的模拟输入通道进行同步采样。当两个 CONVST 引脚(CONVSTA 和 CONVSTB)连在一起时，所有通道同步采样。此共用 CONVST 信号的上升沿启动对所有模拟输入通道的同步采样(V1 至 V8)。

AD7606 内置一个片内振荡器用于转换。所有 ADC 通道的转换时间为 t_{CONV} 。BUSY 信号告知用户正在进行转换，因此当施加 CONVST 上升沿时，BUSY 变为逻辑高电平，在整个转换过程结束时变成低电平。BUSY 信号下降沿用来使所有八个采样保持放大器返回跟踪模式。BUSY 下降沿还表示，现在可以从并行总线 DB[15:0]读取 8 个通道的数据。

17.1.2 AD7606 配置

在 AN706 8 通道的 AD 模块硬件电路设计中，我们对 AD7606 的 3 个配置 Pin 脚通过加上拉或下拉电阻来设置 AD7606 的工作模式。

AD7606 这款芯片支持外部基准电压输入或内部基准电压。如果使用外部基准电压，芯片的 REFIN/REFOUT 需要外接一个 2.5V 的基准源。如果使用内部的基准电压。REFIN/REFOUT 引脚为 2.5V 的内部基准电压输出。REF SELECT 引脚用于选择内部基准电压或外部基准电压。在本模块中，因为考虑到 AD7606 的内部基准电压的精度也非常高 (2.49V~2.505V)，所以电路设计选择使用了内部的基准电压。

Pin 脚名	设置电平	说明
REF SELECT	高电平	使用内部的基准电压 2.5V

AD7606 的 AD 转换数据采集可以采用并行模式或者串行模式，用户可以通过设置 PAR/SER/BYTE SEL 引脚电平来设置通信的模式。我们在设计的时候，选择并行模式读取 AD7606 的 AD 数据。

Pin 脚名	设置电平	说明
PAR/SER/BYTE SEL	低电平	选择并行接口

AD7606 的 AD 模拟信号的输入范围可以设置为 $\pm 5V$ 或者是 $\pm 10V$ ，当设置 $\pm 5V$ 输入范围时， $1LSB=152.58\mu V$ ；当设置 $\pm 10V$ 输入范围时， $1LSB=305.175\mu V$ 。用户可以通过设置 RANGE 引脚电平来设置模拟输入电压的范围。我们在设计的时候，选择 $\pm 5V$ 的模拟电压输入范围。

Pin 脚名	设置电平	说明
--------	------	----

RANGE	低电平	模拟信号输入范围选择: $\pm 5V$
-------	-----	----------------------

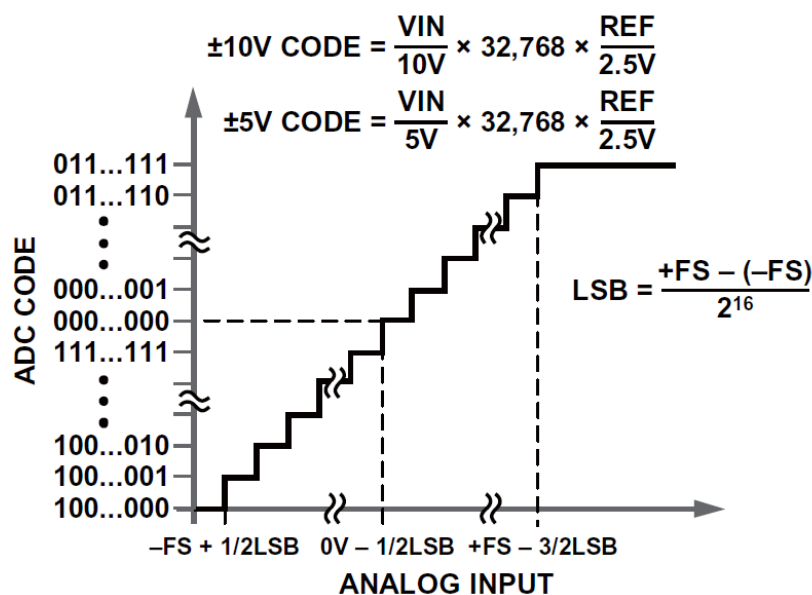
AD7606 内置一个可选的数字一阶 sinc 滤波器, 在使用较低吞吐率或需要更高信噪比的应用中, 应使用滤波器。数字滤波器的过采样倍率由过采样引脚 OS[2:0]控制。下表提供了用来选择不同过采样倍率的过采样位解码。

表9. 过采样位解码						
OS[2:0]	过采样倍率	5 V范围SNR(dB)	10 V范围SNR(dB)	5 V范围3 dB带宽 (kHz)	10 V范围3 dB带宽 (kHz)	最大吞吐量CONVST频率(kHz)
000	No OS	89	90	15	22	200
001	2	91.2	92	15	22	100
010	4	92.6	93.6	13.7	18.5	50
011	8	94.2	95	10.3	11.9	25
100	16	95.5	96	6	6	12.5
101	32	96.4	96.7	3	3	6.25
110	64	96.9	97	1.5	1.5	3.125
111	无效					

在 AN706 模块的硬件设计中, OS[2:0] 已经引到外部的接口中, FPGA 或 CPU 可以通过控制 OS[2:0]的管脚电平来选择是否使用滤波器, 以达到更高的测量精度。

17.1.3 AD7606 AD 转换

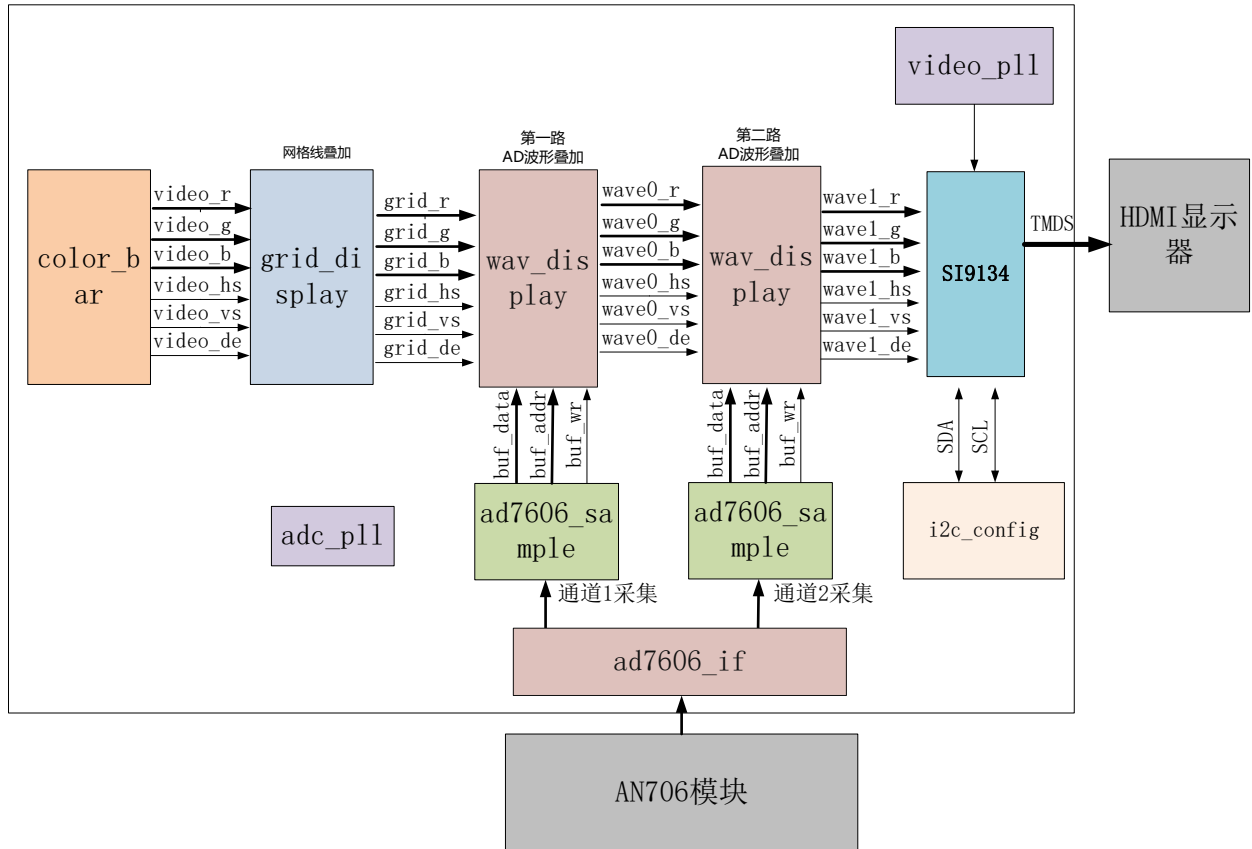
AD7606 的输出编码方式为二进制补码。所设计的码转换在连续 LSB 整数的中间(既 1/2LSB 和 3/2LSB)进行。AD7606 的 LSB 大小为 FSR/65536。AD7606 的理想传递特性如下图所示:



	+FS	MIDSCALE	-FS	LSB
$\pm 10V \text{ RANGE}$	+10V	0V	-10V	305 μ V
$\pm 5V \text{ RANGE}$	+5V	0V	-5V	152 μ V

17.2 程序设计

本实验显示部分是基于前面的 HDMI 显示彩条的实验, 在彩条上叠加网格线和波形, 整个项目的框图如下图所示:



ad7606_if 模块为 AN706 的接口模块，完成 AD706 输入的 8 路 AD 的数据采集，按照 AD706 芯片的时序产生 AD 转换信号 ad_convstab，等待 ADC 忙信号无效后，产生片选信号，依次读取 8 路 AD 数据。

信号名称	方向	宽度 (bit)	说明
clk	in	1	系统时钟
rst_n	in	1	异步复位，低复位
adc_data	in	16	ADC 数据输入
ad_busy	in	1	ADC 忙信号
first_data	in	1	第一通道数据指示信号
ad_os	out	3	ADC 过采样
ad_cs	out	1	ADC 片选
ad_rd	out	1	ADC 读信号
ad_reset	out	1	ADC 复位信号
ad_convstab	out	1	ADC 转换信号
adc_data_valid	in	1	ADC 数据有效
ad_ch1	out	16	ADC 通道 1 数据
ad_ch2	out	16	ADC 通道 2 数据

ad_ch3	out	16	ADC 通道 3 数据
ad_ch4	out	16	ADC 通道 4 数据
ad_ch5	out	16	ADC 通道 5 数据
ad_ch6	out	16	ADC 通道 6 数据
ad_ch7	out	16	ADC 通道 7 数据
ad_ch8	out	16	ADC 通道 8 数据

ad7606_sample 模块主要完成 ad706 的单路数据转换。首先需要对输入数据转换为无符号数，最后的数据只取高 8 位的数据，数据宽度转换到 8bit（为了跟其它 8 位的 AD 模块程序兼容）。另外每次采集 1280 个数据，然后等待一段时间再继续采集下面的 1280 个数据。

信号名称	方向	宽度 (bit)	说明
adc_clk	in	1	adc 系统时钟
rst	in	1	异步复位，高复位
adc_data	in	16	ADC 数据输入
adc_data_valid	in	1	adc 数据有效
adc_buf_wr	out	1	ADC 数据写使能
adc_buf_addr	out	12	ADC 数据写地址
adc_buf_data	out	8	无符号 8 位 ADC 数据

ad7606_sample 模块端口

grid_display 模块主要完成视频图像的网格线叠加，本实验将彩条视频输入，然后叠加一个网格后输出，这一块网格区域提供给后面的波形显示模块使用，这个网格区域是位于显示器水平方向（从左到右）从 9 到 1018，垂直方向（从上到下）从 9 到 308 的视频显示位置。

```
if(pos_y >= 12'd9 && pos_y <= 12'd308 && pos_x >= 12'd9 && pos_x <= 12'd1018)
    region_active <= 1'b1;
```

信号名称	方向	宽度 (bit)	说明
pclk	in	1	像素时钟
rst_n	in	1	异步复位，低电平复位
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	带网格视频行同步输出
o_vs	out	1	带网格视频场同步输出
o_de	out	1	带网格视频数据有效输出
o_data	out	24	带网格视频数据输出

grid_display 模块端口

wav_display 显示模块主要是完成波形数据的叠加显示，模块内含有一个双口 ram，写端口是由 ADC 采集模块写入，读端口是显示模块。在网格显示区域有效的时候，每行显示都会读取 RAM 中存储的 AD 数据值，跟 Y 坐标比较来判断显示波形或者不显示。

```

79      if(region_active == 1'b1)
80          if(12'd287 - pos_y == {4'd0,q})
81              v_data <= wave_color;
82          else
83              v_data <= pos_data;
84      else
85          v_data <= pos_data;

```

信号名称	方向	宽度 (bit)	说明
pclk	in	1	像素时钟
rst_n	in	1	异步复位，低电平复位
wave_color	in	24	波形颜色，rgb
adc_clk	in	1	adc 模块时钟
adc_buf_wr	in	1	adc 数据写使能
adc_buf_addr	in	12	adc 数据写地址
adc_buf_data	in	8	adc 数据，无符号数
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	带网格视频行同步输出
o_vs	out	1	带网格视频场同步输出
o_de	out	1	带网格视频数据有效输出
o_data	out	24	带网格视频数据输出

wav_display 模块端口

RAM 的配置如下：

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

Component Name dpram2048x8

Basic Port A Options Port B Options Other Options Summary

Interface Type Native ☐ Generate address interface with 32 bits

Memory Type Simple Dual Port RAM ☐ Common Clock

ECC Options

ECC Type No ECC

☐ Error Injection Pins Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits) 9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives.
Refer datasheet for more information.

Algorithm Minimum Area

Primitive 8kx2

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

Component Name dpram2048x8

Basic Port A Options Port B Options Other Options Summary

Memory Size

Port A Width 8 Range: 1 to 4608 (bits)

Port A Depth 2048 Range: 2 to 1048576

The Width and Depth values are used for Write Operations in Port A

Operating Mode No Chan... Enable Port Type Use ENA Pin

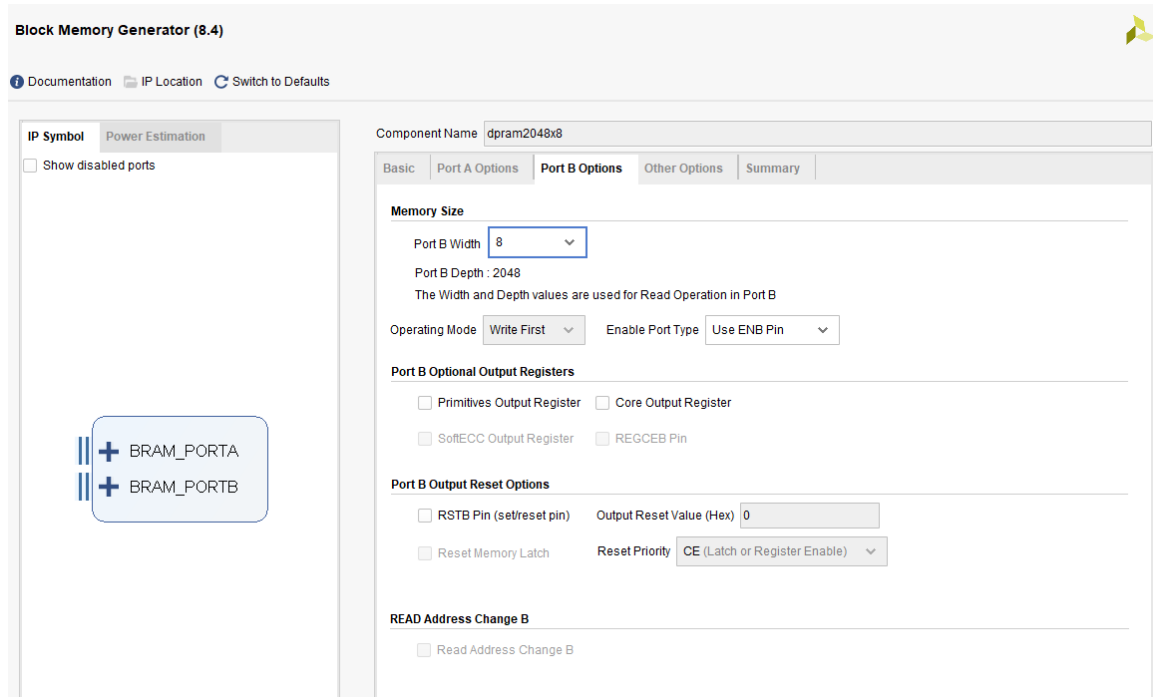
Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

READ Address Change A

☐ Read Address Change A



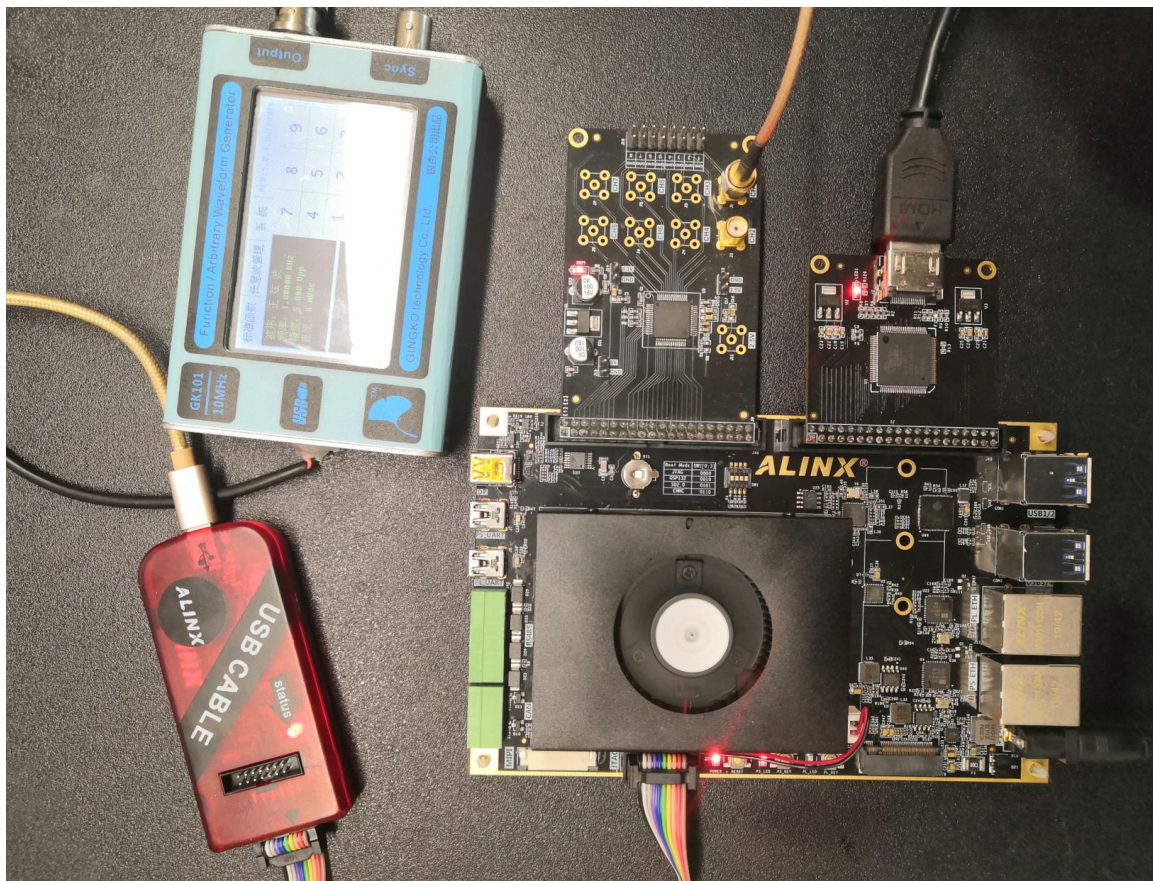
timing_gen_xy 模块为其它模块的子模块，完成视频图像的坐标生成，x 坐标，从左到右增大，y 坐标从上到下增大。

信号名称	方向	宽度 (bit)	说明
clk	in	1	系统时钟
rst_n	in	1	异步复位，低电平复位
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	视频行同步输出
o_vs	out	1	视频场同步输出
o_de	out	1	视频数据有效输出
o_data	out	24	视频数据输出
x	out	12	坐标 x 输出
y	out	12	坐标 y 输出

timing_gen_xy 模块端口

17.3 实验现象

连接电路如下，插入 AN706 模块，连接 SMA 到波形发生器，为了方便观察显示效果，波形发生器采样频率设置范围为 500Hz~10KHz，电压幅度最大为 10V，结果即为本章最前面的效果图。



硬件连接图

第十八章 AD9238 双通道波形显示实验

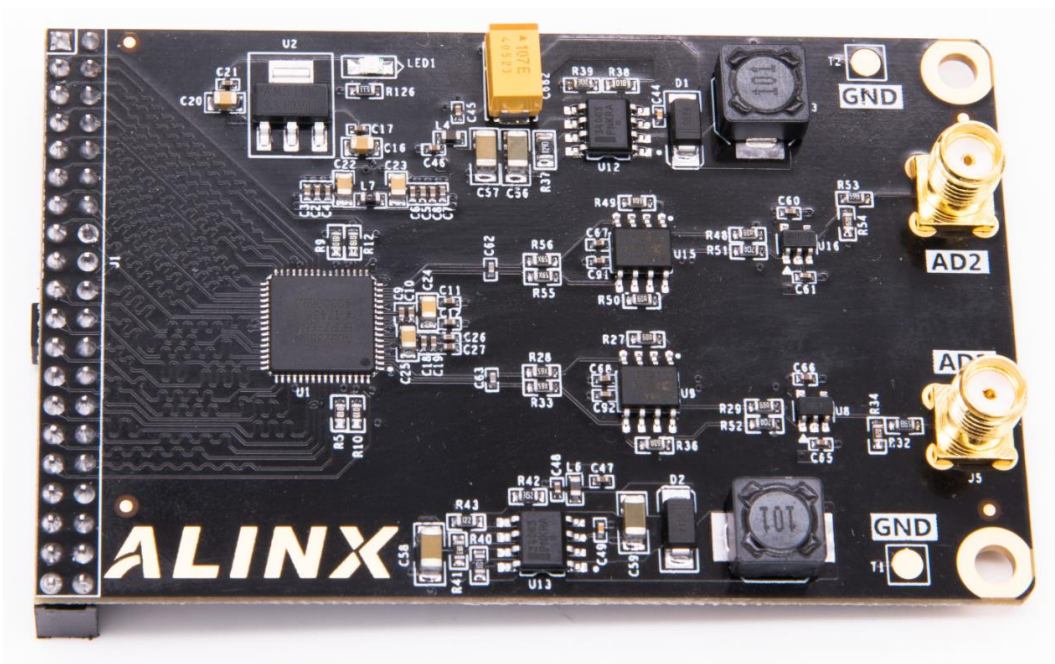
实验 Vivado 工程为 “ad9238_hdmi_test”。

18.1 硬件介绍

18.1.1 两通道 AD 模块说明

黑金高速 AD 模块 AN9238 为 2 路 65MSPS，12 位的模拟信号转数字信号模块。模块的 AD 转换采用了 ADI 公司的 AD9238 芯片，AD9238 芯片支持 2 路 AD 输入转换，所以 1 片 AD9238 芯片一共支持 2 路的 AD 输入转换。模拟信号输入支持单端模拟信号输入，输入电压范围为 -5V~+5V，接口为 SMA 插座。

模块有一个标准 2.54mm 间距的 40 针的排母，用于连接 FPGA 开发板，AN9238 模块实物照片如下：



AN9238 模块实物图

参数说明

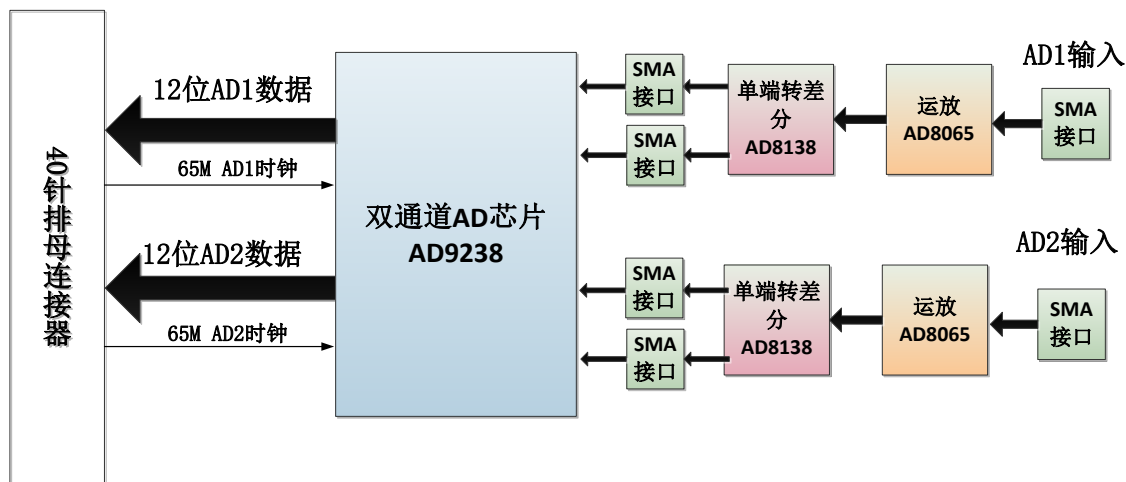
以下为 AN9238 高速 AD 模块的详细参数：

- AD 转换芯片：1 片 AD9238
- AD 转换通道：2 路；
- AD 采样速率：65MSPS；
- AD 采样数据位数：12 位；

- 数字接口电平标准：+3.3V 的 CMOS 电平
- AD 模拟信号输入范围：-5V~+5V；
- 模拟信号输入接口：SMA 接口；
- 测量精度：10mV 左右；
- 工作温度：-40°~85°；

18.1.2 模块功能说明

AN9238 模块的原理设计框图如下：



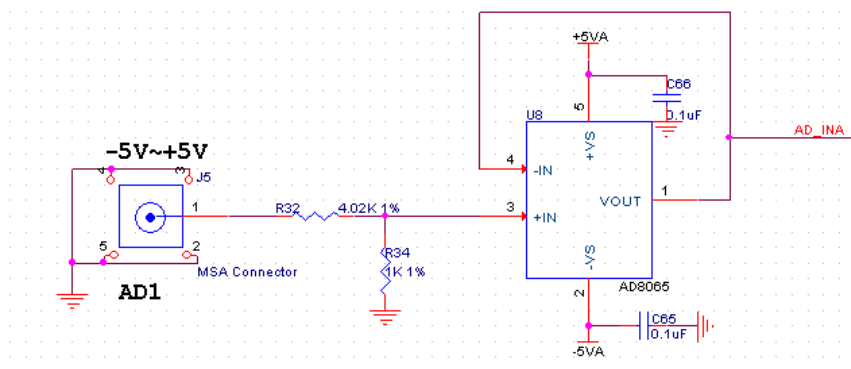
关于 AD9238 的电路具体参考设计请参考 AD9238 的芯片手册。

1) 单端输入及运放电路

单端输入 AD1 和 AD2 通过 J5 或者 J6 两个 SMA 头输入，单端输入的电压为 -5V~+5V。

板上通过运放 AD8065 芯片和分压电阻把-5V~+5V 输入的电压缩小成-1V~+1V。如果用户想输入更宽范围的电压输入只要修改前端的分压电阻的阻值。

$$\text{转换公式: } V_{OUT} = (1.0/5.02) * V_{IN}$$

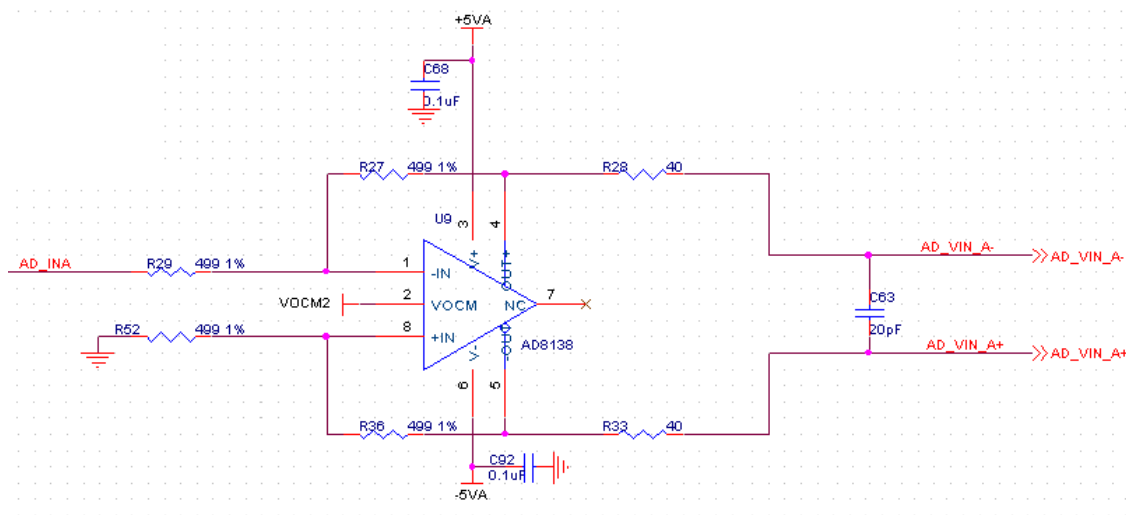


下表为模拟输入信号和 AD8065 运放输出后的电压对照表：

AD 模拟输入值	AD8065 运放输出
-5V	-1V
0V	0V
+5V	+1V

2) 单端转差分及 AD 转换

-1V~+1V 的输入电压通过 AD8138 芯片转换成差分信号 (VIN+ - VIN-), 差分信号的共模电平由 AD 的 CML 管脚决定。



下表为模拟输入信号到 AD8138 差分输出后的电压对照表：

AD 模拟输入值	AD8065 运放输出	AD8138 差分输出 (VIN+ - VIN-)
-5V	-1V	-1V
0V	0V	0V
+5V	+1V	+1V

3) AD9238 转换

默认 AD 是配置成 offset binary 的, AD 转换的值如下图所示：

Table 16. Output Data Format

Input (V)	Condition (V)	Offset Binary Output Mode
VIN+ – VIN–	$< -VREF - 0.5 \text{ LSB}$	0000 0000 0000
VIN+ – VIN–	$= -VREF$	0000 0000 0000
VIN+ – VIN–	$= 0$	1000 0000 0000
VIN+ – VIN–	$= +VREF - 1.0 \text{ LSB}$	1111 1111 1111
VIN+ – VIN–	$> +VREF - 0.5 \text{ LSB}$	1111 1111 1111

在模块电路设计中，AD9238 的 VREF 的值为 1V，这样最终的模拟信号输入和 AD 转换的数据如下：

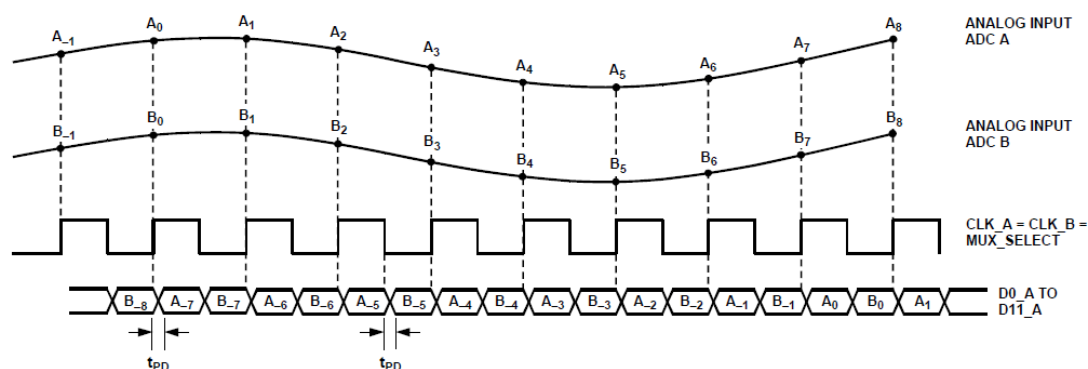
AD 模拟输入值	AD8055 运放输出	AD8138 差分输出 (VIN+ – VIN–)	AD9238 数字输出
-5V	-1V	-1V	000000000000
0V	0V	0V	100000000000
+5V	+1V	+1V	111111111111

从表中我们可以看出，-5V 输入的时候，AD9238 转换的数字值最小，+5V 输入的时候，AD9238 转换的数字值最大。

4) AD9238 数字输出时序

AD9238 双通道 AD 的数字输出为 +3.3V 的 CMOS 输出模式，2 路通道(A 和 B) 独立的数据和时钟。AD 数据在时钟的上沿沿转换数据，FPGA 端可用 AD 时钟的采样 AD 数据。

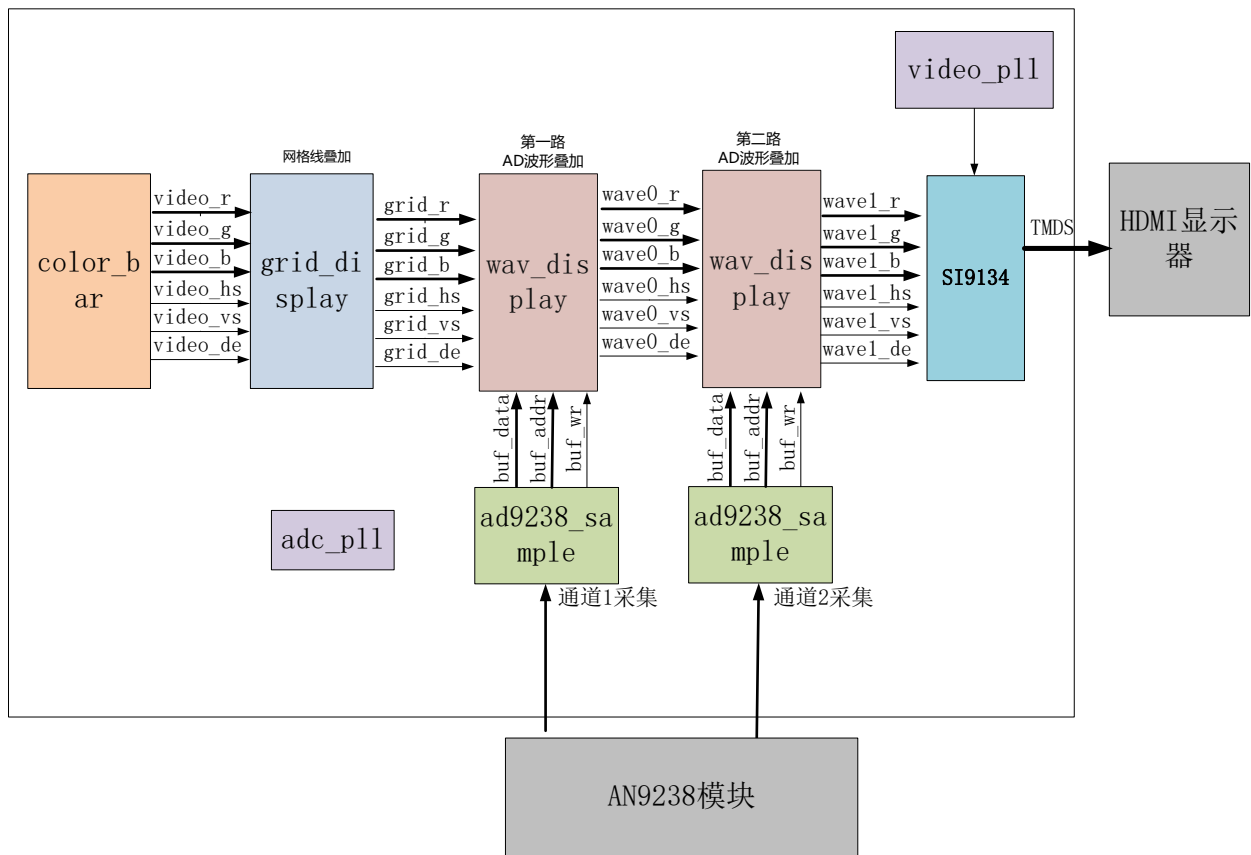
RECOVERING CAPACITORS ON REF1 AND REF2.



18.2 程序设计

本实验显示部分是基于前面的 HDMI 显示彩条的实验，在彩条上叠加网格线和波形，整个

项目的框图如下图所示：



ad9238_sample 模块主要完成 AN9238 的单路数据转换。最后的数据只取高 8 位的数据，数据宽度转换到 8bit（为了跟其它 8 位的 AD 模块程序兼容）。另外每次采集 1280 个数据，然后等待一段时间再继续采集下面的 1280 个数据。

信号名称	方向	宽度 (bit)	说明
adc_clk	in	1	adc 系统时钟
rst	in	1	异步复位，高复位
adc_data	in	12	ADC 数据输入
adc_buf_wr	out	1	ADC 数据写使能
adc_buf_addr	out	12	ADC 数据写地址
adc_buf_data	out	8	无符号 8 位 ADC 数据

ad7606_sample 模块端口

grid_display 模块主要完成视频图像的网格线叠加，本实验将彩条视频输入，然后叠加一个网格后输出，这一块网格区域提供给后面的波形显示模块使用，这个网格区域是位于显示器水平方向（从左到右）从 9 到 1018，垂直方向（从上到下）从 9 到 308 的视频显示位置。

```
if(pos_y >= 12'd9 && pos_y <= 12'd308 && pos_x >= 12'd9 && pos_x <= 12'd1018)
    region_active <= 1'b1;
```

信号名称	方向	宽度 (bit)	说明
pclk	in	1	像素时钟
rst_n	in	1	异步复位，低电平复位
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	带网格视频行同步输出
o_vs	out	1	带网格视频场同步输出
o_de	out	1	带网格视频数据有效输出
o_data	out	24	带网格视频数据输出

grid_display 模块端口

wav_display 显示模块主要是完成波形数据的叠加显示，模块内含有一个双口 ram，写端口是由 ADC 采集模块写入，读端口是显示模块。在网格显示区域有效的时候，每行显示都会读取 RAM 中存储的 AD 数据值，跟 Y 坐标比较来判断显示波形或者不显示。

```

79 if(region_active == 1'b1)
80     if(12'd287 - pos_y == {4'd0,q})
81         v_data <= wave_color;
82     else
83         v_data <= pos_data;
84     else
85         v_data <= pos_data;

```

信号名称	方向	宽度 (bit)	说明
pclk	in	1	像素时钟
rst_n	in	1	异步复位，低电平复位
wave_color	in	24	波形颜色，rgb
adc_clk	in	1	adc 模块时钟
adc_buf_wr	in	1	adc 数据写使能
adc_buf_addr	in	12	adc 数据写地址
adc_buf_data	in	8	adc 数据，无符号数
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	带网格视频行同步输出
o_vs	out	1	带网格视频场同步输出

o_de	out	1	带网格视频数据有效输出
o_data	out	24	带网格视频数据输出

wav_display 模块端口

RAM 的配置如下:

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

Component Name: dpram2048x8

Basic Port A Options Port B Options Other Options Summary

Interface Type: Native ☐ Generate address interface with 32 bits

Memory Type: Simple Dual Port RAM ☐ Common Clock

ECC Options

ECC Type: No ECC

☐ Error Injection Pins: Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits): 9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information.

Algorithm: Minimum Area

Primitive: 8kx2

IP Symbol Power Estimation

☐ Show disabled ports

BRAM_PORTA

BRAM_PORTB

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

☐ Show disabled ports

Component Name: dpram2048x8

Basic Port A Options Port B Options Other Options Summary

Memory Size

Port A Width: 8 Range: 1 to 4608 (bits)

Port A Depth: 2048 Range: 2 to 1048576

The Width and Depth values are used for Write Operations in Port A

Operating Mode: No Chan... Enable Port Type: Use ENA Pin

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

READ Address Change A

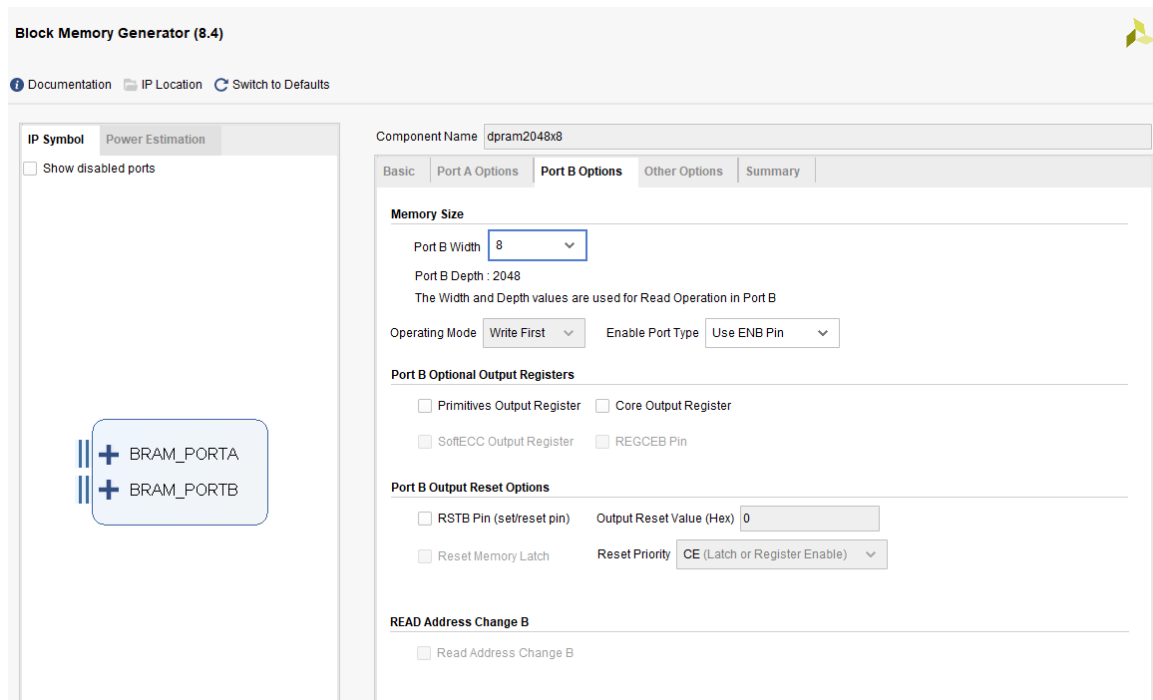
☐ Read Address Change A

IP Symbol Power Estimation

☐ Show disabled ports

BRAM_PORTA

BRAM_PORTB



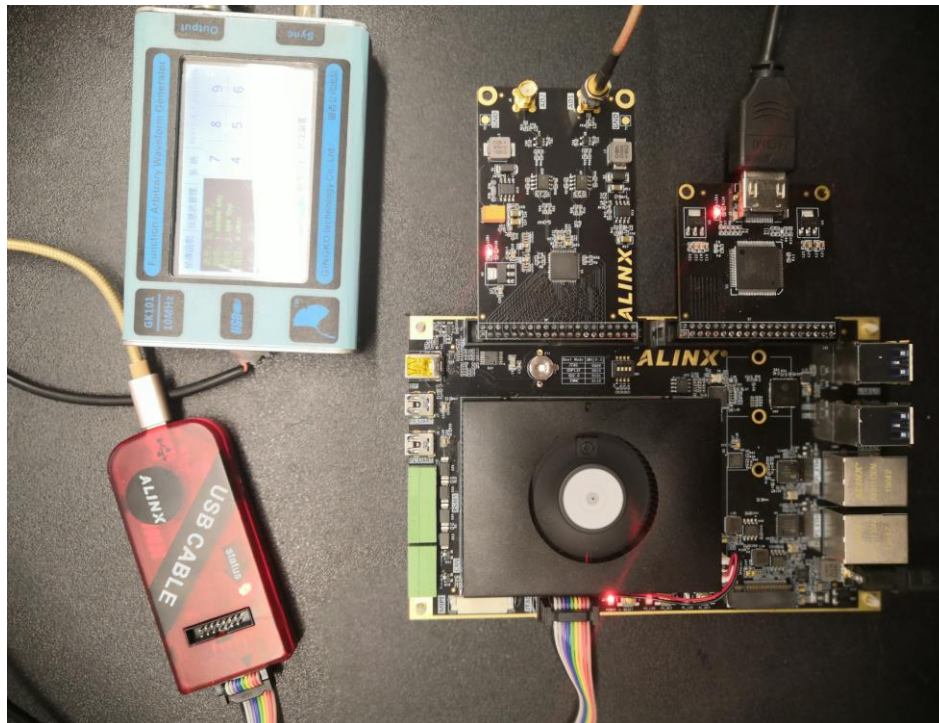
timing_gen_xy 模块为其它模块的子模块，完成视频图像的坐标生成，x 坐标，从左到右增大，y 坐标从上到下增大。

信号名称	方向	宽度 (bit)	说明
clk	in	1	系统时钟
rst_n	in	1	异步复位，低电平复位
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	视频行同步输出
o_vs	out	1	视频场同步输出
o_de	out	1	视频数据有效输出
o_data	out	24	视频数据输出
x	out	12	坐标 x 输出
y	out	12	坐标 y 输出

timing_gen_xy 模块端口

18.3 实验现象

连接电路如下，调节信号发生器的频率和幅度，AN9238 输入范围-5V-5V，为了便于观察波形数据，建议信号输入频率 200KHz 到 1Mhz。观察显示器输出，红色波形为 CH1 输入、蓝色为 CH2 输入、黄色网格最上面横线代表 5V，最下面横线代表-5V，中间横线代表 0V，每个竖线间隔是 10 个采样点。



硬件连接图



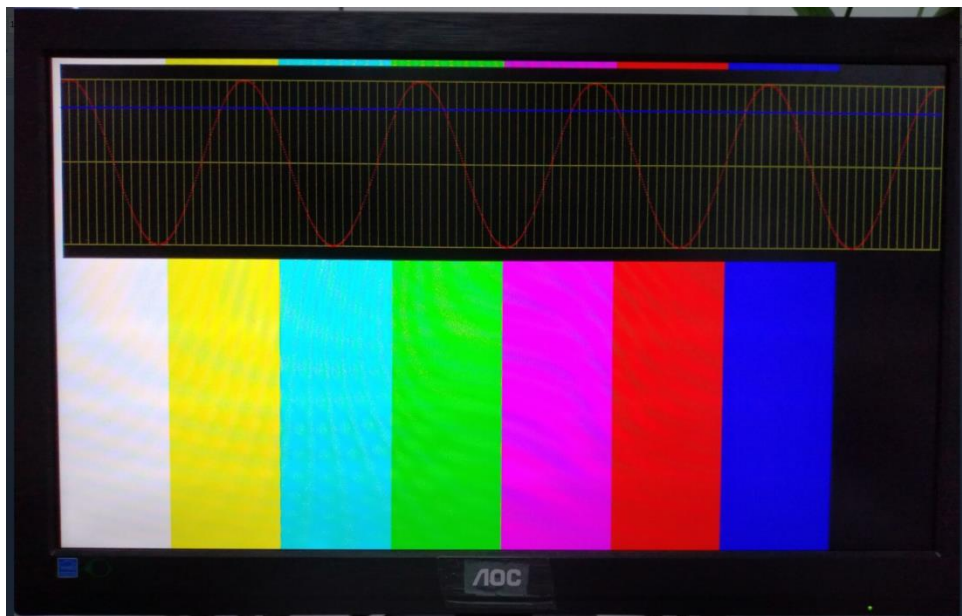
第十九章 ADDA 测试实验

实验 Vivado 工程为 “an108_adda_hdmi_test”。

本实验练习使用 ADC 和 DAC，实验中使用的 ADDA 模块型号为 AN108，ADC 最大采样率 32Mhz，精度为 8 位，DAC 最大采样率 125Mhz，精度为 8 位。实验中用 DAC 输出正弦波，然后使用 ADC 采集并把波形在 HDMI 显示器显示。

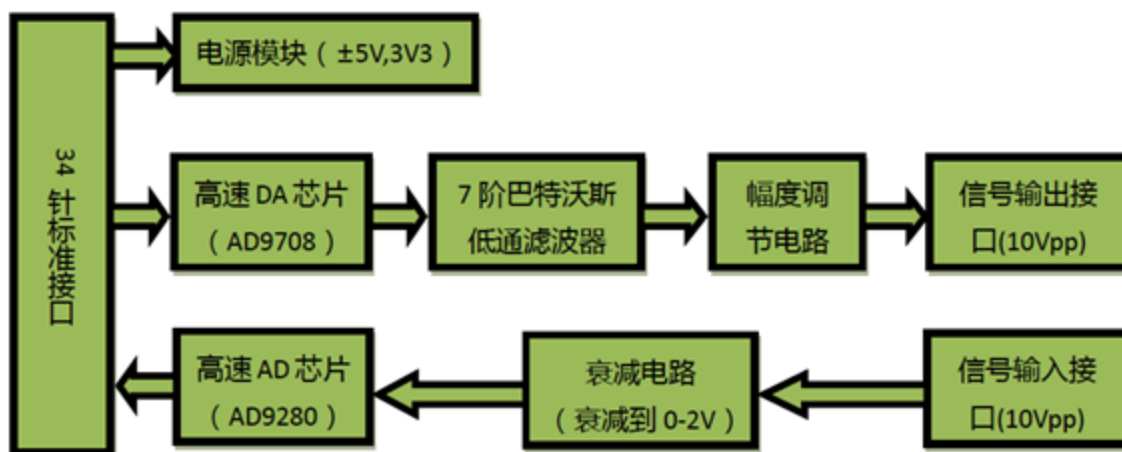


ADDA 模块



实验预期结果

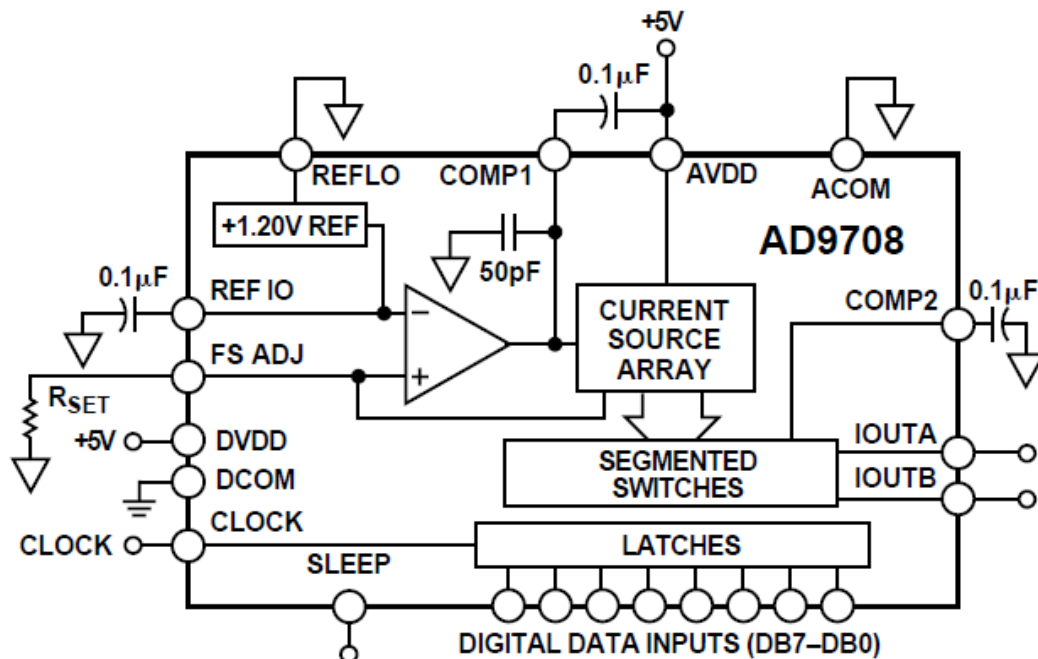
19.1 硬件介绍



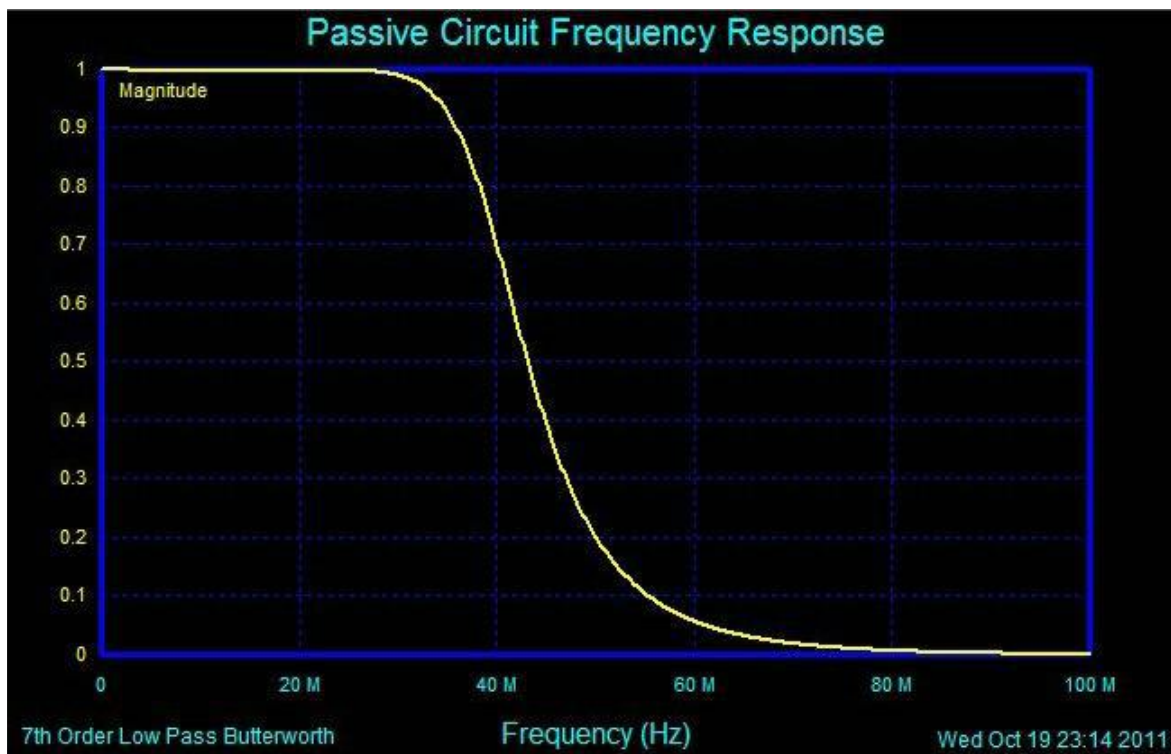
19.1.1 数模转换 (DA) 电路

如硬件结构图所示，DA 电路由高速 DA 芯片、7 阶巴特沃斯低通滤波器、幅度调节电路和信号输出接口组成。

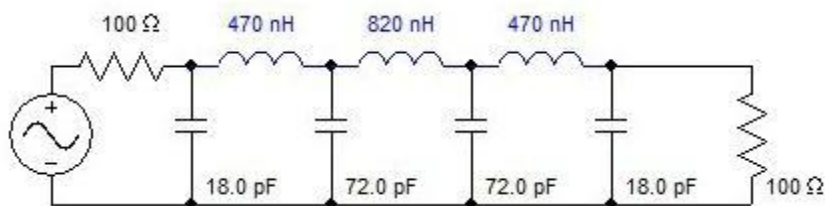
我们使用的高速 DA 芯片是 AD 公司推出的 AD9708。AD9708 是 8 位，125MSPS 的 DA 转换芯片，内置 1.2V 参考电压，差分电流输出。芯片内部结构图如下图所示



AD9708 芯片差分输出以后，为了防止噪声干扰，电路中接入了 7 阶巴特沃斯低通滤波器，带宽为 40MHz，频率响应如下图所示



滤波器参数如下图所示



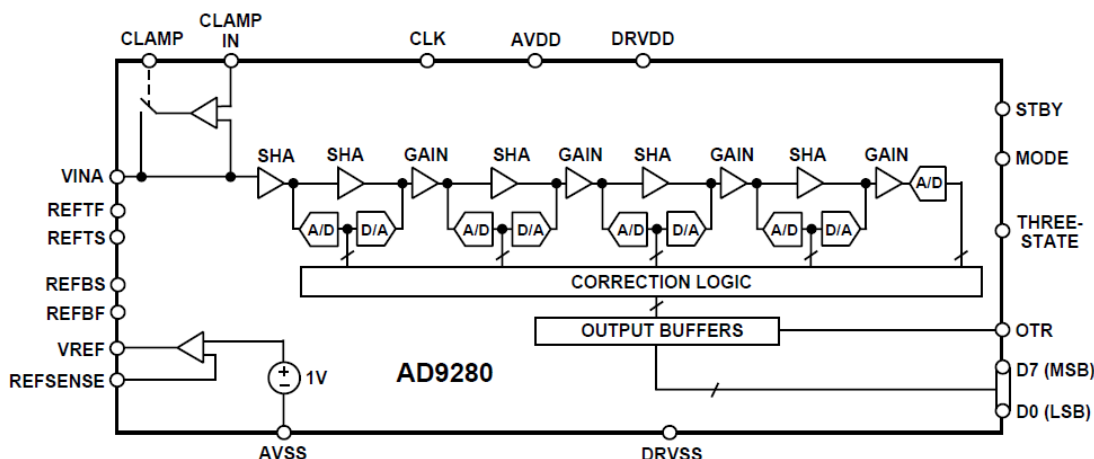
滤波器之后，我们使用了 2 片高性能 145MHz 带宽的运放 AD8056，实现差分变单端，以及幅度调节等功能，使整个电路性能得到了最大限度的提升。幅度调节，使用的是 5K 的电位器，最终的输出范围是-5V~5V（10Vpp）。

注：由于电路器的精度不是很精确，最终的输出有一定误差，有可能波形幅度不能达到 10Vpp，也有可能出现波形削顶等问题，这些都属正常情况。

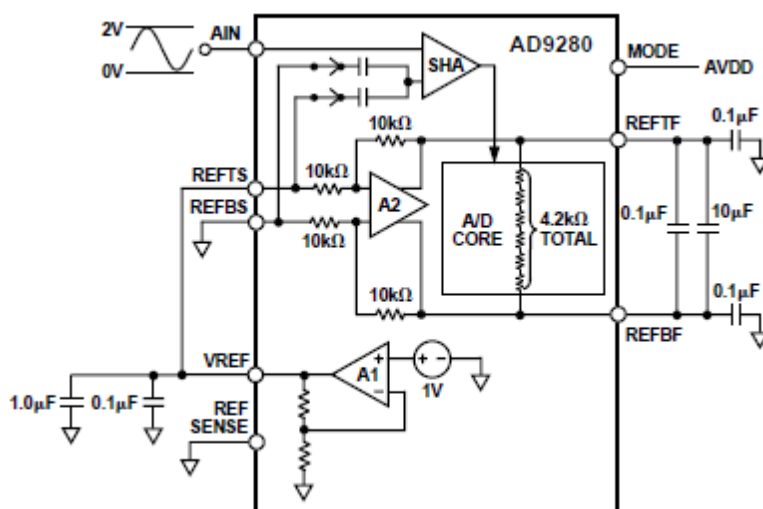
19.1.2 模数转换（AD）电路

如硬件结构图中所示，AD 电路由高速 AD 芯片、衰减电路和信号输入接口组成。

我们使用的高速 AD 芯片是由 AD 公司推出的 8 位，最大采样率 32MSPS 的 AD9280 芯片。内部结构图如下图所示



根据下图的配置，我们将 AD 电压输入范围设置为：0V~2V



在信号进入 AD 芯片之前，我们用一片 AD8056 芯片构建了衰减电路，接口的输入范围是-5V~+5V(10Vpp)。衰减以后，输入范围满足 AD 芯片的输入范围 (0~2V)。转换公式如下：

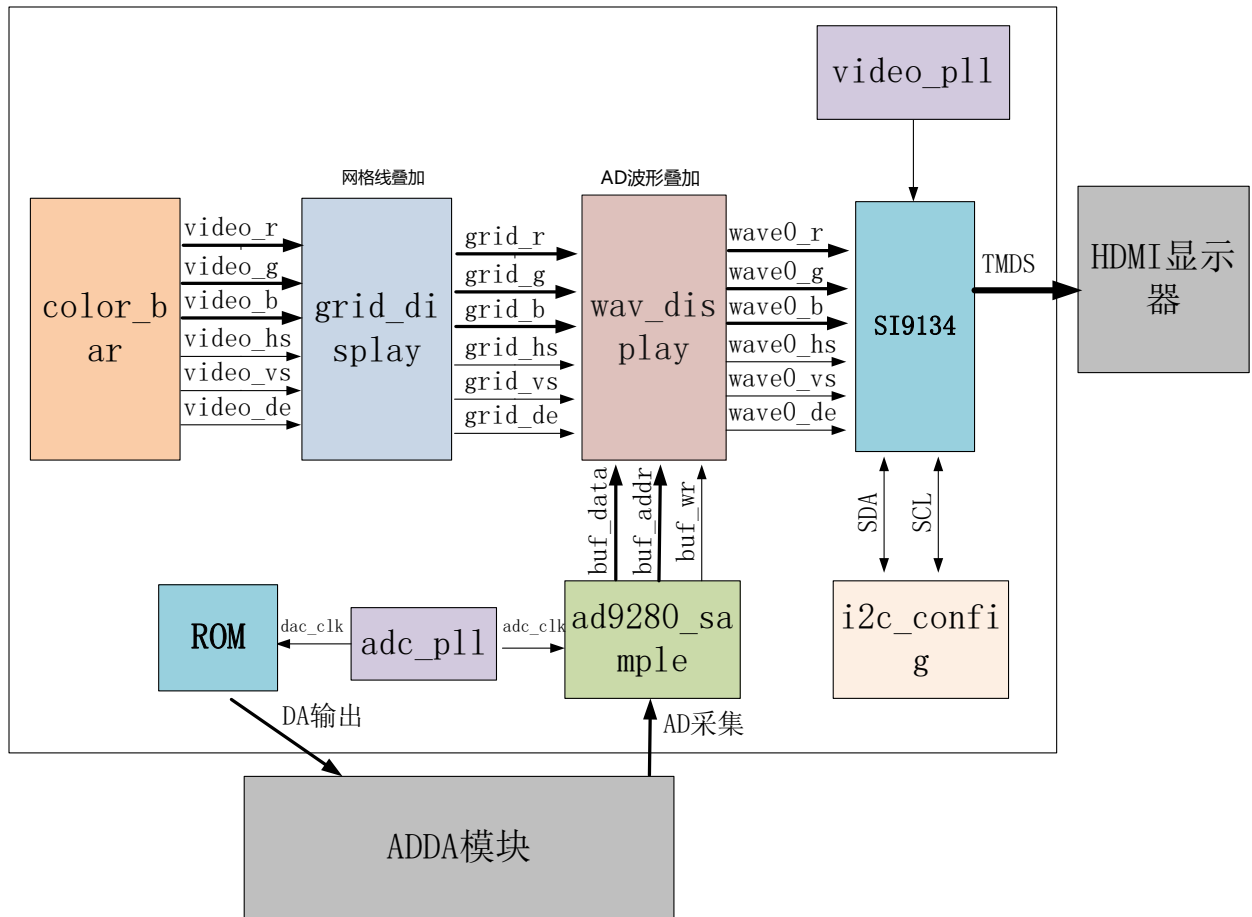
$$V_{AD} = \frac{1}{5} V_{IN} + 1$$

当输入信号 $V_{in}=5(V)$ 的时候，输入到 AD 的信号 $V_{ad}=2(V)$ ；

当输入信号 $V_{in}=-5(V)$ 的时候，输入到 AD 的信号 $V_{ad}=0(V)$ ；

19.2 程序设计

本实验程序设计跟 AN706 波形显示实验基本类似，只是 ADDA 模块是单通道的 AD，这里只是一路采集波形的叠加。另外 FPGA 通过 ROM IP 产生正弦波数据输出到 DA 芯片进行 DA 转换，产生正弦波模拟信号，用户只有用 BNC 线把模块的 AD 和 DA 端口连接起来就形成环路。这样 HDMI 显示器上显示的就是 DA 正弦波信号了。



ad9280_sample 模块主要完成 ad9280 的 AD 8 位数据采集和转换，每次采集 1280 个数据，然后等待一段时间再继续采集下次的 1280 个数据。

信号名称	方向	宽度 (bit)	说明
adc_clk	in	1	adc 系统时钟
rst	in	1	异步复位，高复位
adc_data	in	8	ADC 数据输入
adc_buf_wr	out	1	ADC 数据写使能
adc_buf_addr	out	12	ADC 数据写地址
adc_buf_data	out	8	无符号 8 位 ADC 数据

ad9280_sample 模块端口

grid_display 模块主要完成视频图像的网格线叠加，本实验将彩条视频输入，然后叠加一个网格后输出，这一块网格区域提供给后面的波形显示模块使用，这个网格区域是位于显示器水平方向（从左到右）从 9 到 1018，垂直方向（从上到下）从 9 到 308 的视频显示位置。

```
if(pos_y >= 12'd9 && pos_y <= 12'd308 && pos_x >= 12'd9 && pos_x <= 12'd1018)
    region_active <= 1'b1;
```


信号名称	方向	宽度 (bit)	说明
pclk	in	1	像素时钟
rst_n	in	1	异步复位，低电平复位
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	带网格视频行同步输出
o_vs	out	1	带网格视频场同步输出
o_de	out	1	带网格视频数据有效输出
o_data	out	24	带网格视频数据输出

grid_display 模块端口

wav_display 显示模块主要是完成波形数据的叠加显示，模块内含有一个双口 ram，写端口是由 ADC 采集模块写入，读端口是显示模块。在网格显示区域有效的时候，每行显示都会读取 RAM 中存储的 AD 数据值，跟 Y 坐标比较来判断显示波形或者不显示。

```

79  if(region_active == 1'b1)
80      if(12'd287 - pos_y == {4'd0,q})
81          v_data <= wave_color;
82      else
83          v_data <= pos_data;
84      else
85          v_data <= pos_data;

```

信号名称	方向	宽度 (bit)	说明
pclk	in	1	像素时钟
rst_n	in	1	异步复位，低电平复位
wave_color	in	24	波形颜色，rgb
adc_clk	in	1	adc 模块时钟
adc_buf_wr	in	1	adc 数据写使能
adc_buf_addr	in	12	adc 数据写地址
adc_buf_data	in	8	adc 数据，无符号数
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	带网格视频行同步输出
o_vs	out	1	带网格视频场同步输出

o_de	out	1	带网格视频数据有效输出
o_data	out	24	带网格视频数据输出

wav_display 模块端口

timing_gen_xy 模块为其它模块的子模块，完成视频图像的坐标生成，x 坐标，从左到右增大，y 坐标从上到下增大。

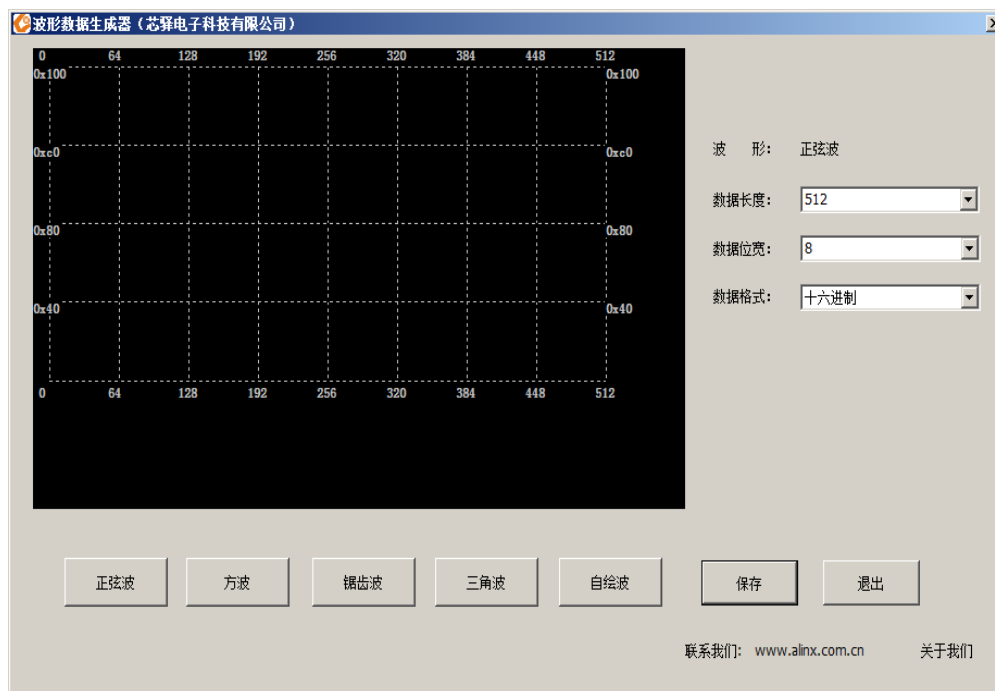
信号名称	方向	宽度 (bit)	说明
clk	in	1	系统时钟
rst_n	in	1	异步复位，低电平复位
i_hs	in	1	视频行同步输入
i_vs	in	1	视频场同步输入
i_de	in	1	视频数据有效输入
i_data	in	24	视频数据输入
o_hs	out	1	视频行同步输出
o_vs	out	1	视频场同步输出
o_de	out	1	视频数据有效输出
o_data	out	24	视频数据输出
x	out	12	坐标 x 输出
y	out	12	坐标 y 输出

timing_gen_xy 模块端口

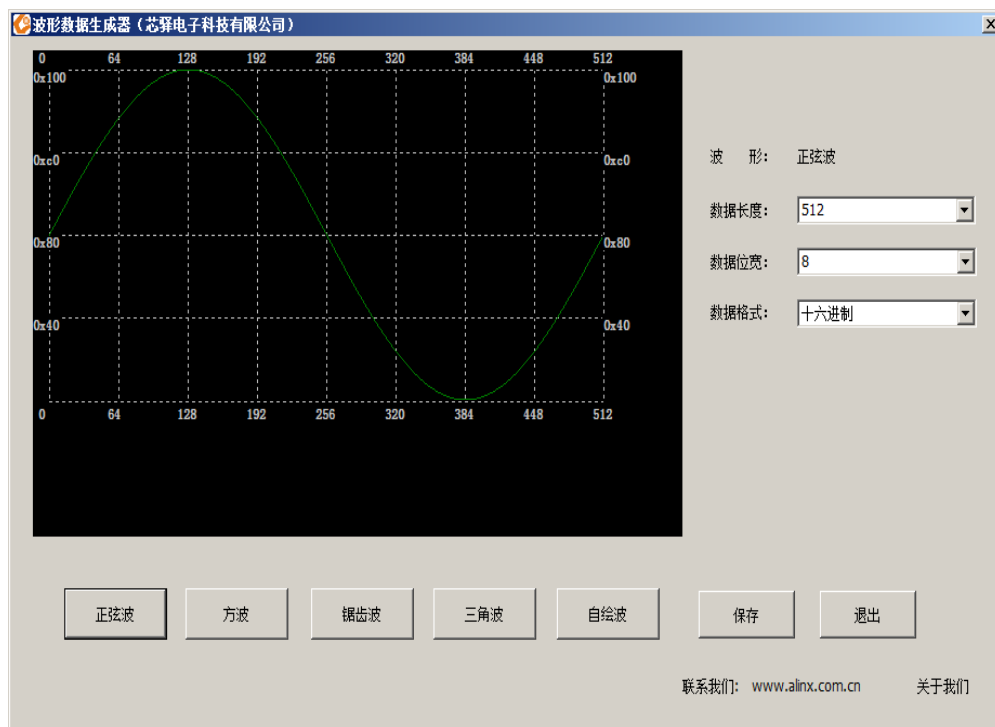
另外在本例程中添加了一个 ROM IP 模块，需要对 ROM IP 初始化数据。这里仅介绍如何使用波形数据生成工具，在软件工具及驱动文件夹下找到工具，其图标如下所示：

 波形数据生成器.exe

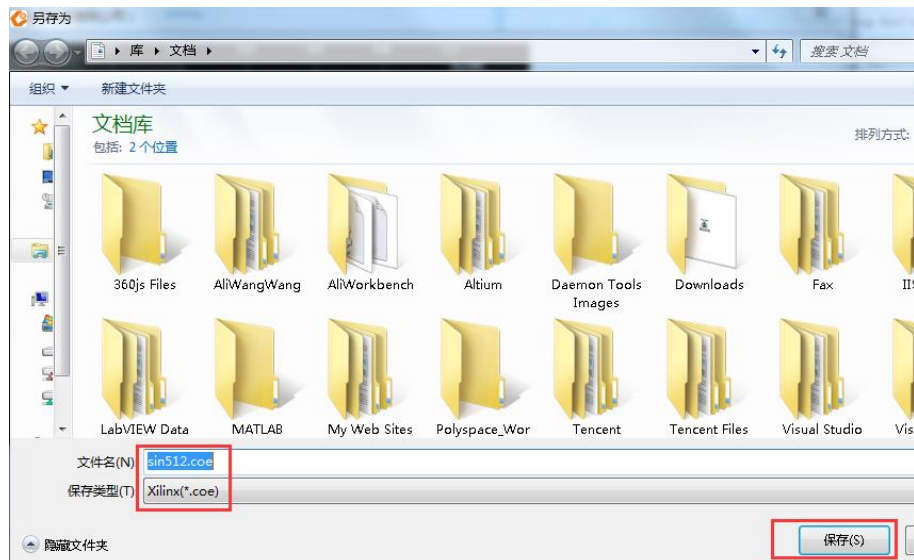
1. 双击.exe 打开工具，打开界面如下：



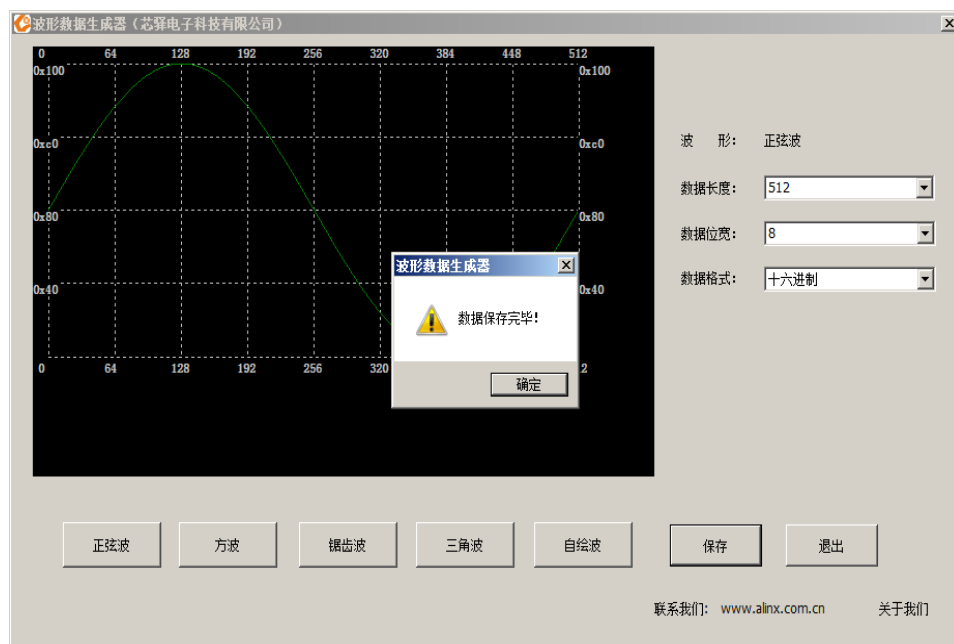
2. 可以根据需要自选波形，本例程中选择正弦波，数据长度和位宽保持默认



3. 点击保存按钮，将生成的数据文件保存到工程目录文件下（注意保存的文件类型）：



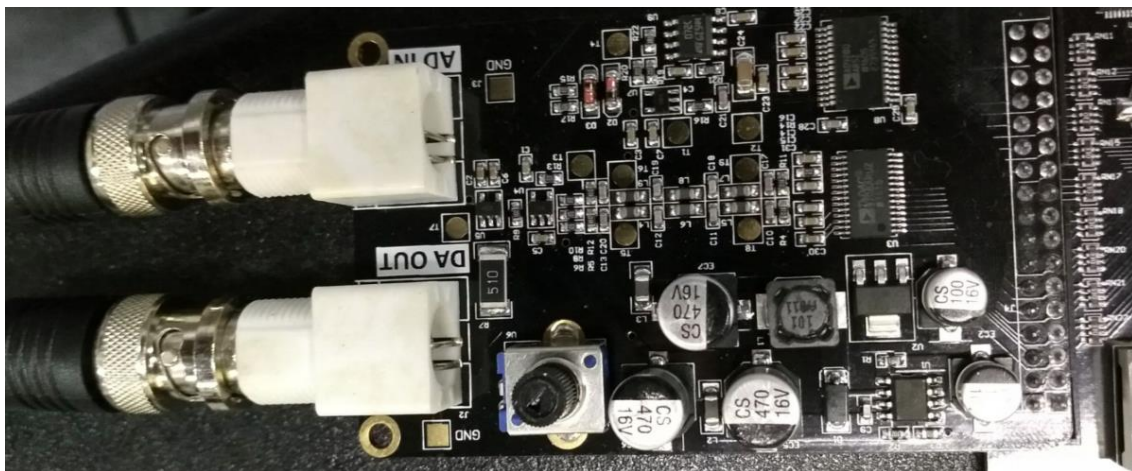
4. 保存后出现如下对话框表示保存成功，点击确定后关闭工具



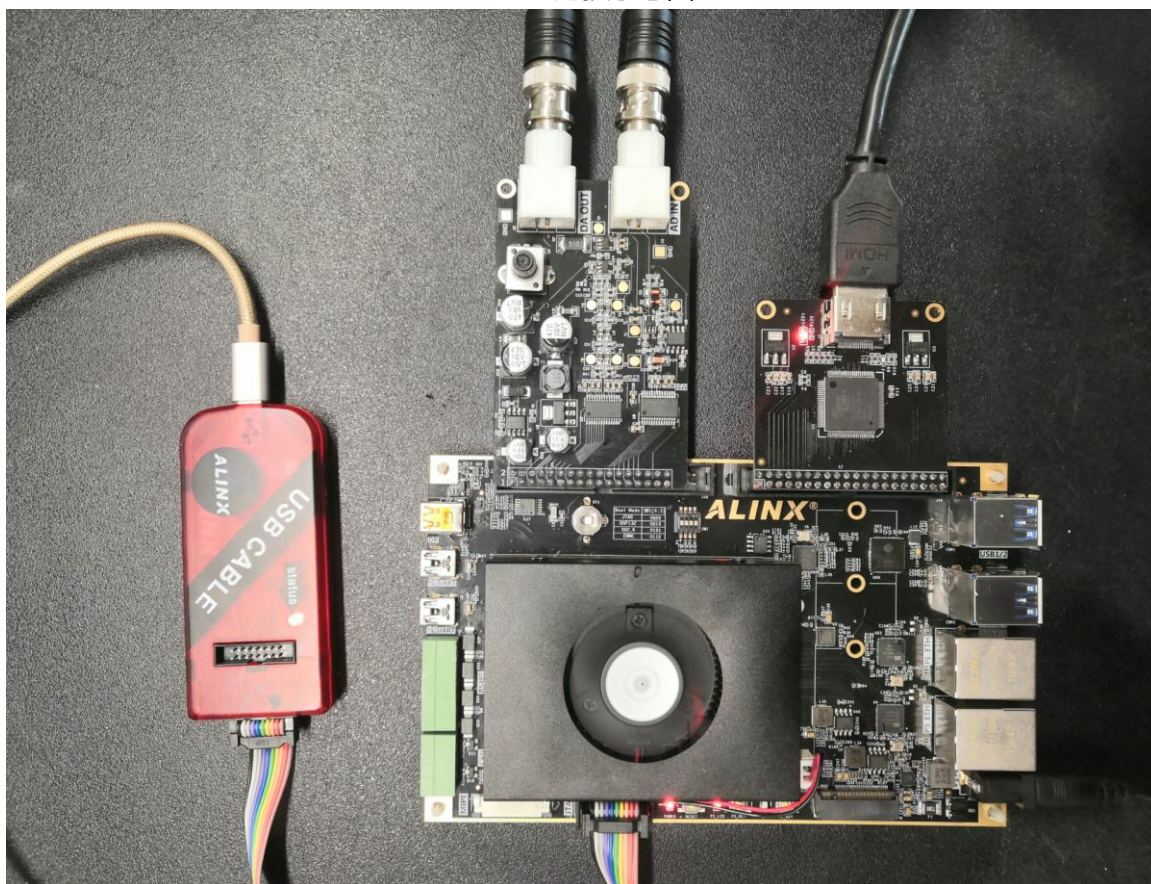
将 .coe 文件保存到生成的 Rom IP 核中即可，这里不再重复介绍

19.3 实验现象

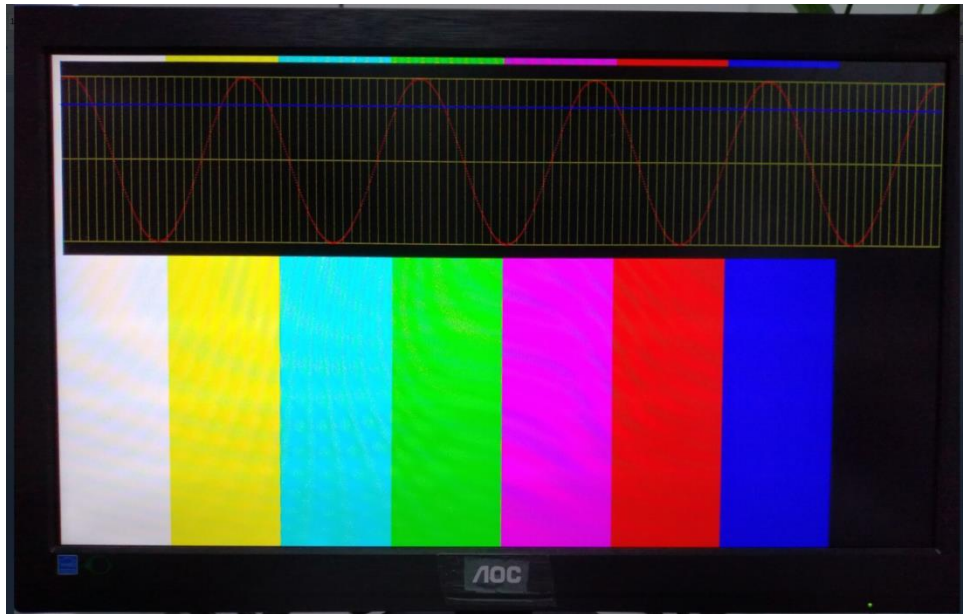
连接 AN108 的 DAC 输入到信号发生器的输出，*这里使用的是专用屏蔽线，如果使用其他线可能会有较大干扰。*



AN108 连接示意图



硬件连接图



第二十章 AD9767 双通道正弦波产生实验

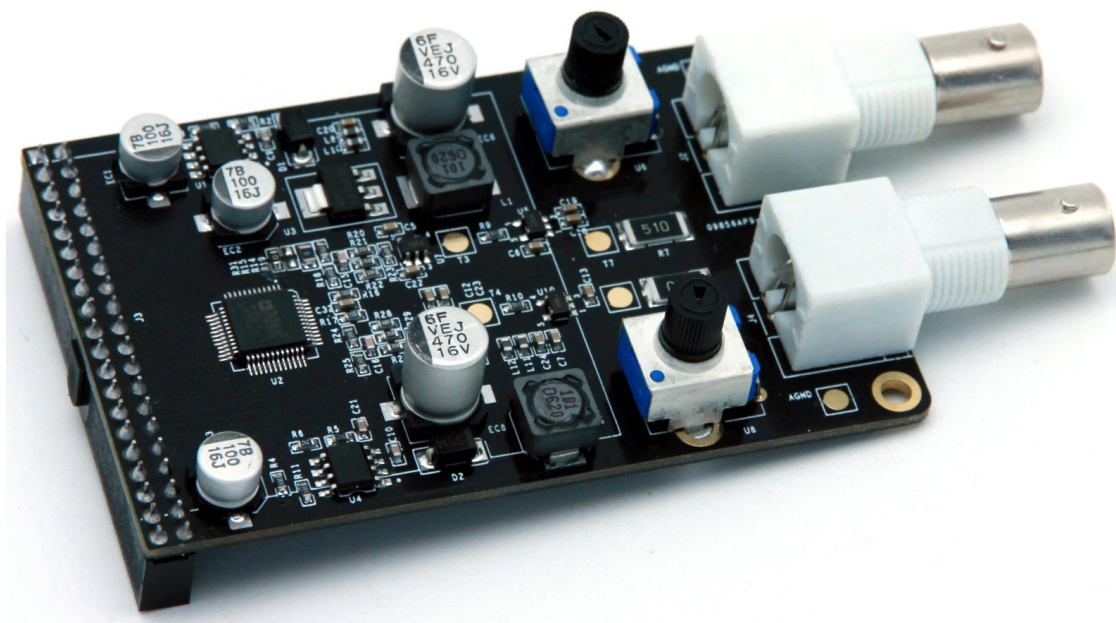
实验 Vivado 工程为 “ad9767_dual_sin_wave”。

本章介绍利用 AN9767 模块实现两路正弦波产生的实验。

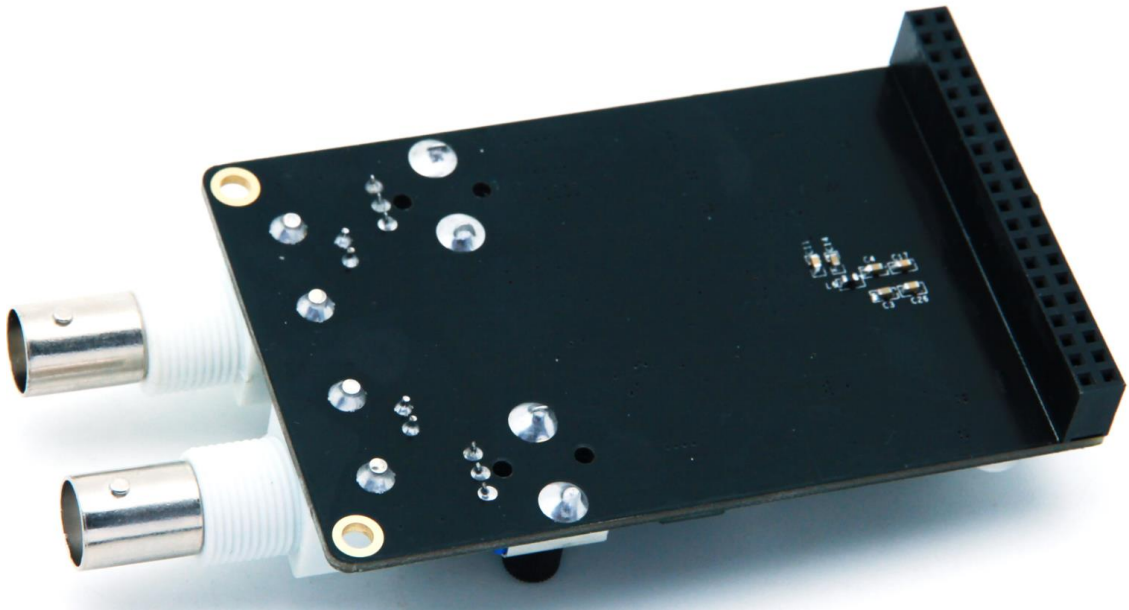
20.1 硬件介绍

双通道 14 位 DA 输出模块 AN9767 采用 ANALOG DEVICES 公司的 AD9767 芯片，支持独立双通道、14 位、125MSPS 的数模转换。模块留有一个 40 针的排母用于连接 FPGA 开发板，2 个 BNC 连接器用于模拟信号的输出。

AN9767 模块实物照片如下：



AN9767 模块正面图



AN9767 模块背面图

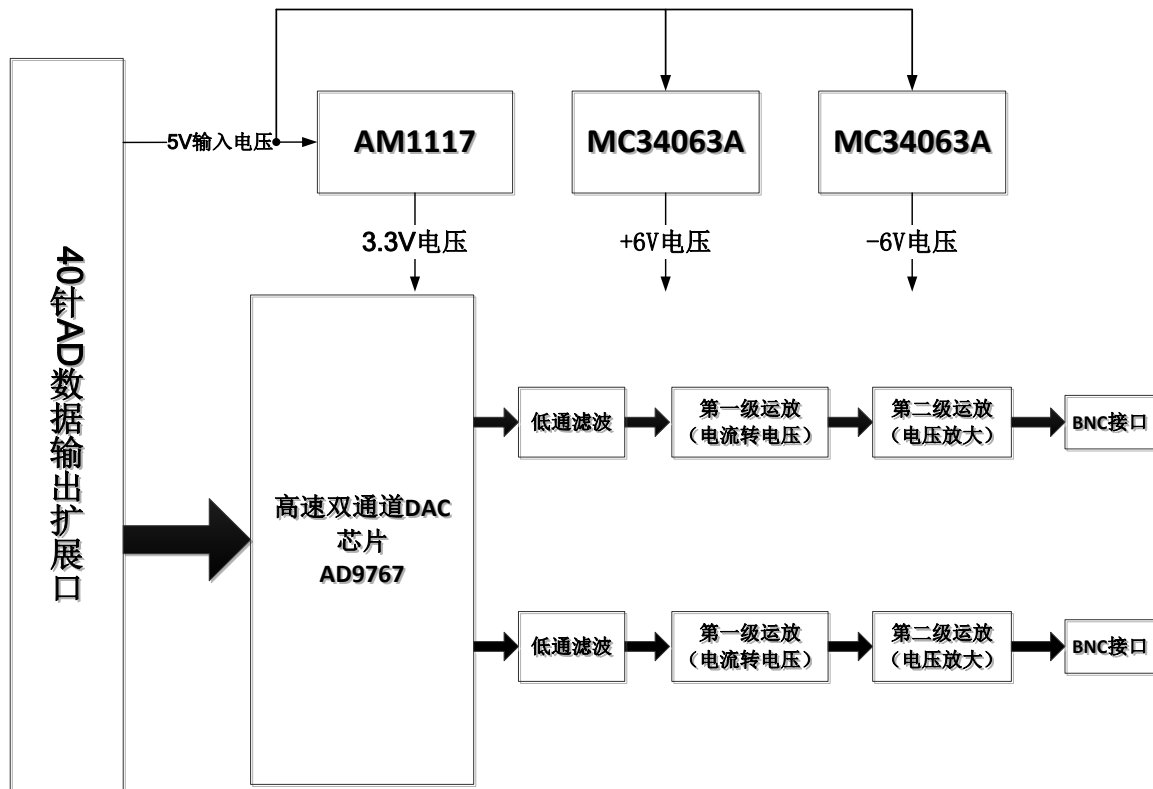
20.1.1 AN9767 模块的参数说明

以下为 AN9767 双通道 DA 模块的详细参数:

- DA 转换芯片: AD9767;
- 通道数: 2 通道;
- DA 转换位数: 14bit;
- DA 更新速率: 125 MSPS;
- 输出电压范围: -5V~+5V;
- 模块 PCB 层数: 4 层, 独立的电源层和 GND 层;
- 模块接口: 40 针 2.54mm 间距排座, 方向向下;
- 工作温度: -40°~85° 模块使用芯片均满足工业级温度范围
- 输出接口: 2 路 BNC 模拟输出接口 (用 BNC 线可以直接连接到示波器);

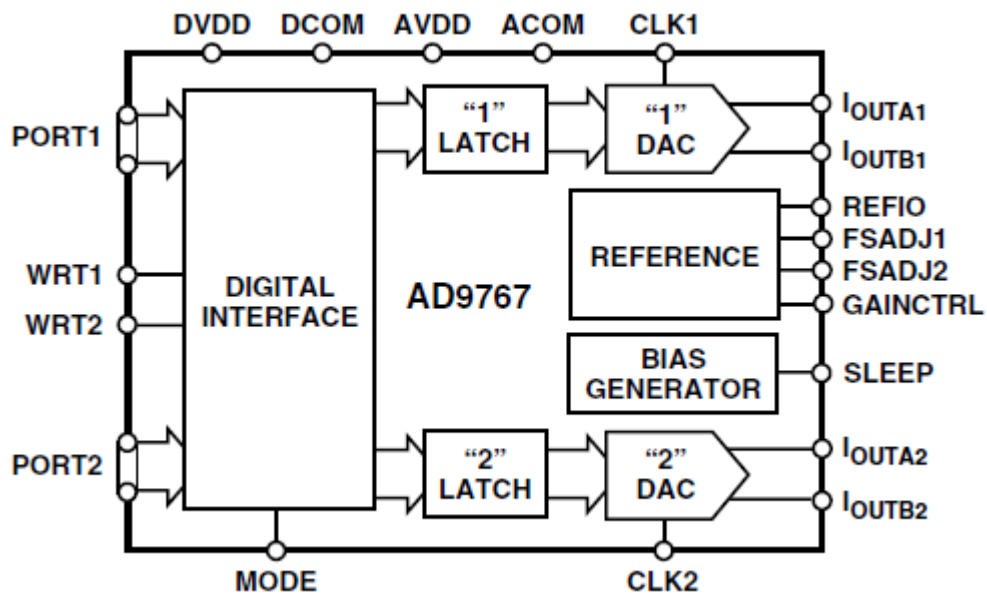
20.1.2 AN9767 模块的原理框图

AN9767 模块的原理设计框图如下:



20.1.3 AD9767 芯片简介

AD9767 是双端口、高速、双通道、14 位 CMOS DAC，芯片集成两个高品质 TxDAC+®内核、一个基准电压源和数字接口电路，采用 48 引脚小型 LQFP 封装。器件提供出色的交流和直流性能，同时支持最高 125 MSPS 的更新速率。AD9767 的功能框图如下：



20.1.4 电流电压转换及放大

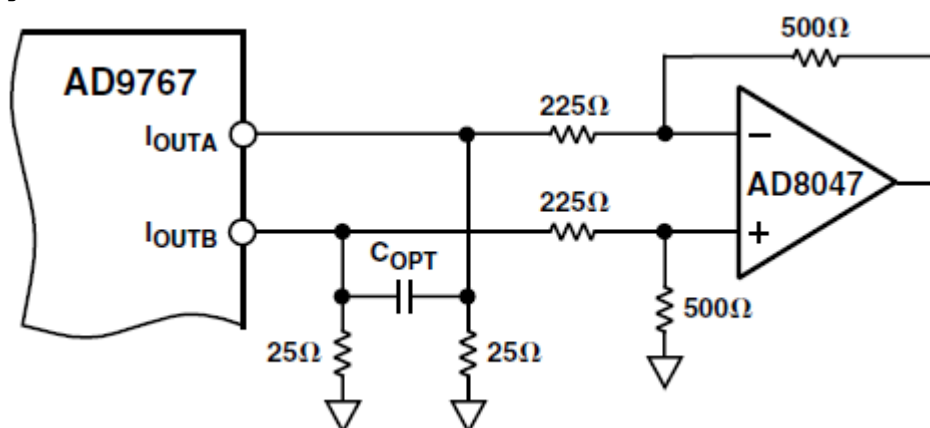
AD9767 的两路 DA 输出都为补码形式的电流输出 IoutA 和 IoutB。当 AD9767 数字输入为满量程时 (DAC 的输入的 14 位数据都为高), IoutA 输出满量程的电流输出 20mA。IoutB 输出的电流为 0mA。具体的电流和 DAC 的数据的关系如下公式所示:

$$I_{OUTA} = (DAC\ CODE / 16384) \times I_{OUTFS}$$

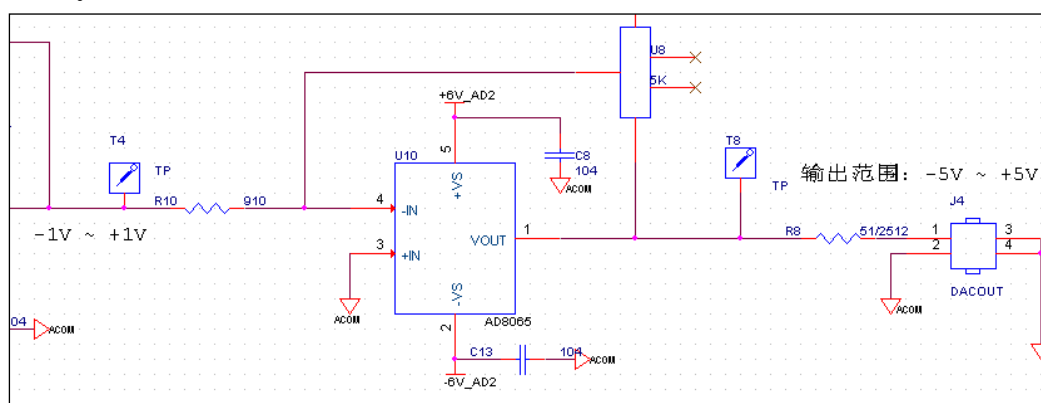
$$I_{OUTB} = (16383 - DAC\ CODE) / 16384 \times I_{OUTFS}$$

其中 IoutFS=32 x Iref, 在 AN9767 模块设计中, Iref 的值由电阻 R16 的值决定, 如果 R16=19.2K, 那 Iref 的值就是 0.625mA。这样 IoutFS 的值就是 20mA。

AD9767 输出的电流通过第一级运放 AD6045 转换成 -1V~+1V 的电压。具体的转换电路如下图所示:



第一级运放转换后的 -1V~+1V 的电压通过第二级运放变换到更高幅度的电压信号, 这个运放的幅度大小可以通过调整板上的可调电阻来改变。通过第二级运放, 模拟信号的输出范围高达 -5V~+5V。



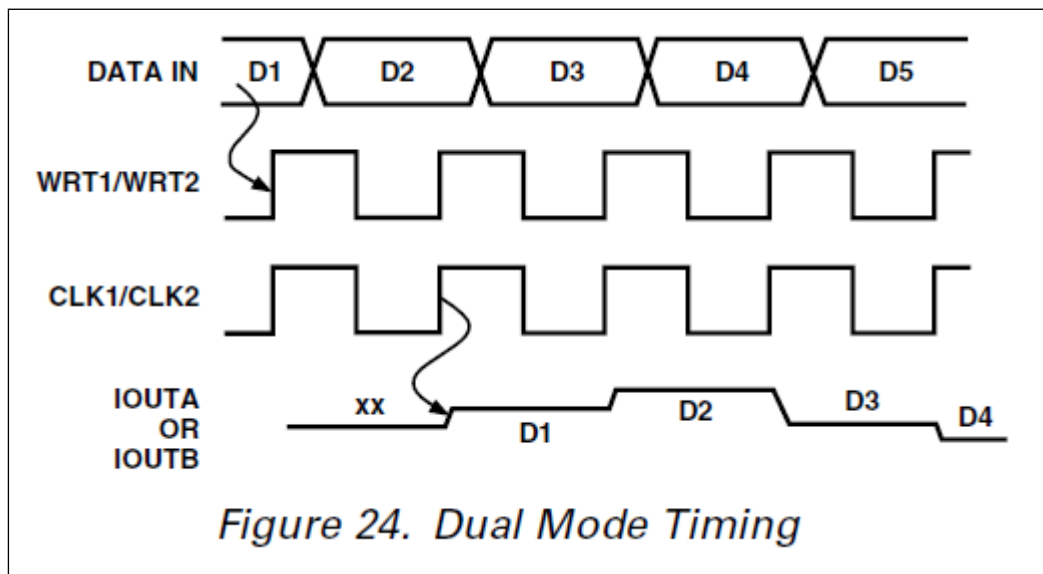
下表为数字输入信号和各级运放输出后的电压对照表:

DAC 数据输入值	AD9767 电流输出	第一级运放输出	第二级运放输出
3fff(14 位全高)	+20mA	-1V	+5V
0(14 位全低)	-20mA	+1V	-5V

2000 (中间值)	0mA	0V	0V
------------	-----	----	----

20.1.5 电流电压转换及放大

AD9767 芯片的数字接口可以通过芯片的模式管脚(MODE)来配置成双端口模式(Dual)或者交叉(Interleaved)模式。在 AN9767 模块设计中, AD9767 芯片是工作在双端口模式, 双通道的 DA 数字输入接口是独立分开的。双端口模式(Dual)的数据时序图如下图所示:

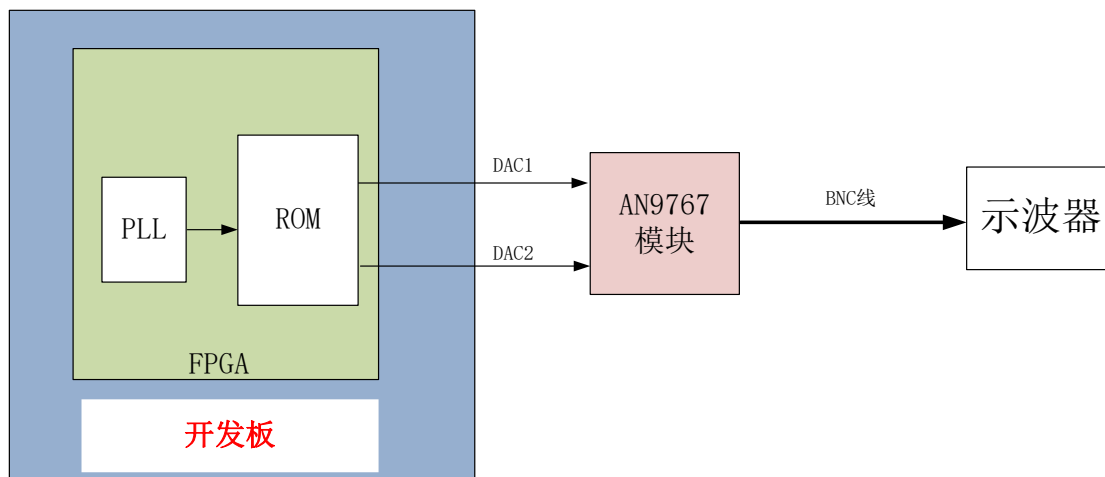


给 AD9767 芯片的 DA 数据通过时钟 CLK 和写信号 WRT 的上升沿输入到芯片进行 DA 转换。

20.2 程序设计

例程中提供了 AN9767 模块的 DA 测试程序, 通过 AN9767 模块来实现正弦波信号的输出。

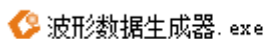
正选波测试程序是通过读取 FPGA 内部的一个 ROM 中存储的正选波数据, 然后把正选波的数据输出到 AN9767 模块进行数模的转换, 从而得到正选波的模拟信号。正选波测试程序的示意图如下:



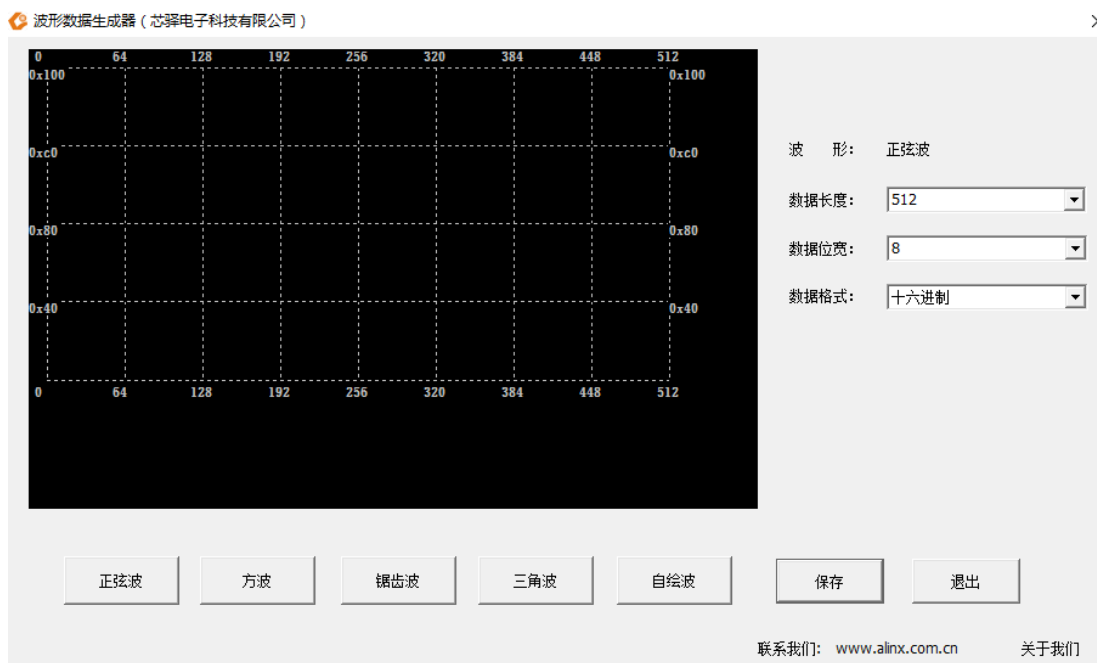
20.2.1 生成 ROM 初始化文件

程序中我们会用到一个 ROM 用于存储 1024 个 14 位的正弦波数据, 首先我们需要准备 ROM 的初始化文件(如果是 ALTERA 开发板的话是 mif 文件, 如果是 Xilinx 开发板的话是 coe 文件)。以下为生成正弦波 ROM 数据文件的方法:

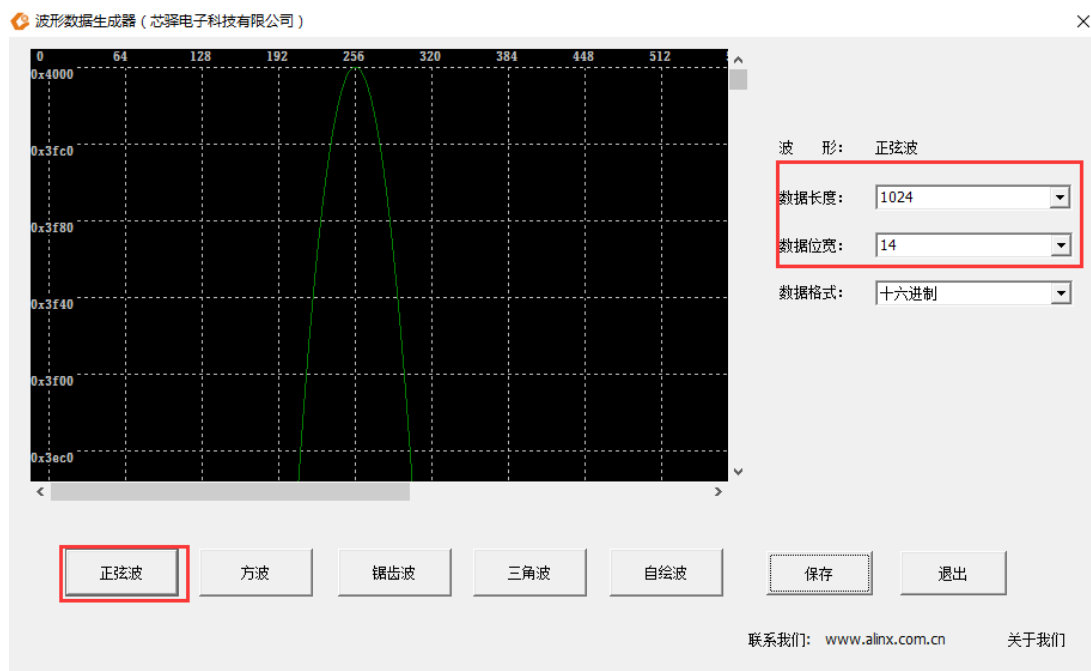
在软件工具及驱动文件夹下找到工具, 其图标如下所示:



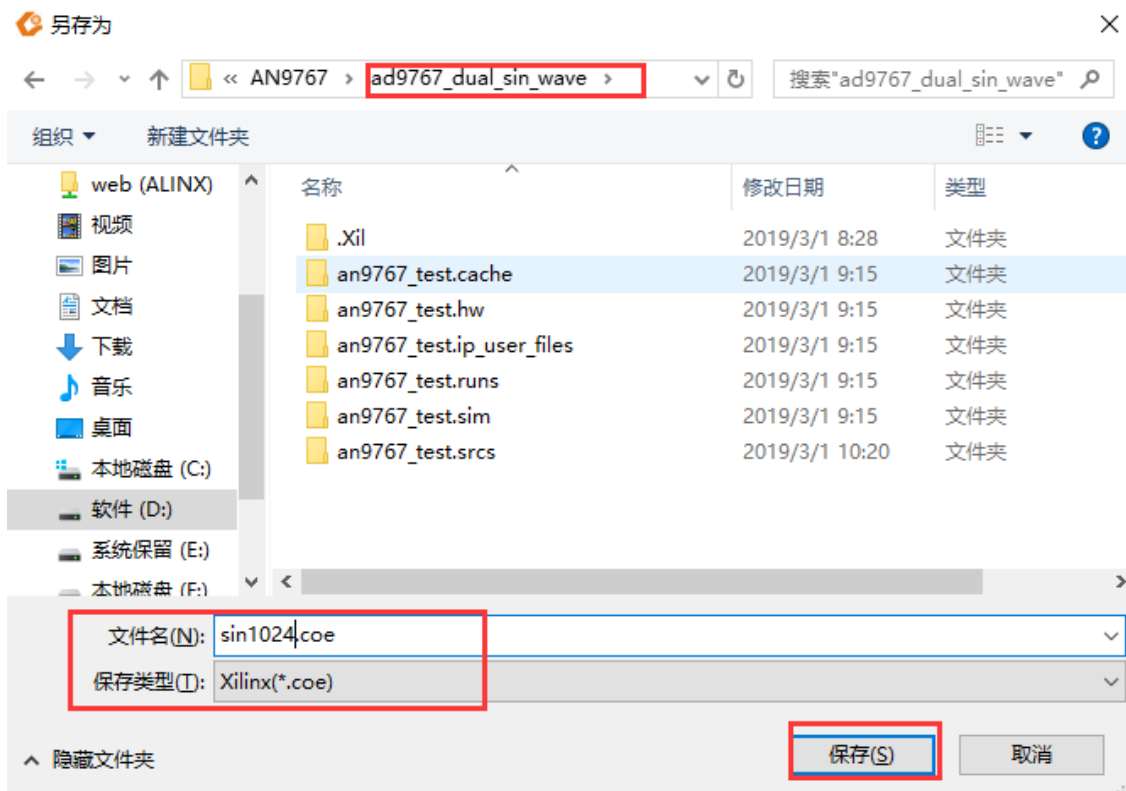
1. 双击.exe 打开工具, 打开界面如下:



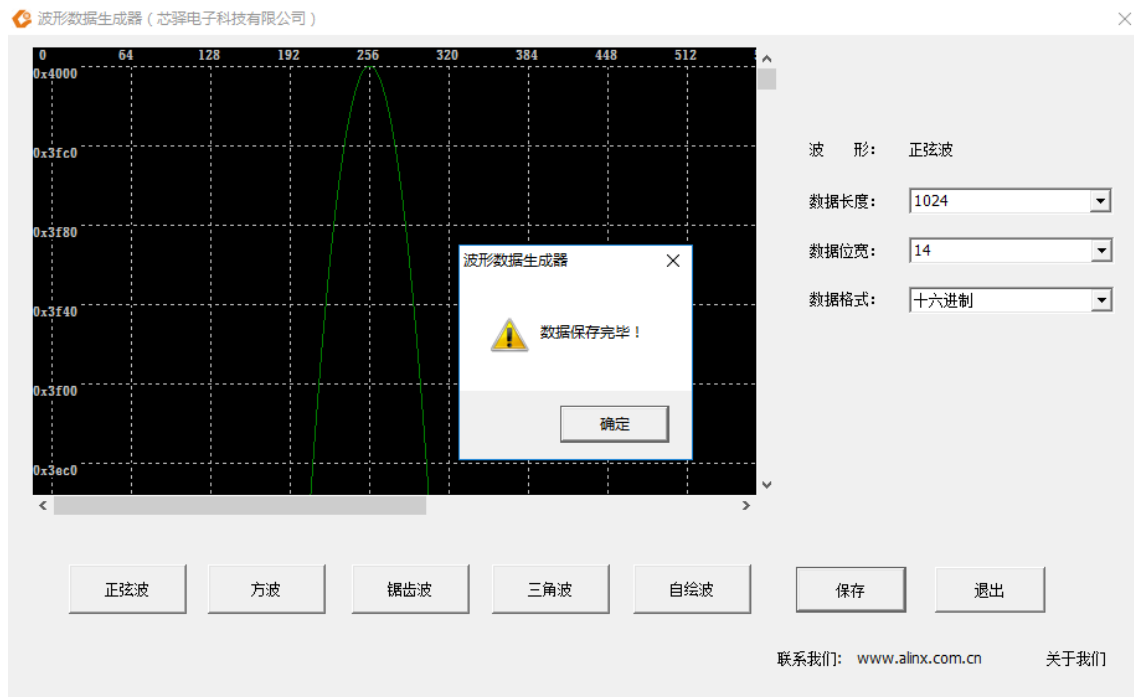
2. 可以根据需要自选波形, 本例程中选择正弦波, 数据长度 1024, 数据位宽 14, 其它默认:



3. 点击保存按钮，将生成的数据文件保存到工程目录文件下（注意保存的文件类型）：



4. 保存后出现如下对话框表示保存成功，点击确定后关闭工具



将 .coe 文件保存到生成的 Rom IP 核中即可，在字符显示实验教程中已做介绍，这里不再重复。

20.2.2 双通道正弦波发生程序

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
//Two sine wave outputs -10V ~ +10V
/////////////////////////////////////////////////////////////////
module ad9767_test
(
//Differential system clock
    input                sys_clk_p,
    input                sys_clk_n,
    output da1_clk,      //AD9767 CH1 clock
    output da1_wrt,      //AD9767 CH1 enable
    output [13:0] da1_data, //AD9767 CH1 data output

    output da2_clk,      //AD9767 CH2 clock
    output da2_wrt,      //AD9767 CH2 enable
    output [13:0] da2_data //AD9767 CH2 data output
);

reg [9:0] rom_addr;

wire [13:0] rom_data;
wire clk_125M;

assign da1_clk=clk_125M;
assign da1_wrt=clk_125M;
assign da1_data=rom_data;

assign da2_clk=clk_125M;
assign da2_wrt=clk_125M;
```

```
assign da2_data=rom_data;

//DA output sin waveform
always @(negedge clk_125M)
begin
    rom_addr <= rom_addr + 1'b1 ;           //The output sine wave frequency is 122Khz
    // rom_addr <= rom_addr + 4 ;           //The output sine wave frequency is 488Khz
    // rom_addr <= rom_addr + 128 ;        //The output sine wave frequency is 15.6Mhz
end

ROM ROM_inst
(
    .clka(clk_125M) , // input clka
    .addra(rom_addr) , // input [8:0] addra
    .douta(rom_data) // output [7:0] douta
);

PLL PLL_inst
(// Clock in ports
    .clk_in1_p (sys_clk_p) , // IN
    .clk_in1_n (sys_clk_n) , // IN
    // Clock out ports
    .clk_out1 ( ) , // OUT
    .clk_out2 (clk_125M) , // OUT
    // Status and control signals
    .reset (1'b0) , // IN
    .locked ( )
);

endmodule
```

程序中通过一个 PLL IP 来产生 125M 的 DA 输出时钟，然后就是循环读取存放在 ROM 中的 1024 个数据，并同时输出到通道 1 和通道 2 的 DA 数据线上。程序中可以通过地址的加 1，加 4，或者加 128 来选择输出不同的频率的正弦波。

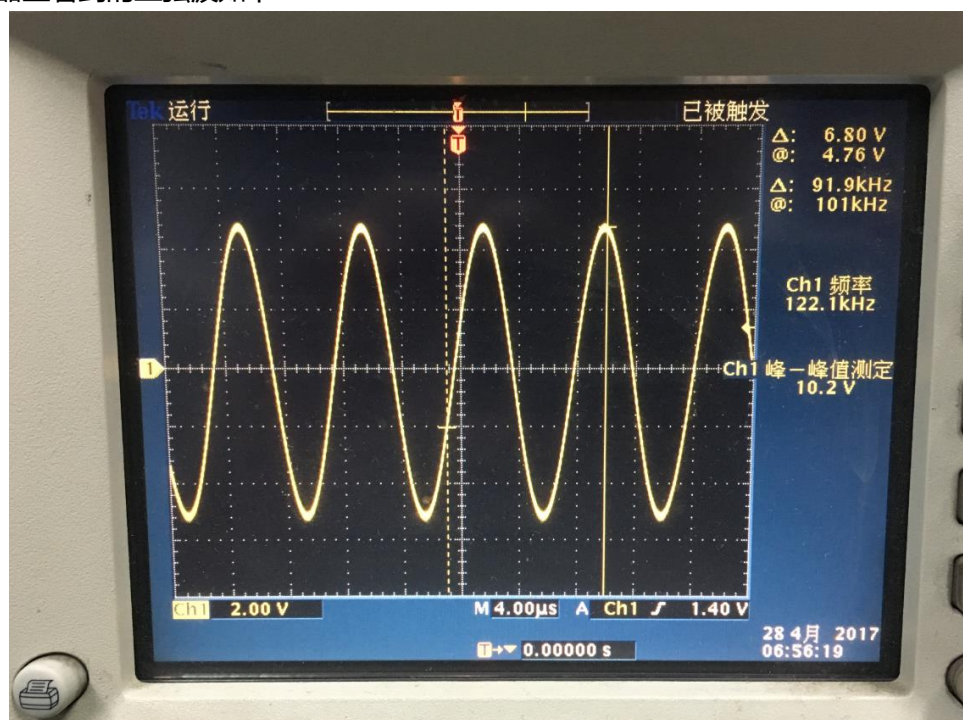
20.3 实验现象

将 AN9767 模块插入开发板的 J11 扩展口，用我们提供的 BNC 线连接 AN9767 的输出到示波器的输入如下图，然后开发板上电，下载程序就可以从示波器上观察从 DA 模块输出的模拟信号的波形了。



硬件连接图

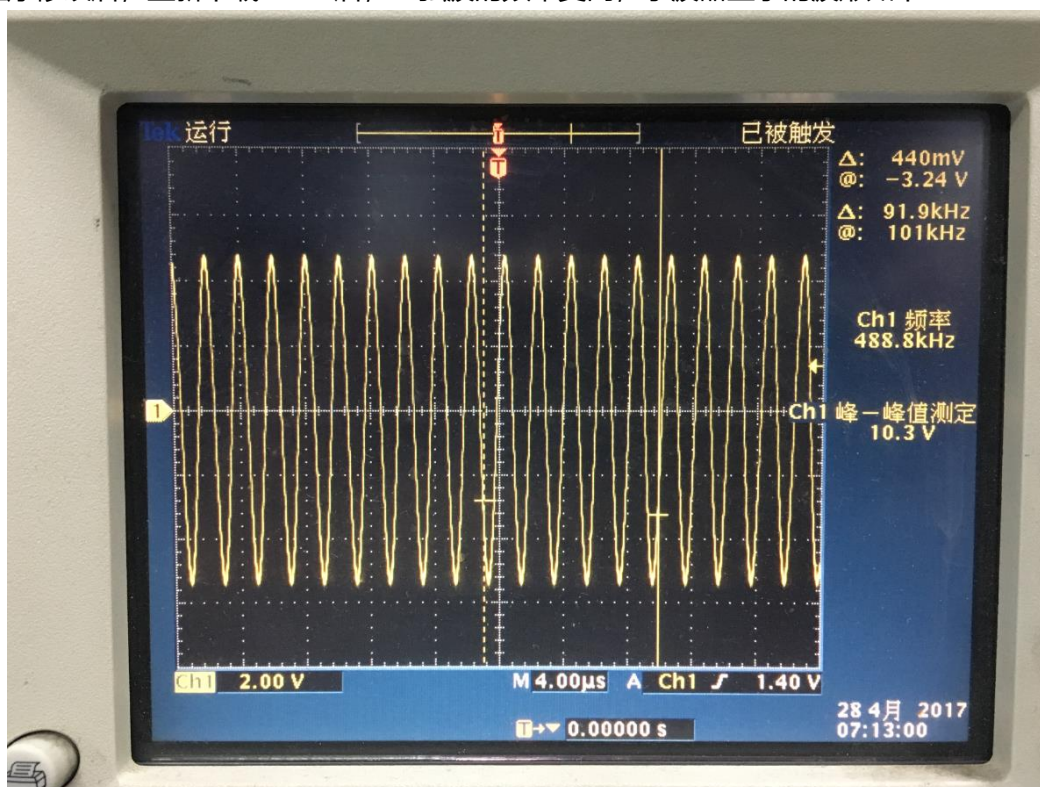
示波器上看到的正弦波如下：



我们可以把程序中的地址修改成+4 的方式，如下修改，这样一个正弦波的输出的点为 256 个，输出的正弦波的频率会提高 4 倍：

```
35  always @(negedge clk_125)
36  begin
37      // rom_addr <= rom_addr + 1'b1 ;
38      rom_addr <= rom_addr + 4 ;
39      // rom_addr <= rom_addr + 128 ;
40
41
42
```

程序修改后，重新下载 FPGA 后，正弦波的频率变高，示波器显示的波形如下：



用户也可以通过调节 AN9767 模块上的可调电阻来改变 2 个通道输出波形的幅度。

